周明德

微型计算机系统原理及应用（第四版）习题集、习题解答与实验指导

（修订版）

清华大学出版社

本书是《微型计算机系统原理及应用》（第四版）的配套习题集、习题解答与实验指导用书。全书分为两大部分：第一部分是教材各章的习题与解答；第二部分是实验指导。实验分为两类：一类是汇编语言程序设计，包括汇编语言程序设计过程中的各种典型问题；另一类是在TPC-H实验装置上开发的各种接口芯片独立的和组合的实验，共16个实验。所有这些，对于巩固和深入理解教材的内容，提高独立思考、独立分析问题的能力，都是十分有益的。

本习题集收题很广、很全面。有的习题有相当难度。本修订版增加了全部习题的解答，便于读者参考与学习。习题解答包含了大量的十分有用的程序，对今后的学习和工作会有很大帮助。

本书适合作为各类高等院校、各种成人教育学校和培训班的教材，也可供广大科技人员参考。

ISBN 7-302-11078-6

9 787302 110781 >

定价：19.00元

周明德

微型计算机系统原理及应用（第四版）习题集、习题解答与实验指导

（修订版）

清华大学出版社
北京

内 容 简 介

本书是《微型计算机系统原理及应用》(第四版)的配套习题集、习题解答与实验指导用书。全书分为两大部分:第一部分是主教材各章的习题与解答;第二部分是实验指导。实验分为两类:一类是汇编语言程序设计,包括汇编语言程序设计过程中的各种典型问题;另一类是在 TPC-H 实验装置上开发的各种接口芯片独立的和组合的实验,共 16 个实验。所有这些,对于巩固和深入理解教材的内容,提高独立思考、独立分析问题的能力,都是十分有益的。

本习题集收题广泛、全面,有的习题有相当难度。本修订版增加了全部习题的解答,以便于读者参考和学习。习题解答中包含了大量的十分有用的程序,对今后的学习和工作会有很大帮助。

本书适合作为各类高等院校、各种成人教育学校和培训班的教材,也可供广大科技人员参考。

版权所有,翻印必究。举报电话:010-62782989 13501256678 13801310933

本书封面贴有清华大学出版社防伪标签,无标签者不得销售。

本书防伪标签采用特殊防伪技术,用户可通过在图案表面涂抹清水,图案消失,水干后图案复现;或将表面膜揭下,放在白纸上用彩笔涂抹,图案在白纸上再现的方法识别真伪。

图书在版编目(CIP)数据

微型计算机系统原理及应用(第四版)习题集、习题解答与实验指导/周明德编著. —修订本. —北京:清华大学出版社,2005.7

ISBN 7-302-11078-6

I. 微… II. 周… III. 微型计算机—高等学校—教学参考资料 IV. TP36

中国版本图书馆 CIP 数据核字(2005)第 050990 号

出 版 者:清华大学出版社 地 址:北京清华大学学研大厦
http://www.tup.com.cn 邮 编:100084
社 总 机:010-62770175 客户服务:010-62776969
责任编辑:张瑞庆
封面设计:常雪影
印 刷 者:北京国马印刷厂
装 订 者:北京鑫海金澳胶印有限公司
发 行 者:新华书店总店北京发行所
开 本:185×260 印张:12.75 字数:295 千字
版 次:2005 年 7 月第 1 版 2005 年 7 月第1 次印刷
书 号:ISBN 7-302-11078-6/TP·7335
印 数:1~8000
定 价:19.00 元

修订版前言

本书自发行以来得到了广大读者的爱护与帮助。有些读者反映，习题有一定的难度，希望能够提供解答，以利于大家的学习。

本书习题部分收题很广、很全面，有相当一部分习题难度较大，本意是供大家思考，检查学习效果，并未要求读者都能做。确实，若读者能把所有习题都做正确了，可以说是真真掌握了教材的内容，而且具有了相当的分析问题和解决问题的能力。

为了满足读者的要求，也为了便于读者参考与学习。本修订版提供了所有习题的解答。希望大家带着问题来看解答，考虑为什么要这样解答。这样，对大家的学习会有更多的帮助。

习题解答中有大量有实用价值的程序，对于读者的今后工作将有较大的帮助。

也许，本书的习题解答中会有不当之处，或不正确甚至错码的地方。恳请读者批评指正。

周明德

2005 年 3 月

前　言

学习微型计算机，实践是非常重要的环节。我们学习微型计算机不仅要掌握其基本原理、基本概念、基本方法，更重要的是要学以致用。目前，微型计算机已经是各行各业中十分重要又十分普遍的工具，我们要用它来解决科技进步中各类专业中的许多问题，以促进各行各业的进一步发展。

要学会应用，实践是十分重要的手段。对于汇编语言程序设计来说，只有多看例程、多做习题，从基本的方法和方面着手，积累汇编语言程序设计的典型例子，才能掌握汇编语言，学会用汇编语言编程的方法和技巧，逐步能得心应手地使用汇编语言。而且，做了习题，最好能上机调试，以确认习题答案是否正确。

对于接口电路，更是只有多做实验才能真正地学会使用。总之，实践是学好微型计算机的重要方法之一。

本实验指导书，是以清华大学同方集团教学仪器设备公司生产的 TPC-H 实验装置为基础，并且采用了他们设计的大部分实验编写而成的。在此向清华大学同方集团教学仪器设备公司 TPC-H 实验装置的开发者致谢！

周明德

2001 年 8 月

目　录

第一部分　习题与解答

第 1 章　概述 …… 2
第 2 章　IA-32 结构微处理器 …… 8
第 3 章　8086 指令系统 …… 12
第 4 章　汇编语言程序设计 …… 20
第 5 章　处理器总线时序和系统总线 …… 68
第 6 章　主存储器 …… 76
第 7 章　输入和输出 …… 82
第 8 章　中断 …… 93
第 9 章　计数器和定时器电路 Intel 8253/8254-PIT …… 106
第 10 章　并行接口芯片 8255 …… 111
第 11 章　串行通信及接口电路 …… 117
第 12 章　数模(D/A)转换与模数(A/D)转换接口 …… 125

第二部分　实 验 指 导

第 13 章　汇编语言程序练习 …… 128
13.1　运行汇编语言的源程序的过程 …… 128
13.1.1　汇编程序 …… 129
13.1.2　连接程序 LINK …… 131
13.2　DOS 功能调用 …… 133
13.2.1　什么是 DOS 功能调用 …… 133
13.2.2　DOS 功能调用清单 …… 133
13.2.3　使用功能调用的方法 …… 134
13.2.4　有关 I/O 的功能调用 …… 145
13.3　汇编语言程序实验 …… 147
13.3.1　字符输入输出及字符串显示 …… 147
13.3.2　码转换程序 …… 149
13.3.3　有关列和表的程序设计 …… 150
第 14 章　接口电路实验 …… 159
14.1　TPC-H 实验装置介绍 …… 159
14.1.1　系统组成与安装 …… 159

14.1.2 实验台结构…… 160
14.2 接口实验…… 165
实验一 I/O 地址译码 …… 166
实验二 简单并行接口…… 167
实验三 可编程计数器/定时器(8253) …… 169
实验四 可编程并行接口(一)(8255 方式 0)…… 171
实验五 可编程并行接口 8255 与七段数码管 …… 172
实验六 用可编程并行接口 8255 控制继电器 …… 173
实验七 交通灯控制实验…… 175
实验八 中断…… 176
实验九 可编程并行接口(二)(8255 方式 1)…… 178
实验十 数模(D/A)转换器…… 179
实验十一 模数(A/D)转换器…… 181
实验十二 数字录音机…… 183
实验十三 串行通信…… 185
实验十四 DMA 传送 …… 186
实验十五 步进电机控制实验…… 188
实验十六 小直流电机转速控制实验…… 191
参考文献…… 193

第一部分

习题与解答

第1章 概 述

1.1 微处理器、微型计算机和微型计算机系统三者之间有什么不同?

【解】 把 CPU(运算器和控制器)用大规模集成电路技术做在一个芯片上,即为微处理器。微处理器加上一定数量的存储器和外部设备(或外部设备的接口)构成了微型计算机。微型计算机与管理、维护计算机硬件以及支持应用的软件相结合就形成了微型计算机系统。

1.2 CPU 在内部结构上由哪几部分组成? CPU 应该具备哪些主要功能?

【解】 CPU 主要由起运算器作用的算术逻辑单元、起控制器作用的指令寄存器、指令译码器、可编程逻辑阵列和标志寄存器等一些寄存器组成。其主要功能是进行算术和逻辑运算以及控制计算机按照程序的规定自动运行。

1.3 微型计算机采用总线结构有什么优点?

【解】 采用总线结构,扩大了数据传送的灵活性、减少了连线。而且总线可以标准化,易于兼容和工业化生产。

1.4 数据总线和地址总线在结构上有什么不同之处? 如果一个系统的数据和地址合用一套总线或者合用部分总线,那么要靠什么来区分地址和数据?

【解】 数据总线是双向的(数据既可以读也可以写),而地址总线是单向的。8086CPU 为了减少芯片的引脚数量,采用数据与地址线复用,既作数据总线也作为地址总线。它们主要靠信号的时序来区分。通常在读写数据时,总是先输出地址(指定要读或写数据的单元),过一段时间再读或写数据。

1.5 控制总线传输的信号大致有哪几种?

【解】 主要是区分读或写存储器还是外部设备、读还是写以及外界输入的 READY 和 INT 等信号。

1.6 在以下 6 个题中所用的模型机的指令系统如表 1-1 所示。

表 1-1 模型机指令系统

指令种类	助记符	机器码	功 能
数据传送	LD A,n	3E n	n→A
	LD H,n	26 n	n→H
	LD A,H	7C	H→A
	LD H,A	67	A→H
	LD A,(n)	3A n	以 n 为地址,把该单元的内容送 A,即(n)→A
	LD (n),A	32 n	把 A 的内容送至以 n 为地址的单元,A→(n)
	LD A,(H)	7E	以 H 的内容为地址,把该单元的内容送 A,(H)→A
	LD (H),A	77	把 A 的内容送至以 H 的内容为地址的单元,A→(H)

续表

指令种类	助记符	机器码	功　能
加　法	ADD　A,n	C6 n	A+n→A
	ADD　A,H	84	A+H→A
	ADD　A,(H)	86	A与以H为地址的单元的内容相加,A+(H)→A
减　法	SUB　n	D6 n	A−n→A
	SUB　H	94	A−H→A
	SUB　(H)	96	A−(H)→A
逻辑与	AND　A	A7	A∧A→A
	AND　H	A4	A∧H→A
逻辑或	OR　A	B7	A∨A→A
	OR　H	B4	A∨H→A
异　或	XOR　A	AF	A⊕A→A
	XOR　H	AC	A⊕H→A
增　量	INC　A	3C	A+1→A
	INC　H	24	H+1→H
减　量	DEC　A	3D	A−1→A
	DEC　H	25	H−1→H
无条件转移	JP　n	C3 n	n→PC
	JP　Z,n	CA n	Z=1,n→PC
	JP　NZ,n	C2 n	Z=0,n→PC
	JP　C,n	DA n	Cy=1,n→PC
	JP　NC,n	D2 n	Cy=0,n→PC
	JP　M,n	FA n	S=1,n→PC
	JP　P,n	F2 n	S=0,n→PC
停机指令	HALT	76	停机

在给定的模型机中,若有以下程序,分析在程序运行后累加器A中的值为多大。若此程序放在以10H为起始地址的存储区内,画出此程序在内存中的存储图。

```
LD      A,20H
ADD     A,15H
LD      A,30H
ADD     A,36H
ADD     A,1FH
HALT
```

【解】 程序在存储器中存放示意图如图1-1所示。

这段程序运行完后累加器 A 中的值为 A = 30H+36H+1FH = 85H。

1.7 条件和要求同题 1.6，程序如下：

```
LD      A,50H
SUB     30H
LD      A,10H
ADD     A,36H
SUB     1FH
HALT
```

分析程序运行后累加器中的值是多少，并且画出该程序在内存中的存储图。

【解】 程序在存储器中存放示意图如图 1-2 所示：

地址	内容	指令
10H	3E	LD A,20H
11H	20	
12H	C6	ADD A,15H
13H	15	
14H	3E	LD A,30H
15H	30	
16H	C6	ADD A,36H
17H	36	
18H	C6	ADD A,1FH
19H	1F	
1AH	76	HALT

图 1-1 题 1.6 程序在存储器中存放示意图

地址	内容	指令
10H	3E	LD A,50H
11H	50	
12H	D6	SUB A,30H
13H	30	
14H	3E	LD A,10H
15H	10	
16H	C6	ADD A,36H
17H	36	
18H	D6	SUB A,1FH
19H	1F	
1AH	76	HALT

图 1-2 题 1.7 程序在存储器中存放示意图

这段程序运行完后累加器 A 中的值为 A = 10H+36H−1FH = 27H。

1.8 在给定的模型机中，写出用累加的办法实现 15×15 的程序。

【解】

```
        LD      A,0
        LD      H,15
LOOP:   ADD     A,15
        DEC     H
        JP      NZ,LOOP
        HALT
```

1.9 在给定的模型机中，写出用累加的办法实现 20×20 的程序。

【解】 模型机的寄存器是 8 位，其最大能表示的值为 256。而 20×20 = 400 超出

了模型机中所能表示的最大值。故此题在模型机中无法做。需要用 8086CPU 中的寄存器才能实现。

1.10 在模型机中，用重复相减的办法实现除法的程序如下：

```
        LD      A,(M2)      ;M2 为放除数的存储单元
        LD      H,A
        XOR     A
LOOP:   LD      (M3),A      ;M3 为放商的存储单元
        LD      A,(M1)      ;M1 为放被除数(或余数)的存储单元
        SUB     H
        JP      C,DONE
        LD      (M1),A
        LD      A,(M3)
        INC     A
        JP      LOOP
DONE:   MALT
```

若此程序放在以 20H 开始的存储区，画出它的存储图。

【解】 程序在存储器中的存储图如图 1-3 所示。

地址	内容	指令
20H	3A	LD A,(M2)
21H	M2	
22H	67	LD H,A
23H	AF	XOR A
24H	32	LD (M3),A
25H	M3	
26H	3A	LD A,(M1)
27H	M1	
28H	94	SUB H
29H	DA	JP C,DONE
2AH	32	
2BH	76	LD (M1),A
2CH	M1	
2DH	3A	LD A, (M3)
2EH	M3	
2FH	3C	INC A
30H	C3	JP LOOP
31H	24	
32H	76	HALT

图 1-3 题 1.10 程序在存储器中存放示意图

1.11　在模型机中，把二进制数转换为 BCD 码的程序流程图如图 1-4 所示。

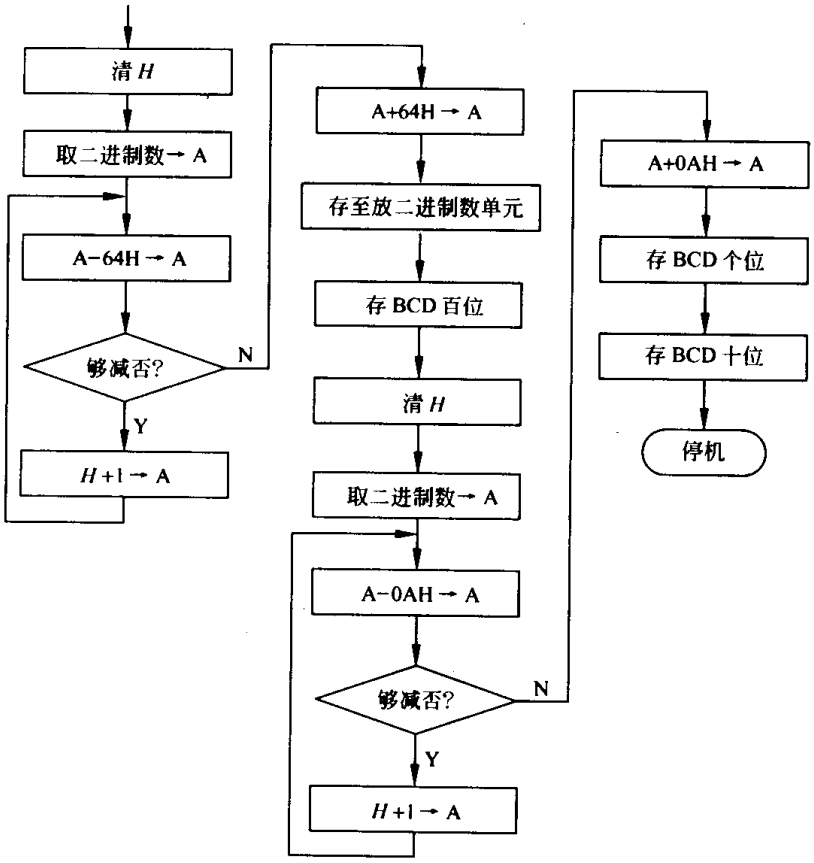


图 1-4　题 1.11 程序流程图

编写出相应的程序。

【解】

```
        LD    H,0          ;计数值 H 清零
        LD    A,(Binary)   ;把待转换的二进制数→A
S1:     SUB   A,64H        ;待转换的二进制数－100
        JP    M,S2         ;结果 < 0,转 S2
        INC   H            ;计数值加 1
        JP    S1
S2:     ADD   A,64H        ;恢复剩余的二进制数
        LD    (Binary),A   ;保存剩余的二进制数
        LD    (BCD3),H     ;保存百位 BCD 码
        LD    H,0          ;计数值 H 清零
        LD    A,( Binary)  ;剩余的二进制数→A
S3:     SUB   A,0AH        ;剩余的二进制数－10
        JP    M,S4         ;结果 < 0,转 S4
        INC   H            ;计数值加 1
```

```
        JP      S3
S4:     ADD     A,0AH       ;恢复剩余的二进制数
        LD      (BCD1),A    ;保存个位 BCD 码
        LD      (BCD2),H    ;保存十位 BCD 码
        HALT
```

第 2 章　IA-32 结构微处理器

2.1　IA-32 结构微处理器直至 Pentium 4，有哪几种？

【解】　80386、80486、Pentium、Pentium Pro、Pentium Ⅱ、Pentium Ⅲ、Pentium 4。

2.2　80386CPU 与 8086CPU 在功能上有哪些主要区别？

【解】　从 8086CPU 到 80386CPU，处理器的功能有了质的飞跃。主要体现在：

(1) 从 16 位寄存器发展为 32 位寄存器。

(2) 地址寄存器也发展为 32 位。可寻址的地址范围达到 4GB，有了巨大的扩展。

(3) 增加了保护方式。使处理器有了两种工作方式：实地址方式和保护虚地址方式。实地址方式用于与 8086 兼容；保护方式才是 32 位处理器能真正发挥其完整功能的工作方式。

(4) 引入了多任务、任务切换的概念。

(5) 引入了四级特权机制，引入了调用门、陷入门、中断门，使程序能在不同特权之间切换。

(6) 引入了存储管理单元(MMU)，使采用 80386 的操作系统能方便地实现请页(每页为 4KB)虚拟存储器管理。

(7) 增加了新指令(主要是保护方式的指令)。

2.3　从功能上，80486CPU 与 80386CPU 有哪些主要区别？

【解】　80486CPU 从功能上来说是 80386＋80387＋片上缓存。

2.4　处理器 Pentium 相对于 80486 在功能上有什么扩展？

【解】　Pentium 增加了第二条流水线，实现每个时钟执行两条指令；片上的一级缓存容量加倍；寄存器仍是 32 位，但内部数据通路是 128 位和 256 位以加速内部数据传送，且猝发的外部数据总线已经增加至 64 位；增加了高级的可编程中断控制器(advanced programmable interrupt controller，APIC)以支持多 Pentium 处理器系统。

2.5　Pentium Ⅱ以上处理器基于什么结构？

【解】　Pentium Ⅱ是基于 Pentium Pro 系列处理器又增加了 MMX 技术。

2.6　IA-32 结构微处理器有哪几种操作模式？

【解】　IA-32 结构支持 3 种操作模式：保护模式、实地址模式和系统管理模式。操作模式确定哪些指令和结构特性是可以访问的。

2.7　什么是基本执行环境？它由哪些部分构成？

【解】　在 IA-32 处理器上执行的程序或任务都给予一组执行指令的资源用于存储代码、数据和状态信息。这些资源构成了 IA-32 处理器的执行环境。

(1) 地址空间。在 IA-32 处理器上运行的任一任务或程序能寻址多至 4GB(2^{32})的线性地址空间(适用于 80386 以上的处理器，而 8086 只有 20 条地址线，只能寻址

1MB)和多至 64GB(2^{36})的物理存储器(Pentium Pro 以上的处理器)。

(2) 基本程序执行寄存器。8 个通用寄存器、6 个段寄存器、标志寄存器 EFLAGS 和 EIP(指令指针)寄存器组成了执行通用指令的基本执行环境。这些指令执行字节、字和双字整型数的基本整数算术运算,处理程序流程控制,在位和字节串上操作并寻址存储器。(这些就是 80386 处理器的操作。)

(3) x87 FPU 寄存器。8 个 80 位的 x87 FPU 数据寄存器, x87 FPU 控制寄存器、状态寄存器、x87 FPU 指令指针寄存器、x87 FPU 操作数(数据)指针寄存器、x87 FPU 标记寄存器和 x87 FPU 操作码寄存器提供执行环境以操作单精度、双精度和双扩展精度浮点数,字、双字和四字整数以及二进制编码的十进制(BCD)值。

(4) MMX 寄存器。8 个 MMX 寄存器支持在 64 位组合的字节、字和双字整数上执行单指令多数据(SIMD)操作。

(5) XMM 寄存器。8 个 XMM 数据寄存器和 MXCSR 寄存器支持在 128 位组合的单精度和双精度浮点值和 128 位组合的字节、字、双字和四字整数上执行 SIMD 操作。

(6) 堆栈(stack)。为支持过程或子例程调用并在过程或子例程之间传递参数,堆栈和堆栈管理资源包含在基本执行环境中。堆栈定位在内存中。

(7) I/O 端口。IA-32 结构支持数据在处理器和输入输出(I/O)端口之间的传送。

2.8 IA-32 结构微处理器的地址空间如何形成?

【解】 由段寄存器确定的段基地址与由各种寻址方式确定的有效地址相加形成了线性地址。若未启用分页机制,线性地址即为物理地址;若启用分页机制,则它把线性地址转换为物理地址。

2.9 基本程序执行寄存器是由哪些寄存器组成?

【解】 基本程序执行寄存器由以下寄存器组成:

(1) 通用寄存器。这 8 个寄存器能用于存放操作数和指针。

(2) 段寄存器。这些寄存器最多能保持 6 个段选择子。

(3) EFLAGS (程序状态和控制) 寄存器。EFLAGS 寄存器报告正在执行的程序的状态并允许有限地(应用程序级)控制处理器。

(4) EIP (指令指针) 寄存器。EIP 寄存器包含下一条要执行的指令的 32 位指针。

2.10 实地址方式的存储器是如何组织的? 地址如何形成?

【解】 IA-32 结构微处理器中的实地址方式是为了与 8086(8088)CPU 兼容而设置的。所以 IA-32 结构微处理器的实地址方式从存储器组织来看与 8086(8088) CPU 的存储器组织是一样的。

8086 微处理器有 20 条地址引线,它的直接寻址能力为 $2^{20}=1$MB。所以,在一个由 8086 组成的系统中,可以有多达 1MB 的存储器。这 1MB 逻辑上可以组织成一个线性矩阵。地址范围为 00000H～FFFFFH。但是,在 8086 内部的 AIU 能进行 16 位运算,有关地址的寄存器如 SP、IP,以及 BP、SI、DI 等也都是 16 位的,因而 8086 对地址的运算也只能是 16 位。这就是说,对于 8086 来说,各种寻址方式,寻找操作数的范围最多只能是 64KB。所以,整个 1MB 存储器以 64KB 为范围分为

若干段。在寻址一个具体物理单元时，必须要由一个基地址再加上由各种寻址方式确定的16位偏移量来形成实际的20位物理地址。段基地址就是由8088微处理器段寄存器中的16位数自动左移4位，然后再与16位偏移量相加，形成20位物理地址。

2.11 通用寄存器起什么作用？

【解】 通用寄存器主要用于：

(1) 逻辑和算术操作的操作数。

(2) 用于地址计算的操作数。

(3) 内存指针。

2.12 指令地址如何形成？

【解】 由段寄存器CS确定的段基地址与由指令指针确定的偏移量相加形成指令地址。

2.13 如何形成指令中的各种条件码？

【解】 指令中的条件码，即标志寄存器中的状态标志。它们主要由算术和逻辑运算指令设置或清除。指令系统中也有设置或清除状态标志的指令。

2.14 8086微处理器的总线接口部件有哪些功能？请逐一说明。

【解】 总线接口单元(BIU)负责与存储器接口；执行单元(EU)执行各种操作。BIU主要由段寄存器、指令指针、加法器(形成20位地址)和预取的指令流队列等组成。

2.15 8086微处理器的总线接口部件由哪几部分组成？

【解】 8086微处理器中的总线接口单元(BIU)负责CPU与存储器之间的信息传送。具体地说，BIU既负责从内存的指定部分取出指令，送至指令队列中排队(8086的指令队列有6个字节，而8088的指令队列只有4个字节)；也负责传送执行指令时所需的操作数。执行单元(EU)负责执行指令规定的操作。

2.16 段寄存器CS＝1200H，指令指针寄存器IP＝FF00H，此时，指令的物理地址为多少？

【解】 指令的物理地址 ＝ 12000H＋FF00H＝21F00H

2.17 8086微处理器的执行部件有什么功能？由哪几部分组成？

【解】 8086微处理器的执行部件负责指令的执行。它主要由算术逻辑单元、标志寄存器和通用寄存器等部分组成。

2.18 状态标志和控制标志有何不同？程序中是怎样利用这两类标志的？8086微处理器的状态标志和控制标志分别有哪些？

【解】 状态标志反映算术和逻辑运算的结果，主要用于各种条件。控制标志实现一些控制作用。EFLAGS寄存器有下面6个状态标志。

(1) 进位标志C(Carry Flag)

当结果的最高位(字节操作时的D7或字操作时的D15、双字操作的D31)产生一个进位或借位，则C＝1，否则为0。这个标志主要用于多字节数的加、减法运算。移位和循环指令也能够把存储器或寄存器中的最高位(左移时)或最低位(右移时)放

入标志 C 中。

(2) 辅助进位标志 A(Auxitiary Carry Flag)

在字节操作时,则由低半字节(一个字节的低 4 位)向高半字节有进位或借位,则 A=1,否则为 0。这个标志用于十进制算术运算指令中。

(3) 溢出标志 O(Overflow Flag)

在算术运算中,带符号数的运算结果超出了 8 位、16 位或 32 位带符号数所能表达的范围,即在字节运算时大于+127 或小于-128,在字运算时大于+32767 或小于-32768,在双字运算时大于$+2^{31}-1$ 或小于-2^{31},则此标志置位,否则复位。一个任选的溢出中断指令,在溢出情况下能产生中断。

(4) 符号标志 S(Sign Flag)

符号标志值与运算结果的最高位相同。即结果的最高位(字操作时为 D15)为 1,则S=1;否则,S=0。

由于在 IA-32 结构微处理器中,符号数是用补码表示的,所以 S 表示了结果的符号,0=正,1=负。

(5) 奇偶标志 P(Parity Flag)

若操作结果中"1"的个数为偶数,则 P=1,否则 P=0。这个标志可用于检查在数据传送过程中是否发生错误。

(6) 零标志 Z(Zero Flag)

若运算的结果为 0,则 Z=1,否则 Z=0

一个控制标志,即

方向标志 D(Direction Flag)

若用指令置 D=1,则引起串操作指令为自动减量指令,也就是从高地址到低地址或是"从右到左"来处理串;若使 D=0,则串操作指令就为自动增量指令。

STD 和 CLD 指令分别地设置和清除 DF 标志。

第3章　8086指令系统

3.1　分别指出下列指令中的源操作数和目的操作数的寻址方式。

(1) MOV　SI,300

(2) MOV　CX,DATA[DI]

(3) ADD　AX,[BX][SI]

(4) AND　AX,CX

(5) MOV　[BP],AX

(6) PUSHF

【解】

(1) 源操作数为立即寻址,目的操作数为寄存器寻址。

(2) 源操作数为变址寄存器加位移量寻址,目的操作数为寄存器寻址。

(3) 源操作数为基址加变址寻址,目的操作数为寄存器寻址。

(4) 源操作数和目的操作数都为寄存器寻址。

(5) 源操作数为寄存器寻址,目的操作数为寄存器间接寻址。

(6) 为堆栈操作。

3.2　试述指令 MOV AX,2000H 和 MOV AX,DS:[2000H]的区别。

【解】　前一条指令是立即寻址,即把立即数2000H传送至寄存器AX。后一条指令是直接寻址,是把数据(DS)段中的地址为2000H单元的内容传送至寄存器AX。

3.3　写出以下指令中内存操作数的所在地址。

(1) MOV　AL,[BX+10]

(2) MOV　[BP+10],AX

(3) INC　BYTE PTR[SI+5]

(4) MOV　DL,ES:[BX+SI]

(5) MOV　BX,[BP+DI+2]

【解】

(1) 数据段BX+10单元。

(2) 堆栈段BP+10单元。

(3) 数据段SI+5字节单元。

(4) 附加段(ES段)BX+SI单元。

(5) 堆栈段BP+DI+2单元。

3.4　判断下列指令书写是否正确。

(1) MOV　AL,BX

(2) MOV　AL,CL

(3) INC　[BX]

(4) MOV 5,AL

(5) MOV [BX],[SI]

(6) MOV BL,0F5H

(7) MOV DX,2000H

(8) POP CS

(9) PUSH CS

【解】

(1) 不正确,AL 与 BX 数据宽度不同。

(2) 正确。

(3) 不正确,因为不明确是增量字节还是字。

(4) 不正确,立即数不能作为目的操作数。

(5) 不正确,因为不明确要传送的是字节还是字。

(6) 正确。

(7) 正确。

(8) 不正确,CS 不能作为 POP 指令的操作数。

(9) 不正确,CS 不能作为 PUSH 指令的操作数。

3.5 设堆栈指针 SP 的初值为 1000H,AX=2000H,BX=3000H,试问:

(1) 执行指令 PUSH AX 后 SP 的值是多少?

(2) 再执行 PUSH BX 及 POP AX 后,SP、AX 和 BX 的值各是多少?

【解】

(1) SP=0FFEH。

(2) SP=0FFEH,AX=3000H,BX=3000H。

3.6 要想完成把[3000H]送[2000H]中,用指令:

```
MOV [2000H],[3000H]
```

是否正确? 如果不正确,应该用什么方法实现?

【解】 不正确。

正确的方法是:

```
MOV AL,[3000H]
MOV [2000H],AL
```

3.7 假如想从 200 中减去 AL 中的内容,用 SUB 200,AL 是否正确? 如果不正确,应该用什么方法?

【解】 不正确。

正确的方法是:

```
MOV     BL,200
SUB     BL,AL
```

3.8 试用两种方法写出从 80H 端口读入信息的指令。再用两种方法写出从 40H 口输

出 100H 的指令。

【解】

(1) IN AL,80H

(2) MOV DX,80H
IN AL,DX

(3) MOV AL,100H
OUT 40H,AL

(4) MOV AL,100H
MOV DX,40H
OUT DX,AL

3.9 假如:AL=20H,BL=10H,当执行 CMP AL,BL 后,问:

(1) AL、BL 中的内容是两个无符号数,比较结果如何? 影响哪几个标志位?

(2) AL、BL 中的内容是两个有符号数,结果又如何,影响哪几个标志位?

【解】

(1) AL=20H,BL=10H,O=0,S=0,Z=0,A=0,P=0,C=0。

(2) 因为两个都是符号正数,其结果与(1)相同。

3.10 若要使 AL×10,有哪几种方法,试编写出各自的程序段。

【解】

(1) 使用乘法指令:

```
MOV  BL,10
MUL  BL
```

(2) 使用移位指令:

```
SHL  AL,1
MOV  BL,AL
SHL  AL,2
ADD  AL,BL
```

(3) 使用加法指令:

```
ADD  AL,AL
MOV  BL,AL
ADD  AL,AL
ADD  AL,AL
ADD  AL,BL
```

3.11 8086 汇编语言指令的寻址方式有哪几类? 哪种寻址方式的指令执行速度最快?

【解】 寻址方式分为:立即数寻址方式、寄存器操作数寻址方式和存储器操作数寻址方式。其中,寄存器操作数寻址方式的指令执行速度最快。

3.12 在直接寻址方式中,一般只指出操作数的偏移地址,那么,段地址如何确定? 如果要用某个段寄存器指出段地址,指令中应该如何表示?

【解】 默认的数据访问,操作数在 DS 段;堆栈操作在 SS 段;串操作源操作数(SI)在 DS 段,目的操作数(DI)在 ES 段;用 BP 作为指针在 SS 段。如果要显式地指定

段地址，则在操作数中规定段寄存器。例如：

MOV AX,ES:(BX+10H)

3.13 在寄存器间接寻址方式中，如果指令中没有具体指明段寄存器，那么如何确定段地址？

【解】 在寄存器间接寻址方式中，如果指令中没有具体指明段寄存器，段地址是隐含约定的，也就是隐含地选择段寄存器。如操作类型为指令，则指定代码段寄存器 CS，如操作类型为堆栈操作，则指定堆栈段寄存器 SS，……，如表 3-1 中所示。

表 3-1 段寄存器的约定

操作类型	约定段寄存器	可修改的段寄存器	逻辑地址
指令	CS	无	IP
堆栈操作	SS	无	SP
源串地址	DS	CS、ES、SS	SI
目的串地址	ES	无	DI
BP 用作基址寄存器	SS	CS、DS、ES	有效地址
通用数据读写	DS	CS、ES、SS	有效地址

当需要超越约定时，通常用段替代前缀加冒号“:”来表示段超越，它允许程序设计者偏离任何约定的段。例如：

MOV ES:[EBX], EAX

这时数据将从寄存器 EAX 传送至附加段中由 EBX 寻址的存储单元，而不是传送到数据段中。

3.14 采用寄存器间接寻址方式时，BX、BP、SI、DI 分别针对什么情况来使用？这 4 个寄存器组合间接寻址时，地址是怎样计算的？请举例说明。

【解】 在寄存器间接寻址方式下，BX 和 BP 作为间址寄存器使用，而 SI、DI 作为变址寄存器使用。除 BP 间址默认的段为堆栈段，其他的都默认为数据段。它们都可以单独使用，或加上偏移量或组合使用。如：

[BX + n]
[BP + n]
[SI + n]
[DI + n]
[BX + SI + n]
[BX + DI + n]
[BP + SI + n]
[BP + DI + n]

3.15 设 DS=2100H，SS=5200H，BX=1400H，BP=6200H，说明下面两条指令所进行

的具体操作：

```
MOV BYTE PTR[BP],200
MOV WORD PTR[BX],2000
```

【解】 前一条指令是把立即数(应是字节)200，传送至堆栈段(BP 的默认段)偏移量由 BP 规定的字节单元，地址为：52000H + 6200H = 58200H 。

第二条指令是把立即数 2000，传送至数据段(BX 的默认段)偏移量由 BX 规定的字单元，地址为：21000H + 1400H = 22400H。

3.16 使用堆栈操作指令时要注意什么问题？传送指令和交换指令在涉及内存操作数时应该分别要注意什么问题？

【解】 使用堆栈指令可以把内存单元作为一个操作数(从内存到内存)。但堆栈固定在堆栈段且只能由 SP 指向。且堆栈操作要修改堆栈指针。MOV 指令不能实现内存单元间的数据传送。XCHG 指令是交换，有一个操作数必须是寄存器。

3.17 下面这些指令中哪些是正确的？哪些是错误的？若是错误的，请说明原因。

(1) XCHG CS,AX
(2) MOV [BX],[1000]
(3) XCHG BX,IP
(4) PUSH CS
(5) POP CS
(6) IN BX,DX
(7) MOV BYTE[BX],1000
(8) MOV CS,[1000]

【解】

(1) 错误，CS 不能交换。
(2) 错误，MOV 指令不能在内存间传送。
(3) 错误，IP 不能交换。
(4) 错误，CS 可以作为 PUSH 指令的操作数。
(5) 错误，CS 可以作为 POP 指令的操作数。
(6) 错误，IN 指令的目的操作数是累加器。
(7) 错误，目的操作数是字节单元。
(8) 错误，CS 不能作为 MOV 指令的目的操作数。

3.18 以下是格雷码的编码表，

0 0000
1 0001
2 0011
3 0010
4 0110
5 0111

6 0101

7 0100

8 1100

9 1101

请用换码指令和其他指令设计一个程序段，以实现由格雷码向 ASCII 码的转换。

【解】

```
        MOV   BX,TABLE
        MOV   SI,ASCII_TAB
        MOV   AL,0
        MOV   CX,10
TRAN:   XLAT  TABLE
        MOV   DL,AL
        ADD   DL,30H
        MOV   [SI],DL
        INC   AL
        LOOP  TRAN
```

3.19 使用乘法指令时，特别要注意先判断是用有符号数乘法指令还是用无符号数乘法指令，这是为什么？

【解】 因为有符号数和无符号数，其数的乘法是一样的。但结果的符号取决于两个操作数的符号。

3.20 字节扩展指令和字扩展指令一般用在什么场合？举例说明。

【解】 主要用于字节相除和字相除之前，把被除数扩展为两倍宽度的操作数。

3.21 什么叫 BCD 码？什么叫组合的 BCD 码？什么叫非组合的 BCD 码？8086 汇编语言在对 BCD 码进行加、减、乘、除运算时，采用什么方法？

【解】 BCD 码为十进制编码的二进制数。组合的 BCD 数是把两位 BCD 加在一个字节中，高位的在高 4 位。非组合的 BCD 码是把一位 BCD 数放在一个字节的低 4 位，高 4 位为 0。8086 在 BCD 加、减和乘法运算以后用 BCD 调整指令把结果调整为正确的 BCD 数。在 BCD 除法之前先用 BCD 调整指令再做除法。

3.22 用普通运算指令执行 BCD 码运算时，为什么要进行十进制调整？具体地讲，在进行 BCD 码的加、减、乘、除运算时，程序段的什么位置必须加上十进制调整指令？

【解】 因为 8086 指令把操作数作为二进制数进行二进制运算，要得到正确的 BCD 结果，需要进行调整。在加、减、乘法指令之后加上 BCD 调整指令，而在除法指令之前先用 BCD 调整指令再用除法指令。

3.23 普通移位指令和循环移位指令(带 CF 的和不带 CF 的两类)在执行操作时，有什么差别？在编制乘、除法程序时，为什么常用移位指令来代替乘、除法指令？试编写一个程序段，实现将 BX 中的数乘以 10，结果仍放在 BX 中的操作。

【解】 这两种指令的区别在于是否把标志位(CF)的值放入循环中。左移 1 位相当于乘 2，右移 1 位相当于除 2。故左移和右移可实现乘或除 2 的幂。而移位指令

比乘法和除法指令要快得多。要实现 BX 中的数乘 10，可以用乘(2 + 8)实现，乘 2 即左移 1 位，乘 8 即左移 3 位，把左移 1 位和左移 3 位的数相加就行。

```
SHL     BX,1
MOV     AX,BX
SHL     BX,2
ADD     BX,AX
```

3.24 使用串操作指令时，要特别注意与 SI、DI 这两个寄存器及方向标志 DF 密切相关。请具体就指令 MOVSB/MOVSW、CMPSB/CMPSW、SCASB/SCASW、LODSB/LODSW，列表说明与 SI、DI 及 DF 的关系。

【解】 这些指令与 SI、DI 及 DF 的关系如表 3-2 所示。

表 3-2 题 3.24 中的指令与 SI、DI 及 DF 的关系

指令	实现操作	DF	SI	DI
MOVSB	DS:[SI]→ES:[DI]	0	+1	+1
		1	−1	−1
MOVSW	DS:[SI]→ES:[DI]	0	+2	+2
		1	−2	−2
CMPSB	DS:[SI]−ES:[DI]	0	+1	+1
		1	−1	−1
CMPSW	DS:[SI]−ES:[DI]	0	+2	+2
		1	−2	−2
SCASB	AL−ES:[DI]	0	+1	+1
		1	−1	−1
SCASW	AX−ES:[DI]	0	+2	+2
		1	−2	−2
LODSB	DS:[SI]→AL	0	+1	+1
		1	−1	−1
LODSW	DS:[SI]→AX	0	+2	+2
		1	−2	−2

3.25 用串操作指令设计实现如下功能的程序段：首先将 100H 个数从 2170H 处搬到 1000H 处；然后，从中检索出相等于 AL 中字符的单元，并将此单元的值换成空格符。

【解】

```
        MOV     SI,2170H
        MOV     DI,1000H
```

```
        MOV     CX,100H
REP     MOVS    DI,SI
NEXT:   SCASB
        JZ      N1
        LOOP    NEXT
N1:     DEC     DI
        MOV     DI,20H
        JMP     NEXT
```

3.26 在使用条件转移指令时，要特别注意它们均为相对转移指令，请解释“相对转移”的含义。如果要向较远的地方进行条件转移，那么，程序中应该怎样设置？

【解】 相对转移即为相对于指令指针的转移。通常，偏移量是 8 位数，这是短(short)转移；它可以规定 16 位偏移量，这就是段内(near)转移。

3.27 带参数的返回指令用在什么场合？设栈顶地址为 3000H，当执行 RET 0006 后，SP 的值为多少？

【解】 带参数的返回指令用于跳过在调用时通过堆栈传送的参数。执行 RET 0006 指令后，SP = 2FF6H。

3.28 在执行中断指令时，堆栈的内容有什么变化？中断处理子程序的入口地址是怎样得到的？

【解】 在执行中断指令后，处理器先推入标志寄存器，再推入断点处的指令指针和 CS 值。中断处理程序的入口地址是由中断向量和中断向量(描述符)表得到的。

3.29 在执行中断返回指令 IRET 和普通子程序返回指令 RET 时，具体操作内容有什么不同？

【解】 IRET 除了弹出中断要返回的指令的地址(CS 和 IP)外，还要恢复中断前的标志寄存器。

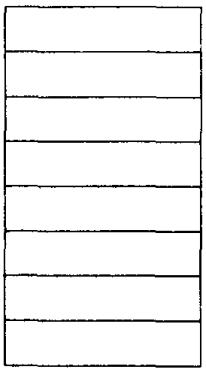
3.30 设当前 SS＝2010H，SP＝FE00H，BX＝3457H，计算当前栈顶地址为多少？当执行 PUSH BX 指令后，栈顶地址和栈顶两个字节的内容分别是什么？

【解】 当前的栈顶为：20100H＋FE00H＝2FF00H。执行 PUSH BX 后，栈顶为 2FF00H－2＝2FDFE，而 2FF00H 中的内容为 57H，2FDFF 中的内容为 34H。

第 4 章　汇编语言程序设计

4.1　在下列程序运行后，给相应的寄存器及存储单元填入运行的结果：

```
MOV     AL,10H
MOV     CX,1000H
MOV     BX,2000H
MOV     [CX],AL
XCHG    CX,BX
MOV     DH,[BX]
MOV     DL,01H
XCHG    CX,BX
MOV     [BX],DL
HLT
```



【解】　寄存器及存储单元的内容如下：

AL = 10H
BL = 00H
BH = 20H
CL = 00H
CH = 10H
DL = 01H
DH = 10H
(1000H) = 10H
(2000H) = 01H

4.2　要求同题 4.1，程序如下：

```
MOV     AL,50H
MOV     BP,1000H
MOV     BX,2000H
MOV     [BP],AL
MOV     DH,20H
MOV     [BX],DH
MOV     DL,01H
MOV     DL,[BX]
MOV     CX,3000H
HLT
```

【解】　寄存器及存储单元的内容如下：

AL = 50H

BL = 00H
BH = 20H
CL = 00H
CH = 30H
DL = 20H
DH = 20H
BP = 1000H
(1000H) = 50H
(2000H) = 20H

4.3 自 1000H 单元开始有一个 100 个数的数据块，若要把它传送到自 2000H 开始的存储区中去，可以采用以下 3 种方法实现，试分别编制程序以实现数据块的传送。

(1) 不用数据块传送指令；

(2) 用单个传送的数据块传送指令；

(3) 用数据块成组传送指令。

【解】

```
(1)     LEA   SI,1000H
        LEA   DI,2000H
        MOV   CX,100
L1:     MOV   AX,[SI]
        MOV   [DI],AX
        LOOP  L1
        HLT

(2)     LEA   SI,1000H
        LEA   DI,2000H
        MOV   CX,100
        CLD
L1:     MOVSB
        LOOP  L1
        HLT

(3)     LEA   SI,1000H
        LEA   DI,2000H
        MOV   CX,100
        CLD
REP     MOVSB
        HLT
```

4.4 利用变址寄存器，编写一个程序，把自 1000H 单元开始的 100 个数传送到自 1070H 开始的存储区中去。

【解】

```
    LEA   SI,1000H
```

```
        LEA     DI,1070H
        MOV     CX,100
        CLD
REP MOVSB
        HLT
```

4.5 要求同题 4.4，源地址为 2050H，目的地址为 2000H，数据块长度为 50。

【解】

```
        LEA     SI,2050H
        LEA     DI,2000H
        MOV     CX,50
        STD
REP MOVSB
        HLT
```

4.6 编写一个程序，把自 1000H 单元开始的 100 个数传送至 1050H 开始的存储区中（注意：数据区有重叠）。

【解】

```
        LEA     SI,1000H
        LEA     DI,1050H
        ADD     SI,63H
        ADD     DI,63H
        MOV     CX,100
        STD
REP MOVSB
        HLT
```

4.7 在自 0500H 单元开始，存有 100 个数。要求把它传送到 1000H 开始的存储区中，但在传送过程中要检查数的值，遇到第一个零就停止传送。

【解】

```
        LEA     SI,0500H
        LEA     DI,1000H
        MOV     CX,100
N1:     MOV     AL,[SI]
        TEST    AL,0
        JZ,N2
        MOV     [DI],AL
        INC     SI
        INC     DI
        LOOP    N1
N2:     HLT
```

4.8 条件同题 4.7，但在传送过程中检查数的值，零不传送，不是零则传送到目的区。

【解】

```
        LEA     SI,0500H
        LEA     DI,1000H
        MOV     CX,100
N1:     MOV     AL,[SI]
        TEST    AL,0
        JZ      N2
        MOV     [DI],AL
N2:     INC     SI
        INC     DI
        LOOP    N1
        HLT
```

4.9 把在题 4.7 中指定的数据块中的正数，传送到自 1000H 开始的存储区。

【解】

```
        LEA     SI,0500H
        LEA     DI,1000H
        MOV     CX,100
N1:     MOV     AL,[SI]
        TEST    AL,0
        JS      N2
        MOV     [DI],AL
N2:     INC     SI
        INC     DI
        LOOP    N1
        HLT
```

4.10 把在题 4.7 中指定的数据块中的正数，传送到自 1000H 开始的存储区；而把其中的负数，传送到自 1100H 开始的存储区。且分别统计正数和负数的个数，分别存入 1200H 和 1201H 单元中。

【解】

```
        LEA     SI,0500H
        LEA     DI,1000H
        LEA     BX,1100H
        MOV     CX,100
        MOV     DX,0
N1:     MOV     AL,[SI]
        TEST    AL,0
        JS      N2
        MOV     [DI],AL
        INC     DI
        INC     SI
        INC     DI
        JMP     N3
N2:     MOV     [BX],AL
```

```
        INC     DH
        INC     SI
        INC     BX
N3:     LOOP    N1
        LEA     BX,1200H
        MOV     [BX],DL
        MOV     [BX+1],DH
        HLT
```

4.11 自0500H单元开始，有10个无符号数，编写一个程序求这10个数的和(用8位数运算指令)，把和放到050A及050B单元中(和用两个字节表示)，且高位在050B单元。

【解】

```
        LEA     BX,0500H
        MOV     AX,0
        MOV     CL,10
N1:     MOV     DL,[BX]
        ADD     AL,DL
        ADC     AH,0
        INC     BX
        DEC     CL
        JNZ     N1
        MOV     050AH,AX
```

4.12 自0200H单元开始，有100个无符号数，编写一个程序求这100个数的和(用8位数运算指令)，把和放在0264H和0265H单元(和用两字节表示)，且高位在0265H单元。

【解】

```
        LEA     BX,0200H
        MOV     AX,0
        MOV     CL,100
N1:     MOV     DL,[BX]
        ADD     AL,DL
        ADC     AH,0
        INC     BX
        DEC     CL
        JNZ     N1
        MOV     0264H,AX
```

4.13 要求同题4.12，只是在累加时用16位运算指令编程序。

【解】

```
        LEA     BX,0200H
        MOV     AX,0
        MOV     CL,100
```

```
N1: MOV    DX,[BX]
    ADD    AX,DX
    INC    BX
    DEC    CL
    JNZ    N1
    MOV    0264H,AX
```

4.14 若在0500H单元中有一个数 x：

(1) 利用加法指令把它乘2，且送回原存储单元(假定 $x*2$ 后仍为一个字节)；

(2) $x*4$；

(3) $x*10$(假定 $x*10\leqslant 255$)。

【解】

```
(1)    LEA    BX,0500H
       MOV    AL,[BX]
       ADD    AL,AL
       MOV    [BX],AL

(2)    LEA    BX,0500H
       MOV    AL,[BX]
       ADD    AL,AL
       ADD    AL,AL
       MOV    [BX],AL

(3)    LEA    BX,0500H
       MOV    AL,[BX]
       ADD    AL,AL
       MOV    DL,AL
       ADD    AL,AL
       ADD    AL,AL
       ADD    AL,DL
       MOV    [BX],AL
```

4.15 题意与要求同题4.14，只是 $x*2$ 后可能为两个字节。

【解】

```
       LEA    BX,0500H
       MOV    AL,[BX]
       ADD    AL,AL
       ADC    AH,0
       MOV    [BX],AX
```

4.16 若在存储器中有两个数 a 和 b(它们所在地址用符号表示，下同)，编写一个程序实现 $a*10+b$，其中 $a*10$ 以及“和”用两字节表示。

【解】

```
       LEA    SI,DATA1
```

```
        LEA     DI,DATA2
        MOV     AX,[SI]
        ADD     AX,AX
        MOV     DX,AX
        ADD     AX,AX
        ADD     AX,AX
        ADD     AX,DX
        ADD     AX,[DI]
```

4.17 若在存储器中有数 a、b、c、d(它们连续存放),编写一个程序实现下列算式:

$$((a*10+b)*10+c)*10+d \qquad (和\leqslant 65535)$$

【解】

```
        LEA     SI,DATA
        MOV     AX,[SI]
        MOV     CX,2
N1:     ADD     AX,AX
        MOV     DX,AX
        ADD     AX,AX
        ADD     AX,AX
        ADD     AX,DX
        ADD     AX,[SI+1]
        INC     SI
        LOOP    N1
```

4.18 在 0100H 单元和 010AH 单元开始,存放两个各为 10 个字节的 BCD 数(地址最低处放的是最低字节),求出它们的和,并且把和放在 0114H 开始的存储单元中。

【解】

```
        LEA     SI,0100H
        LEA     DI,0114H
        CLC
        MOV     CL,10
N1:     MOV     AL,[SI]
        MOV     BL,10[SI]
        ADC     AL,BL
        DAA
        MOV     [DI],AL
        INC     SI
        INC     DI
        DEC     CL
        JNZ     N1
```

4.19 在 0200H 单元和 020AH 单元开始,存放两个各为 10 个字节的二进制数(地址最低处放的是最低字节),求出它们的和,并且把和放在 0214H 开始的存储单元中。

【解】

```
        LEA     SI,0200H
        LEA     DI,0214H
        CLC
        MOV     CL,10
N1:     MOV     AL,[SI]
        MOV     BL,10[SI]
        ADC     AL,BL
        MOV     [DI],AL
        INC     SI
        INC     DI
        DEC     CL
        JNZ     N1
```

4.20 在0200H单元开始放有数A(低位在前)：

$$A=95\ 43\ 78\ 62\ 31\ 04\ 56\ 28\ 91\ 01$$

在020AH单元开始放有数B：

$$B=78\ 96\ 42\ 38\ 15\ 40\ 78\ 21\ 84\ 50$$

求两数之差，且把差值送入自0200H开始的存储区。

【解】

```
        LEA     SI,0200H
        LEA     DI,020AH
        CLC
        MOV     CL,10
N1:     MOV     AL,[SI]
        MOV     BL,[DI]
        SBC     AL,BL
        MOV     [SI],AL
        INC     SI
        INC     DI
        DEC     CL
        JNZ     N1
```

4.21 若在0500H单元有一个数x，把此数的前4位变0，后4位维持不变，送回同一单元。

【解】

```
        MOV     AL,[0500H]
        AND     0FH
        MOV     [0500H],AL
```

4.22 条件同题4.21，要求最高位不变，后7位都为0。

【解】

```
        MOV     AL,[0500H]
        AND     80H
```

```
        MOV     [0500H],AL
```

4.23 若在0500H单元有一个数x，把此数的前4位变“1”，后4位维持不变，送回同一单元。

【解】

```
        MOV     AL,[0500H]
        OR      0F0H
        MOV     [0500H],AL
```

4.24 把x的最低位变“1”，高7位不变，送至0600H单元。

【解】

```
        MOV     AL,[0500H]
        OR      01H
        MOV     [0500H],AL
```

4.25 若在0500H单元有一个数x，把此数的前4位变反，后4位维持不变，送回同一单元。

【解】

```
        MOV     AL,[0500H]
        XOR     0F0H
        MOV     [0500H],AL
```

4.26 把x的最高位取反，后7位不变，送至0600H单元。

【解】

```
        MOV     AL,[0500H]
        XOR     80H
        MOV     [0600H],AL
```

4.27 从0200H单元读入一个数，检查它的符号，且在0300H单元为它建立一个符号标志(正为00H，负为FFH)。

【解】

```
        MOV     AL,[0200H]
        AND     AL,AL
        JS      MINUS
        MOV     [0300H],00H
        JMP     NEXT
MINUS:  MOV     [0300H],0FFH
NEXT:
```

4.28 若从0200H单元开始有100个数，编写一个程序检查这些数，正数保持不变，负数都取补后送回。

【解】

```
        LEA     BX,0200H
        MOV     CX,100
```

```
CHE:    MOV   AL,[BX]
        AND   AL,AL
        JS    MINUA
        INC   BX
        JMP   L1
MINUS:  NEG   AL
        MOV   [BX],AL
        INC   BX
L1:     LOOP  CHE
        HLT
```

4.29 把题 4.28 中的负数取补后送至 0300H 单元开始的存储区。

【解】

```
        LEA   BX,0200H
        LEA   DI,0300H
        MOV   CX,100
CHE:    MOV   AL,[BX]
        AND   AL,AL
        JS    MINUA
        INC   BX
        JMP   L1
MINUS:  NEG   AL
        MOV   [DI],AL
        INC   BX
        INC   DI
L1:     LOOP  CHE
        HLT
```

4.30 若在 0200H 和 0201H 单元中有一个双字节数，编写一个程序对它求补。

【解】

```
        MOV   AX,[0200H]
        NEG   AX
        MOV   [0200H],AX
```

4.31 在 BX 寄存器对中有一个双字节数，对它求补。

【解】

```
        NEG   BX
```

4.32 若在 0200H～0203H 单元中有一个四字节数，编写一个程序对它求补。

【解】

```
        LEA   BX,0200H
        MOV   AX,[BX]
        NEG   AX
        MOV   [BX],AX
        MOV   AX,[BX+2]
```

```
        MOV     DX,0
        SBC     DX,AX
        MOV     [BX+2],DX
```

4.33 若在0200H和0201H单元中有两个正数，编写一个程序比较它们的大小，并且把大的数放在0201H单元中。

【解】

```
        MOV     AL,[0200H]
        CMP     AL,[0201H]
        JNS     LAG
        JMP     NEXT
LAG:    XCHG    AL,[0201H]
NEXT:
```

4.34 条件同题4.33，把较小的数放在0201H单元中。

【解】

```
        MOV     AL,[0200H]
        CMP     AL,[0201H]
        JS      LESS
        JMP     NEXT
LESS:   XCHG    AL,[0201H]
NEXT:
```

4.35 条件与要求同题4.33，只是两个数为无符号数。

【解】

```
        MOV     AL,[0200H]
        CMP     AL,[0201H]
        JNC     LAG
        JMP     NEXT
LAG:    XCHG    AL,[0201H]
NEXT:
```

4.36 条件与要求同题4.33，相比较的是两个带符号数。

【解】

```
        MOV     AL,[0200H]
        CMP     AL,[0201H]
        JG      LAG
        JMP     NEXT
LAG:    XCHG    AL,[0201H]
NEXT:
```

4.37 若自0500H单元开始有1000个带符号数，把它们的最小值找出来，并且放在1000H单元中。

【解】

```
        LEA     BX,[0500H]
        MOV     AX,[BX]
        INC     BX
        INC     BX
        MOV     CX,999
AGAIN:  CMP     AX.[BX]
        JL      NEXT
        MOV     AX.[BX]
NEXT:   INC     BX
        INC     BX
        LOOP    AGAIN
        MOV     [1000H],AX
        HLT
```

4.38 若自 1000H 单元开始有 1000 个无符号数，把它们的最大值找出来，放在 2000H 单元中。

【解】

```
        LEA     BX,[1000H]
        MOV     AX,[BX]
        INC     BX
        INC     BX
        MOV     CX,999
AGAIN:  CMP     AX.[BX]
        JNC     NEXT
        MOV     AX.[BX]
NEXT:   INC     BX
        INC     BX
        LOOP    AGAIN
        MOV     [2000H],AX
        HLT
```

4.39 若在 0200H 单元中有一个数 x，用移位方法实现：

(1) $x*2$；

(2) $x*4$ ($x*4\leqslant 255$)。

且送回原单元。

【解】

```
        SHL     [0200H],1
        SHL     [0200H],1
```

4.40 编写一个程序，使寄存器对 BX 中的数整个左移一位。

【解】

```
        SHL     BX,1
```

4.41 编写一个程序，使寄存器对 BP 中的数整个右移一位(最高位维持不变)。

【解】

```
        SAR     BP,1
```

4.42 在0200H单元中有一个数 x，利用移位和相加的办法，使 $x*10$（假定 $x*10\leqslant 255$）后送回原单元。

【解】

```
        MOV     AL,[0200H]
        SHL     AL,1
        MOV     DL,AL
        SHL     AL,2
        ADD     AL,DL
        MOV     [0200H],AL
```

4.43 条件和要求同题4.42，但 $x*10$ 后可大于255。

【解】

```
        MOV     AX,[0200H]
        SHL     AX,1
        MOV     DX,AX
        SHL     AX,2
        ADD     AX,DX
        MOV     [0200H],AX
```

4.44 在0200H和0201H单元中，存有一个两字节数（高位在后），编写一个程序把它们整个右移一位。

【解】

```
        SAR     [0201H],1
        RCR     [0200H],1
```

4.45 在自BUFFER单元开始，放有一个数据块，BUFFER和BUFFER+1单元中放的是数据块的长度，自BUFFER+2开始存放的是以ASCII码表示的十进制数码，把它们转换为BCD码，并且把两个相邻单元的数码并成一个单元（地址高的放在高4位），放到自BUFFER+2开始的存储区中。

【解】

```
        MOV     CX,[BUFFER]
        LEA     SI,BUFFER+2
        LEA     DI,BUFFER+2
TRAN:   MOV     AL,[SI]
        SUB     AL,30H
        MOV     BL,AL
        INC     SI
        MOV     AL,[SI]
        SUB     AL,30H
        SHL     AL,4
        ADD     AL,BL
```

```
        MOV     [DI],AL
        INC     SI
        INC     DI
        DEC     CX
        DEC     CX
        JNZ     TRAN
```

4.46 在自 BUFFER 单元开始，放有一个数据块，BUFFER 和 BUFFER+1 单元中放的是数据的长度，自 BUFFER+2 开始存放数据，每一单元放的是两位 BCD 码，把它们分别转换为 ASCII 码，放到自 BLOCK 开始的存储区中（低 4 位 BCD 码转换成的 ASCII 码放在地址低的单元），而 BLOCK 和 BLOCK+1 放转换成的 ASCII 码的长度。

【解】

```
        MOV     CX,[BUFFER]
        MOV     DX,CX
        ADD     DX,DX
        LEA     SI,BUFFER+2
        LEA     DI,BLOCK+2
        MOV     [BLOCK],DX
TRAN:   MOV     AL,[SI]
        MOV     BL,AL
        AND     AL,0FH
        ADD     AL,30H
        MOV     [DI],AL
        INC     DI
        MOV     AL,BL
        SHR     AL,4
        ADD     AL,30H
        MOV     [DI],AL
        INC     SI
        INC     DI
        DEC     CX
        JNZ     TRAN
```

4.47 在自 BUFFER 单元开始，放有一个数据块，BUFFER 和 BUFFER+1 单元中放的是数据块的长度，自 BUFFER+2 开始存放的是以 ASCII 码表示的十六进制数码（即 0～9，A～F），把它们转换为十六进制数码，并且放在同一单元中。

【解】

```
        MOV     CX,[BUFFER]
        LEA     SI,BUFFER+2
TRAN:   MOV     AL,[SI]
        SUB     AL,30H
        MOV     [SI],AL
```

```
        INC     SI
        LOOP    TRAN
```

4.48 条件同题 4.47，把转换以后的两个相邻的十六进制数并在一个存储单元中。

【解】

```
        MOV     CX,[BUFFER]
        LEA     SI,BUFFER+2
        LEA     DI,BUFFER+2
TRAN:   MOV     AL,[SI]
        SUB     AL,30H
        MOV     BL,AL
        INC     SI
        MOV     AL,[SI]
        SUB     AL,30H
        SHL     AL,4
        ADD     AL,BL
        MOV     [DI],AL
        INC     SI
        INC     DI
        DEC     CX
        DEC     CX
        JNZ     TRAN
```

4.49 若在某存储区中已输入 4 个以 ASCII 码表示的十六进制数码(高位在前)，把它们转换为二进制数放入 BX 寄存器对中。

【解】

```
        LEA     SI,DATA
        MOV     AL,[SI]
        SUB     AL,30H
        MOV     BL,AL
        MOV     AL,[SI+1]
        SUB     AL,30H
        SHL     AL,4
        AND     BL,AL
        MOV     AH,[SI+2]
        SUB     AH,30H
        MOV     BH,AH
        MOV     AH,[SI+3]
        SUB     AH,30H
        SHL     AH,4
        AND     BH,AH
```

4.50 在自 BUFFER 单元开始的数据块中，前两个单元放的是数据块的长度，自 BUFFER+2 开始存放的是二进制的数据块。把每一个存储单元的两位十六进制

数，分别转换为各自的 ASCII 码，并且放到自 BLOCK 开始的存储区中（开始两个单元放新的数据块的长度）。

【解】

```
        MOV   CX,[BUFFER]
        MOV   DX,CX
        ADD   DX,DX
        LEA   SI,BUFFER+2
        LEA   DI,BLOCK+2
        MOV   [BLOCK],DX
TRAN:   MOV   AL,[SI]
        MOV   DL,AL
        AND   AL,0FH
        ADD   AL,30H
        MOV   [DI],AL
        INC   DI
        MOV   AL,DL
        SHR   AL,4
        ADD   AL,30H
        MOV   [DI],AL
        INC   SI
        INC   DI
        LOOP  TRAN
```

4.51 在题 4.45 中，把相邻单元的两个数码看成是两位十进制数（后面的为 10 位数），把它们转换为相应的二进制数，并且放到自 BUFFER+2 开始的存储区中。

【解】

```
        MOV   CX,[BUFFER]
        LEA   SI,BUFFER+2
        LEA   DI,BUFFER+2
TRAN:   MOV   AL,[SI]
        SUB   AL,30H
        MOV   BL,AL
        INC   SI
        MOV   AL,[SI]
        SUB   AL,30H
        ADD   AL,AL
        MOV   DL,AL
        ADD   AL,AL
        ADD   AL,AL
        ADD   AL,DL
        ADD   AL,BL
        MOV   [DI],AL
        INC   SI
```

```
        INC     DI
        DEC     CX
        DEC     CX
        JNZ     TRAN
```

4.52 在题 4.50 中，把数据块中的每一单元的二进制数转换为相应的 BCD 码(每一字节的二进制数，对应 3 位 BCD 码)，再把它们转换为 ASCII 码放到自 BLOCK 开始的存储区中(开始两个单元放新的数据块的长度)。

【解】

```
        MOV     CX,[BUFFER]
        MOV     DX,CX
        ADD     DX,DX
        ADD     DX,CX
        LEA     SI,BUFFER+2
        LEA     DI,BLOCK+2
        MOV     [BLOCK],DX
        MOV     DL,0
TRAN:   MOV     AL,[SI]
TRAN1:  SUB     AL,64H
        JS      N1
        INC     DL
        JMP     TRAN1
N1:     ADD     AL,64H
        ADD     DL,30H
        MOV     [DI],DL
        INC     DI
        MOV     DL,0
N2:     SUB     AL,0AH
        JS      N3
        INC     DL
        JMP     N2
N3:     ADD     AL,0AH
        ADD     DL,30H
        MOV     [DI],DL
        INC     DI
        ADD     AL,30H
        MOV     [DI],AL
        INC     SI
        INC     DI
        LOOP    TRAN
```

4.53 若在 BX 寄存器中放有 4 位 BCD 码，把它们转换为相应的二进制数，并放在 DX 寄存器中。

【解】 采用(((千位数×10)+百位数)×10+十位数)×10+个位数的方法转换。

```
        MOV     AX,BX
        AND     AX,0F000H
        SHR     AX,12
        MOV     DX,AX
        CALL    CHANGE
        MOV     AX,DX
        MOV     CX,BX
        AND     CX,0F00H
        SHR     CX,8
        ADD     AX,CX
        MOV     DX,AX
        CALL    CHANGE
        MOV     AX,DX
        MOV     CX,BX
        AND     CX,00F0H
        SHR     CX,4
        ADD     AX,CX
        MOV     DX,AX
        CALL    CHANGE
        MOV     AX,DX
        AND     BX,000FH
        ADD     AX,BX
        MOV     DX,AX
CHANGE  PROC
        ADD     DX,DX
        MOV     CX,DX
        ADD     DX,DX
        ADD     DX,DX
        ADD     DX,CX
        RET
CHANGE  ENDP
```

4.54 若在 BX 寄存器中有一个 16 位无符号数，把它们转换为相应的 BCD 码，并放到自 DATA 开始的存储区中(每一位 BCD 码占一个存储单元，高位在前)。

【解】 用在 BX 寄存器的 16 位无符号数有多少个 10000、1000、100、10，再把它们以 BCD 数表示来转换。

```
        LEA     DI,DATA
        MOV     DL,0
T1:     SUB     BX,10000
        JC      N1
        INC     DL
        JMP     T1
N1:     ADD     BX,10000
        MOV     [DI],DL
```

```
        INC     DI
        MOV     DL,0
T2:     SUB     BX,1000
        JC      N2
        INC     DL
        JMP     T2
N2:     ADD     BX,1000
        MOV     [DI],DL
        INC     DI
        MOV     DL,0
T3:     SUB     BX,100
        JC      N3
        INC     DL
        JMP     T3
N3:     ADD     BX,100
        MOV     [DI],DL
        INC     DI
        MOV     DL,0
T4:     SUB     BX,10
        JC      N4
        INC     DL
        JMP     T4
N4:     ADD     BX,10
        MOV     [DI],DL
        INC     DI
        MOV     [DI],BL
```

4.55 若在 BX 寄存器中有一个 16 位带符号数，把它们转换为相应的 BCD 码，并放在自 DATA 开始的存储区中（符号占一个单元，每一位 BCD 码占一个存储单元，高位在前）。

【解】

```
        LEA     DI,DATA
        MOV     DL,0
        AND     BX,BX
        JS      MINUS
        MOV     AL,00H
        MOV     [DI],AL
        JMP     PLUS
MINUS:  MOV     AL,0FFH
        MOV     [DI],AL
PLUS:   INC     DI
        AND     BX,7FFFH
T1:     SUB     BX,10000
        JC      N1
```

```
        INC     DL
        JMP     T1
N1:     ADD     BX,10000
        MOV     [DI],DL
        INC     DI
        MOV     DL,0
T2:     SUB     BX,1000
        JC      N2
        INC     DL
        JMP     T2
N2:     ADD     BX,1000
        MOV     [DI],DL
        INC     DI
        MOV     DL,0
T3:     SUB     BX,100
        JC      N3
        INC     DL
        JMP     T3
N3:     ADD     BX,100
        MOV     [DI],DL
        INC     DI
        MOV     DL,0
T4:     SUB     BX,10
        JC      N4
        INC     DL
        JMP     T4
N4:     ADD     BX,10
        MOV     [DI],DL
        INC     DI
        MOV     [DI],BL
```

4.56 若自 STRING 单元开始存放一个字符串(以字符 $ 结尾),编写一个程序统计这个字符串的长度(不包括 $ 字符);并且把字符串的长度,放在 STRING 单元,把整个字符串往下移两个存储单元。

【解】

```
        LEA     DI,STRING
        XOR     DX,DX
        MOV     AL,'$'
        CLD
AGAIN:  SCASB
        JE      N1
        INC     DX
        JMP     AGAIN
N1:     LEA     SI,STRING+2
```

```
        XCHG    SI,DI
        MOV     CX,DX
    REP MOVSB
        MOV     [STRING],DX
```

4.57 若自 STRING 单元开始存放一个字符串(以字符空格引导,以 $ 结尾),编写一个程序统计这个字符串的长度(忽略前导空格和结尾的 $ 字符)。

【解】

```
        LEA     DI,STRING
        XOR     DX,DX
        MOV     AL,20H
        CLD
AGAIN:  SCASB
        JNE     N1
        JMP     AGAIN
N1:     MOV     AL,'$'
A1:     SCASB
        JE      N2
        INC     DX
        JMP     A1
N2:     MOV     [LENGTH],DX
```

4.58 在题 4.57 的字符串中,统计数字字符('0'～'9')的个数。

【解】

```
        LEA     DI,STRING
        XOR     DX,DX
        MOV     AL,'$'
        CLD
AGAIN:  SCASB
        JE      N1
        CMP     AL,'0'
        JL      A1
        CMP     AL,'9'
        JG      A1
        INC     DX
A1:     JMP     AGAIN
N1:     MOV     [RESULT],DX
```

4.59 在题 4.57 的字符串中,统计十六进制字符(即'0'～'9','A'～'F')的个数。

【解】

```
        LEA     DI,STRING
        XOR     DX,DX
        MOV     AL,'$'
        CLD
AGAIN:  SCASB
```

```
        JE      N1
        CMP     AL,'0'
        JL      A1
        CMP     AL,'9'
        JG      A2
        INC     DX
A2:     CMP     AL,'A'
        JL      A1
        CMP     AL,'F'
        JG      A1
        INC     DX
A1:     JMP     AGAIN
N1:     MOV     [RESULT],DX
```

4.60 在题 4.57 的字符串中，把十进制数字字符('0'～'9')传送至 DATA 开始的存储区中，在 DATA 和 DATA+1 单元存放的是这个数字字符串的长度，自 DATA+2 单元开始存放字符。

【解】

```
        LEA     DI,STRING
        LEA     BX,DATA+2
        XOR     DX,DX
        MOV     AL,'$'
        CLD
AGAIN:  SCASB
        JE      N1
        CMP     AL,'0'
        JL      A1
        CMP     AL,'9'
        JG      A1
        MOV     [BX],AL
        INC     BX
        INC     DX
A1:     JMP     AGAIN
N1:     MOV     [DATA],DX
```

4.61 条件和要求同题 4.60，只是传送的是十六进制数字字符。

【解】

```
        LEA     DI,STRING
        LEA     BX,DATA+2
        XOR     DX,DX
        MOV     AL,'$'
        CLD
AGAIN:  SCASB
        JE      N1
```

```
        CMP     AL,'0'
        JL      A1
        CMP     AL,'9'
        JG      A2
        INC     DX
        MOV     [BX],AL
        INC     BX
        JMP     A1
A2:     CMP     AL,'A'
        JL      A1
        CMP     AL,'F'
        JG      A1
        INC     DX
        MOV     [BX],AL
        INC     BX
A1:     JMP     AGAIN
N1:     MOV     [DATA],DX
```

4.62 在自 STRI1 和 STRI2 开始各有一个由 10 个字符组成的字符串，检查这两个字符串是否相等，在 STFLAG 单元中建立一个标志(相等为 00H，不等为 FFH)。

【解】

```
        LEA     SI,STR1
        LEA     DI,STR2
        MOV     CX,10
        CLD
    REPZ CMPSB
        JNE     UNMAT
        MOV     AL,0
        JMP     OUTPT
UNMAT:  MOV     AL,0FFH
OUTPT:  MOV     [STFLAG],AL
```

4.63 编写一个程序，统计一个 8 位二进制数中的为“1”的位的个数。

【解】 用对二进制数移位，检查 CF 标志确定是否为“1”。

```
        LEA     BX,DATA
        MOV     DL,0
        MOV     AL,[BX]
        MOV     CL,8
A1:     SHL     AL,1
        JNC     A2
        INC     DL
A2:     DEC     CL
        JNZ     A1
        MOV     [RESULT],DL
```

4.64 编写一个程序，统计一个16位二进制数中的为“1”的位的个数。

【解】

```
        LEA   BX,DATA
        MOV   DL,0
        MOV   AX,[BX]
        MOV   CL,16
A1:     SHL   AX,1
        JNC   A2
        INC   DL
A2:     DEC   CL
        JNZ   A1
        MOV   [RESULT],DL
```

4.65 在自STRI1开始有一个字符串(前两个字节为字符串长度)，对每一个字符配上偶检验位，送回原处。

【解】

```
        LEA   SI,STRI1+2
        MOV   CX,[STRI1]
CYC:    MOV   AL,[SI]
        AND   AL,AL
        JPE   N1
        OR    AL,80H
        MOV   [SI],AL
N1:     INC   SI
        DEC   CX
        JNZ   CYC
```

注：以下各题所编写的程序必须包含必要的伪指令。

4.66 自NUMBER单元开始放有两个多字节的用BCD码表示的十进制数，NUMBER单元放的是字节数，NUMBER+1开始连续存放两个多字节数(高位在后)，编写一个程序把这两个多字节数相加，和接着原来的数连续存放。

【解】

```
        NAME  ADD_BCD
DATA    SEGMENT
NUMBER  DW    LENGTH
        DB    'x1,x2,…,xn'
        DB    'y1,y2,…,yn'
        DB    LENGFH DUP(?)
DATA    ENDS
STACK   SEGMENT  PARA STACK 'STACK'
        DB    100 DUP(?)
STACK   ENDS
CODE    SEGMENT
        ASSUME  CS:CODE,DS:DATA,SS:STACK
```

```
START   PROC   FAR
BEGIN:  PUSH   DS
        MOV    AX,0
        PUSH   AX
        MOV    AX,DATA
        MOV    DS,AX
        MOV    ES,AX
        LEA    SI,NUMBER+2
        MOV    BX,SI
        MOV    CX,[NUMBER]
        ADD    BX,CX
        MOV    DX,BX
        ADD    BX,CX
        MOV    DI,BX
        MOV    BX,DX
        CLC
AGAIN:  MOV    AL,[SI]
        ADC    AL,[BX]
        DAA
        MOV    [DI],AL
        INC    SI
        INC    BX
        INC    DI
        LOOP   AGAIN
        RET
START   ENDP
CODE    ENDS
        END    BEGIN
```

4.67 编写一个程序，使放在 DATA 及 DATA+1 单元的两个 8 位带符号数相乘，并将乘积放在 DATA+2 及 DATA+3 单元中(高位在后)。

【解】

```
        NAME   MUL
DATA1   SEGMENT
DATA    DB     X1,X2
        DB     ?,?
DATA1   ENDS
STACK   SEGMENT PARA STACK 'STACK'
        DB     100 DUP(?)
STACK   ENDS
CODE    SEGMENT
        ASSUME CS:CODE,DS:DATA1,SS:STACK
START   PROC   FAR
BEGIN:  PUSH   DS
```

```
        MOV     AX,0
        PUSH    AX
        MOV     AX,DATA
        MOV     DS,AX
        MOV     AL,[DATA]
        IMUL    [DATA+1]
        MOV     [DATA+2],AL
        MOV     [DATA+3],AH
        RET
START   ENDP
CODE    ENDS
        END     BEGIN
```

4.68 编写一个程序，使放在 DATA(被除数)及 DATA+1(除数)的两个 8 位带符号数相除，商放在 DATA+2 单元，余数放在 DATA+3 单元中。

【解】

```
        NAME    DIV
DATA1   SEGMENT
DATA    DB      X1,X2
        DB      ?,?
DATA1   ENDS
STACK   SEGMENT PARA STACK 'STACK'
        DB      100 DUP(?)
STACK   ENDS
CODE    SEGMENT
        ASSUME CS:CODE,DS:DATA1,SS:STACK
START   PROC    FAR
BEGIN:  PUSH    DS
        MOV     AX,0
        PUSH    AX
        MOV     AX,DATA
        MOV     DS,AX
        MOV     AL,[DATA]
        CBW
        IDIV    [DATA+1]
        MOV     [DATA+2],AL
        MOV     [DATA+3],AH
        RET
START   ENDP
CODE    ENDS
        END     BEGIN
```

4.69 编写一个程序，使放在 DATA 和 DATA+1 的两字节无符号数与在 DATA+2 及 DATA+3 中的无符号数相乘，乘积接着原来的数存放(高位在后)。

【解】

```
        NAME  MULW
DATA1   SEGMENT
DATA    DW    X1,X2
        DB    ?,?,?,?
DATA1   ENDS
STACK   SEGMENT PARA STACK 'STACK'
        DB    100 DUP(?)
STACK   ENDS
CODE    SEGMENT
        ASSUME CS:CODE,DS:DATA1,SS:STACK
START   PROC  FAR
BEGIN:  PUSH  DS
        MOV   AX,0
        PUSH  AX
        MOV   AX,DATA
        MOV   DS,AX
        LEA   BX,DATA
        MOV   AX,[BX]
        MUL   2[BX]
        MOV   4[BX],AX
        MOV   6[BX],DX
        RET
START   ENDP
CODE    ENDS
        END   BEGIN
```

4.70 编写一个 24 位无符号数相乘的程序。

【解】 在 32 位处理器中，可以用 32 位乘法指令。把 24 位无符号数高位添 8 位“0”放在 32 位寄存器中。相乘以后的 64 位结果存放在 EDX:EAX 中。

```
        NAME  24MUL
DATA1   SEGMENT
DATA    DB    X1,X2,X3,0
        DB    Y1,Y2,Y3,0
        DB    8 DUP(?)
DATA1   ENDS
STACK   SEGMENT PARA STACK 'STACK'
        DB    100 DUP(?)
STACK   ENDS
CODE    SEGMENT
        ASSUME CS:CODE,DS:DATA1,SS:STACK
START   PROC  FAR
BEGIN:  PUSH  DS
        MOV   EAX,0
```

```
        PUSH    EAX
        MOV     AX,WORD PTR DATA
        MOV     DS,AX
        LEA     EBX,DATA
        MOV     EAX,[EBX]
        MUL     4[EBX]
        MOV     8[EBX],EAX
        MOV     12[EBX],EDX
        RET
START   ENDP
CODE    ENDS
        END     BEGIN
```

4.71 编写一个程序，使题 4.69 中的两个 16 位无符号数相除，商和余数接着原来的数存放(先放商，高位在后)。

【解】

```
        NAME    DIV_16
DATA1   SEGMENT
DATA    DW      X1,X2
        DB      ?,?,?,?
DATA1   ENDS
STACK   SEGMENT PARA STACK 'STACK'
        DB      100 DUP(?)
STACK   ENDS
CODE    SEGMENT
        ASSUME CS:CODE,DS:DATA1,SS:STACK
START   PROC    FAR
BEGIN:  PUSH    DS
        MOV     AX,0
        PUSH    AX
        MOV     AX,DATA
        MOV     DS,AX
        MOV     AX,[DATA]
        CWD
        IDIV    [DATA+2]
        MOV     [DATA+4],AX
        MOV     [DATA+6],DX
        RET
START   ENDP
CODE    ENDS
        END     BEGIN
```

4.72 在题 4.69 中，相乘的是两个 16 位的带符号数，计算这两个数的乘积。

【解】

```
        NAME  IMULW
DATA1   SEGMENT
DATA    DM    X1,X2
        DB    ?,?,?,?
DATA1   ENDS
STACK   SEGMENT PARA STACK 'STACK'
        DB    100 DUP(?)
STACK   ENDS
CODE    SEGMENT
        ASSUME CS:CODE,DS:DATA1,SS:STACK
START   PROC  FAR
BEGIN:  PUSH  DS
        MOV   AX,0
        PUSH  AX
        MOV   AX,DATA
        MOV   DS,AX
        LEA   BX,DATA
        MOV   AX,[BX]
        IMUL  2[BX]
        MOV   4[BX],AX
        MOV   6[BX],DX
        RET
START   ENDP
CODE    ENDS
        END   BEGIN
```

4.73 若在自 DATA 开始的 7 个单元中，第一个单元存放数的符号(用 ASCII 码表示)，接下来存放最多为 5 位用 ASCII 码表示的十进制数码(高位在前)，且数值在±32 768范围内，最后存放非数字字符(表示数的结束)。编写一个程序，把它转换为相应的 16 位二进制数(用补码表示)，放在 BX 寄存器中。

【解】

```
        NAME  BCD_TO_B
DATA1   SEGMENT
DATA    DB    X1,X2,X3,X4,X5,X6,X7
DATA1   ENDS
STACK   SEGMENT PARA STACK 'STACK'
        DB    100 DUP(?)
STACK   ENDS
CODE    SEGMENT
        ASSUME CS:CODE,DS:DATA1,SS:STACK
START   PROC  FAR
BEGIN:  PUSH  DS
        MOV   AX,0
        PUSH  AX
```

```
        MOV     AX,DATA
        MOV     DS,AX
        MOV     BL,0AH
        MOV     CX,0
        MOV     SI,1
        MOV     DX,0
LOP:    MOV     AL,DATA[SI]
        CMP     AL,30H
        JL      N1
        CMP     AL,39H
        JG      N1
        INC     CX
        INC     SI
        JMP     LOP
N1:     SUB     SI,CX
        MOV     AX,0
LOP1:   MOV     DL,DATA[SI]
        AND     DL,0FH
        ADD     AX,DX
        DEC     CX
        JZ      N3
        CALL    MUL10
        INC     SI
        JMP     LOP1
N3:     MOV     BL,DATA
        CMP     BL,'+'
        JZ      N2
        NEG     AX
N2:     MOV     BX,AX
        RET
START   ENDP
MUL10   PROC
        ADD     AX,AX
        PUSH    DX
        MOV     DX,AX
        ADD     AX,AX
        ADD     AX,AX
        ADD     AX,DX
        POP     DX
        RET
MUL10   ENDP
CODE    ENDS
        END     BEGIN
```

4.74 把在 BX 寄存器对中的 16 位带符号二进制数转换为十进制数，且把符号位及各位

十进制码转换为相应的 ASCII 码，放到自 DATA 开始的 7 个存储单元中(高位在前)，最后填以空格。

【解】

```
        NAME   H_TO_BCD
DATA1   SEGMENT
DATA    DB     7 DUP(?)
DATA1   ENDS
STACK   SEGMENT PARA STACK 'STACK'
        DB     100 DUP(?)
STACK   ENDS
CODE    SEGMENT
        ASSUME CS:CODE,DS:DATA1,SS:STACK
START   PROC   FAR
BEGIN:  PUSH   DS
        MOV    AX,0
        PUSH   AX
        MOV    AX,DATA
        MOV    DS,AX
        LEA    DI,DATA
        MOV    DL,0
        AND    BX,BX
        JS     MINUS
        MOV    AL,'+'
        MOV    [DI],AL
        JMP    PLUS
MINUS:  MOV    AL,'-'
        MOV    [DI],AL
PLUS:   INC    DI
        AND    BX,7FFFH
T1:     SUB    BX,10000
        JC     N1
        INC    DL
        JMP    T1
N1:     ADD    BX,10000
        ADD    DL,30H
        MOV    [DI],DL
        INC    DI
        MOV    DL,0
T2:     SUB    BX,1000
        JC     N2
        INC    DL
        JMP    T2
N2:     ADD    BX,1000
        ADD    DL,30H
```

```
        MOV     [DI],DL
        INC     DI
        MOV     DL,0
T3:     SUB     BX,100
        JC      N3
        INC     DL
        JMP     T3
N3:     ADD     BX,100
        ADD     DL,30H
        MOV     [DI],DL
        INC     DI
        MOV     DL,0
T4:     SUB     BX,10
        JC      N4
        INC     DL
        JMP     T4
N4:     ADD     BX,10
        ADD     DL,30H
        MOV     [DI],DL
        INC     DI
        ADD     BL,30H
        MOV     [DI],BL
        INC     DI
        MOV     AL,20H
        MOV     [DI],AL
        RET
START   ENDP
CODE    ENDS
        END     BEGIN
```

4.75 若自 STRING 开始有一个字符串(以'#'号作为字符串的结束标志),编写一个程序,查找此字符串中有没有字符$,有多少个$(放在 NUMBER 单元中,没有$则其为0,否则即为$的个数);并且把每一个$字符所存放的地址,放到自 POINTR 开始的连续的存储单元中。

【解】

```
        NAME    S_STRING
DATA1   SEGMENT
STRING  DB      ',,…,,#'
NUMBER  DW      ?
POINTR  DB      100 DUP(?)
DATA1   ENDS
STACK   SEGMENT PARA STACK 'STACK'
        DB      100 DUP(?)
STACK   ENDS
```

```
CODE    SEGMENT
        ASSUME CS:CODE,DS:DATA1,SS:STACK
START   PROC   FAR
BEGIN:  PUSH   DS
        MOV    AX,0
        PUSH   AX
        MOV    AX,DATA
        MOV    DS,AX
        MOV    ES,AX
        LEA    DI,STRING
        LEA    SI,NUMBER
        LEA    BX,POINTR
        CLD
        MOV    DL,0
LOP1:   SCASB
        CMP    AL,'#'
        JE     DONE
        CMP    AL,'$'
        JE     N1
        JMP    LOP1
N1:     INC    DL
        DEC    DI
        MOV    [BX],DI
        INC    BX
        INC    DI
        JMP    LOP1
DONE:   MOV    [SI],DL
        RET
START   ENDP
CODE    ENDS
        END    BEGIN
```

4.76 某一个监控程序中，有 10 个命令，分别以字母 A、B、C、D、E、F、G、H、I、J 表示。这 10 个命令有 10 个处理程序，它们的入口地址形成一个表格 CMDTBL。

```
3000        CMDTBL
3000        3500        ; A 命令入口
3002        3550        ; B 命令入口
3004        3600        ; C 命令入口
3006        3640        ; D 命令入口
3008        3670        ; E 命令入口
300A        36B0        ; F 命令入口
300C        3700        ; G 命令入口
300E        3730        ; H 命令入口
```

```
3010        3760            ;I命令入口
3012        37A0            ;J命令入口
```

若打入的命令字已在累加器 A 中，试编写一个程序，根据输入的命令字转至相应的处理程序。

【解】

```
            NAME    CMD_LIST
DATA1       SEGMENT
            ORG     3000
CMDTBL      DW      3500,3550,3600,3640,3670,36B0.3700,3730,3760,37A0
            ORG     3500
CMDA        DW      X1
            ORG     3550
CMDB        DW      X2
            ORG     3600
CMDC        DW      X3
            ORG     3640
CMDD        DW      X4
            ORG     3670
CMDE        DW      X5
            ORG     36B0
CMDF        DW      X6
            ORG     3700
CMDG        DW      X7
            ORG     3730
CMDH        DW      X8
            ORG     3760
CMDI        DW      X9
            ORG     37A0
CMDJ        DW      X10
DATA1       ENDS
STACK       SEGMENT PARA STACK 'STACK'
            DB      100 DUP(?)
STACK       ENDS
CODE        SEGMENT
            ASSUME CS:CODE,DS:DATA1,SS:STACK
START       PROC    FAR
BEGIN:      PUSH    DS
            MOV     AX,0
            PUSH    AX
            MOV     AX,DATA
            MOV     DS,AX
            CMP     AL,'A'
            JE      TRAN1
```

```
        CMP     AL,'B'
        JE      TRAN2
        CMP     AL,'C'
        JE      TRAN3
        CMP     AL,'D'
        JE      TRAN4
        CMP     AL,'E'
        JE      TRAN5
        CMP     AL,'F'
        JE      TRAN6
        CMP     AL,'G'
        JE      TRAN7
        CMP     AL,'H'
        JE      TRAN8
        CMP     AL,'I'
        JE      TRAN9
        CMP     AL,'J'
        JE      TRAN10
        JMP     DONE
TRAN1:  JMP     CMDA
TRAN2:  JMP     CMDB
TRAN3:  JMP     CMDC
TRAN4:  JMP     CMDD
TRAN5:  JMP     CMDE
TRAN6:  JMP     CMDF
TRAN7:  JMP     CMDG
TRAN8:  JMP     CMDH
TRAN9:  JMP     CMDI
TRAN10: JMP     CMDJ
DONE:   RET
START   ENDP
CODE    ENDS
        END     BEGIN
```

4.77 条件同题 4.76，只是有一个转移指令表，如下所示：

```
3000        CMDJPT
3000        JP 3500         ;转至 A 命令处理程序入口
3003        JP 3550         ;转至 B 命令处理程序入口
3006        JP 3600         ;转至 C 命令处理程序入口
3009        JP 3640         ;转至 D 命令处理程序入口
300C        JP 3670         ;转至 E 命令处理程序入口
300F        JP 36B0         ;转至 F 命令处理程序入口
3012        JP 3700         ;转至 G 命令处理程序入口
3015        JP 3730         ;转至 H 命令处理程序入口
```

```
3018        JP 3760          ;转至I命令处理程序入口
301B        JP 37A0          ;转至J命令处理程序入口
```

要求同题 4.76。

【解】

```
            NAME    JMP_LIST
DATA1       SEGMENT
            ORG     3000
CMDJPT      DB      /4,FF
            DW      3500
            DB      /4.FF
            DW      3550
            DB      /4,FF
            DW      3600
            DB      /4,FF
            DW      3640
            DB      /4,FF
            DW      3670
            DB      /4,FF
            DW      36B0
            DB      /4,FF
            DW      3700
            DB      /4,FF
            DW      3730
            DB      /4,FF
            DW      3760
            DB      /4,FF
            DW      37A0
DATA1       ENDS
STACK       SEGMENT PARA STACK 'STACK'
            DB      100 DUP(?)
STACK       ENDS
CODE        SEGMENT
            ASSUME  CS:CODE,DS:DATA1,SS:STACK
START       PROC    FAR
BEGIN:      PUSH    DS
            MOV     AX,0
            PUSH    AX
            MOV     AX,DATA
            MOV     DS,AX
            LEA     SI,CMDJPT
            CMP     AL,'A'
            JE      TRAN1
            CMP     AL,'B'
```

```
                JE      TRAN2
                CMP     AL,'C'
                JE      TRAN3
                CMP     AL,'D'
                JE      TRAN4
                CMP     AL,'E'
                JE      TRAN5
                CMP     AL,'F'
                JE      TRAN6
                CMP     AL,'G'
                JE      TRAN7
                CMP     AL,'H'
                JE      TRAN8
                CMP     AL,'I'
                JE      TRAN9
                CMP     AL,'J'
                JE      TRAN10
                JMP     DONE
TRAN1:          JMP     [SI]
TRAN2:          JMP     4[SI]
TRAN3:          JMP     8[SI]
TRAN4:          JMP     12[SI]
TRAN5:          JMP     16[SI]
TRAN6:          JMP     20[SI]
TRAN7:          JMP     24[SI]
TRAN8:          JMP     28[SI]
TRAN9:          JMP     32[SI]
TRAN10:         JMP     36[SI]
DONE:           RET
START           ENDP
CODE            ENDS
                END     BEGIN
```

4.78 某一个操作系统有 6 个内部命令，把这 6 个内部命令及相应的处理程序的入口地址，组成一个内部命令表 INCMDT，如下所示：

```
                ORG     2000H
INCMDT:         DB      'ATRIB'
                DB      00
                DW      2020
                DB      'BYE'
                DB      00
                DW      2050
                DB      'DIR'
```

```
        DB      00
        DW      20A0
        DB      'ERA'
        DB      00
        DW      2100
        DB      'REN'
        DB      00
        DW      2140
        DB      'SAVE'
        DB      00
        DW      2180
        DB      'TYPE'
        DB      00
        DW      21E0
```

若输入的内部命令字符(以00字节作为命令字符的结束标志),已在BUFFER开始的存储区中。编写一个程序根据输入的命令字转至相应的处理程序。

【解】 存在内部命令表和BUFFER中的命令,只是它们的首字符,例如'ATRIB',为'A'。

```
        NAME    ICMD_LIST
DATA1   SEGMENT
        ORG     2000H
INCMDT: DB      'ATRIB'
        DB      00
        DW      2020
        DB      'BYE'
        DB      00
        DW      2050
        DB      'DIR'
        DB      00
        DW      20A0
        DB      'ERA'
        DB      00
        DW      2100
        DB      'REN'
        DB      00
        DW      2140
        DB      'SAVE'
        DB      00
        DW      2180
        DB      'TYPE'
        DB      00
        DW      21E0
```

```
DATA1   ENDS
STACK   SEGMENT PARA STACK 'STACK'
        DB      100 DUP(?)
STACK   ENDS
CODE    SEGMENT
        ASSUME CS:CODE,DS:DATA1,SS:STACK
START   PROC    FAR
BEGIN:  PUSH    DS
        MOV     AX,0
        PUSH    AX
        MOV     AX,DATA
        MOV     DS,AX
        LEA     SI,BUFFER
        LEA     BX,INCMDT
        MOV     AL,[SI]
        CMP     AL,[BX]
        JE      TRAN1
        CMP     AL,4[BX]
        JE      TRAN2
        CMP     AL,8[BX]
        JE      TRAN3
        CMP     AL,[BX+0C]
        JE      TRAN4
        CMP     AL,[BX+10]
        JE      TRAN5
        JMP     DONE
TRAN1:  JMP     2[BX]
TRAN2:  JMP     6[BX]
TRAN3:  JMP     [BX+0A]
TRAN4:  JMP     [BX+0E]
TRAN5:  JMP     [BX+12]
DONE:   RET
START   ENDP
CODE    ENDS
        END     BEGIN
```

4.79 条件同题 4.78，但在 BUFFER 开始的存储区中的命令字，若是一个内部命令，则应转至相应的处理程序；若不是内部命令，则在程序中应建立一个标志。

【解】 存在内部命令表和 BUFFER 中的命令，只是它们的首字符，例如'ATRIB'，为'A'。

```
        NAME    ICMD_LIST1
DATA1   SEGMENT
        ORG     2000H
INCMDT  DB      'ATRIB'
```

```
        DB      00
        DW      2020
        DB      'BYE'
        DB      00
        DW      2050
        DB      'DIR'
        DB      00
        DW      20A0
        DB      'ERA'
        DB      00
        DW      2100
        DB      'REN'
        DB      00
        DW      2140
        DB      'SAVE'
        DB      00
        DW      2180
        DB      'TYPE'
        DB      00
        DW      21E0
FLG     DB      ?
DATA1   ENDS
STACK   SEGMENT PARA STACK 'STACK'
        DB      100 DUP(?)
STACK   ENDS
CODE    SEGMENT
        ASSUME  CS:CODE,DS:DATA1,SS:STACK
START   PROC    FAR
BEGIN:  PUSH    DS
        MOV     AX,0
        PUSH    AX
        MOV     AX,DATA
        MOV     DS,AX
        LEA     SI,BUFFER
        LEA     BX,INCMDT
        MOV     AL,[SI]
        CMP     AL,[BX]
        JE      TRAN1
        CMP     AL,4[BX]
        JE      TRAN2
        CMP     AL,8[BX]
        JE      TRAN3
        CMP     AL,[BX+0C]
        JE      TRAN4
```

```
        CMP     AL,[BX+10]
        JE      TRAN5
        JMP     DONE
TRAN1:  JMP     2[BX]
TRAN2:  JMP     6[BX]
TRAN3:  JMP     [BX+0A]
TRAN4:  JMP     [BX+0E]
TRAN5:  JMP     [BX+12]
DONE:   MOV     DL,0FFH
        MOV     FLG,DL
        RET
START   ENDP
CODE    ENDS
        END     BEGIN
```

4.80 在自 BLOCK 开始的存储区中有 1000 个带符号数。试用气泡排序法编写一个程序，使它们排列有序。

【解】

```
        NAME    BOBBLE_SORT
DATA1   SEGMENT
BLOCK   DW      X1,X2,…,X999
DATA1   ENDS
STACK   SEGMENT PARA STACK 'STACK'
        DB      100 DUP(?)
STACK   ENDS
CODE    SEGMENT
        ASSUME  CS:CODE,DS:DATA1,SS:STACK
START   PROC    FAR
BEGIN:  PUSH    DS
        MOV     AX,0
        PUSH    AX
        MOV     AX,DATA
        MOV     DS,AX
CONTI:  MOV     BL,0
        MOV     CX,1000
        ADD     CX,CX
        MOV     SI,CX
        DEC     SI
        DEC     SI
AGAIN:  MOV     AX,BLOCK[SI]
        CMP     AX,BLOCK[SI-2]
        JGE     NEXT
        XCHG    AX,BLOCK[SI-2]
        MOV     [SI],AX
```

```
        MOV     BL,-1
NEXT:   DEC     SI
        DEC     SI
        LOOP    AGAIN
        CMP     BL,0
        JNE     CONTI
        RET
START   ENDP
CODE    ENDS
        END     BEGIN
```

4.81 在自 TABLE 开始的存储区中有一个有序排列(带符号数,大的在后面)的数据块,其中 TABLE 及 TABLE+1 中存放数据块长度 N(可以大于 255),若要搜索的关键字已输入至累加器 A 中。编写一个程序用对分搜索法搜索此关键字,在程序中建立一个标志,表示是否有此关键字。

【解】

```
        NAME    B_SEARCH
DATA1   SEGMENT
TABLE   DW      N
        DW      X1,X2,…,X(N-1)
FLG     DW      ?
DATA1   ENDS
STACK   SEGMENT PARA STACK 'STACK'
        DB      100 DUP(?)
STACK   ENDS
CODE    SEGMENT
        ASSUME  CS:CODE,DS:DATA1,SS:STACK
START   PROC    FAR
BEGIN:  PUSH    DS
        MOV     AX,0
        PUSH    AX
        MOV     AX,DATA
        MOV     DS,AX
        LEA     SI,TABLE+2
        MOV     CX,[TABLE]
        MOV     DI,SI
        MOV     DI,CX
        MOV     DX,1
CONTI:  MOV     BX,SI
        ADD     BX,DI
        SHR     BX,1
        CMP     AX,[BX]
        JE      FOUND
        PUSHF
```

```
            CMP     BX,SI
            JE      NOFOD
            POPF
            JLE     LESS
            MOV     SI,BX
            JMP     NEXT
LESS:       MOV     DI,BX
NEXT:       INC     DX
            JMP     CONTI
NOFOD:      MOV     DX,-1
FOUND:      MOV     FLG,DX
            RET
START       ENDP
CODE        ENDS
            END     BEGIN
```

4.82 条件及要求同题 4.81，只是已排序的数据块是大的数在前面(地址低处)。

【解】

```
            NAME    B_SEARCH1
DATA1       SEGMENT
TABLE       DW      N
            DW      X1,X2,…,X(N-1)
FLG         DW      ?
DATA1       ENDS
STACK       SEGMENT PARA STACK 'STACK'
            DB      100 DUP(?)
STACK       ENDS
CODE        SEGMENT
            ASSUME  CS:CODE,DS:DATA1,SS:STACK
START       PROC    FAR
BEGIN:      PUSH    DS
            MOV     AX,0
            PUSH    AX
            MOV     AX,DATA
            MOV     DS,AX
            LEA     SI,TABLE+2
            MOV     CX,[TABLE]
            MOV     DI,SI
            MOV     DI,CX
            MOV     DX,1
CONTI:      MOV     BX,SI
            ADD     BX,DI
            SHR     BX,1
            PUSH    DX
```

```
        MOV     DH,[BX]
        MOV     DL,[BX+1]
        CMP     AX,DX
        POP     DX
        JE      FOUND
        PUSHF
        CMP     BX,SI
        JE      NOFOD
        POPF
        JLE     LESS
        MOV     SI,BX
        JMP     NEXT
LESS:   MOV     DI,BX
NEXT:   INC     DX
        JMP     CONTI
NOFOD:  MOV     DX,-1
FOUND:  MOV     FLG,DX
        RET
START   ENDP
CODE    ENDS
        END     BEGIN
```

4.83 把数据块传送程序编写为一个子程序 MOVE，源地址、目的地址和传送的字节数放在自 ADDR 开始的存储单元中。

【解】

```
        NAME    SUBPROGRAM_1
DATA1   SEGMENT
ADDR    DW      S_ADDR
        DW      D_ADDR
        DW      LEN
        ⋮
DATA1   ENDS
STACK   SEGMENT PARA STACK 'STACK'
        DB      100 DUP(?)
STACK   ENDS
CODE    SEGMENT
        ASSUME  CS:CODE,DS:DATA1,SS:STACK
START   PROC    FAR
BEGIN:  PUSH    DS
        MOV     AX,0
        PUSH    AX
        MOV     AX,DATA
        MOV     DS,AX
        ⋮
```

```
        CALL    MOV1
        RET
START   ENDP
MOV1    PROC
        LEA     BX,ADDR
        MOV     SI,[BX]
        MOV     DI,[BX+2]
        MOV     CX,[BX+4]
        CLD
        REP MOVS S_ADDR,D_ADDR
        RET
MOV1    ENDP
CODE    ENDS
        END     BEGIN
```

4.84 把 8 位无符号数乘法程序编写为一个子程序，被乘数、乘数和乘积放在自 DATA 开始的存储单元中。

【解】

```
        NAME    SUBPROGRAM_2
DATA1   SEGMENT
DATA    DB      Y
        DB      X
        DW      ?
        ⋮
DATA1   ENDS
STACK   SEGMENT PARA STACK 'STACK'
        DB      100 DUP(?)
STACK   ENDS
CODE    SEGMENT
        ASSUME  CS:CODE,DS:DATA1,SS:STACK
START   PROC    FAR
BEGIN:  PUSH    DS
        MOV     AX,0
        PUSH    AX
        MOV     AX,DATA
        MOV     DS,AX
        ⋮
        CALL    MUL1
        RET
START   ENDP
MUL1    PROC
        LEA     BX,DATA
        MOV     AL,[BX]
        MUL     [BX+1]
```

```
        MOV     [BX+2],AX
        RET
MUL1    ENDP
CODE    ENDS
        END     BEGIN
```

4.85 把 8 位无符号数除法程序编写为一个子程序，被除数、除数、商和余数放在自 DATA 开始的存储单元中。

【解】

```
        NAME    SUBPROGRAM_3
DATA1   SEGMENT
DATA    DB      Y
        DB      X
        DW      ?
        ⋮
DATA1   ENDS
STACK   SEGMENT PARA STACK 'STACK'
        DB      100 DUP(?)
STACK   ENDS
CODE    SEGMENT
        ASSUME  CS:CODE,DS:DATA1,SS:STACK
START   PROC    FAR
BEGIN:  PUSH    DS
        MOV     AX,0
        PUSH    AX
        MOV     AX,DATA
        MOV     DS,AX
        ⋮
        CALL    DIV1
        RET
START   ENDP
DIV1    PROC
        LEA     BX,DATA
        MOV     AL,[BX]
        CBW
        DIV     [BX+1]
        MOV     [BX+2],AL
        MOV     [BX+3],AH
        RET
MUL1    ENDP
CODE    ENDS
        END     BEGIN
```

4.86 写一个宏定义，使 8088CPU 的 8 位寄存器之间的数据能实现任意传送。

【解】 宏定义如下：

```
X_MOVE    MACRO     P1,P2
          MOV       P1,P2
          ENDM
```

4.87 写一个宏定义，使 8088CPU 的 16 位寄存器的数据互换。

【解】 宏定义如下：

```
X_CH      MACRO     P1,P2
          XCHG      P1,P2
          ENDM
```

4.88 写一个宏定义，能把任一个寄存器的最高位移至另一个寄存器的最低位中。

【解】 宏定义如下：

```
X_SHR     MACRO     P1,P2
          SHL       P1,1
          RCL       P2,1
          ENDM
```

4.89 写一个宏定义，能把任一个内存单元中的最高位移至另一个内存单元的最低位中。

【解】 宏定义如下：

```
X_SHM     MACRO     P1,P2
          LEA       SI,P1
          LEA       DI,P2
          SHL       [SI],1
          RCL       [DI],1
          ENDM
```

4.90 写一个宏定义，能把任一个寄存器的最低位移至另一个寄存器的最高位中。

【解】 宏定义如下：

```
X_SHR1    MACRO     P1,P2
          SAR       P1,1
          RCR       P2,1
          ENDM
```

4.91 写一个宏定义，能把任一个内存单元中的最低位移至另一个内存单元的最高位中。

【解】 宏定义如下：

```
X_SHM1    MACRO     P1,P2
          LEA       SI,P1
          LEA       DI,P2
          SAR       [SI],1
          RCR       [DI],1
          ENDM
```

4.92 用宏定义写一个数据块传送指令。

【解】 宏定义如下：

```
X_MOVS    MACRO     P1,P2,P3
          LEA       SI,P1
          LEA       DI,P2
          MOV       CX,P2
          CLD
          REP MOVSW
          ENDM
```

4.93 写一个宏定义，能使任意一个寄存器向左或向右移位指定的次数。

【解】 宏定义如下：

```
X_SH1     MACRO     P1,P2,P3
          MOV       CL,P1
          S&P2      P3,CL
          ENDM
```

第 5 章　处理器总线时序和系统总线

5.1　总线周期的含义是什么？8086/8088CPU 的基本总线周期由几个时钟组成？如果一个 CPU 的时钟频率为 8MHz，那么，它的一个时钟周期是多少？一个基本总线周期是多少？如果主频为 5MHz 呢？

【解】 CPU 访问总线(从内存储器或 I/O 端口读/写字节或字)所需的时间称为总线周期。8086/8088CPU 的基本总线周期由 4 个时钟组成。若 CPU 的时钟频率为 8(5)MHz，时钟周期为 1/8 MHz = 125(1/5MHz = 200)ns，基本总线周期为 4×125(200)ns = 500(800)ns。

5.2　在总线周期的 T_1、T_2、T_3、T_4 状态，CPU 分别执行什么动作？什么情况下需要插入等待状态 T_W？T_W 在哪儿插入？怎样插入？

【解】 下面以存储器读为例进行说明。在 T_1 周期，输出地址信号、地址锁存允许信号、内存或 I/O 端口访问控制信号；在 T_2 周期，CPU 开始执行数据传送操作，此时，8086CPU 内部的多路开关进行切换，将地址/数据线 AD_{15}～AD_0 上的地址撤销，切换为数据总线，为读写数据作准备。8288 总线控制器发出数据总线允许信号和数据发送/接收控制信号 DT/$\overline{R}$ 允许数据收发器工作，使数据总线与 8086CPU 的数据线接通，并控制数据传送的方向。同样，把地址/状态线 A_{19}/S_6～A_{16}/S_3 切换成与总线周期有关的状态信息，指示若干与周期有关的情况。

在 T_3 周期开始的时钟下降沿上，8086CPU 采样 READY 线。如果 READY 信号有效(高电平)，则在 T_3 状态结束后进入 T_4 状态。在 T_4 状态开始的时钟下降沿，把数据总线上的数据读入 CPU 或写到地址选中的单元。在 T_4 状态中结束总线周期。如果访问的是慢速存储器或是外设接口，则应该在 T_1 状态输出的地址，经过译码选中某个单元或设备后，立即驱动 READY 信号到低电乎。8086CPU 在 T_3 状态采样到 BEADY 信号无效，就会插入等待周期 T_W，在 T_W 状态 CPU 继续采样 READY 信号；直至其变为有效后再进入 T_4 状态，完成数据传送，结束总线周期。

在 T_4 状态，8086CPU 完成数据传送，状态信号 $\overline{S}_2$～$\overline{S}_0$ 变为无操作的过渡状态。在此期间，8086CPU 结束总线周期，恢复各信号线的初态，准备执行下一个总线周期。

5.3　8086CPU 和 8088CPU 是怎样解决地址线和数据线的复用问题的？$\overline{ALE}$信号何时处于有效电平？

【解】 8086CPU 和 8088CPU 是利用信号的时序来解决地址线和数据线的复用的。无论是内存或 I/O 端口访问，CPU 总是先输出要访问的内存单元或 I/O 端口的地址，使内存和 I/O 端口有时间进行地址译码以找到要访问的单元，然后才能进行数据传送。所以 CPU 在 T_1 周期输出地址，同时输出$\overline{ALE}$信号以锁存在复用线上输出的地址。在 T_2 周期，CPU 开始执行数据传送操作，此时，8086CPU 内部的多路开关进行切换，将地址/数据线 AD_{15}～AD_0 上的地址撤销，切换为数据总线，为读写

数据作准备。

5.4 T_1 状态下，数据/地址线上是什么信息？用哪个信号将此信息锁存起来？数据信息是在什么时候给出的？用时序图表示出来。

【解】 在 T_1 周期，数据/地址线上输出的是地址。用$\overline{\text{ALE}}$信号把地址锁存。在 T_2 周期在 CPU 内部已经把数据/地址线切换为数据线。时序图如图 5-1 所示。

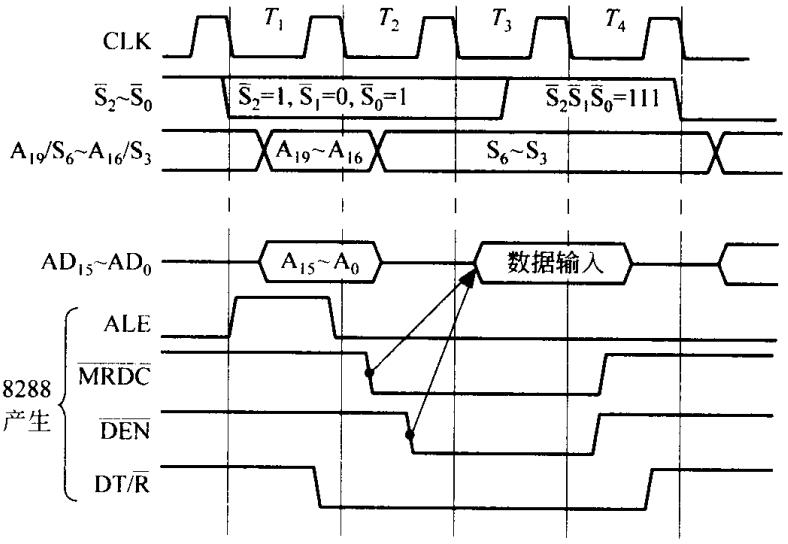


图 5-1 题 5.4 的时序图

5.5 若已有两个数：$a=200$，$b=150$。用累加的办法求 $x=a*b$，可以有两种编写程序的方法：

(1) 用一个起始值为 0 的 16 位部分积寄存器，把被乘数(即 200)加 150 次(即次数由乘数决定)，即直接用 150 次加法指令。

(2) 方法同上，但用循环程序，用乘数作循环次数。

分别编写出这两种程序，比较这两种程序的执行时间(指令的执行时间见附录)。

【解】

(1) 用累加的方法：

```
        MOV     BX,a
        MOV     AX,0
        ADD     AX,BX
        ADD     AX,BX
        ⋮
        ADD     AX,BX
```

(2) 用循环的方法：

```
        MOV     BX,a
        MOV     CX,b
        MOV     AX,0
LOP1:   ADD     AX,BX
        LOOP    LOP1
```

(1) 方法所需时间为:前两条立即数传送时钟周期数为4,加法指令的时钟周期数为3,故程序段的总时钟周期数为 $4+4+3\times150=458$

(2) 方法所需时间为:前3条立即数传送时钟周期数为4,加法指令的时钟周期数为3,LOOP指令的时钟周期数为17,故程序段的总时钟周期数为 $4+4+4+17\times150=2562$。显然,第二种方法缩短了程序的长度,但时钟周期数却大大增加了。

5.6 若用部分积右移的办法来编写乘法程序,写出程序,计算这种方法所用的执行时间,并与题5.5中的结果相比较。

【解】 用部分积右移的算法是:置部分积为0,由乘数的最低位判断,若其为1,则把被乘数加至部分积(乘数为0,则不加被乘数),然后部分积和乘数都右移一位,重复上述的操作,重复次数为乘数的位数。相应的程序为:

```
        MOV     AX,0
        MOV     DL,10010110B
        MOV     BL,200
        MOV     CL,8
LOP1:   SHR     DL,1
        JNC     N1
        ADD     AH,BL
N1:     SAR     AX,1
        DEC     CL
        JNZ     LOP1
        RET
```

此程序段的时钟周期数 $=4+4+4+4+(2+16+3+2+16)\times8=328$。比上述两种方法都快。

5.7 编出用被乘数左移的方法实现乘法的程序,计算它的执行时间,并与题5.5中的结果相比较。

【解】 用被乘数左移的算法是:置部分积为0,由乘数的最低位判断,若其为1,则把被乘数加至部分积(乘数为0,则不加被乘数),然后被乘数左移一位,乘数右移一位,重复上述的操作,重复次数为乘数的位数。相应的程序为:

```
        MOV     AX,0
        MOV     DL,10010110B
        MOV     BL,200
        MOV     CL,8
LOP1:   SHR     DL,1
        JNC     N1
        ADD     AL,BL
N1:     SHL     BL,1
        DEC     CL
        JNZ     LOP1
        RET
```

此程序段的时钟周期数 $=4+4+4+4+(2+16+3+2+16)\times8=328$。与部分积右

移的结果一样。

5.8 下面是两个能实现数据块传送的程序：

```
        MOV   BX,AREA1                MOV   SI,0
        MOV   DI,AREA2                MOV   DI,0
        MOV   CX,100                  MOV   CX,100
LOOP1:  MOV   AL,[BX]          LOP1:  MOV   AL,AREA1(SI)
        MOV   [DI],AL                 MOV   AREA2(DI),AL
        INC   BX                      INC   SI
        INC   DI                      INC   DI
        LOOP  LOOP1                   LOOP  LOP1
        HLT                           HLT
```

比较这两个程序的执行时间。

【解】 这两个程序段执行时间的主要区别在于循环中传送内存操作数的寻址方式。左边的是采用基址寻址，每条指令为 5 个时钟周期，而右边的采用偏移量加基址或变址寻址，每条指令是 9 个时钟周期。故左边的程序段比右边的要快。

5.9 下面是两个能把累加器 A 中的数乘以 10(* 10 后仍<255)的程序：

```
SAL   AL                ADD   AL,AL
MOV   CL,AL             MOV   CL,AL
SAL   AL                ADD   AL,AL
SAL   AL                ADD   AL,AL
ADD   AL,CL             ADD   AL,CL
```

比较这两个程序的执行时间。

【解】 这两个程序段执行时间的主要区别是用移位还是用加法指令实现×2、×4或×8，移位指令用 2 个时钟周期，加法指令用 3 个时钟周期。故左边的程序段比右边的快。

5.10 编写一个能用软件实现延时 20ms 的子程序。

【解】 若延时 20ms，可用指令的循环实现，最简单的循环是对 CX 寄存器装入一立即数，然后用 LOOP 循环。装入立即数为 4 个时钟周期，LOOP 是 17 个时钟周期。若一个时钟周期是 125ns，则需循环 20ms/(21 × 125ns) = 762。故程序段为：

```
        MOV   CX,762
LOP1    LOOP  LOP1
```

5.11 编写一个能用软件实现延时 100ms 的子程序。

【解】 要延时 100ms，则只需使延时 20ms 的程序段执行 5 次。

```
        MOV   BL,5
DEL20:  MOV   CX,762
LOP1:   LOOP  LOP1
        DEC   BL
```

```
        JNZ     DEL20
```

5.12 编写一个能用软件实现延时 1s 的子程序。

【解】 要延时 1s，则执行 10 次 100ms，也即需使延时 20ms 的程序段执行 50 次。

```
        MOV     BL,50
DEL20:  MOV     CX,762
LOP1:   LOOP    LOP1
        DEC     BL
        JNZ     DEL20
```

5.13 若在 TIME 开始的存储区中，已输入了以 BCD 码表示的时、分、秒的起始值（共用 3 个存储单元，时在前），利用延时 1s 的子程序，以 CPU 内部的 3 个寄存器中，产生实时时钟。

【解】

```
        NAME    CLOCK
DATA1   SEGMENT
TIME    DB      Y
        DB      X
        DW      Z
DATA1   ENDS
STACK   SEGMENT PARA STACK 'STACK'
        DB      100 DUP(?)
STACK   ENDS
CODE    SEGMENT
        ASSUME  CS:CODE,DS:DATA1,SS:STACK
START   PROC    FAR
BEGIN:  PUSH    DS
        MOV     AX,0
        PUSH    AX
        MOV     AX,DATA
        MOV     DS,AX
        LEA     BX,TIME
        MOV     CH,[BX]
        MOV     DH,[BX+1]
        MOV     DL,[BX+2]
AGAIN:  CALL    DELAY
        MOV     AL,DL
        ADD     AL,1
        DAA
        MOV     DL,AL
        CMP     AL,60
        JNE     N1
        MOV     DL,0
```

```
        MOV     AL,DH
        ADD     AL,1
        DAA
        MOV     DH,AL
        CMP     AL,60
        JNE     N1
        MOV     DH,0
        MOV     AL,CH
        ADD     AL,1
        DAA
        MOV     CH,AL
        CMP     AL,24
        JNE     N1
        MOV     CH,0
N1:     JMP     AGAIN
START   ENDP
DELAY   PROC
        PUSH    BX
        PUSH    CX
        MOV     BL,50
DEL20:  MOV     CX,762
LOP1:   LOOP    LOP1
        DEC     BL
        JNZ     DEL20
        POP     CX
        POP     BX
        RET
DELAY   ENDP
CODE    ENDS
        END     BEGIN
```

5.14 在存储器读周期，画出 $A_{15} \cdot A_{14} \cdot IO/\overline{M}$ 的波形。

【解】 由主教材第 5 章中的图 5-6 可以看到 $IO/\overline{M}$ 信号在地址信号有效前已经有效，且一直维持至 T_4 周期。故 $A_{15} \cdot A_{14} \cdot IO/\overline{M}$ 的波形完全取决于地址信号的波形。

5.15 在存储器写周期，画出 $IO/\overline{M} \cdot \overline{WR}$的波形。

【解】 在最大组态下，存储器写周期的 $IO/\overline{M} \cdot \overline{WR}$信号由 8288 组合为$\overline{MWTC}$信号，如主教材第 5 章中图 5-10 中所示。

5.16 在输入周期，画出 $IO/\overline{M} \cdot \overline{RD}$的波形，标出它们的时间关系（什么时刻有效，什么时刻无效）。

【解】 在最大组态下，输入周期的 $IO/\overline{M} \cdot \overline{RD}$信号由 8288 组合为$\overline{IORC}$信号，如主教材第 5 章中图 5-11 所示。它在 T_2 周期的时钟下降沿后有效，延续至 T_4 周期的时钟下降沿后无效。

5.17 在输出周期，画出 $IO/\overline{M}\cdot\overline{WR}$的波形，标出它们的时间关系。

【解】 在最大组态下，输出周期的 $IO/\overline{M}\cdot\overline{WR}$信号由 8288 组合为$\overline{AIOWC}$信号，如主教材第 5 章中图 5-11 所示。它在 T_2 周期的时钟下降沿后有效，延续至 T_4 周期的时钟下降沿后无效。

5.18 若 CPU 为 8088，时钟周期 $T=200ns$，工作在最小组态存储器读周期，计算自 $IO/\overline{M}$ 有效，至 T_3 周期时钟下降沿之间的时间间隔。

【解】 由主教材的图 6-17 可以看出，从 T_1 周期时钟下降沿至 $IO/\overline{M}$ 开始有效的最长延迟时间是 110ns，则 T_1 周期剩下的时间为(200－110)ns＝90ns，至 T_3 周期的时钟下降沿，要经过整个 T_2 周期。故，从 $IO/\overline{M}$ 有效至 T_3 的时钟下降沿的时间间隔为(90＋200)ns＝290ns。

5.19 条件同题 5.18，计算自地址有效至 T_3 周期时钟下降沿之间的时间间隔。

【解】 由主教材的图 6-17 可以看出，从 T_1 周期时钟下降沿至地址有效的最长延迟时间也是 110ns。故，从地址有效至 T_3 的时钟下降沿的时间间隔也为(90＋200)ns＝ 290ns。

5.20 计算自地址有效至 T_3 周期时钟下降沿之间，以及自$\overline{RD}$有效至 T_3 周期时钟下降沿之间的时间间隙。

【解】 如题 5.19 计算的从地址有效至 T_3 的时钟下降沿的时间间隔为(90＋200)ns＝290ns。而$\overline{RD}$是从 T_2 周期的时钟下降沿后经 TCLRL 延迟(最大为 165ns)变为有效。故，它与 T_3 周期的时钟下降沿的时间间隔为(200－165)ns＝35ns。

5.21 在存储器写周期，计算自地址有效至$\overline{WR}$信号有效之间的时间间隔；$\overline{WR}$有效至 T_3 周期时钟下降沿之间的时间间隔。

【解】 由主教材的图 6-20 可以看出在存储器写周期，地址信号从 T_1 周期时钟下降沿开始最长经过 110ns 延迟变为有效，则 T_1 周期剩下的时间为(200－110)ns＝90ns。而$\overline{WR}$信号是从 T_2 周期的时钟下降沿经 TCVCTV(最小为 10ns)延迟后有效。故地址有效至$\overline{WR}$有效的时间间隔为(90＋10)ns＝100ns。$\overline{WR}$有效至 T_3 周期时钟下降沿的时间间隔为 200ns－TCVCTV(最大为 110ns)＝ 90ns。

5.22 条件同题 5.21，计算$\overline{WR}$无效至地址无效之间的时间间隔。计算自 $IO/\overline{M}$ 有效至$\overline{WR}$有效之间的时间间隔及 $IO/\overline{M}$ 与$\overline{WR}$共同有效的时间。

【解】 由主教材的图 6-20 可以看到，在存储器写周期，地址信号持续有效 4 个时钟周期，即它在下一总线周期的 T_1 周期时钟下降沿最短经过 10ns 延迟变为无效。而$\overline{WR}$的有效持续时间为 TWLWH(2×200ns－60ns＝340ns)，故从 T_2 周期时钟下降沿经过 TCVCTV ＋ TWLWH(110ns＋340ns＝450ns)，$\overline{WR}$变为无效。故地址无效与$\overline{WR}$无效的时间间隔为(3×200 ＋10)ns－450ns＝160ns。$IO/\overline{M}$ 有效至$\overline{WR}$有效的时间间隔为 90ns ＋10ns＝100ns。而 $IO/\overline{M}$ 与$\overline{WR}$信号共同有效的时间，取决于$\overline{WR}$的宽度，即为 340ns。

5.23 输入周期，计算自 $IO/\overline{M}$ 有效至 T_3 周期时钟下降沿之间；$IO/\overline{M}$ 有效至 T_W 周期时钟下降沿之间；$IO/\overline{M}\cdot\overline{RD}$有效至 T_4 周期时钟下降沿之间的时间间隔。

【解】 由主教材的图 6-17 可以看到，从 T_1 周期时钟下降沿至 IO/$\overline{M}$ 开始有效的最长延迟时间是 110ns，则 T_1 周期剩下的时间为 (200－110)ns＝90ns，至 T_3 周期的时钟下降沿，要经过整个 T_2 周期。故，从 IO/$\overline{M}$ 有效至 T_3 的时钟下降沿的时间间隔为(90＋200)ns＝290ns。IO/$\overline{M}$ 有效至 T_w 周期时钟下降沿的时间间隔为(290＋200)ns＝490ns。IO/$\overline{M}$ · $\overline{RD}$共同有效时间取决于$\overline{RD}$的有效，它从 T_2 周期时钟下降沿经 TCLRL(最大 165ns)延迟后有效，在 T_2 周期还剩下(200－165)ns＝35ns。故 IO/$\overline{M}$ · $\overline{RD}$有效至 T_4 周期时钟下降沿之间的时间间隔为(200＋35)ns＝235ns。在输入周期还要加上 T_w 周期。故为(235＋200)ns＝435ns。

5.24 输出周期，计算 IO/$\overline{M}$ · $\overline{WR}$信号同时有效距 T_1 状态时钟下降沿之间及与 T_3 状态时钟下降沿之间的时间间隔。

【解】 由主教材的图 6-20 可以看到，在输出周期，IO/$\overline{M}$ 信号在 T_1 周期有效，而$\overline{WR}$信号在 T_2 周期有效，它们共同有效距 T_1 周期时钟下降沿的时间为 200ns＋TCVCTV(最长为 110ns)＝ 310ns。与 T_3 周期时钟下降沿的时间间隔为$\overline{WR}$有效在 T_2 同期余下的时间 200ns－110ns＝90ns。

第6章　主存储器

6.1　若有一单板机，具有用8片2114构成的4KB RAM，连线如图6-1所示。

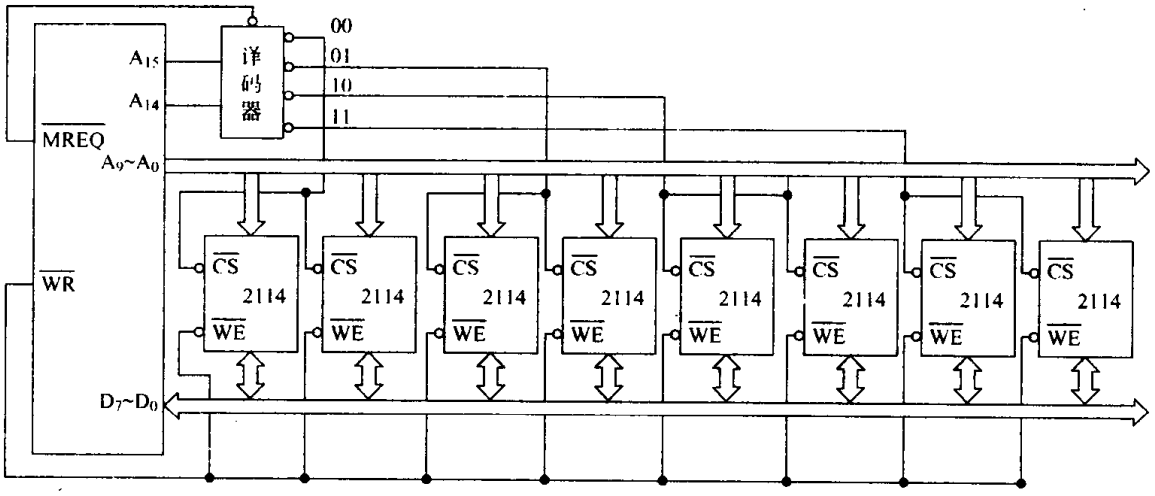


图6-1　单板机与用8片2114芯片构成4KB RAM的连线图

若以每1KB RAM作为一组，则此4组RAM的基本地址是什么？地址有没有重叠区？每一组的地址范围为多少？

【解】 RAM的基本地址为：

第一组　　0000H～03FFH

第二组　　4000H～43FFH

第三组　　8000H～83FFH

第四组　　C000H～C3FFH

地址有重叠区。

每一组的地址范围为000H～3FFH(1024个字节)。

6.2　若某一单板机的存储器连线如图6-2所示。

其中，地址译码器的输出与输入编码之间的关系如表6-1所示。

表6-1　题6.2译码器的输出与输入编码之间的关系

A_{15}～A_{11}	译码器输出	A_{15}～A_{11}	译码器输出
00111	$\overline{Y}_7$　未用	00111	$\overline{Y}_3$　未用
00110	$\overline{Y}_6$　未用	00010	$\overline{Y}_2$　PROM2选择
00101	$\overline{Y}_5$　RAM2选择	00001	$\overline{Y}_1$　PROM1选择
00100	$\overline{Y}_4$　RAM1选择	00000	$\overline{Y}_0$　ROM选择

分析图上的ROM、PROM1、PROM2和各组RAM(1KB为一组)的地址范围；每一种存储器的地址有没有重叠？重叠区是什么？

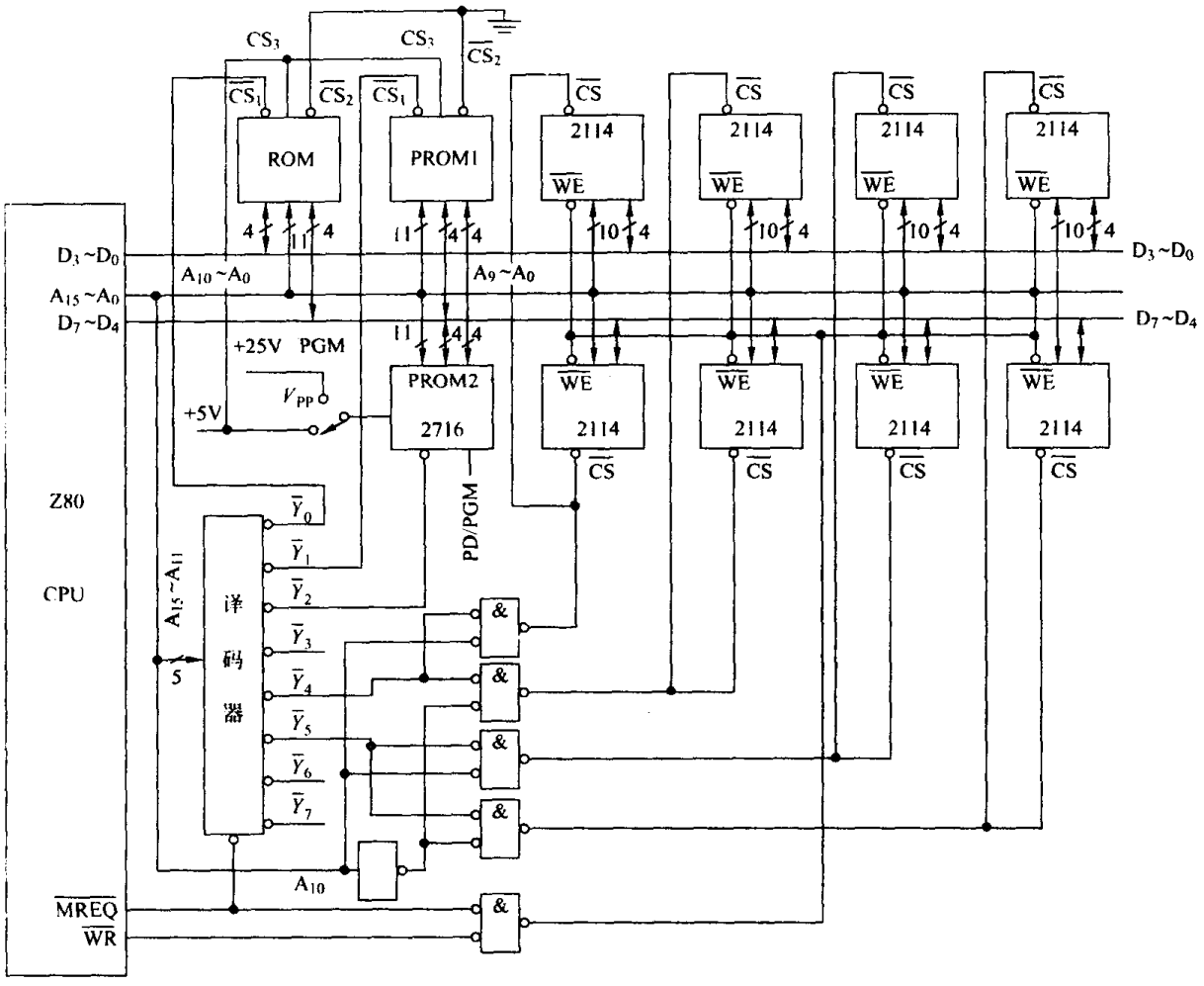


图 6-2　某单板机的存储器的连线图

【解】

ROM	0000H～07FFH
PROM 1	0800H～0FFFH
PROM 2	1000H～17FFH
RAM 1（第 1 组）	2000H～23FFH
（第 2 组）	2400H～27FFH
RAM 2（第 3 组）	2800H～2BFFH
（第 4 组）	2C00H～2FFFH

每一种存储器的地址都没有重叠，重叠区是指一个存储单元出现多个地址的现象。

6.3　在题 6.2 中，若用 2114 片子扩展 4KB RAM 有没有可能？片子的各引线应该如何连接？

【解】　用 2114 芯片子扩展 4KB RAM 是可能的。

新扩展的 8 片 2114 芯片的地址线和数据线的连接方法与图 6-2 中原有的 2114 芯片完全相同，只是组选择信号的连接不同而已。例如，可以如下安排：

新 1 组 RAM 的组选择（A_{15}～A_{10}）001100

新 2 组 RAM 的组选择(A_{15}～A_{10}) 001101

新 3 组 RAM 的组选择(A_{15}～A_{10}) 001110

新 4 组 RAM 的组选择(A_{15}～A_{10}) 001111

6.4 若要扩充 1KB RAM(用 2114 片子),规定地址为 8000H～83FFH,地址线应该如何连接?

【解】 扩充 1KB RAM 至规定地址 8000H～83FFH,其地址线的低 10 位接芯片,高 6 位地址(A_{15}～A_{10}=100000)产生组选择信号。

6.5 若要用 2114 片子扩充 2KB RAM,规定地址为 4000H～47FFH,地址线应该如何连接?

【解】 扩充 2KB RAM 至规定地址 4000H～47FFH,则把它们分为两组,它们的地址线的低 10 位接芯片,高 6 位地址产生两组译码输出:A_{15}～A_{10}=100000 和 100001,产生两组选择信号。

6.6 如何检查扩展的 RAM 工作是否正常?

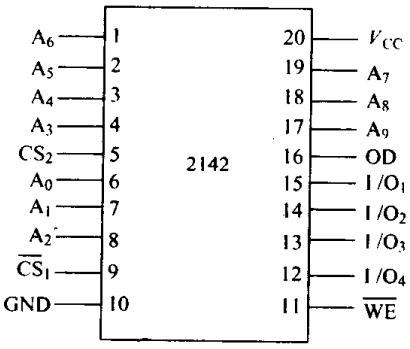
【解】 可以编写一个程序向 RAM 写一个数,然后再从 RAM 读出。检查是否相等。

6.7 编写一个简单的 RAM 检查程序,此程序能记录有多少个 RAM 单元工作有错,且能把出错的 RAM 单元的地址记录下来。

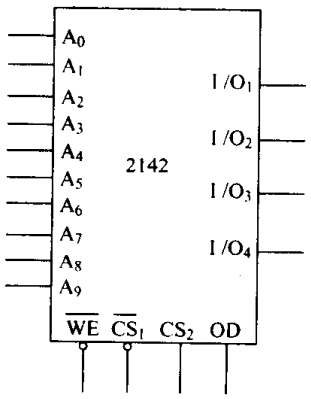
【解】

```
            NAME    RAMCK
DATA1       SEGMENT
RAM1        DB      1024 DUP(?)
ADERR       DB      2050 DUP(?)
DATA1       ENDS
STACK       SEGMENT PARA STACK 'STACK'
            DB      100 DUP(?)
STACK       ENDS
CODE        SEGMENT
            ASSUME  CS:CODE,DS:DATA1,SS:STACK
START       PROC    FAR
BEGIN:      PUSH    DS
            MOV     AX,0
            PUSH    AX
            MOV     AX,DATA
            MOV     DS,AX
            MOV     AL,05
            MOV     CX,1024
            LEA     SI,RAM1
            MOV     DI,SI
            LEA     BX,ADERR+2
            MOV     DX,0
            CLD
      REP   STOSB
            MOV     CX,1024
CK1:        LODSB
```

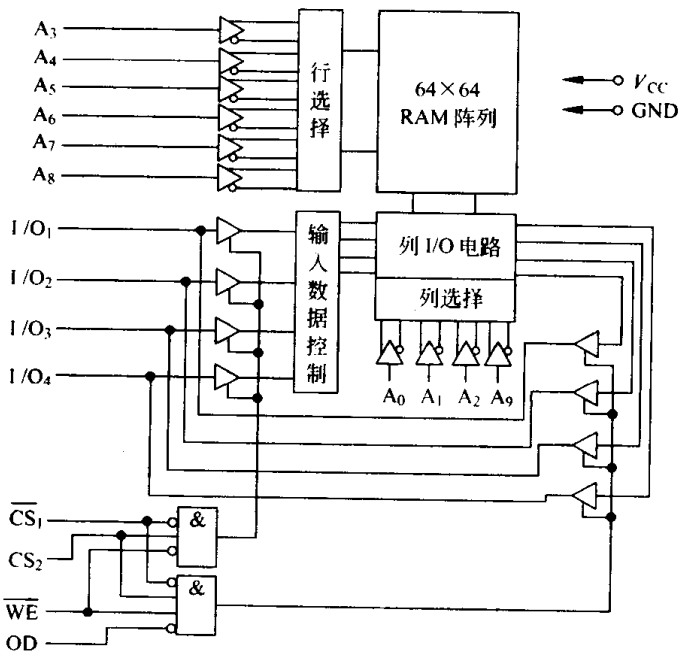
```
        CMP     AL,05
        JE      N1
        INC     DX
        DEC     SI
        MOV     [BX],SI
        INC     SI
        INC     BX
        INC     BX
N1:     LOOP    CK1
        MOV     ADERR,DX
        RET
START   ENDP
CODE    ENDS
        END     BEGIN
```



引脚图



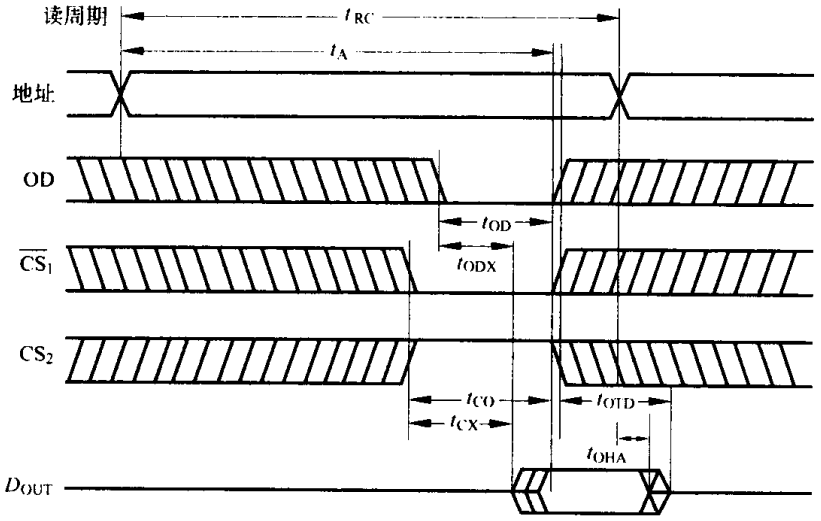
逻辑符号



A_0~A_9 地址输入	I/O_1~I/O_4 地址输入
$\overline{WE}$ 写允许	OD 输出禁止
$\overline{CS_1}$ CS_2 选中	V_{CC} 5V 电源，GND 地

图 6-3　静态 RAM 芯片 2142 的方框图

6.8 若有型号为 2142 的 1024×4 位静态 RAM 片子，其方框图、读出和写入的时序如图 6-3、图 6-4 和图 6-5 所示。若与 8088 CPU 相配合构成 2KB RAM。分析此存储器与 CPU 如何连线；分析存储器读周期以及写周期的时序能否与 CPU 相配合，是否要插入 T_W 状态。



符号	参　数	2142-2		2142-3,2142L3		2142,2142L		单位
		最小	最大	最小	最大	最小	最大	
t_{RC}	读周期	200		300		450		ns
t_A	存取时间		200		300		450	ns
t_{OD}	输出允许到输出稳定		70		100		120	ns
t_{ODX}	输出允许到输出有效	20		20		20		ns
t_{CO}	片选到输出稳定		70		100		120	ns
t_{CX}	片选到输出有效	20		20		20		ns
t_{OTD}	从禁止到输出三态		60		80		100	ns
t_{OHA}	从地址改变到输出保持	50		50		50		ns

图 6-4　静态 RAM 芯片 2142 的读出时序

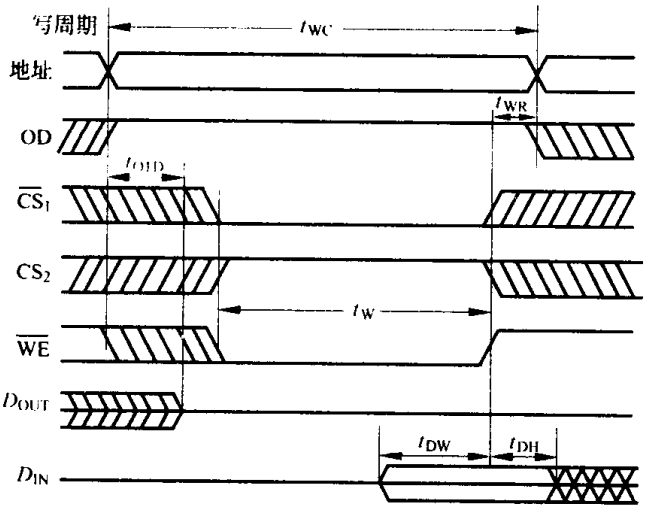


图 6-5　静态 RAM 芯片 2142 的写入时序

符号	参　数	2142-2		2142-3,2142L3		2142,2142L		单位
		最小	最大	最小	最大	最小	最大	
t_{WC}	写周期	200		300		450		ns
t_W	写时间	120		150		200		ns
t_{WR}	写释放时间	0		0		0		ns
t_{OTD}	从禁止到输出三态		60		80		100	ns
t_{DW}	数据与写信号的重叠时间	120		150		200		ns
t_{DH}	写无效后的数据保持	0		0		0		ns

图 6-5（续）

【解】 把 2KB 2142 分为两组，每页为 1KB RAM。2142 的 10 条地址线直接带地址总线。地址的高 6 位 A_{15}～A_{10}经译码分别接至$\overline{CS_1}$，$\overline{CS_2}$ 接至电源，$\overline{WE}$端接 CPU 的$\overline{WR}$输出。存储器不禁止输出，则 OD 端接地。

在存储器读周期，存储器的地址和选片信号都是接至地址总线。从 CPU 来说，从 T_1 周期开始至地址有效的最大延迟时间为 TALAVmax = 110ns。从存储器来说要数据读出至数据总线需要两个条件，一个是从地址有效至数据能稳定读出需要 t_A=200ns 则从 T_1 算起，需要 110ns+200ns=310ns；另一是从选片有效开始，经过 t_{CO}=70ns，三态门才能打开。选片也是用地址信号，故从 T_1 开始，经过 110ns+70ns=180ns，三态门可靠打开。它小于存储器读出时间。而从 CPU 来说，从 T_1 开始至 T_4 的前沿采样数据，其时间间隔为 $3\times T$－TDVCL=3×200ns－30ns=570ns。因此，8088CPU 与 2142 存储器的读周期能很好配合。

在存储器写周期，由于是用 CPU 的地址信号作为选片信号，2142 对$\overline{CS}$的要求总能满足。输出数据在 T_2 周期，CPU 内部的多路开关切换后也已输出，也能满足。关键是写信号有效的宽度能否满足 2142 的 t_W = 120ns 的要求。CPU 的$\overline{WR}$信号的宽度为 TWLWH = 2TCLCL－60ns= 2×200ns－ 60ns=340ns，完全满足要求。

总之，8088 与 2142 在存储器读/写周期能很好配合，不需插入 T_W 周期。

第 7 章　输入和输出

7.1　外部设备为什么要通过接口电路和主机系统相连？

【解】　因为外部设备种类繁多，输入信息可能是数字量、模拟量或开关量，而且输入速度、电平、功率与 CPU 差距很大。所以，通常要通过接口电路与主机系统相连。

7.2　接口电路的作用是什么？按功能可以分为几类？

【解】　接口电路使主机与不同种类的设备、不同信号类型、不同工作速度与不同电平和功率的外部设备能够正确连接，以保证信息的正确传送。接口电路按功能可分为并行接口电路和串行接口电路两大类。

7.3　数据信息有哪几类？举例说明它们各自的含义。

【解】　数据信息可分为 3 种基本类型：

(1) 数字量

由键盘等输入的信息是以二进制形式表示的数或以 ASCII 码表示的数或字符。

(2) 模拟量

当计算机用于控制时，大量的现场信息经过传感器把非电量(例如温度、压力、流量、位移等)转换为电量，并经放大即得模拟电压或电流。这些模拟量必须先经过 A/D 转换才能输入计算机(位数由 A/D 转换的精度确定)；计算机的控制输出也必须先经过 D/A 转换才能去控制执行机构。

(3) 开关量

这是一些两个状态的量，例如电机的运转与停止、开关的合与断、阀门的打开与关闭等。这些量只要用 1 位二进制数即可表示，故字长 8 位的机器一次输入或输出可控制 8 个这样的开关量。

7.4　CPU 和输入输出设备之间传送的信息有哪几类？

【解】　CPU 和输入输出设备之间传送的信息主要有 3 类。

(1) 数据(Data)

在微型计算机中，数据通常为 8 位、16 位或 32 位。

(2) 状态信息(Status)

在输入时，有输入设备的信息是否准备好(Ready)；在输出时有输出设备是否有空(Empty)，若输出设备正在输出信息，则以忙(Busy)指示等。

(3) 控制信息(Control)

例如，控制输入输出设备启动或停止等。

7.5　什么叫端口？通常有哪几类端口？计算机对 I/O 端口编址时通常采用哪两种方法？在 8086/8088 系统中，用哪种方法对 I/O 端口进行编址？

【解】　端口是信息输入或输出的通路。在计算机中用地址来区分不同的端口。计算机对 I/O 端口编址时通常采用两种方法。

(1) 存储器对应输入输出方式

在这种方式中，把一个外设端口作为存储器的一个单元来对待，故每一个外设端口占有存储器的一个地址。从外部设备输入一个数据，作为一次存储器读的操作；而向外部设备输出一个数据，则作为一次存储器写的操作。

(2) 端口寻址的输入输出方式

在这种工作方式中：CPU 有专门的 I/O 指令，用地址来区分不同的外设。但要注意实际上是以端口(Port)作为地址的单元，因为一个外设不仅有数据寄存器还有状态寄存器和控制命令寄存器，它们各需要一个端口才能加以区分，故一个外设往往需要数个端口地址。CPU 用地址来选择外设。

在 8086/8088 中，采用端口寻址的输入输出方式。

7.6 为什么有时候可以使两个端口对应同一个地址？

【解】 若两个端口的数据传送方向不同，或两个端口所使用的信息，占用端口的不同位，则它们可以对应同一个地址。

7.7 CPU 和外设之间的数据传送方式有哪几种？实际选择某种传输方式时，主要依据是什么？

【解】 CPU 和外设之间的数据传送方式主要有：

(1) 查询传送方式；

(2) 中断传送方式；

(3) 直接数据通道传送(DMA)方式。

具体采用哪种传送方式，取决于数据传送的速度和数量。当外部设备的信息传送速度较低或要求实时处理时，采用中断方式；当外设速度与 CPU 相当时，采用查询方式；当要求传送速度快且是批量传送时，采用 DMA 方式。

7.8 条件传送方式的工作原理是怎样的？主要用在什么场合？画出条件传送(查询)方式输出过程的流程图。

【解】 条件传送方式(即查询方式)，主要是在数据传送前，先查询传送的条件是否满足(输入的信息是否已就绪，输出设备是否忙)。若未满足就继续查询；当查询到条件满足时，就进行数据传送。主要用于输入输出信息的速度与 CPU 的速度相当的场合。流程图如图 7-1 所示。

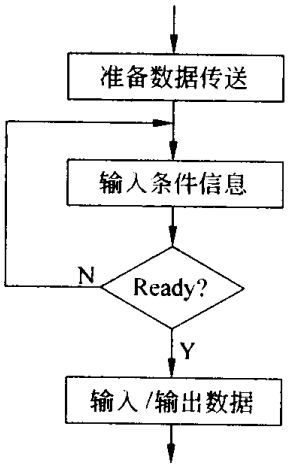


图 7-1 题 7.8 条件传送(查询)方式输出过程流程图

7.9 设一个接口的输入端口地址为 0100H，状态端口地址为 0104H，状态端口中第 5 位为 1 表示输入缓冲区中有一个字节准备好，可以输入。设计具体程序以实现查询式输入。

【解】 查询输入的程序段为：

```
POL1:   IN      AL,0104H
        AND     AL,20H
        JZ      POL1
        IN      AL,0100H
```

7.10 查询式传送方式有什么优缺点？中断方式为什么能弥补查询方式的缺点？

【解】 查询方式是程序不停地主动地对输入状态信息进行查询。优点是保证在条件具备时才进行输入或输出，传送可靠。但若输入速度较慢，则 CPU 的大量时间用于查询等待，而只有较少的时间实现输入输出，这样使得 CPU 的效率很低。

中断方式是在程序启动输入输出过程后，程序不等待，继续执行……直至外部设备条件具备后，由外设通过中断信号通知 CPU，条件已经具备，则 CPU 暂停程序的执行，处理输入输出工作。故 CPU 的效率高。

7.11 与 DMA 方式比较，中断传输方式有什么不足之处？

【解】 中断传送仍是由 CPU 通过程序来传送，每次要保护断点、保护现场都需用多条指令，每条指令要有取指和执行时间。这对于一个高速 I/O 设备，以及成组交换数据的情况，例如磁盘与内存间的信息交换，就显得速度太慢了。

所以希望用硬件在外设与内存间直接进行数据交换(DMA)而不通过 CPU，这样数据传送的速度的上限就取决于存储器的工作速度。但是，通常系统的地址和数据总线以及一些控制信号线(例如 IO/$\overline{M}$、$\overline{RD}$、$\overline{WR}$等)是由 CPU 管理的，在 DMA 方式时，就希望 CPU 把这些总线让出来(即 CPU 连到这些总线上的线处于第三态——高阻状态)，而由 DMA 控制器接管，控制传送的字节数，判断 DMA 是否结束，以及发出 DMA 结束等信号。这些都是由硬件实现的。

7.12 简述采用 DMA 方式传输单个数据的全过程。

【解】 用 DMA 方式传输单个数据的过程为：

① 当要传送数据时，向 CPU 发出 HOLD 信号，即向 CPU 发出 DMA 请求信号。

② 当 CPU 接收到 HOLD 信号，在当前总线周期结束时，CPU 向外设发出 DMA 请求的响应信号 HLDA。DMA 控制器接收到 HLDA 信号后，接管对总线的控制，进入 DMA 方式。

③ DMA 控制器发出地址信息，对存储器寻址。

④ DMA 控制器能发出读或写等控制信号，读或写数据。

⑤ 读或写数据后，向 CPU 发出 DMA 结束信号，使 CPU 恢复正常工作状态。

7.13 DMA 控制器的地址线为什么是双向的？什么时候向 DMA 控制器传输地址？什么时候 DMA 控制器向地址总线传输地址？

【解】 $A_0 \sim A_3$(地址，输入/输出)——这 4 条最低 4 位地址线是双向三态信号线。在空闲周期，这 4 条是地址输入线，CPU 用这 4 条地址线选择 8237 内部不同的寄存器。在 DMA 传送时，由这 4 条线输出要访问的存储单元的最低 4 位地址。

$A_4 \sim A_7$(地址，输出)——这是 4 条三态的地址输出线，只用于 DMA 传送时，由它们输出要访问的存储单元地址低 8 位中的高 4 位。

7.14 在设计 DMA 传输程序时，要有哪些必要的程序模块？设计一个用 DMA 方式实现数据块输出的程序段。

【解】 要进行 DMA 传送，需要启动 DMA 请求、等待 CPU 的 DMA 响应，然后就是数据传送(这是主体部分)，数据传送完成后就要结束 DMA 工作。用 DMA 方式实现数据块输出的程序流程图如图 7-2 所示。

7.15 在查询方式、中断方式和 DMA 方式中，分别采用什么方法启动数据传输过程？

【解】 查询方式在要进行数据输入输出的程序中，用程序查询条件，当条件满足时，程序输入或输出数据。在中断方式，由中断请求信号，使 CPU 转至中断服务程序，在中断服务程序中完成数据传统。在 DMA 方式中，DMA 控制器向 CPU 发出 DMA 请求，CPU 响应 DMA 请求后，由 DMA 控制器启动和完成数据传送。

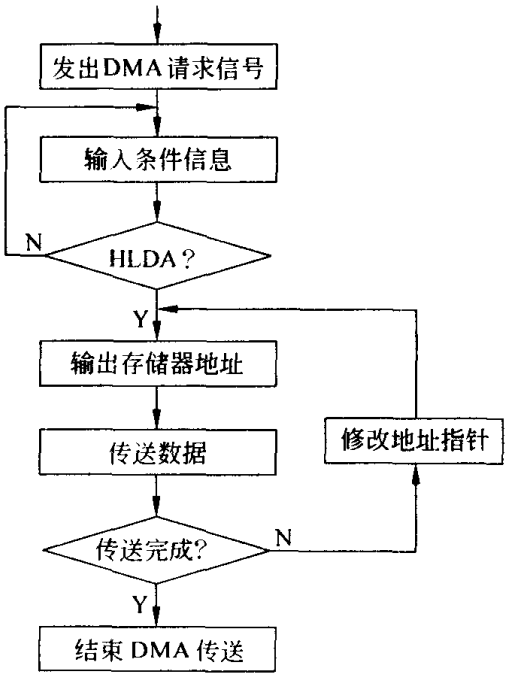


图 7-2 题 7.14DMA 方式传送数据的流程图

7.16 若有一个 CRT 终端，它的输入输出数据的端口地址为 01H，状态端口的地址为 00H，其中 D_7 位为 TBE，若其为 1，则表示发送缓冲区空，CPU 可向它输出新的数据；D_6 位为 RDA，若其为 1，则表示输入数据有效，CPU 可以输入该有效数据。

(1) 编写一个程序，从终端上输入 100 个字节的字符，送到以 BUFFER 开始的内存缓冲区中去。

(2) 若已经有一个能用查询方法从键盘输入一个字符放于累加器 A 中的子程序 GETCH，利用此子程序，完成本题(1)中提出的要求。

(3) 编写一个程序，把内存中自 BLOCK 开始的 100 个字节的数据块，通过终端显示。

(4) 若已有一个能用查询方式，把累加器 AL 中的字符输出的子程序 PUTCH，利用此子程序，完成本题(3)中的要求。

(5) 编写一个程序，能从终端上输入一个字符，放入寄存器 CL。

(6) 编写一个程序，使能把在寄存器 CL 中的一个字符输出给终端。

(7) 编写一个程序，能把内存中以 STRING 开始的一个字符串(以'$'字符作为结束标志)通过终端输出。

(8) 编写一个程序，利用子程序 PUTCH 实现本题(7)的要求。

(9) 编写一个程序，能从终端输入一个字符串(以回车键作为字符串的结束标志)放到内存中以 BUFFER 开始的缓冲区。

(10) 编写一个程序，先向终端输出一个提示符'>'，表示要求输入一个十进制数；然后能从终端上输入一个带符号的最多为 5 位的十进制数(但数值范围在 ±32 768以内)，以非数字字符作为输入的十进制数的结束标志，把这个十进制数连同符号放至内存中以 DATA 开始的缓冲区(高位在前)。

(11) 条件及要求同本题(10)，在输入十进制数的程序中，允许输入的十进制数码多于 5 个，但程序中取后输入的 5 个。

(12) 编写一个程序，能把累加器 A 中的 8 位带符号二进制数，以十进制数形式输出(注意：在终端上输入输出的字符都以 ASCII 码表示)。

(13) 编写一个程序，把在 AX 寄存器中的 16 位带符号二进制数，以十进制数的形式输出。

【解】 下面大部分只给出了相应的程序段。

(1)

```
        LEA     BX,BUFFER
        MOV     CX,100
POL1:   IN      AL,00H
        AND     AL,40H
        JE      POL1
        IN      AL,01H
        MOV     [BX],AL
        INC     BX
        LOOP    POL1
```

(2)

```
        LEA     BX,BUFFER
        MOV     CX,100
POL1:   CALL    GETCH
        MOV     [BX],AL
        INC     BX
        LOOP    POL1
```

(3)

```
        LEA     BX,BLOCK
        MOV     CX,100
POL1:   IN      AL,00H
        AND     AL,80H
        JE      POL1
        MOV     AL,[BX]
        OUT     01H,AL
        INC     BX
        LOOP    POL1
```

(4)

```
        LEA     BX,BLOCK
        MOV     CX,100
POL1:   CALL    PUTCH
        INC     BX
        LOOP    POL1
```

(5)

```
POL1:   IN      AL,00H
        AND     AL,40H
        JE      POL1
        IN      AL,01H
        MOV     CL,AL
```

(6)

```
POL1:   IN      AL,00H
        AND     AL,80H
        JE      POL1
        MOV     AL,CL
        OUT     01H,AL
```

(7)

```
        LEA     BX,STRING
POL1:   IN      AL,00H
        AND     AL,80H
        JE      POL1
        MOV     AL,[BX]
        CMP     AL,'$'
        JE      N1
        OUT     01H,AL
        INC     BX
        JMP     POL1
N1:     OUT     01H,AL
        RET
```

(8)

```
        LEA     BX,STRING
POL1:   CALL    PUTCH
        MOV     AL,[BX]
        CMP     AL,'$'
        JE      N1
        OUT     01H,AL
        INC     BX
        JMP     POL1
N1:     OUT     01H,AL
        RET
```

(9)

```
        LEA     BX,BUFFER
POL1:   IN      AL,00H
        AND     AL,40H
        JE      POL1
```

```
        IN      AL,01H
        CMP     AL,'CR'
        JE      N1
        MOV     [BX],AL
        INC     BX
        JMP     POL1
N1:     MOV     [BX],AL
        RET
```

(10)

```
        NAME    GET_DECIMAL
DATA1   SEGMENT
DATA    DB      6 DUP(?)
DATA1   ENDS
STACK   SEGMENT PARA STACK 'STACK'
        DB      100 DUP(?)
STACK   ENDS
CODE    SEGMENT
        ASSUME  CS:CODE,DS:DATA1,SS:STACK
START   PROC    FAR
BEGIN:  PUSH    DS
        MOV     AX,0
        PUSH    AX
        MOV     AX,DATA
        MOV     DS,AX
        MOV     DL,'>'
        MOV     AH,2
        INT     21H
        LEA     BX,DATA
        MOV     AH,10
        INT     21H
LOP1:   MOV     AL,[BX]
        CMP     AL,'+'
        JE      N1
        CMP     AL,'-'
        JE      N1
        CMP     AL,30H
        JL      N2
        CMP     AL,39H
        JG      N2
        AND     AL,0FH
        MOV     [BX],AL
        INC     BX
        JMP     LOP1
```

```
N1:     MOV     [BX],AL
        INC     BX
        JMP     LOP1
N2:     RET
START   ENDP
CODE    ENDS
        END     BEGIN
```

(11)

```
        NAME    GET_DECIMAL
DATA1   SEGMENT
DATA    DB      6 DUP(?)
BUFFER  DB      10 DUP(?)
DATA1   ENDS
STACK   SEGMENT PARA STACK 'STACK'
        DB      100 DUP(?)
STACK   ENDS
CODE    SEGMENT
        ASSUME  CS:CODE,DS:DATA1,SS:STACK
START   PROC    FAR
BEGIN:  PUSH    DS
        MOV     AX,0
        PUSH    AX
        MOV     AX,DATA
        MOV     DS,AX
        MOV     DL,'>'
        MOV     AH,2
        INT     21H
        LEA     BX,DATA
        MOV     AH,10
        INT     21H
        LEA     SI,BUFFER
        MOV     DL,0
LOP1:   MOV     AL,[BX]
        CMP     AL,'+'
        JE      N1
        CMP     AL,'-'
        JE      N1
        CMP     AL,30H
        JL      N2
        CMP     AL,39H
        JG      N2
        AND     AL,0FH
        INC     DL
```

```
            MOV     [SI],AL
            INC     SI
            INC     BX
            JMP     LOP1
N1:         MOV     [BX],AL
            INC     BX
            JMP     LOP1
N2:         CMP     DL,5
            JLE     LOP2
            MOV     DL,5
LOP2:       MOV     AL,[SI-1]
            MOV     [BX-1],AL
            DEC     SI
            DEC     BX
            DEC     DL
            JNZ     LOP2
            RET
START       ENDP
CODE        ENDS
            END     BEGIN
```

7.17 试说明在 DMA 方式时由内存向外设传输数据的过程。

【解】 当 CPU 响应 DMA 请求发出 HLDA 信号后，DMA 控制器接受对总线的控制，向地址线发生要寻址的内存的地址，并发出读/写控制信号。把内存的数据输出至外设。然后判断是否已传送完。若未完，修改内存地址，重复输出过程。直至输出完成，结束 DMA 请求。

7.18 对一个 DMA 控制器的初始化工作包括哪些内容？

【解】 DMA 控制器的初始化工作包括：

① 输出主清除命令；

② 写入基与现行地址寄存器；

③ 写入基与现行字节数寄存器；

④ 写入模式寄存器；

⑤ 写入屏蔽寄存器；

⑥ 写入命令寄存器；

⑦ 写入请求寄存器。若有软件请求，就写入至指定通道，就可以开始 DMA 传送的过程。若无软件请求，则在完成了①～⑥的编程后，由通道的 DREQ 启动 DMA 传送过程。

7.19 DMA 控制器 8237A 何时作为主模块工作？何时作为从模块工作？在这两种情况下，各控制信号处于什么状态？试作说明。

【解】 除了在级连方式，作为第二级或其后级时，8237 工作在从模式。其他情况下都工作在主模式。在级连情况下，第一级只起优先权网络的作用，除了由某一个

二级的请求向 CPU 输出 HRQ 信号外，并不输出任何其他信号。实际的操作是由第二级的片子完成的。若有需要还可由第二级扩展到第三级等。

7.20 8237A 有哪几种工作模式？各自用在什么场合？

【解】 8237A 有 4 种工作模式。

(1) 单字节传送模式

这种模式是编程为一次只传送一个字节。数据传送后字节计数器减量，地址要相应修改(增量或减量取决于编程)。HRQ 变为无效，释放系统总线。若传送使字节数减为 0，TC 发生或者终结 DMA 传送，或重新初始化。

(2) 块传送方式

在这种传送方式，8237 由 DREQ 启动就连续地传送数据，直至字节数计数器减到零产生 TC(Terminal Count)，或者由外部输入有效的$\overline{\text{EOP}}$信号来终结 DMA 传送。

(3) 请求传送方式

在这种工作方式下，8237 可以进行连续的数据传送。当出现以下 3 种情况之一时停止传送。

① 字节数计数器减到 0，发生 TC；

② 由外界送来一个有效的$\overline{\text{EOP}}$信号；

③ 外界的 DREQ 信号变为无效(外设的数据已传送完)。

(4) 级连方式

这种方式用于通过级连以扩展通道。第二级的 HRQ 和 HLDA 信号连到第一级的 DREQ 和 DACK 上。

7.21 什么叫 DMA 控制器的自动预置功能？这种功能用得很普遍，举一个例子说明它的使用场合。

【解】 自动预置功能即自动初始化功能，即若在模式寄存器的编程中允许自动初始化，则在每次$\overline{\text{EOP}}$后，现行地址寄存器初始化为它的初始值(即保存在基地址寄存器中的值)。现行字节数寄存器它自动初始化至它的起始状态(即保存在基字节数寄存器中的值)。这种功能使用在从一内存地址传送相同字节数的场合。

7.22 DMA 控制器进行内存到内存的传输时，有什么特点？

【解】 8237 可以编程工作在存储器到存储器之间传送。这时就要用到两个通道，通道 0 的地址寄存器编程为源区地址；通道 1 的地址寄存器编程为目的区地址，字节数寄存器编程为传送的字节数。传送由设置一个通道 0 的软件 DREQ 启动，8237 按正常方式向 CPU 发出 DMA 请求信号 HRQ，待 CPU 用 HLDA 信号响应后传送就可以开始，每传送一个字节要用 8 个时钟周期，其中，4 个时钟周期以通道 0 为地址从源区读数据送入 8237 的临时寄存器，另 4 个时钟周期以通道 1 为地址把临时寄存器中的数据写入目的区。每传送一个字节，源地址和目的地址都要修改(可增量也可减量修改)，字节数减量。传送一直进行到通道 1 的字节数计数器减到零，产生 TC 引起在$\overline{\text{EOP}}$端输出一个脉冲，结束 DMA 传送。

7.23 DMA 控制器 8237A 是怎样进行优先级管理的？

【解】 8237 有两种优先权方式可供选择。一种是固定优先权，在这种方式下通道的优先权是固定的，通道 0 的优先权最高，通道 3 的优先权最低；另一种方式是优先权旋转，在这种方式下刚服务过的通道的优先权变为最低的，其他通道的优先权也作相应的旋转。

7.24 设计 8237A 的初始化程序。其中，8237A 的端口地址为 0000H～000FH，设通道 0 工作在块传输模式，地址加 1 变化，自动预置功能；通道 1 工作于单字节读传输，地址减 1 变化，无自动预置功能；通道 2、通道 3 和通道 1 工作于相同方式。然后对 8237A 设控制命令，使 DACK 为高电平有效，DREQ 为低电平有效，用固定优先级方式，并启动 8237A 工作。

【解】 若高 4 位经译码连至$\overline{CS}$的地址为 5。

```
OUT     5DH,AL
MOV     AL,94H
OUT     5BH,AL
MOV     AL,75H
OUT     5BH,AL
MOV     AL,76H
OUT     5BH,AL
MOV     AL,77H
OUT     5BH,AL
MOV     AL,00H
OUT     5AH,AL
MOV     AL,0E0H
OUT     58H,AL
```

第8章 中 断

8.1 在中断响应过程中，8086CPU 向 8259A 发出的两个$\overline{\text{INTA}}$信号分别起什么作用？

【解】 CPU 发出的第一个$\overline{\text{INTA}}$脉冲告诉外部电路，其提出的中断请求已被响应，应准备将类型号发给 CPU，8259A 接到了这个$\overline{\text{INTA}}$脉冲时，把中断的最高优先级请求置入中断状态寄存器(ISR)中，同时把 IRR(中断请求寄存器)中的相应位复位。CPU 发出的第二个$\overline{\text{INTA}}$脉冲告诉外部电路将中断的类型号放在数据总线上。

8.2 8086CPU 最多可以有多少个中断类型？按照产生中断的方法分为哪两大类？

【解】 8086CPU 最多可以有 256 个中断类型。按照产生中断的方法，可以分为内部中断(软件中断)和外部(硬件中断)中断两大类。

8.3 非屏蔽中断有什么特点？可屏蔽中断有什么特点？它们分别用在什么场合？

【解】 非屏蔽中断不受 CPU 中断标志的影响，无论中断标志是允许中断或禁止中断，在非屏蔽中断引线上的中断请求，CPU 在当前指令执行完以后，都要响应此中断。可屏蔽中断则受中断标志影响，只是在开中断(允许中断)情况下，CPU 才响应可屏蔽中断。非屏蔽中断，使用在 CPU 必须要响应的场合，例如电源故障等。

8.4 什么叫中断向量？它放在哪里？对应于 1CH 的中断向量存放在哪里？如果 1CH 的中断处理子程序从 5110H:2030H 开始，则中断向量应该怎样存放？

【解】 中断向量是区分不同中断和获取相应的中断服务程序的一个字节的量，它由 CPU 产生(对于软件中断)或由外设在中断响应周期输送至 CPU 的。8086 在内存的前 1KB 有一个中断向量表，存放最多为 256 个中断服务程序的入口。题中的中断在中断向量表中的存放。如图 8-1 所示。

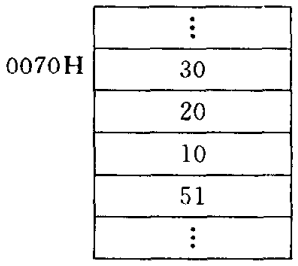


图 8-1 中断向量的存放

8.5 从 8086/8088CPU 的中断向量表中可以看到，如果一个用户想定义某个中断，应该选择在什么范围？

【解】 应该在向量 32～256 之间选择。

8.6 非屏蔽中断处理程序的入口地址怎样寻找？

【解】 CPU 已经给定非屏蔽中断的中断向量为 2。由中断向量 2 从中断向量表中查找。

8.7 简述可屏蔽中断的响应过程，一个可屏蔽中断或者非屏蔽中断响应后，堆栈顶部 6 个单元中的内容是什么？

【解】 CPU 响应可屏蔽中断的过程如下。

① 推入标志寄存器。

② 关中断。

8086 在 CPU 响应中断后，发出中断响应信号$\overline{\text{INTA}}$的同时，内部自动实现关中断。

③ 保留断点。

CPU 响应中断，封锁 IP+1，且把 IP 和 CS 推入堆栈保留，以备中断处理完毕后能返回主程序。

④ 保护现场。

为了使中断处理程序不影响主程序的运行，故要把断点处的有关的各个寄存器的内容和标志位的状态，推入堆栈保护起来。8086 是由软件（即在中断服务程序中）把要用到的寄存器的内容用 PUSH 指令推入堆栈。

⑤ 给出中断入口，转入相应的中断服务程序。

8086 是由中断源提供的中断矢量形成中断入口地址（即中断服务程序的起始地址）。

在中断服务程序完成后，还要做下述的(5)、(6)两步。

⑥ 恢复现场。

把所保存的各个内部寄存器的内容和标志位的状态，从堆栈弹出，送回 CPU 中的原来位置。这个操作在 8086 中也是由服务程序中用 POP 指令来完成的。

⑦ 开中断与返回。

在中断服务程序的最后，要开中断（以便 CPU 能响应新的中断请求）和安排一条返回指令，将堆栈内保存的 IP、CS 值和标志寄存器内容弹出，运行就恢复到主程序。

CPU 响应中断要保护标志寄存器和断点，故堆栈顶部的内容为标志寄存器 IP 和 CS。

8.8 一个可屏蔽中断请求到来时，通常只要中断允许标志为 1，便可以在执行完当前指令后响应，在哪些情况下会有例外？

【解】 若中断标志为 0，即禁止中断，则 CPU 就不响应可屏蔽中断。或此中断有屏蔽设置，屏蔽此中断。

8.9 在编写中断处理子程序时，为什么要在子程序中保护许多寄存器？

【解】 因为在用户程序运行时，会在寄存器中有中间结果，当在中断服务程序中要使用这些寄存器前要把这些寄存器的内容推至堆栈保存（称为保护现场）。在从中断服务程序返回至用户程序时把这些内容从堆栈恢复至寄存器中（称为恢复现场）。

8.10 中断指令执行时，堆栈的内容有什么变化？中断处理子程序的入口地址是怎样得到的？

【解】 因为要保护现场，故堆栈中会保存现场的内容。中断处理子程序的入口地址是由中断向量从中断向量表中获取。

8.11 中断返回指令 IRET 和普通子程序返回指令 RET 在执行时，具体操作内容有什么不同？

【解】 因为在响应中断时，CPU 保护标志寄存器和保护断点。所以，IRET 指令要恢复断点和标志寄存器。

8.12 若在一个系统中有 5 个中断源，其优先权排列为：1、2、3、4、5，它们的中断服务程序的入口地址分别为：3000H、3020H、3050H、3080H、30A0H。编写一个程序，当有中断请求 CPU 响应时，能用查询方式转至申请中断的优先权最高的源的中断服务程序。

【解】 若 5 个中断源的中断请求放在一中断状态寄存器中，按优先权分别放在状

态的位 7(优先权最高)至位 3 中。查询方法的程序段为:

```
        IN      AL,STATUS
        CMP     AL,80H
        JNE     N1
        JMP     3000H
N1:     IN      AL,STATUS
        CMP     AL,40H
        JNE     N2
        JMP     3020H
N2:     IN      STATUS
        CMP     AL,20H
        JNE     N3
        JMP     3050H
N3:     IN      STATUS
        CMP     AL,10H
        JNE     N4
        JMP     3080H
N4:     IN      STATUS
        CMP     AL,08H
        JNE     N5
        JMP     30A0H
N5:     RET
```

8.13 设置中断优先级的目的是什么?

【解】 若有多个外设同时请求中断,它们之间是有轻重缓急的,设置中断优先级就能使 CPU 按照轻重缓急要求响应中断。而且,设置了中断优先级可以使优先级高的中断比优先级低的中断优先处理,实现中断嵌套。

8.14 软中断(两字节 INT *N* 指令)的功能调用与子程序调用有何异同?

【解】 软中断的功能与子程序调用相当。只是子程序是调用用户程序,而软中断是调用系统程序。

8.15 可编程中断控制器 8259A 在中断处理时,协助 CPU 完成哪些功能?

【解】 8259A 有中断请求寄存器、优先权电路和中断在服务寄存器,可接收多个中断请求以及优先权处理。8259A 还可以对每个中断源进行屏蔽。

8.16 8259A 中断控制器具有哪些中断操作功能?请指出与这些功能相对应的控制字(ICW/OCW)的内容。

【解】 ICW_1 的 D_4 必须为 1。D_0 确定是否送 ICW_4,若根据选择 ICW_4 的各位应为 0,则可令 D_0 位(即 IC_4)为 0,则不送 ICW_4。D_1 位 SNGL,规定系统中是单片 8259A 工作还是级连工作。D_2 位 ADI,规定 CALL 地址的间隔,$D_2=1$,则间隔为 4,这适用于建立一个转移指令表;$D_2=0$,则间隔为 8。D_3 位 LTIM,规定中断请求输入线的触发方式,$D_3=1$ 为电平触发方式,此时边沿检测逻辑断开;$D_3=0$ 则为边沿触发方式。

ICW_2 当应用于 8088/8086 系统中时，ICW_2 的 $D_7 \sim D_3$ 用以确定中断向量的 $T_7 \sim T_3$，此时 ICW_2 的 $D_2 \sim D_0$ 位无用。

ICW_3 用于 8259A 的级连，若系统中只有一片 8259A，则不用 ICW_3；若有多片 8259A 级连，则主 8259A 和每一片从 8259A 都必须用 ICW_3。8259A 最多允许有一片主 8259A 和 8 片从 8259A，使中断源扩展至 64 个。主和从 8259A 的 ICW_3 有所不同，对于主 8259A（由 $\overline{SP}=1$ 或由 ICW_4 中的 M/S=1 规定在缓冲方式所决定），ICW_3 的每一位对应于一片从 8259A，即若有一片从 8259A，则可令 ICW_3 的 $S_0=1$，其他的位全为 0；若有两片从 8259A，则可令 $S_0=1$，$S_1=1$，其他的位全为 0……。若是从 8259A，则 ICW_3 中只有低 3 位（位 2～0）作为这个从 8259A 的标识符（ID），高 5 位全为 0。在中断响应周期中，主 8259A 通过级连线输送申请中断的源中优先权最高的源所在的从 8259A 的标识符，每个从 8259A 拿这个标识符与自己编程时 ICW_3 中所规定的标识符相比较，只有两者相符合的这片从 8259A，能在下个中断响应周期输送入一个字节的中断向量。

ICW_4 的 D_0 位 μPM，用于规定所用的微处理器。$\mu PM=0$，则规定 8259A 用于 MCS-80/85 系统中；若 $\mu PM=1$，则规定用于 MCS-86 系统中。

D_1 位 AEOI，规定结束中断的方式，若 AEOI=1，则为自动结束中断方式。

D_2 位 M/S，它与 D_3 位 BUF 配合使用，若 BUF=1，选择为缓冲模式，则 M/S=1 确定为主 8259A；若 M/S=0，则为从 8259A。若 BUF=0，则 M/S 位不起作用。

D_3 位 BUF，若 BUF=1，则为缓冲模式，此时 $\overline{SP}/\overline{EN}$ 变为允许输出线，同时由 M/S 确定是主 8259A 还是从 8259A。

D_4 位 SFNM，若 SFNM=1，则规定为特殊的全嵌套模式。

OCW_1 是中断屏蔽命令字，命令字的每一位可以对相应的中断请求输入线进行屏蔽。OCW_1 的某一位为“1”则相应的输入线被屏蔽；若某一位为“0”，则相应的输入线的中断就允许。

OCW_2 说明了 R、SL、EOI 3 位的功用，它们的不同组合决定了几种不同的工作方式。在其中的 3 种工作方式中要用到 OCW_2 的最低 3 位即 L_2、L_1、L_0，这 3 位二进制编码决定了 8 个中断源的某一个被 SEOI 信号复位，或规定某一个的优先权最低。D_4、D_3 为 00 是写入 OCW_2 的标志。

OCW_3 的最低两位决定下一个操作是否为读操作（RR=1），以及是读中断请求寄存器 IRR（若 RIS=0），还是读中断服务寄存器（ISR）（若 RIS=1）。

D_2 位 P，决定是查询命令（P=1），还是非查询命令（P=0）。

D_1、D_3 位为 01 为写入 OCW_3 的标志。

D_6、D_5 这两位决定是否工作于特殊屏蔽模式，当 D_6、D_5 为 11 时，则允许特殊屏蔽模式；而 D_6、D_5 为 10 时撤除特殊屏蔽模式返回正常的屏蔽模式。若 D_6 位 ESMM=0，则 D_5 位 SMM 不起作用。

8.17 什么是中断响应周期？在中断响应中，8086CPU 和 8259A 一般完成哪些工作？

【解】 中断响应周期是从用户程序至中断服务程序的过渡周期。在第一个中断响应周期，8259A 确定中断的优先级；在第二个中断响应周期，8259A 送出中断向

量，CPU 获取此向量转至中断服务程序的入口。

8.18 若有一中断源，当其有中断请求时，要求 CPU 把一个 100 个字节的数据块从 AREA1 开始的存储区传送至 AREA2 开始的存储区。要求编写出主程序（与中断有关的部分），中断服务程序。所有程序的入口地址（包括中断服务程序入口地址表）由伪指令给定（具体的值可由自己指定）。

【解】 假定中断向量为 100，中断服务程序的入口为 2000:1000。

```
        NAME    INT_1
DATA1   SEGMENT
        ORG     0000H
INT_T   DD      X1,X2,…,X99
        DW      1000,2000
AREA1:  DB      D1,D2,…,D100
AREA2:  DB      100 DUP(?)
DATA1   ENDS
STACK   SEGMENT PARA STACK 'STACK'
        DB      100 DUP(?)
STACK   ENDS
        ORG     2000:0000H
CODE    SEGMENT
        ASSUME  CS:CODE,DS:DATA1,SS:STACK
START   PROC    FAR
BEGIN:  PUSH    DS
        MOV     AX,0
        PUSH    AX
        MOV     AX,DATA
        MOV     DS,AX
        STI
HERE:   JMP     HERE
        ⋮
        RET
START   ENDP
        ORG     1000H
INT_S   PROC
        PUSH    AX
        PUSH    SI
        PUSH    BX
        PUSH    CX
        MOV     CX,100
        LEA     SI,AREA1
        LEA     BX,AREA2
LOP1:   MOV     AL,[SI]
        MOV     [BX],AL
        INC     SI
```

```
        INC     BX
        LOOP    LOP1
        POP     CX
        POP     BX
        POP     SI
        POP     AX
        IRET
INT_S   ENDP
CODE    ENDS
        END     BEGIN
```

8.19 若在内存中自 7000H 单元开始有一个 1000 个字节的信息组要存入磁盘。存入磁盘的操作是在中断服务程序中完成的，但磁盘的写入每次只写入一个记录即 128 字节；而且是从指定的磁盘缓冲区(例如起始地址为 0080H)把信息写入磁盘的。所以在每次写入磁盘以前，要把一个记录的信息从它所在的存储区传送至磁盘缓冲区。编写出中断服务程序中能起这样传送作用的程序段。

【解】

```
        NAME    INT_2
DATA1   SEGMENT
        ORG     0080H
BUFFER  DB      128 DUP(?)
        ORG     7000H
AREA1:  DB      D1,D2,…,D1000
DATA1   ENDS
STACK   SEGMENT PARA STACK 'STACK'
        DB      100 DUP(?)
STACK   ENDS
CODE    SEGMENT
        ASSUME  CS:CODE,DS:DATA1,SS:STACK
START   PROC    FAR
BEGIN:  PUSH    DS
        MOV     AX,0
        PUSH    AX
        MOV     AX,DATA
        MOV     DS,AX
        MOV     ES,AX
        STI
HERE:   JMP     HERE
        ⋮
        RET
START   ENDP
INT_S   PROC
        PUSH    SI
```

```
            PUSH    DI
            PUSH    CX
            MOV     CX,128
            LEA     SI,AREA1
            LEA     DI,BUFFER
            CLD
        REP MOVSB
            CALL    WRDISK
            POP     CX
            POP     DI
            POP     SI
            IRET
INT_S       ENDP
CODE        ENDS
            END     BEGIN
```

8.20 若要把磁盘上一个 1KB 的文件读入内存自 7000H 开始的存储区中，读盘的操作是在中断服务程序中完成的，但读盘每次读入一个记录(128B)放在磁盘缓冲区中。所以在每次读入一个记录后，要把信息自磁盘缓冲区传送至它的存储区。编写出中断服务程序中能完成这样传送的程序段。

【解】

```
            NAME    INT_3
DATA1       SEGMENT
            ORG     0080H
BUFFER      DB      128 DUP(?)
            ORG     7000H
AREA1:      DB      1024 DUP(?)
DATA1       ENDS
STACK       SEGMENT PARA STACK 'STACK'
            DB      100 DUP(?)
STACK       ENDS
CODE        SEGMENT
            ASSUME  CS:CODE,DS:DATA1,SS:STACK
START       PROC    FAR
BEGIN:      PUSH    DS
            MOV     AX,0
            PUSH    AX
            MOV     AX,DATA
            MOV     DS,AX
            MOV     ES,AX
            STI
HERE:       JMP     HERE
            ⋮
```

```
        RET
START   ENDP
INT_S   PROC
        PUSH    SI
        PUSH    DI
        PUSH    CX
        CALL    RDDISK
        MOV     CX,128
        LEA     SI,BUFFER
        LEA     DI,AREA1
        CLD
    REP MOVSB
        POP     CX
        POP     DI
        POP     SI
        IRET
INT_S   ENDP
CODE    ENDS
        END     BEGIN
```

8.21 8086CPU 有哪几种中断？哪些是硬件中断？哪些是软件中断？

【解】 8086CPU 有两类中断：软件中断——由指令的执行所引起的；硬件中断——由外部(主要是外设)的请求所引起的。

(1) 外部中断

8086 有两条外部中断请求线：NMI——Non Maskable Interrupt(非屏蔽中断)和 INTR(可屏蔽中断)。

(2) 内部中断

8086 可以有几种产生内部中断的情况：DIV 或 IDIV 指令、INT 指令、INTO 指令和单步执行。

8.22 什么是 8086 的中断向量？中断向量表是什么？8086CPU 系统的中断向量表放在何处？

【解】 8086 的中断向量是区分不同中断和获取相应的中断服务程序的一个字节的量。它或是由 CPU 产生(对于软件中断)或由外设在中断响应周期输送至 CPU 的。中断向量表是存放 256 个中断服务程序入口的跳转表，由中断向量从表中取出相应的中断服务程序的入口。8086 在内存的前 1KB 有一个中断向量表，存放最多为 256 个中断服务程序的入口地址。

8.23 8259A 的初始化命令字和操作命令字有什么差别？它们分别对应于编程结构中哪些内部寄存器？

【解】 8259A 的初始化命令字是使 8259A 在开始工作前处在正确的起始状态。初始化命令主要规定 8259A 的工作模式和中断向量。初始化命令字写至 $ICW_1 \sim ICW_4$。

8259A 操作命令字主要用于规定 8259A 的工作方式，例如中断屏蔽、结束中断、优先权旋转和中断状态。

工作命令字可在 8259A 已经初始化以后的任何时间写入。

8.24 8259A 的中断屏蔽寄存器 IMR 和 8086/8088CPU 的中断允许标志 I 有什么差别？在中断响应过程中，它们是怎样配合工作的？

【解】 8086/8088CPU 的中断允许标志 I，用于确定是否允许或禁止可屏蔽中断。多个外设都可以连至 CPU 的可屏蔽中断输入线。利用 8259A 的中断屏蔽寄存器 IMR 可以对外设的中断请求，分别地予以屏蔽，增大了灵活性。

8.25 8259A 的全嵌套方式和特殊全嵌套方式有什么差别？各自用在什么场合？

【解】

(1) 全嵌套方式

当工作在全嵌套模式时，在初始化编程以后，中断优先权是固定的，且 IR_0 优先权最高，IR_7 优先权最低(除非用优先权旋转的办法来改变)。当 CPU 响应中断时，申请中断的优先权最高的源在 ISR 中的相应位置位，而且把它的中断矢量送至数据总线(在 MCS-80/85 中为 CALL 指令)。在此中断源的中断服务程序完成之前，与它同级或优先权更低的中断源的申请就被屏蔽，只有优先权比它高的源的中断申请才是允许的(当然 CPU 是否响应取决于 CPU 是否处在开中断状态)。

(2) 特殊全嵌套模式

若在一个大的系统中，8259A 用在级连情况下，就需要采用特殊全嵌套模式。在这种模式下与全嵌套模式的工作情况基本上是相同的，只有以下两点不同。

- 当某一个从 8259A 有中断请求，CPU 响应以后，这个从 8259A 的中断并没有被屏蔽，即这个从 8259A 中优先权高于正在处理的源的中断申请仍可以产生(在全嵌套模式中这样的中断是被屏蔽的)。
- 当某个中断源要退出中断服务程序前，用软件检查它是否这个从 8259A 中的惟一的中断源。检查的办法是送一个非特殊中断结束(EOI)命令给这个从 8259A，然后读它的 ISR，检查它是否为 0，若为 0，则这个从 8259A 中的中断源是惟一的；否则就不是惟一的；只有在读回的 ISR 为 0 时，再把另一个非特殊 EOI 命令送至主 8259A，结束此从 8259A 的中断。

8.26 8259A 的优先级自动旋转方式和优先级特殊旋转方式有什么差别？

【解】

(1) 自动旋转

在某些应用情况下，若干个中断源有相等的优先权。因此，当某一个中断源服务完以后，它的优先权应该变成最低的。这样，某个中断源的请求必须等待，在最坏情况下，必须等待其他 7 个源都服务一次以后才能再服务。

(2) 特殊旋转方式

自动旋转方式，适用于设备的优先权相等的情况下。在特殊旋转方式，可用程序来改变优先权。可以用 OCW_2 来设置最低优先权的源，则其他的输入线的优先权也就相应固定了。例如，设置 IR_5 为最低优先权，则 IR_6 的优先权就变为最高的了。

8.27 8259A 的特殊屏蔽方式和普通屏蔽方式相比，有什么不同之处？特殊屏蔽方式一般用在什么场合？

【解】 8259A 的 8 个中断请求线的每一条都可根据需要单独屏蔽，OCW_1 写入主屏蔽字寄存器，它的每一位可对相应的请求线实现屏蔽。

在某些应用场合，可能要求能在软件的控制下动态地改变系统的优先权结构。也就是说，若 CPU 正处在中断服务过程中，希望能屏蔽一些较低优先权的源的中断，而允许一些优先权更低的源申请中断。但是在通常的工作方式下，当较高优先权的源正处在中断服务的过程中，所有优先权较低的源的中断全给屏蔽了，达不到上述的要求。为此，8259A 中有一种特殊屏蔽模式。若在 OCW_3 中的 D_6 位 ESMM＝1，且 D_5 位 SMM＝1，则使 8259A 工作在特殊的屏蔽模式。此时，由 OCW_1 写入的屏蔽字中为“1”的这些位的中断被屏蔽，而为“0”的这些位的中断不管其优先权如何，在任何情况下都可申请中断。

8.28 8259A 有几种结束中断处理的方式？各自应用在什么场合？除了中断自动结束方式以外，其他情况下如果没有在中断处理程序中发出中断结束命令，会出现什么问题？

【解】 当某一个中断源的服务完成时，必须给 8259A 一个中断结束命令，使这个源在 ISR 中的相应位复位。否则，8259A 就不是处在正常的起始状态。在不同的工作情况下，8259A 可以有几种不同的给出中断结束命令的方法。

(1) 自动中断结束模式(AEOI)

可以在 ICW_4 中规定工作在这种模式，则在最后一个中断响应周期(对于 MCS-86 为第二个周期)的$\overline{INTA}$信号的后沿 8259A 自动地使中断源在 ISR 中的相应位复位。这种方式显然只能用于不要求中断嵌套的情况下。

(2) 非自动中断结束方式(EOI)

在这种工作方式下，当中断服务完成从中断服务程序返回之前，必须发送中断结束(EOI)命令。若是工作在 8259A 级连的情况下，则必须发送两个 EOI 命令，一个送给从 8259A，另一个送给主 8259A，若是在特殊嵌套模式下，在发送了第一个 EOI 命令后，必须经过检查确定这一从 8259A 的所有申请中断的源都已经服务了，才向主 8259A 送出另一个 EOI 命令。

8.29 8259A 引入中断请求的方式有哪几种？如果对 8259A 用查询方式引入中断请求，会有什么特点？中断查询方式一般用在什么场合？

【解】 一片 8259A 有 8 个中断请求输入线，有一条中断输出线，通过 8259A 的级连，最多可以扩展为 64 个输入请求。主 8259A 的中断输出线连至 CPU 的可屏蔽中断输入线。若当外部中断源超过 64 个时，则 8259A 片子可工作在查询方式。此时，在 8259A 的编程中，使 OCW_3 的 D_2 位 P 置为 1。程序中令 CPU 关中断，用查询对外设进行服务。

在令 OCW_3 的 D_2 位 P 置为 1 后的下一个读命令，8259A 可看作是中断响应信号，使最高优先权的 ISR 的相应位置位。读命令从数据总线上读取一个字节，其内容为

D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
I	—	—	—	—	W_2	W_1	W_0

其中，I=1，则表示此片 8259A 有中断请求；I=0 则无中断，可查询别的片子。在 I=1时，$W_2 \sim W_0$为有中断请求的最高优先权源的编码。

8.30 8259A 的初始化命令字有哪些？它们各自有什么含义？哪几个应写入奇地址？哪几个应写入偶地址？

【解】 8259A 的初始化命令字有 4 个，但并不是所有的初始化编程都要写此 4 个 ICW。ICW_1 和 ICW_2 是必须送的，而 ICW_3 和 ICW_4 是由工作方式来选择的。

ICW_1 的 D_4 必须为 1。D_0 确定是否送 ICW_4，若根据选择 ICW_4 的各位应为 0，则可令 D_0 位（即 IC_4）为 0，则不送 ICW_4。D_1 位 SNGL，规定系统中是单片 8259A 工作还是级连工作。D_2 位 ADI，规定 CALL 地址的间隔，$D_2=1$，则间隔为 4，这适用于建立一个转移指令表；$D_2=0$，则间隔为 8。D_3 位 LTIM，规定中断请求输入线的触发方式，$D_3=1$ 为电平触发方式，此时边沿检测逻辑断开；$D_3=0$ 则为边沿触发方式。D_7、D_6、D_5 这 3 位在 8086 中无用。

ICW_2 的 $D_7 \sim D_3$ 用以确定中断向量的 $T_7 \sim T_3$，此时 ICW_2 的 $D_2 \sim D_0$ 位无用。

ICW_3 用于 8259A 的级连，若系统中只有一片 8259A，则不用 ICW_3；若有多片 8259A 级连，则主 8259A 和每一片从 8259A 都必须用 ICW_3。主和从 8259A 的 ICW_3 有所不同：对于主 8259A（由 SP=1 或由 ICW_4 中的 M/S=1 规定在缓冲方式所决定），ICW_3 的每一位对应于一片从 8259A，即若有一片从 8259A，则可令 ICW_3 的 $S_0=1$，其他位全为 0；若有两片从 8259A，则可令 $S_0=1$，$S_1=1$，其他位全为 0……。若是从 8259A，则 ICW_3 中只有低 3 位（位 2～0）作为这个从 8259A 的标识符（ID），高 5 位全为 0。在中断响应周期中，主 8259A 通过级连线输送申请中断的源中优先权最高的源所在的从 8259A 的标识符，每个从 8259A 拿这个标识符与自己编程时 ICW_3 中所规定的标识符相比较，只有两者相符合的这片从 8259A，能在下两个中断响应周期输送一个字节的中断向量。

ICW_4 的 D_0 位 μPM，用于规定所用的微处理器。$\mu PM=0$，则规定 8259A 用于 MCS-80/85 系统中；若 $\mu PM=1$，则规定用于 MCS-86 系统中。

D_1 位 AEOI，规定结束中断的方式，若 AEOI=1，则为自动结束中断方式。

D_2 位 M/S，它与 D_3 位 BUF 配合使用，若 BUF=1，选择为缓冲模式，则 M/S=1 确定为主 8259A；若 M/S=0，则为从 8259A。若 BUF=0，则 M/S 位不起作用。

D_3 位 BUF，若 BUF=1，则为缓冲模式，此时$\overline{SP}/\overline{EN}$变为允许输出线，同时由 M/S 确定是主还是从 8259A。

D_4 位 SFNM，若 SFNM=1，则规定为特殊的全嵌套模式。

ICW_1 是偶数地址，而 ICW_2、ICW_3 和 ICW_4 是奇数地址。

8.31 中断控制器 8259A 的 ICW_2 设置了中断类型码的哪几位？试说明对 8259A 分别设置 ICW_2 为 30H、38H、36H 有什么差别？

【解】 在 8259A 系统中，ICW_2 规定了中断向量的高 5 位。ICW_2 为 30H 和 38H

设置了两个不同的中断向量。而 36H 与 30H 一样，是同一个向量。

8.32 8259A 通过 ICW_4 可以给出哪些重要信息？什么情况下不需要用 ICW_4？什么情况下要设 ICW_4？

【解】 ICW_4 确定所有的 CPU、中断结束方式、缓存方式和嵌套方式。若选择 ICW_1 的各位为 0，则可以不写 ICW_4。

8.33 试按照如下要求对 8259A 设置初始化命令字：系统中有一片 8259A，中断请求信号用电平触发方式，下面要用 ICW_4，中断类型码为 60H、61H、62H、…、67H，用特殊全嵌套方式，不用缓冲方式，采用中断自动结束方式。8259A 的端口地址为 93H、94H。

【解】 按题目的要求，ICW_1 为：19H，ICW_2 为：60H，ICW_4 为：13H。相应为程序段为：

```
MOV     AL,19H
OUT     94H,AL
MOV     AL,60H
OUT     95H,AL
MOV     AL,13H
OUT     95H,AL
```

8.34 怎样用 8259A 的屏蔽命令字来禁止 IR_3 和 IR_5 引脚上的请求？又怎样撤销这一禁止命令？设 8259A 的端口地址为 93H、94H。

【解】 除了初始化编程外，要屏蔽 IR_3 和 IR_5 有中断请求，需输出 OCW_1，屏蔽 IR_3 和 IR_5 的 $OCW_1 = 28H$。撤销屏蔽的 $OCW_1 = 00H$。相应的程序段为：

```
MOV     AL,28H
OUT     95H,AL
:
MOV     AL,00H
OUT     95H,AL
```

8.35 试用 OCW_2 对 8259A 设置中断结束命令，并使 8259A 按优先级自动循环方式工作。

【解】 $OCW_2 = 40H$。

8.36 用流程图来表示特殊全嵌套方式时的工作过程。设主程序运行时先在 IR_2 端有请求，接着 IR_2 端又有请求，而此时前一个 IR_2 还未结束，后来 IR_3 端有请求，再后来 IR_1 端有请求。

【解】 假定工作在特殊全嵌套方式，固定优先权，即 IR_0 的优先级最高、IR_7 的优先级最低。按题的要求的流程图如图 8-2 所示。

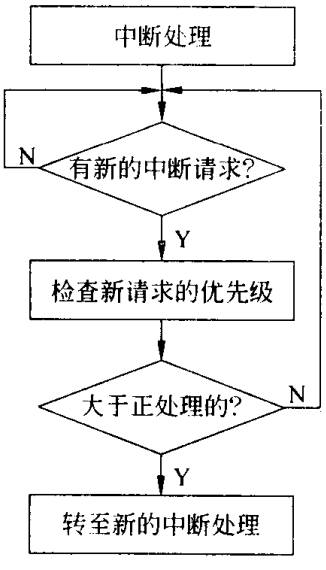


图 8-2 题 8.36 的流程图

8.37 说明特殊屏蔽方式的使用方法。为什么要用“或”的方法来设置屏蔽字？

【解】 在某些应用场合，可能要求能在软件的控制下动态地改变系统的优先权结构。也就是说，若 CPU 正处在中断服务过程中，希望能屏蔽一些较低优先权的源的中断，而允许一些优先权更低的源申请中断。但是在通常的工作方式下，当较高优先权的源正处在中断服务的过程中，所有优先权较低的源的中断全给屏蔽了，达不到上述的要求。为此，8259A 中有一种特殊屏蔽模式。若在 OCW_3 中的 D_6 位 ESMM＝1，且 D_5 位 SMM＝1，则使 8259A 工作在特殊的屏蔽模式。此时，由 OCW_1 写入的屏蔽字中为“1”的这些位的中断被屏蔽，而为“0”的这些位的中断不管其优先权如何，在任何情况下都可申请中断。

8.38 在 80386 系统中，中断控制器 8259A 采用了级连方式，试说明在主从式中断系统中 8259A 的主片和从片的连接关系。

【解】 在级连方式下，外设的中断请求，连至从片的 $IR_0 \sim IR_7$，从片的 INT 输出线连至主片的 $IR_0 \sim IR_7$ 中的某一条。所以，主 8259A 可与 8 个从片相连。主片的 INT 输出线连至 CPU 的 INTR 端。

第 9 章　计数器和定时器电路 Intel 8253/8254-PIT

9.1　定时与计数技术在微型计算机系统中有什么作用?

【解】　在控制系统中,常常要求有一些实时时钟以实现定时或延时控制,如定时中断、定时检测、定时扫描等,也往往要求有计数器能对外部事件计数。

9.2　计数器/定时器 8253 有哪几种工作方式? 各有何特点? 其用途如何?

【解】　计数器/定时器 8253 有以下 6 种工作方式:

方式 0——计完最后一个数时中断,用于只计数一次;

方式 1——可编程序的单拍脉冲;

方式 2——速率发生器;

方式 3——方波速率发生器;

方式 4——软件触发选通;

方式 5——硬件触发选通。

方式 2、4、5 的输出波形是相同的,都是宽度为一个 CLK 周期的负脉冲。但方式 2 是连续工作,方式 4 由软件(设置计数值)触发启动,而方式 5 由门控脉冲触发启动。方式 5(硬件触发选通)与方式 1(硬件再触发单拍脉冲),工作方式基本相同,但输出波形不同,方式 1 输出的为宽度是 N 个 CLK 脉冲的低有效脉冲(计数过程中输出为低),而方式 5 输出的为宽度是 1 个 CLK 脉冲的负脉冲(计数过程中输出为高)。

9.3　在某一应用系统中,计数器/定时器 8253 地址为 340H～343H,定时器 0 用作分频器(N 为分频系数),定时器 2 用作外部事件计数器,如何编制初始化程序?

【解】　定时器 0 用作分频器,工作在方式 2,其初始化程序为:

```
MOV     AL,34H
OUT     343H,AL
MOV     AL,N1
OUT     342H,AL
MOV     AL,N2
OUT     342H,AL
```

定时器 2 用作外部事件计数器,工作在方式 0,其初始化程序为:

```
MOV     AL,0B0H
OUT     343H,AL
MOV     AL,N1
OUT     342H,AL
MOV     AL,N2
OUT     342H,AL
```

9.4　若已有一频率发生器,其频率为 1MHz,若要求通过计数器/定时器 8253,产生每秒

一次的信号，8253 应如何连接？编写出初始化程序。

【解】 1MHz 的信号要变为每秒一次，则需经过 10^6 分频。显然一个通道的计数为 16 位最大为 65536。故需两个通道级连。则每个通道计数为 1000。用通道 0 和通道 1 级连。都工作在方式 2。初始化程序为：

```
MOV     AL,34H
OUT     343H,AL
MOV     AL,0E8H
OUT     342H,AL
MOV     AL,03H
OUT     342H,AL
MOV     AL,74H
OUT     343H,AL
MOV     AL,0E8H
OUT     342H,AL
MOV     AL,03H
OUT     342H,AL
```

9.5 条件同上，若要求每隔 5 秒钟产生一个正脉冲，8253 应该如何连接？编写出初始化程序。

【解】 要求每隔 5 秒钟产生一个正脉冲，仍用两个通道级连，第一个通道计数为 5000。要求输出正脉冲，比较合适的方法是 8253 通道 1 的输出经过反相后得到。初始化程序为：

```
MOV     AL,34H
OUT     343H,AL
MOV     AL,13H
OUT     342H,AL
MOV     AL,88H
OUT     342H,AL
MOV     AL,74H
OUT     343H,AL
MOV     AL,0E8H
OUT     342H,AL
MOV     AL,03H
OUT     342H,AL
```

9.6 计数器/定时器 8253 中，时钟信号 CLK 和门脉冲信号 GATE 分别起什么作用？

【解】 CLK 信号是 8253 的输入时钟信号，8253 对 CLK 信号计数或分频。GATE 信号是 8253 的门控信号，控制 8253 是否计数。在各种方式下，门控信号的作用如表 9-1 所示。

表 9-1　门控信号的作用

方式	功　能	GATE		
		低或变为低	上升沿	高
0	计完最后一个数中断	禁止计数	—	允许计数
1	硬件再触发单拍脉冲	—	① 启动计数 ② 下一个 CLK 脉冲使输出变低	—
2	速率发生器	① 禁止计数 ② 立即使输出为高	① 重新装入计数值 ② 启动计数	允许计数
3	方波速率发生器	① 禁止计数 ② 立即使输出为高	启动计数	允许计数
4	软件触发选通	禁止计数	—	允许计数
5	硬件触发选通	—	启动计数	—

9.7　说明计数器/定时器 8253 在 6 种工作模式下的特点，并举例说明使用场合。

【解】　见题 9.2。

9.8　计数器/定时器 8253 工作于模式 4 和模式 5 时有什么不同？

【解】　工作模式 4 与模式 5 的主要区别在于门控信号的控制作用。在模式 4，GATE 应保持为“1”，由软件启动计数。而模式 5 是由门控脉冲的上升沿触发启动计数。

9.9　编程将计数器/定时器 8253 计数器 0 设置为模式 1，计数初值为 3000H；计数器 1 设置为模式 2 计数初值为 2010H；计数器 2 设置为模式 4，计数初值为 4030H；计数器 3 设置为模式 3，计数初值为 5060H。

【解】　若端口地址为：F8H～FBH，初始化程序为：

```
MOV     AL,32H
OUT     0FBH,AL
MOV     AL,00H
OUT     0F8H,AL
MOV     AL,30H
OUT     0F8H,AL
MOV     AL,74H
OUT     0FBH,AL
MOV     AL,10H
OUT     0F8H,AL
MOV     AL,20H
OUT     0F8H,AL
MOV     AL,0B8H
OUT     0FBH,AL
```

```
MOV     AL,30H
OUT     0F8H,AL
MOV     AL,40H
OUT     0F8H,AL
MOV     AL,0F6H
OUT     0FBH,AL
MOV     AL,60H
OUT     0F8H,AL
MOV     AL,50H
OUT     0F8H,AL
```

9.10 下面是一个计数器/定时器 8253 的初始化程序段。8253 的控制口地址为 46H,3 个计数器端口地址分别为 40H、42H、44H。在 8253 初始化前,先将 8259A 的所有中断进行屏蔽,8259A 的奇地址端口为 81H。请对下面程序段加上详细注释,并以十进制数表示出各计数器初值。

【解】

```
INI:    CLI
        MOV     AL,0FFH
        OUT     81H,AL      ;屏蔽 8259A 的所有中断
        MOV     AL,36H      ;通道 0 先输出低 8 位后输出高 8 位,方式 3,二进制计数
        OUT     46H,AL
        MOV     AL,0
        OUT     40H,AL
        MOV     AL,40H
        OUT     40H,AL      ;计数值为 4000H
        MOV     AL,54H      ;通道 1 只写低 8 位,方式 2,二进制计数
        OUT     46H,AL
        MOV     AL,18H      ;计数值为十进制 24
        OUT     42H,AL
        MOV     AL,0B6H     ;通道 2 先写低 8 位后写高 8 位,方式 3,二进制计数
        OUT     46H,AL
        MOV     AL,46H
        OUT     44H,AL
        MOV     AL,80H
        OUT     44H,AL
```

9.11 下面是一个用 8253 作为定时器的发音程序,程序中已加了部分注释。请对计数器/定时器 8253 的有关程序段加上注释。设 8253 的控制口地址为 46H,3 个计数器端口地址分别为 40H、42H、44H,8255A 的 B 端口接扬声器驱动电路,B 端口的地址为 61H。

【解】

```
SOUND:  PUSHF
        CLI
```

```
        OR      DH,DH       ;DH 中为发长音的个数
        JZ      K3          ;如不发长音,则转 K3
K1:     MOV     BL,6        ;如发长音,则置长音计数器
        CALL    BEEL        ;调用发音程序
K2:     LOOP    K2          ;两音之间留一点间隙
        DEC     DH          ;长音发完否
        JNZ     K1          ;否,则继续
K3:     MOV     BL,1        ;如发完长音,则置短音计数器
        CALL    BEEL        ;调用发音程序
K4:     LOOP    K4          ;发音之间留一点间隙
        DEC     DL          ;继续发短音吗
        JNZ     K3          ;是,则继续
K5:     LOOP    K5          ;否,则留一个间隙
        POPF                ;标志恢复
        RET                 ;返回
BEEL:   MOV     AL,B6H
        OUT     46H,AL
        MOV     AX,533H
        OUT     44H,AL
        MOV     AL,AH
        OUT     44H,AL
        IN      AL,61H      ;取扬声器驱动信息
        MOV     AH,AL
        OR      AH,03       ;接通扬声器
        OUT     61H,AL      ;扬声器驱动
        SUB     CX,CX       ;一次发音时间定时
K7:     LOOP    K7
        DEC     BL          ;BL 中为发音计数值
        JNZ     K7          ;如未结束,则继续发音
        MOV     AL,AH       ;如发音结束,则恢复 B 端口信息
        OUT     61H,AL
        RET
```

第 10 章　并行接口芯片 8255

10.1　接口部件的输入输出操作具体对应哪些功能？请举例说明。

【解】 CPU 总是通过接口电路(Interface)才能与外设连接。所以，接口电路一边与 CPU 连接，另一边与外设连接。

在接口电路中要有输入输出数据的锁存器和缓冲器；要有状态(如 READY、BUSY 等)和控制命令的寄存器，以便于 CPU 与接口电路之间用应答方式(程序查询式)来交换信息，也便于接口电路与外设间传送信息。接口电路中还要有端口的译码和控制电路，以及为了与 CPU 用中断方式交换信息所需要的中断请求触发器、中断屏蔽触发器、中断优先权排队电路和能向 CPU 发出中断矢量的电路等。这样才能解决 CPU 的驱动能力问题、时序的配合问题和实现各种控制，保证 CPU 能正确、可靠地与外设交换信息。

10.2　从广义上说，接口部件有哪些功能？

【解】 从广义上说接口部件应有以下功能：

(1) 两个或两个以上的具有锁存器或缓冲器的数据端口。

(2) 每个数据端口都有与 CPU 用应答方式交换信号所必须的控制和状态信息。

(3) 每个数据端口通常还具有能用中断方式与 CPU 交换信息所必需的电路。

(4) 选片和控制电路。

(5) 这类接口芯片通常可用程序选择数据端口；选择端口的传送方向(输入、输出或双向)；选择与 CPU 交换信息的方法(查询、中断)等。故，芯片中要有能实现这些选择的控制字寄存器，它可由 CPU 用输出指令来写入。

10.3　在输入过程和输出过程中，并行接口分别起什么作用？

【解】 在输入过程中并行接口主要起输入数据的锁存器作用。在输出过程中并行接口主要起输出数据的缓冲器作用。

10.4　可编程并行接口芯片 8255A 的 3 个端口在使用时有什么差别？

【解】 通常端口 A 或 B 作为输入输出的数据端口(端口 A 还可以作为双向数据端口)，而端口 C 作为控制或状态信息的端口，它在“方式 ”字的控制下，可以分成两个 4 位的端口。每个端口包含一个 4 位锁存器。它们分别与端口 A 和 B 配合使用，可用以作为控制信号输出，或作为状态信号输入。

10.5　当数据从 8255A 的端口 C 向数据总线上传送时，8255A 的控制信号$\overline{CS}$、A_1、A_0、$\overline{RD}$、$\overline{WR}$分别是什么状态？

【解】 这时，$\overline{CS}=0$，$A_1=1$，$A_0=0$，$\overline{RD}=0$，$\overline{WR}=1$。

10.6　可编程并行接口芯片 8255A 的方式选择控制字和置 1/置 0 控制字都是写入控制端口的，那么，它们是由什么来区分的？

【解】 用控制字的 D_7 确定。$D_7=1$，写入的是方式控制字；而 $D_7=0$，写入的是按

位置位/复位字。

10.7 8255A 有哪几种基本工作方式？对这些工作方式有什么规定？

【解】 8255A 有 3 种基本的工作方式：

方式 0(Mode 0)—— 基本输入输出；

方式 1(Mode 1)—— 选通输入输出；

方式 2(Mode 2)—— 双向传送。

10.8 设置 8255A 工作方式，8255A 的控制口地址为 00C6H。要求端口 A 工作在方式 1，输入；端口 B 工作在方式 0，输出；端口 C 的高 4 位配合端口 A 工作；低 4 位为输入。

【解】 按要求的方式控制字为：10111001B。编程语句为：

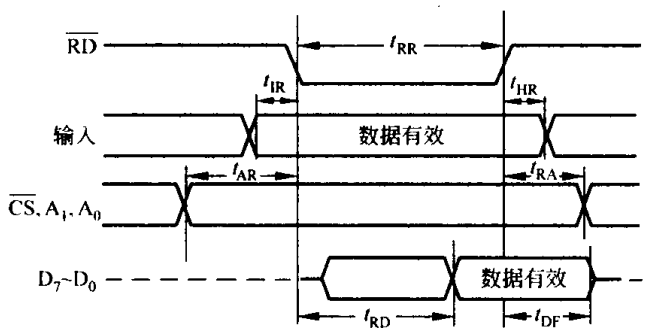
```
MOV     AL,0B8H
OUT     00C6H,AL
```

10.9 设可编程并行接口芯片 8255A 的 4 个端口地址为 00C0H、00C2H、00C4H、00C6H，要求用置 0/置 1 方式对 PC_6 置 1，对 PC_4 置 0。

【解】 对端口 C 的 PC_6 置 1 的控制字为：00001101B，对 PC_4 置 0 的控制字为：00001000B。程序段为：

```
MOV     AL,0DH
OUT     00C6H,AL
MOV     AL,08H
OUT     00C6H,AL
```

10.10 可编程并行接口芯片 8255A 在方式 0 时，如进行读操作，CPU 和 8255A 分别要发出什么信号？对这些信号有什么要求？据此画出 8255A 方式 0 的输入时序。



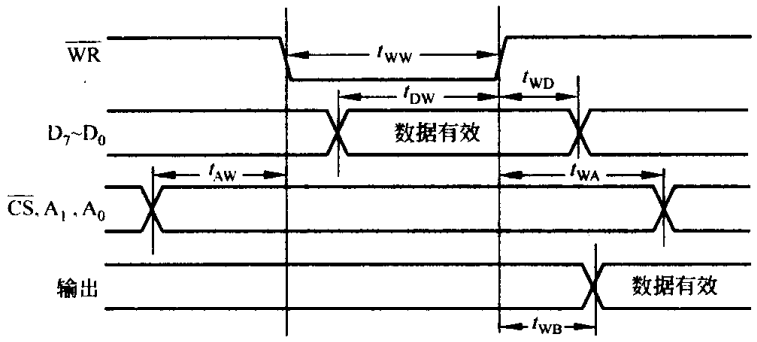
符号	参　数	8255A		单位
		最小	最大	
t_{RR}	读脉冲宽度	300		ns
t_{IR}	输入领先于$\overline{RD}$的时间	0		ns
t_{HR}	输入滞后$\overline{RD}$的时间	0		ns
t_{AR}	地址稳定领先读信号的时间	0		ns
t_{RA}	读信号无效后地址保持时间	0		ns
t_{RD}	从读信号有效到数据稳定		250	ns
t_{DF}	读信号去除后至数据浮空	10	150	ns
t_{RY}	在两次读(或写)之间的时间间隔	850		ns

图 10-1　方式 0 的输入时序

【解】 CPU 发出地址信号连至 8255 的地址和选片线，CPU 还要发出读信号。则在读信号有效后经 t_{RD} 延迟后把输入设备来的数据（若已按要求有效）输出至数据总线 $D_7 \sim D_0$。其时序如图 10-1 所示。

10.11 可编程并行接口芯片 8255A 在方式 0 时，如进行写操作，CPU 和 8255A 分别要发什么信号？画出这些信号之间的时序关系。

【解】 CPU 发出地址信号连至 8255 的地址和选片线，CPU 把要写的数据输出至数据总线 $D_7 \sim D_0$，还要发出写信号。在写信号结束后，经 t_{WA} 延迟后 8255 把数据输出至外设。其时序如图 10-2 所示。



符号	参　数	8255A		单位
		最小	最大	
t_{AW}	地址稳定领先写信号的时间	0		ns
t_{WA}	写信号后地址保持时间	20		ns
t_{WW}	写脉冲宽度	400		ns
t_{DW}	从写信号到数据有效	100		ns
t_{WD}	数据保持时间	30		ns
t_{WB}	从写信号结束到输出		350	ns

图 10-2　8255A 的输出时序

10.12 可编程并行接口芯片 8255A 的方式 0 一般使用在什么场合？在方式 0 时，如何使用应答信号进行联络？

【解】 8255A 的方式 0 一般在普通的输入和输出使用，它没有固定的与外设之间的联络信号线。若在方式 0，要使用联络信号就可以利用端口 C 的一些引线来实现。

10.13 可编程并行接口芯片 8255A 的方式 1 有什么特点？参考教材中的说明，用控制字设定 8255A 的 A 口工作于方式 1，并作为输入口；B 口工作于方式 1，并作为输出口，用文字说明各个控制信号和时序关系。假定 8255A 的端口地址为 00C0H、00C2H、00C4H、00C6H。

【解】 8255A 方式 1 的特点为：

- 用作一个或两个选通端口。
- 每一个端口包含有：8 位的数据端口，3 条控制线（是固定指定的，不能用程序改变），提供中断逻辑。

- 任一个端口都可作为输入或输出。
- 若只有一个端口工作于方式 1，余下的 13 位，可以工作在方式 0（由控制字决定）。
- 若两个端口都工作于方式 1，端口 C 还留下两位，这两位可以由程序指定作为输入或输出，也具有置位/复位功能。

题中要求的方式控制字为：10111100B，相应的初始化程序段为：

```
MOV     AL,0BCH
OUT     00C6H,AL
```

在方式 1 输入时，当外设的数据已经输至 8255A 的端口数据线上，用选通信号$\overline{STB}$把数据锁入 8255A 的输入锁存器，选通信号的宽度至少为 500ns。选通信号经过时间 t_{SIB} 后，IBF 信号有效，输给外设，阻止外设输入新的数据，也可供 CPU 查询；在选通信号结束后，经过 t_{SIT} 向 CPU 发出 INTR 信号（中断是允许的话）。CPU 响应中断，发出$\overline{RD}$信号，把数据读入 CPU。在$\overline{RD}$信号有效后经过 t_{RIT} 就清除中断请求，当$\overline{RD}$信号结束后，数据已读至 CPU，使 IBF 变低。表示输入缓冲器已空，通知外设可输入新的数据。

在方式 1 输出时，用在中断控制方式时，输出过程由 CPU 响应中断开始，在中断服务程序中，CPU 输出数据和发出$\overline{WR}$信号，$\overline{WR}$信号一方面清除 INTR（经过时间 t_{WIT}），另外在$\overline{WR}$上升沿，使$\overline{OBF}$有效，通知外设接受数据，实质上$\overline{OBF}$信号是外设的一个选通命令。在$\overline{WR}$上升沿后经过 t_{WB} 时间数据就输出了，当外设接受数据后，发出$\overline{ACK}$信号，它一方面使$\overline{OBF}$无效（经过 t_{AOB}），另一方面在$\overline{ACK}$的上升沿使 INTR 有效（经过 t_{AIT}），发出新的中断请求。

10.14 可编程并行接口芯片 8255A 的方式 2 用在什么场合？说明端口 A 工作于方式 2 时各信号之间的时序关系。

【解】 8255A 的方式 2 主要用在要求通道双向工作的场合。方式 2 各信号的时序是方式 1 的输入与输出方式的组合，故各个时间参数的意义也相同。输出是由 CPU 执行输出指令（给出$\overline{IOWR}$）信号开始的，输入是由选通信号开始的。

输入、输出的顺序是任意的，只要$\overline{WR}$在$\overline{ACK}$以前发生；$\overline{STB}$在$\overline{RD}$以前发生就行。

10.15 现有 4 种简单外设：① 一组 8 位开关；② 一组 8 位 LED 指示灯；③ 一个按钮开关；④ 一个蜂鸣器。要求：

(1) 用 8255A 作为接口芯片，将这些外设构成一个简单的微机应用系统，画出接口连接图。

(2) 编制 3 种驱动程序，每个程序必须包括至少有两种外设共同作用的操作。给出程序清单。

【解】

(1) 用 8255A 的端口 B 作为 8 位开关输入；端口 A 作为 8 位 LED 指示灯的输出；用 PC_0 连至按钮开关；PC_7 输出接蜂鸣器。接线如图 10-3 所示。

(2) 编制由开关输入经 LED 输出显示；按钮输入的 L_0 显示和按钮输入，下面蜂

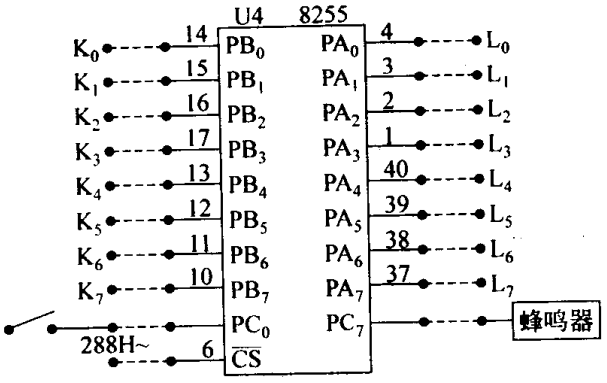


图 10-3　题 10.15 接口电路连线图

鸣器响 1 秒的 3 种程序。

①

```
CODE    SEGMENT
        ASSUME   CS:CODE
START   MOV      DX,28BH        ;8255A 控制端口地址
        MOV      AL,10000011B   ;端口 A 方式 0 输出,端口 B 方式 0 输入
        OUT      DX,AL
INOUT:  MOV      DX,289H        ;端口 B 地址
        IN       AL,DX          ;从端口 B 输入
        MOV      DX,288H        ;端口 A 地址
        OUT      DX,AL
        MOV      DL,0FFH        ;判断是否有按键
        MOV      AH,06H
        INT      21H
        JZ       INOUT          ;若无,继续
        MOV      AH,4CH         ;否则,返回 DOS
CODE    ENDS
        END      START
```

②

```
CODE    SEGMENT
        ASSUME   CS:CODE
START   MOV      DX,28BH        ;8255A 控制端口地址
        MOV      AL,10000011B   ;端口 A 方式 0 输出,PC0 输入
        OUT      DX,AL
INOUT:  MOV      DX,28AH        ;端口 C 地址
        IN       AL,DX          ;从端口 C 输入
        MOV      DX,288H        ;端口 A 地址
        OUT      DX,AL          ;从端口 A 输出
        MOV      DL,0FFH        ;判断是否有按键
        MOV      AH,06H
        INT      21H
```

```
        JZ      INOUT       ;若无,继续
        MOV     AH,4CH      ;否则,返回 DOS
CODE    ENDS
        END     START
```

③

```
CODE    SEGMENT
        ASSUME  CS:CODE
START   MOV     DX,28BH     ;8255A 控制端口地址
        MOV     AL,10000011B ;端口 PC7 输出,PC0 输入
        OUT     DX,AL
INOUT:  MOV     DX,28AH     ;端口 C 地址
        IN      AL,DX       ;从端口 C 输入
        OUT     DX,AL       ;从端口 C 输出
        MOV     BL,50       ;延时 1 秒
DEL20:  MOV     CX,762
LOP1:   LOOP    LOP1
        DEC     BL
        JNZ     DEL20
        MOV     DL,0FFH     ;判断是否有按键
        MOV     AH,06H
        INT     21H
        JZ      INOUT       ;若无,继续
        MOV     AH,4CH      ;否则,返回 DOS
CODE    ENDS
        END     START
```

第 11 章　串行通信及接口电路

11.1　为什么串行接口部件中的 4 个寄存器可以只用 1 位地址来进行区分？

【解】　串行接口部件中有两类端口，即控制(控制字和状态字)端口和数据端口，故要由地址来区分。而控制端口的控制字和状态字，数据端口的发送和接收，都可由输入和输出来区分。

11.2　在数据通信系统中，什么情况下可以采用全双工方式，什么情况下可以采用半双工方式？

【解】　若通信双方的接收和发送不需要同时进行，就可以用半双工方式。否则，就要用全双工方式。

11.3　什么叫同步通信方式？什么叫异步通信方式？它们各有什么优缺点？

【解】　同步通信是通信双方用相同的频率来发送和接收信息，且在数据块传送前用同步字符标志数据块。而异步通信所传送的每个字符都需要用起始位标志信息的开始，用停止位标识字符的结束。所以，在数据块传送时，适宜用同步传送。

11.4　什么叫波特率因子？什么叫波特率？设波特率因子为 64，波特率为 1200，那么时钟频率是多少？

【解】　波特率因子是异步传送中采样信息的时钟频率与信号的波特率之比。

$$\text{时钟频率} = \text{波特率因子} \times \text{波特率} = 64 \times 1200 = 76800(\text{Hz})$$

11.5　标准波特率系列指的是什么？

【解】　标准波特率是指在异步通信中传统上常用的波特率，例如：50、110、300、600、1200、2400、4800、9600 等。

11.6　设异步传输时，每个字符对应 1 位起始位、7 位信息位、1 位奇/偶校验位和 1 位停止位，如果波特率为 9600，则每秒钟能传输的最大字符数是多少个？

【解】　波特率是 9600，则每位的时间为 1/9600 = 10.1(ms)，一个字符是 10 位，即传送一个字符约为 100ms。故每秒钟能传送 10 个字符。

11.7　在 RS-232-C 标准中，信号电平与 TTL 电平不兼容，问：RS-232-C 标准的“1”和“0”分别对应什么电平？RS-232-C 的电平和 TTL 电平之间通常用什么器件进行转换？

【解】　RS-232-C 标准的“1”，为 −12V；而 RS-232-C 的“0”，为 +12V。它们通过芯片 1488 和芯片 1489 与 TTL 电平相连。

11.8　在 8251A 的编程结构中，8251A 有几个寄存器和外部电路有关？一共需要几个端口地址？为什么？

【解】　在 8251A 编程结构中，寄存器有控制字寄存器、命令字寄存器、状态字寄存器、发送器和接收器。控制端口(控制字、命令字和状态字)和数据端口(发送器和接收器)是不同的端口地址。在控制端口上，方式控制字和状态字是同一端口，由

IN 和 OUT 区分。控制字和命令字是同一端口，又都是由 CPU 写入，它们由次序区分，在写入方式控制字后的写至控制端口的是命令字。而发送器与接收器由数据端口的读和写区分。

11.9 8251A 内部有哪些功能模块？其中读/写控制逻辑电路的主要功能是什么？

【解】 整个 8251 可以分成 5 个主要部分：接收器、发送器，调制控制，读写控制以及 I/O 缓冲器。而 I/O 缓冲器由状态缓冲器、发送数据/命令缓冲器和接收数据缓冲器 3 个部分组成。8251 的内部由内部数据总线实现相互之间的通信。

读/写控制逻辑对 CPU 输出的控制信号进行译码以实现 CPU 从 8251A 读数据和状态；CPU 写数据和命令到 8251A。8251A 是以$\overline{\text{READ}}$或$\overline{\text{WRITE}}$信号中的一个为“0”来实现 I/O 操作的。若两者中无一为“0”，则 8251A 不执行 I/O 操作；若两者全为“0”，这是一种无确定结果的非法状态。

11.10 什么叫异步工作方式？画出异步工作方式时 8251A 的 TxD 和 RxD 线上的数据格式。

【解】 异步工作方式用一个起始位表示字符的开始，用停止位表示字符的结束构成一帧。如图 11-1 所示。

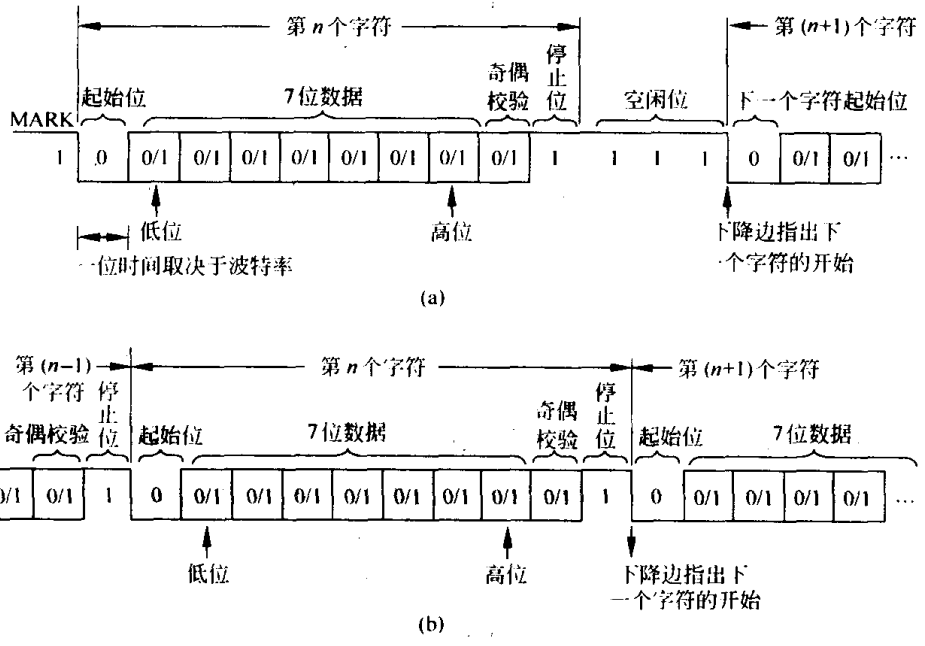


图 11-1 异步通信的格式

超始位占用一位，字符编码可为 5～8 位，下一位为奇偶校验位，加上这一位使字符中为“1”的位为奇数(或偶数)，停止位可以是一位，一位半或两位。

用这样的方式表示字符，则字符可以一个挨着一个传送。

在 RxD 线上接收异步数据时要用时钟信号采样，如图 11-2 所示。

11.11 什么叫同步工作方式？什么叫双同步字符方式？外同步和内同步有什么区别？画出双同步工作时 8251A 的 TxD 线和 RxD 线上的数据格式。

【解】 同步方式时发送与接收方用同步时钟发送和接受信息。为表示数据块的开始，用一个或两个(称为双同步字符)同步字符作为标志。当接收方接收到规定

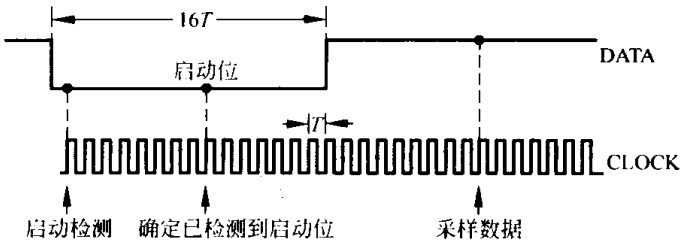


图 11-2　异步工作时的采样

的同步字符后，就用同步时钟接收数据。外同步方式不是以同步字符作为数据块的开始，在外同步方式，在外同步信号有效后就开始接收数据。双同步的数据格式如图 11-3 所示。

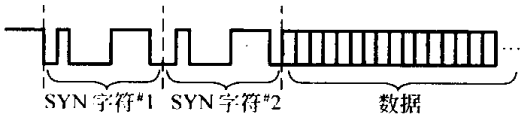


图 11-3　双同步数据格式

11.12　8251A 和 CPU 之间有哪些连接信号？其中，C/$\overline{D}$ 和$\overline{RD}$、$\overline{WR}$如何结合起来完成对命令、数据的写入以及状态、数据的读出？

【解】　在 8251A 与 CPU 中信息的传送如表 11-1 所示。

表 11-1　8251A 与 CPU 之间传送的信息

CE	C/$\overline{D}$	$\overline{READ}$	$\overline{WRITE}$	功　能
0	0	0	1	CPU 从 USART 读数据
0	1	0	1	CPU 从 USART 读状态
0	0	1	0	CPU 写数据到 USART
0	1	1	0	CPU 写命令到 USART
1	×	×	×	USART 总线浮空(无操作)

11.13　在 8086/8088 系统中，8251A 的 C/$\overline{D}$ 端应当和哪个信号相连，以便实现状态端口、数据端口、控制端口的读和写操作？

【解】　8251A 的 C/$\overline{D}$ 端，通常接 8086/8088 的地址信号 A_0，以区分控制端口($A_0=1$)和数据端口($A_0=0$)。

11.14　8251A 和外设之间有哪些连接信号？

【解】　8251A 与外设之间的接口信号有：

① $\overline{DTR}$(Data Terminal Ready)——数据终端准备好。这是一个通用的输出信号，低电平有效。它能由命令指令的位 1 置"1"变为有效，用以表示 CPU 准备就绪。

② $\overline{DSR}$(Data Set Ready)——数据装置准备好。这是一个通用的输入信号，低电平有效。用以表示调制器或外设已准备好。CPU 可通过读入状态操作，在状态寄存器的位 7 检测这个信号。$\overline{DTR}$与$\overline{DSR}$是一组信号，通常用于接收器。

③ $\overline{\text{RTS}}$(Reguest To Send)——读求传送，这是一个输出信号等效于$\overline{\text{DTR}}$。这个信号用于通知调制器 CPU 准备好发送。可由命令指令的位 5 置 1 来使其有效(低电平有效)。

④ $\overline{\text{CTS}}$(Clear To Send)——准许传送，这是调制器对 USART 的$\overline{\text{RTS}}$信号的响应，当其有效时(低电平)USART 发送数据。

⑤ $\overline{\text{RxC}}$(Receiver Clock)——接收器时钟。这个时钟控制 USART 接收字符的速度。

在同步方式，$\overline{\text{RxC}}$等于波特率，由调制解调器供给。

在异步方式，$\overline{\text{RxC}}$是波特率的 1、16 或 64 倍，由方式控制指令预先选择。USART 在$\overline{\text{RxC}}$的上升沿采样数据。

⑥ $\overline{\text{RxD}}$(Receiver Data)——接收器数据，字符在这条线上串行地被接收，在 USART 中转换为并行的字符。高电平表示 Mark 即“1”。

⑦ $\overline{\text{TxC}}$(Transmitter Clock)——发送器时钟，这个时钟控制 USART 发送字符的速度。时钟速度与波特率之间的关系同$\overline{\text{RxC}}$。数据在$\overline{\text{TxC}}$的下降沿由 USART 移位输出。

⑧ $\overline{\text{TxD}}$(Transmitter Data)——发送器数据。由 CPU 送来的并行的字符在这条线上被串行地发送。高电平代表 Mark 即“1”。

11.15 为什么 8251A 要提供$\overline{\text{DTR}}$、$\overline{\text{DSR}}$、$\overline{\text{RTS}}$和$\overline{\text{CTS}}$4 个信号作为和外设的联络信号？平常使用时是否可以只用其中两个或者全部不用？要注意什么？说明$\overline{\text{CTS}}$端的连接方法。

【解】 为保证 8251A 与外设之间信息的可靠传送，所以提供 4 个联络信号。其中，$\overline{\text{DTR}}$和$\overline{\text{DSR}}$用于接收器的联络信号；$\overline{\text{RTS}}$和$\overline{\text{CTS}}$是用于发送器的联络信号。为了可靠工作，这些信号都要用。$\overline{\text{CTS}}$是外设做好了接收 8251A 输出的信号的准备，允许 8251A 发送的联络信号，由外设输送给 8251A，低电平有效。

11.16 8086 在系统中采取什么措施来实现 8 位接口芯片和低 8 位数据线的连接且满足对奇/偶端口地址的读/写？这样做的道理是什么？

【解】 8251A 是 8 位接口芯片，它与 CPU 之间传送 8 位数据。所以，8251A 的 8 位数据总线连至 8086 的低 8 位数据线。8251 与 CPU 之间的控制信息(方式字、命令字和状态字)和数据(发送和接收的数据)都是通过这 8 位数据总线传送的。控制信息与数据是通过端口地址区分的。信息是从 CPU 传送至 8251 还是从 8251 传送至 CPU，是通过读/写信号区分的。

11.17 对 8251A 进行编程时，必须遵守哪些约定？

【解】 由于 8251A 只有一个控制端口地址，方式字、同步方式的同步字和命令字都是写向同一个端口地址。它们就用写的次序来区分。先写方式字；若是同步方式，接着写一个或两个同步字符；然后(或在异步方式，在方式字之后)是命令字。

11.18 8251A 的模式字格式如何？参照教材上给定格式编写如下模式字：异步方式，1 位停止位，偶校验，7 位数据位，波特率因子为 16。

【解】 8251A 的模式字格式如图 11-4 所示。

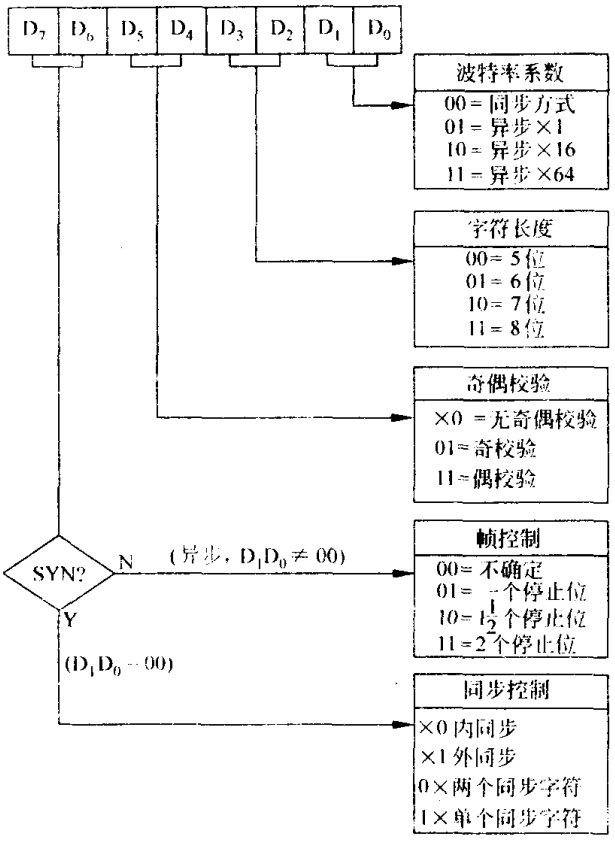


图 11-4　8251A 模式字

满足题中要求的模式字为:01111010B。

11.19　8251A 控制字的格式如何？参照教材上列出的格式给出如下控制字:发送允许,接收允许,$\overline{DTR}$端输出低电平,TxD 端发送空白字符,$\overline{RTS}$端输出低电平,内部不复位,出错标志复位。

【解】　8251A 的命令字如图 11-5 所示。

满足题中要求的命令字为:00110111B。

11.20　8251A 的状态字格式如何？哪几位和引脚信号有关？状态位 TxRDY 和引脚信号 TxRDY 有什么区别？它们在系统设计中有什么用处？

【解】　8251A 的状态字格式如图 11-6 所示。

图 11-6 中的 D_7、D_6、D_2 和 D_1 与管脚同义。状态寄存器的 TxRDY,只要数据缓冲器一空就置位;而引脚 TxRDY 只能当条件:

$$\text{数据缓冲器空} \cdot \overline{CTS} \cdot \overline{TxEN}$$

成立时,才置位。状态字的 TxRDY 用以反映数据缓冲器是否空而引脚 TxRDY 有效就能用于通知 CPU,8251A 已准备好接收一个数据。当 CPU 与 8251 之间用查询方式(Polling)交换信息时,此信号可作为一个"状态"信号(hand shake);在用中断方式交换信息时,此信号可作为 8251 的一个中断请求信号。

11.21　参考初始化流程,用程序段对 8251A 进行同步模式设置。奇地址端口地址为 66H,规定用内同步方式,同步字符为 2 个,用奇校验,7 位数据位。

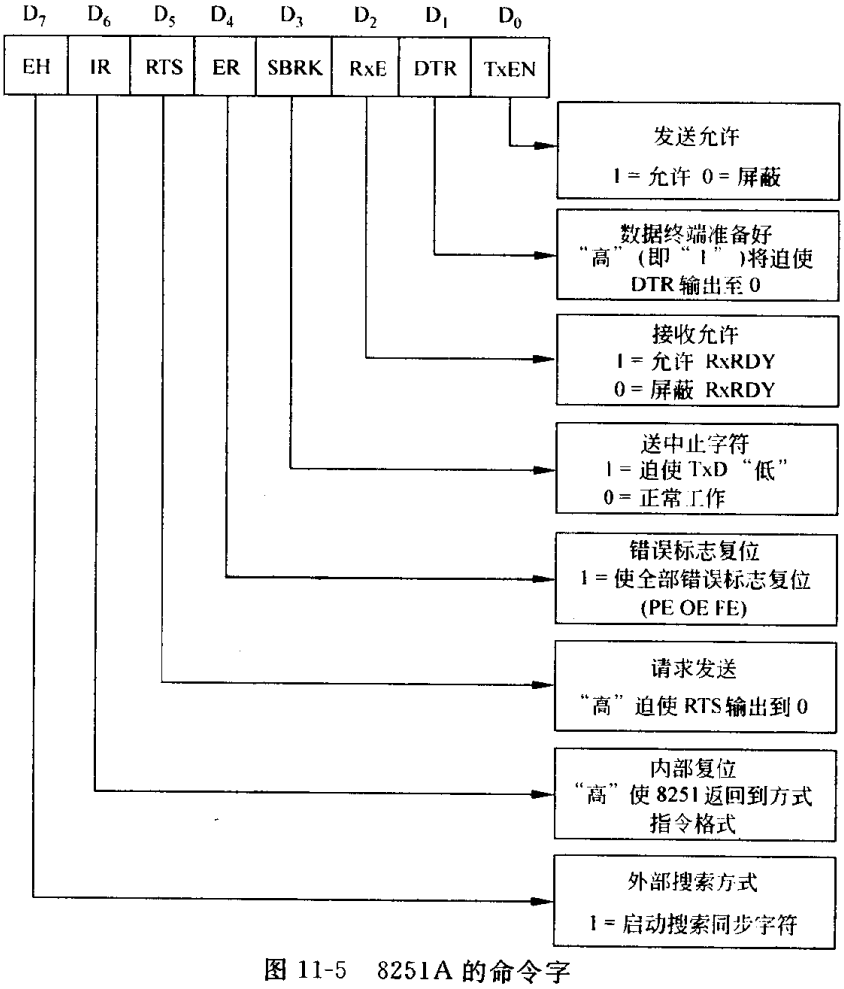


图 11-5　8251A 的命令字

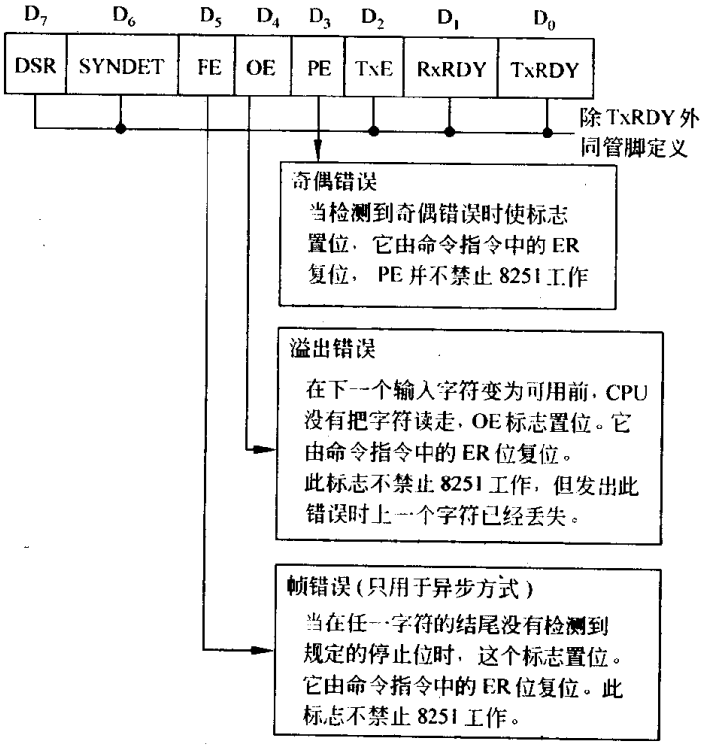


图 11-6　8251A 的状态字

【解】 按题要求的方式字为:00011000B

相应的初始化程序段为:

```
MOV     AL,18H
OUT     67H,AL
MOV     AL,SYNC1
OUT     67H,AL
MOV     AL,SYNC2
OUT     67H,AL
```

11.22 设计一个采用异步通信方式输出字符的程序段,规定波特率因子为 64,7 位数据位,1 位停止位,用偶校验,端口地址为 40H、42H,缓冲区首址为 2000H:3000H。

【解】

```
        ORG     2000H
DATA1   SEGMENT
        ORG     3000H
BUFFER  DB      D1,D2,…,Dn
DATA1   ENDS
STACK   SEGMENT PARA STACK 'STACK'
        DB      100 DUP(?)
STACK   ENDS
CODE    SEGMENT
        ASSUME  CS:CODE,DS:DATA1,SS:STACK
START   PROC    FAR
BEGIN:  PUSH    DS
        MOV     AX,0
        PUSH    AX
        MOV     AX,DATA
        MOV     DS,AX
        MOV     ES,AX
        MOV     DX,41H          ;控制口地址
        MOV     AL,01111011B    ;方式字(异步、1 位停止位、字符长度为 7 位,偶
                                 校验,波特率因子为 64)
        OUT     DX,AL
        MOV     AL,37H          ;命令字(RTS、RR、RxE、DTR 和 TxEN 均置 1)
        OUT     DX,AL
        MOV     CX,2DH          ;传送字节数
        MOV     SI,3000H        ;发送区首地址
L1:     MOV     DX,309H         ;状态口地址
        IN      AL,DX           ;输入状态
        TEST    AL,38H          ;检查 3 个出错标志位
        JNZ     ERR             ;有错,转出错处理
        AND     AL,01H
        JZ      L1              ;发送未准备好,则等待
```

```
        MOV     DX,40H          ;数据口地址
        MOV     AL,[SI]         ;取发送数据
        OUT     DX,AL           ;输出
        INC     SI
        DEC     CX
        JNZ     L1              ;未发送完,循环
ERR:    (略)
        MOV     AX,4C00H
        INT     21H             ;发送完返回 DOS
START   ENDP
CODE    ENDS
        END     BEGIN
```

第 12 章　数模(D/A)转换与模数(A/D)转换接口

12.1　D/A 转换器接口的任务是什么？它和微处理器连接时，一般有哪几种接口形式？

【解】　D/A 转换器接口是要把 CPU 数据总线的输出连到 D/A 的数字输入上。但是，由于 CPU 要进行各种信息的加工处理，它的数据总线上的数据是不断改变的，它输给 D/A 的数据只在输出指令的几个微秒中出现在数据总线上。所以，必须有一个锁存器，把 CPU 输给 D/A 转换的数据锁存起来，直至输入新的数据为止。D/A 转换器与 CPU 连接时有单缓冲器和多缓冲器等不同接口形式。

12.2　DAC 分辨率和微型计算机系统数据总线宽度相同或高于系统数据总线宽度时，其连接方式有何不同？

【解】　当 DAC 分辨率和微型计算机系统数据总线宽度相同时，通常采用单缓冲器结构就可以把 CPU 输出的数据可靠地传送给 DAC。但当 CPU 的数据总线宽度小于 DAC 的位数时，数据就要分多次输出。这时，若仍采用单缓冲器结构，由于输出时间的不一致，在 DAC 的输出电平上会产生毛刺。因此，至少要采用双缓冲器结构。

12.3　利用主教材中的图 12-1 电路，编制在输出端得到锯齿波和梯形波的程序。

【解】　锯齿波的产生可用先置 AL 的值为 0，然而增量 AL，当其增量至最大值(255)后再增量就溢出为 0，如此循环就形成了锯齿波。相应的程序段如下：

```
        MOV     AL,0
LOP1:   OUT     P1,AL
        INC     AL
        JMP     LOP1
```

梯形波是先由线性上升，当达到上限时，延时一段时间；然后线性下降，当它达到下限时又延时一段时间后再上升。如此循环就形成了。所以，梯形波要指定上下限及延时时间。相应的程序段为：

```
        MOV     AL,L1
        OUT     P1,AL
LOP1:   INC     AL
        OUT     P1,AL
        CMP     AL,L2
        JL      LOP1
        CALL    DELAY
LOP2:   DEC     AL
        OUT     P1,AL
        CMP     AL,L1
        JG      LOP2
```

```
        CALL    DELAY
        JMP     LOP1
```

12.4 用带两级数据缓冲器的 D/A 转换器时，为什么有时要用 3 条输出指令才能完成 16 位或 12 位数据转换？

【解】 第一条输出指令把数据的低 8 位输出至第一级缓冲器。第二条输出指令把数据的高 4 位或高 8 位输出至另一个第一级缓冲器。第三条输出指令把第一级缓冲器中的 12 位或 16 位数据同时输送至第二级缓冲器实现 D/A 转换。

12.5 设计一个电路和相应程序完成一个锯齿波发生器的功能，使锯齿波呈负向增长，并且锯齿波周期可调。

【解】 主教材中的图 12-1 就是能实现锯齿波功能的电路。锯齿波负向生长，只要从最大值减量就行。周期可调就要引入延时电路。相应的程序段如下：

```
        MOV     AL,0FFH
LOP1:   OUT     P1,AL
        DEC     AL
        CALL    DELAY
        JMP     LOP1
```

12.6 A/D 转换器接口电路一般应该完成哪些任务？

【解】 A/D 转换通常是把现场的缓慢变化的模拟量经过采样，转换为电量，再经过 A/D 转换把模拟量变换为数字量送入至计算机处理。由于 A/D 转换总需要一定时间(转换的方法不同，转换的时间也有较大的差别)，所以要转换的模拟量必须通过采样保持电路再送至 A/D 转换器的输入端。

12.7 A/D 转换器与 CPU 之间采用查询方式和采用中断方式下，接口电路有什么不同？

【解】 在查询方式，必须有 A/D 转换器是否完成的状态信息输入至 CPU 的接口电路。程序中读入此状态信息进行查询。而中断方式不需要此状态信息的接口电路。只需要 A/D 转换器在转换完成后有一个输出信号，以此信号作为 CPU 的中断请求信号。

第二部分

实验指导

第 13 章　汇编语言程序练习

13.1　运行汇编语言的源程序的过程

当用户编制好汇编语言的源程序以后，要在机器上运行，必须经过如图 13-1 所示的以下几个步骤：

(1) 调用 MS-DOS 下的屏幕编辑程序 EDIT(或其他的字处理软件)来建立和修改汇编语言源程序。

在 EDIT 下，通常用键盘打入汇编语言的源程序，此时，源程序是以 ASCII 字符的形式存放在内存缓冲区中。若输入过程有错，可以利用 EDIT 的命令来改正。改正后，可以用命令使源程序存盘，于是在盘上就建立了一个汇编语言的源程序文件。

若汇编语言的源程序在汇编过程中有错，或在运行过程中发现有错，则还需要在编辑程序 EDIT 下来加以修改。

这样，在盘上就有了源文件和它的后备文件(修改以前的文件)。

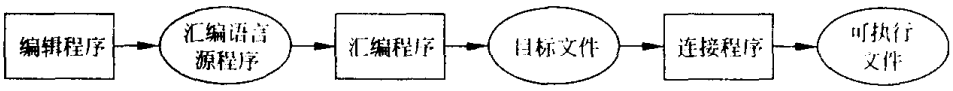


图 13-1　汇编语言程序的建立过程

(2) 源文件必须经过汇编，变为机器码的目标文件，机器才可能运行。

汇编是由调用在 MS-DOS 下的宏汇编(MACRO assembler)MASM 来实现的。

为了适应编制多模块的大的程序和调用 MS-DOS 支持下的公共子程序的需要，汇编以后的目标程序中的地址部分仍不是可执行的绝对地址，而是可浮动的相对地址。

(3) 必须经过连接，把程序的各个模块连接在一起，或把要调用的公共子程序库中的子程序与主程序连接在一起，把相对地址变为绝对地址，形成可执行的文件。

连接是由调用在 MS-DOS 下的 LINK 程序来实现的。

(4) 运行程序。

经过了上述过程，在盘上有了可执行文件，则可在 DOS 的提示符下，直接打入文件名(不要扩展名 EXE 或 COM)，MS-DOS 把可执行文件从盘上调入内存，并且立即执行此程序。

若在程序中，有向 CRT 输出结果的语句，则结果就会显示在 CRT 上(也可以在打印机上打印硬拷贝)。

通常一个较复杂、较长的用汇编语言编写的源程序，希望它一点错误也没有，并且一次通过的可能性是很小的。这样就需要调用 MS-DOS 支持下的排错程序 DEBUG 来调试目标程序。在 DEBUG 的管理下，可以单步执行程序，也可在程序中设置断点；可以显示和修改指定的内存单元和区域的内容；可以显示和修改 CPU 内部寄存器和标志位的内容等。这样就便于寻找程序中的错误。

在发现错误后，通常还需要重复上述的编辑、汇编、连接和运行程序的全过程，直至程序运行正确。

下面对汇编程序和连接程序的使用过程作简要的介绍。

13.1.1 汇编程序

经过编辑命令建立和修改后存盘的是汇编语言的源程序，要在机器上运行，必须先由汇编程序把它汇编为机器码的目标程序。IBM 的汇编程序在汇编后于盘上建立 3 个文件，如图 13-2 所示。

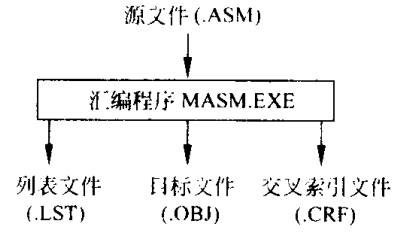


图 13-2 汇编程序的输入输出文件

第一个是扩展名为.OBJ 的目标程序，在此程序中，操作码部分已变为机器码，但地址操作数只是一个可浮动的相对地址，而不是在内存中的绝对地址。第二个是扩展名为.LST 的可打印文件，它把源程序(包括注释)和汇编后的目标程序都打印出来，以供检查用。第三个是扩展名为.CRF 的可以对符号(symbol)进行前后对照的文件。所以，在汇编时，汇编程序对要不要建立这些文件，要建立时的文件名进行提问。如下所示：

```
A>masm
Microsoft ® Macro Assembler Version 5.00
Copyright © Microsoft Corp 1981－1985,1987,All rights reserved.
Source filename [.ASM]:exam
Object filename [EXAM.OBJ]
Source listing [NUL.LST]:exam
Cross reference [NUL.CRF]:exam
                0 Warning Errors
                0 Severe Errors
```

在 DOS 提示符下，输入 masm 来调用汇编程序，在汇编程序调入后，先显示版本号、版权信息，然后出现第一行提示，询问要汇编的源程序文件名。用户输入文件名(汇编语言的源程序的扩展名必须为.ASM，在打下源文件名时可省去扩展名.ASM)，则出现第二个提示，询问目标程序的文件名，括号中的是机器由输入的源文件名导出的默认的文件名。通常直接按〈CR〉键，表示采用默认。接着出现第三个提示，询问是否要建立可打印文件(默认为空)，若需要则输入文件名。最后发出第四个提示，询问是否要建立前后对照文件(默认为空)，若需要则输入文件名。在回答了第四个询问后，汇编程序就对源程序进行汇编。若汇编过程中发现源程序中有语法错误，则列出有错误的语句和错误的代码(在 masm 中，还直接指出是什么性质的错误)，最后列出错误的总数。此时，就要分析错误，继而再调用编辑程序 EDIT 来加以修正，修正后再重新汇编，直至汇编后无错误。

汇编后建立的.LST 文件，可用于打印，以了解汇编后的情况。下面附上一个.LST 文件。

```
The IBM Personal Computer MACRO Assembler     01-01-80
1                                             name    output_character_0_9
```

```
2       0000                            stack   segment para stack 'stack'
3       0000    64[                     stapn   db      100  dup(?)
4               ??
5                   ]
6
7       =0064                           top     equ     length stapn
8       0064                            stack   ends
9       0000                            coseg   segment
10                                      assume cs:coseg,ss:stack
11      0000    B8  -R                  start:  mov ax, stack
12      0003    8E  D0                          mov ss, ax
13      0005    B8  0064                        mov ss, top
14      0008    8B  E0                          mov sp, ax
15      000A    B3  00                          mov bl, 0
16      000C    53                              push bx
17      000D    B2  20                  goon:   mov dl, 20h
18      000F    B4  02                          mov ah, 2
19      0011    CD  21                          int 21
20      0013    5B                              pop bx
21      0014    8A  C3                          mov al, bl
22      0016    FE  C0                          inc al
23      0018    27                              daa
24      0019    24  0F                          and al,`ofh
25      001B    8A  D8                          mov bl, al
26      001D    53                              push bx
27      001E    0C  30                          or al, 30h
28      0020    8A  D0                          mov dl, al
29      0022    B4  02                          mov ah, 2
30      0024    CD  21                          int 21h
31      0026    B9  FFFF                        mov cx, 0ffffh
32      0029    49                  again:  dec cx
33      002A    75  FD                          jne again
34      002C    EB  DF                          jmp goon
35      002E                            coseg   ends
36                                              end
The IBM Personal Computer MACRO Assembler  01-01-80    PAGE   Symbols-1
Segments and groups:
                Name                    Size        align       combine class
COSEG……                                 002E        PARA        NONE
STACK……                                 0064        PARA        STACK   'STACK'
Symbols:
                Name                    Type        Value       Attr
AGAIN……                                 L NEAR      0029        COSEG
GOON ……                                 L NEAR      000D        COSEG
```

```
STAPN……                    L BYTE    0000    STACK Length=0064
START……                    L NEAR    0000    COSEG
TOP  ……                    Number    0064
Warning Severe
Errors Errors
0       0
```

汇编后建立的.CRF 文件，是用来了解源程序中的符号(包括变量)定义时和引用时的情况。

但要打印出变量(符号)表时，要用到 Cref 文件，其过程如下：

```
Cref
Cref filename[.CRF]:examl
List filename[EXAM1.REF]:
A>type examl.ref
```

在 DOS 下输入 Cref，调用 Cref 文件，在 Cref 文件调入后，提示询问文件名，在输入文件名后，出现第二个提示询问 List 文件名，可用〈CR〉键来承认 default 的 List 文件名，于是就建立了一个扩展名为.REF 的文件，Cref 就返回 DOS。

在 DOS 下，用 type 命令可打印此文件，得出类似如下的显示：

```
Symbol Cross Reference                    (# is definition)  Cref-1
AGAIN ……32#    33
COSEG ……9#     10    35
GOON  ……17#    34
STACK ……2#     2     8     10    11
STAPN ……3#     7
START ……11#
TOP   ……7#     13
```

其中，带#号的是此符号定义时的语句号，后面是引用此符号时的语句号。

13.1.2 连接程序 LINK

汇编后建立的目标码文件必须经过连接才能成为一个可执行的文件。即使用高级语言编写的程序，在经过编译后所建立的目标码文件也要经过连接才能成为一个可执行的文件。LINK 的输入文件和输出文件如图 13-3 所示。

启动 LINK 有 3 种方式：提问方式、命令行方式和命令文件方式。若目标码文件、连接程序都在同一目录下，则启动 LINK 的 3 种方式具体介绍如下。

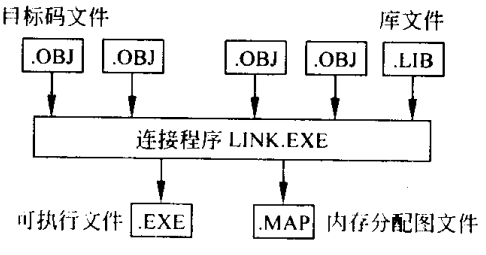


图 13-3 连接程序的输入输出文件

1. 提问方式

在 DOS 提示符下输入命令：

```
>LINK
```

就以提问方式启动了 LINK 程序。在显示版

本信息后，依次提出 4 个问题，以 Prog.OBJ为被连接的目标文件：

```
Object Modules [.OBJ]:Prog
Run file [Prog.BXE]:
List file [NUL.MAP]:Prog
Libraries [.Lib]:
```

第一行是询问被连接的各目标码文件名(隐含的扩展名为.OBJ)。当输入不止一个文件名时，中间以＋(加号)相连，最后按回车键。

第二行是询问连接产生的可执行文件名，方括号内为默认值。

第三行是询问连接产生的内存分配图文件名，NUL.MAP 表示不要内存分配图文件。

第四行询问欲连接的库文件名，也可能是多个，若无库文件连接则按回车键即可。4 个问题回答完，LINK 立即开始工作。

2. 命令行方式

格式如下：

```
>LINK 目标文件名,执行文件名,列表文件名;
```

也可用简短命令：

```
>LINK Prog;
```

使用该命令时，若以分号结尾，则不显示提问，直接执行连接任务。若 3 种文件未全指定，则表示使用默认值。

3. 命令文件方式

将 LINK 程序提问的各文件名按顺序写成一个命令文件形式。设命令文件名为 RT，用 COPY 命令建立：

```
>COPY CON:RT
PROG
PROG
PROG
^Z
```

然后在 LINK 程序中执行这个命令文件，其格式为：

```
>LINK@RT
```

此时屏幕上显示 4 个提问行，由 RT 文件回答，用户无需再按键即可完成连接。这种方式在源程序调试时比较方便，因要反复编辑、汇编、连接，直至没错为止，所以每次就可以连续调用已建立的连接命令文件，无需每次都要回答。

经过连接后的文件就可以直接运行了。而连接以后，若已要求产生内存分配图文件(MAP)，则可用 TYPE 命令将其打印出来，该文件提供了各段的名字、起始地址和长度，以及程序入口地址。

当需要不断地执行汇编、连接进行调试时，可将上述各过程写成一个批文件，使之连续执行，免得多次操作键盘，以提高调试效率。

13.2 DOS 功能调用

13.2.1 什么是 DOS 功能调用

MS-DOS(PC-DOS)的核心部分是由一连串的有关文件管理、磁盘读写、设备驱动等子程序组成。DOS 的各种命令就是由适当调用这些子程序实现的。为了方便程序员使用,这些子程序编成相对独立的程序模块而且编了号,可由程序员用汇编语言实现调用。显然,这些程序是精心编写的,而且经过了大量的各种应用范围的实践考验的。程序员调用这些子程序就可以大大精简应用程序的编写,大大扩展对 DOS 的应用。这些编了号的可由程序员调用的子程序就称为 DOS 的功能调用(function call)或称为系统调用(system call)。一般认为 DOS 的各种命令是操作员与 DOS 的接口,而功能调用则是程序员与 DOS 的接口。

13.2.2 DOS 功能调用清单

MS-DOS 2.0 版以前版本是由 CP/M 操作系统脱胎而来的,保留着许多 CP/M 的痕迹,所以 DOS 功能调用中的低序号的功能调用 00H～24H 是与 CP/M 兼容的。DOS 2.0 版以上又参照了 Xenix 的思想所以功能调用中又有一部分是与 Xenix 兼容的。

以功能调用号为序的 DOS 功能调用清单如下:

功能号	功能名
00H	终结程序
01H	读键盘并回显(echo)
02H	输出字符
03H	辅助输入
04H	辅助输出
05H	打印字符
06H	直接控制台输入输出(I/O)
07H	直接控制台输入
08H	读键盘
09H	显示字符串
0AH	具有缓冲区的键盘输入
0BH	检查键盘状态
0CH	清输入缓冲后输入
0DH	重启磁盘
0EH	选择当前盘驱动器
0FH	打开文件
10H	关闭文件
11H	搜索第一个匹配文件
12H	搜索下一个匹配文件
13H	删除文件
14H	顺序读文件

功能号	功能名
15H	顺序写文件
16H	建立文件
17H	文件改名
18H	保留
19H	获取当前驱动器
1AH	设置磁盘缓冲区地址
1BH	获取当前驱动器的分配信息
1CH	获取指定驱动器的分配信息
1DH	保留
1EH	保留
1FH	保留
20H	保留
21H	随机读文件
22H	随机写文件
23H	确定文件长度
24H	设置随机记录号
25H	设置中断向量
26H	创建程序段前缀
27H	随机块读
28H	随机块写
29H	分析文件名

功能号	功能名	功能号	功能名
2AH	获取日期	47H	返回当前目录
2BH	设置日期	48H	分配内存
2CH	获取时间	49H	释放所分配的内存
2DH	设置时间	4AH	修改所分配的内存
2EH	设置确认标志	4BH	装入和执行程序
2FH	获取磁盘缓冲区地址	4CH	文件终结
30H	获取 DOS 版本号	4DH	获取返回码
31H	结束并驻留	4EH	寻找第一个匹配文件
32H	保留	4FH	寻找下一个匹配文件
33H	获取或设置 Ctrl+Break 标志	50H	保留
34H	保留	51H	保留
35H	获取中断向量	52H	保留
36H	获取磁盘自由空间	53H	保留
37H	保留	54H	获取确认标志
38H	返回与地点有关的信息	55H	保留
39H	建立子目录	56H	重命名文件
3AH	删除子目录	57H	获取/设置文件的日期/时间
3BH	改变当前工作目录	58H	获取或设置分配策略
3CH	建立文件	59H	获取扩展的出错信息
3DH	打开文件	5AH	创建临时文件
3EH	关闭文件	5BH	创建新文件
3FH	从文件/设备读	5CH	控制记录存取
40H	写至文件/设备	5DH	保留
41H	删除文件	5EH	获取机器名
42H	移动文件指针	5FH	重定向功能
43H	获取或设置文件属性	60H	保留
44H	设备驱动控制	61H	保留
45H	复制文件描述字	62H	获取程序段前缀地址
46H	强迫复制文件描述字		

13.2.3 使用功能调用的方法

大部分的 DOS 功能调用，要求在寄存器中传送一些参数。在规定的寄存器中设置了适当的值以后，可以用以下方法之一来实现调用：

- 在 AH 中放功能调用号，然后执行长调用至程序段前缀中的偏移量 50H。注意，在 MS-DOS 版本小于 2.0 时，不能采用这种方法。
- 在 AH 中放功能号，然后引用中断 21H 指令。这是向读者推荐的方法，本章中的所有例子都采用这种方法。

系统调用的宏指令

```
                NAME        B:MACRO.LIB
READ_KBD_AND_ECHO  MACRO
                MOV         AH,1
                INT         21H
```

```
                ENDM
DISPLAY_CHAR    MACAO   CHARACTER
                MOV     DL, CHARACTER
                MOV     AH, 2
                INT     21H
                ENDM
AUX_INPUT       MACRO
                MOV     AH, 3
                INT     21H
                ENDM
AUX_OUTPUT      MACRO
                MOV     AH, 4
                INT     21H
                ENDM
PRINT_CHAR      MACRO   CHARACTER
                MOV     DL, CHARACTER
                MOV     AH, 5
                INT     21H
                ENDM
DIR_CONSOLE_IO  MACRO   SWITCH
                MOV     DL, SWITCH
                MOV     AH, 6
                INT     21H
                ENDM
DIR_CONSOE_IN_PUT MACRO
                MOV     AH, 7
                INT     21H
                ENDM
READ_KBD        MACRO
                MOV     AH, 8
                INT     21H
                ENDM
DISPLAY         MACRO   STRING
                MOV     DX, OFFSET STRING
                MOV     AH, 9
                INT     21H
                ENDM
GET_STRING      MACRO   STRING
                MOV     DX, OFFSET STRING
                MOV     AH, 0AH
                INT     21H
                ENDM
CHECK_KBD_STATUS MACRO
                MOV     AH, 0BH
```

```
                    INT         21H
                    ENDM
FLUSH_AND_READ_KBD  MACRO       SWITCH
                    MOV         AL, SWITCH
                    MOV         AH, 0CH
                    INT         21H
                    ENDM
RESET_DISK          MACRO
                    MOV         AH, 0DH
                    INT         21H
                    ENDM
SELECT_DISK         MACRO       DISK
                    MOV         DL, DISK[-65]
                    MOV         AH, 0EH
                    INT         21H
                    ENDM
OPEN                MACRO       FCB
                    MOV         DX, SEG FCB
                    MOV         DS, DX
                    MOV         DX, OFFSET FCB
                    MOV         AH, 0FH
                    INT         21H
                    ENDM
CLOSE               MACRO       FCB
                    MOV         DX, SEG FCB
                    MOV         DS, DX
                    MOV         DX, OFFSET FCB
                    MOV         AH, 10H
                    INT         21H
                    ENDM
SEARCH_FIRST        MACRO       FCB
                    MOV         DX, SEG FCB
                    MOV         DS, DX
                    MOV         DX, OFFSET FCB
                    MOV         AH, 11H
                    INT         21H
                    ENDM
SEARCH_NEXT         MACRO       FCB
                    MOV         DX, SEG FCB
                    MOV         DS, DX
                    MOV         DX, OFFSET FCB
                    MOV         AH, 12H
                    INT         21H
                    ENDM
```

```
DELETE          MACRO       FCB
                MOV         DX, SEG FCB
                MOV         DS, DX
                MOV         DX, OFFSET FCB
                MOV         AH, 13H
                INT         21H
                ENDM
READ_SEQ        MACRO       FCB
                MOV         DX, SEG FCB
                MOV         DS, DX
                MOV         DX, OFFSET FCB
                MOV         AH,14H
                INT         21H
                ENDM
WRITE_SEQ       MACRO       FCB
                MOV         DX, SEG FCB
                MOV         DS, DX
                MOV         DX, OFFSET FCB
                MOV         AH, 15H
                INT         21H
                ENDM
CREATE          MACRO       FCB
                MOV         DX, SEG FCB
                MOV         DS, DX
                MOV         DX, OFFSET FCB
                MOV         AH,16H
                INT         21H
                ENDM
RENAME          MACRO       FCB NEWNAME
                MOV         DX, SEG FCB
                MOV         DS, DX
                MOV         DX, OFFSET FCB
                MOV         AH, 17H
                INT         21H
                ENDM
CURRENT_DISK    MACRO
                MOV         AH, 19H
                INT         21H
                ENDM
SET_ DTA        MACRO       BUFFER
                MOV         DX, SEG BUFFER
                MOV         DS, DX
                MOV         DX, OFFSET BUFFER
                MOV         AH, 1AH
```

```
                    INT         21H
                    ENDM
ALLOC_TABLE         MACRO
                    MOV         AH, 1BH
                    INT         21H
                    ENDM
GET_SDK_MSG         MACRO       DISK
                    MOV         DL, DISK
                    MOV         AH, 1CH
                    INT         21H
                    ENDM
READ_RAN            MACRO       FCB
                    MOV         DX, SEG FCB
                    MOV         DS, DX
                    MOV         DX, OFFSET FCB
                    MOV         AH, 21H
                    INT         21H
                    ENDM
WRITE_RAN           MACRO       FCB
                    MOV         DX, SEG FCB
                    MOV         DS, DX
                    MOV         DX, OFESET FCB
                    MOV         AH, 22H
                    INT         21H
                    ENDM
FILE_SIZE           MACRO       FCB
                    MOV         DX, SEG FCB
                    MOV         OS, DX
                    MOV         DX, OFFSET FCB
                    MOV         AH, 23H
                    INT         21H
                    ENDM
SET_RELATIVE_RECORD MACRO       FCB
                    MOV         DX, SEG FCB
                    MOV         DS, DX
                    MOV         DX, OFFSET FCB
                    MOV         AH, 24H
                    INT         21H
                    ENDM
SET_VECTOR          MACRO       INTERRUPT, SEG_ ADDR, OFF_ ADDR
                    PUSH        DS
                    MOV         AX, SEG_ADDR
                    MOV         DS, AX
                    MOV         DX, OFF_ ADDR
```

```
                    MOV     AL, INTERRUPT
                    MOV     AH, 25H
                    INT     21H
                    ENDM
CREAT_PROG_SEG      MACRO   SEG_ADDR
                    MOV     DX, SEG_ADDR
                    MOV     AH, 26H
                    INT     21H
                    ENDM
RAN_BLOCK_READ      MACRO   PCB, COUNT, REC_SIZE
                    MOV     DX, OFFSET FCB
                    MOV     CX, COUNT
                    MOV     WORD PTR FCB[14], REC_SIZE
                    MOV     AH, 27H
                    INT     21H
                    ENDM
RAN_BLOCK_WRITE     MACRO   FCB, COUNT, REC_SIZE
                    MOV     DX, OFFSET FCB
                    MOV     CX, COUNT
                    MOV     WORD PTR FCB[14], REC_SIZE
                    MOV     AH, 28H
                    INT     21H
                    ENDM
PARSE               MACRO   FILENAME, FCB
                    MOV     SI, OFFSET FILENAME
                    MOV     DI, OFFSET FCB
                    PUSH    ES
                    PUSH    DS
                    POP     ES
                    MOV     AL,15
                    MOV     AH, 29H
                    INT     21H
                    POP     ES
                    ENDM
GET_DATE            MACRO
                    MOV     AH, 2AH
                    INT     21H
                    ENDM
SET_DATE            MACRO   YEAR, MONTH, DAY
                    MOV     CX, YEAR
                    MOV     BH, MONTH
                    MOV     DL, DAY
                    MOV     AH, 2BH
                    INT     21H
```

```
                ENDM
GET_TIME        MACRO
                MOV         AH, 2CH
                INT         21H
                ENDM
SET_TIME        MACRO       HOUR, MINUTES, SECONDS,
                            HUNDREDTHES
                MOV         CH, HOUR
                MOV         CL, MINUTES
                MOV         DH, SECONDS
                MOV         DL, HUNDREDTHES
                MOV         AH, 2DH
                INT         21H
                ENDM
VERIFY          MACRO       SWITCH
                MOV         AL, SWITCH
                MOV         AH, 2EH
                INT         21H
                ENDM
GET_ DTA        MACRO
                MOV         AH, 2FH
                INT         21H
                ENDM
CET_DOS_VER     MACRO
                MOV         AH, 30H
                INT         21H
                ENDM
KEEP_PROC       MACRO       EXITCODE, PARASIZE
                MOV         AL, EXITCODE
                MOV         DX, PARASIZE
                MOV         AH, 31H
                INT         21H
                ENDM
CHECK_CT_C      MACRO       VAL, FUNC
                MOV         DL, VAL
                MOV         AL, FUNC
                MOV         AH, 33H
                INT         21H
                ENDM
GET_INT_VEC     MACRO       INTN
                MOV         AL, INTN
                MOV         AH, 35H
                INT         21H
                ENDM
```

```
GET_FREE_DK     MACRO     DISK
                MOV       DL, DISK
                MOV       AH, 36H
                INT       21H
                ENDM
GET_AREA_MSG    MACRO     BUFFER, AREA_CODE
                MOV       DX, SEG BUFFER
                MOV       DS, DX
                MOV       DX, OFFSET BUFFER
                MOV       AL, AREA_CODE
                MOV       AH, 38H
                INT       21H
                ENDM
CREAT_CDIR      MACRO     PATH
                MOV       DX, SEG PATH
                MOV       DS, DX
                MOV       DX, OFFSET PATH
                MOV       AH, 39H
                INT       21H
                ENDM
MV_DIR          MACRO     PATH
                MOV       DX, SEG PATH
                MOV       DS, DX
                MOV       DX, OFFSET PATH
                MOV       AH, 3AH
                INT       21H
                ENDM
CH_DIR          MACRO     PATH
                MOV       DX, SEG PATH
                MOV       DS, DX
                MOV       DX, OFFSET PATH
                MOV       AH, 3BH
                INT       21H
                ENDM
CREAT_FILE      MACRO     PATH, ATTR
                MOV       DX, SEG PATH
                MOV       DS, DX
                MOV       DX, OFFSET PATH
                MOV       CX, ATTR
                MOV       AH, 3CH
                INT       21H
                ENDM
OPEN_FILE       MACRO     PATH, ACCESS
                MOV       DX, SEG PATH
```

```
                MOV     DS, DX
                MOV     DX, OFFSET PATH
                MOV     AL, ACCESS
                MOV     AH, 3DH
                INT     21H
                ENDM
CLOSE_FILE      MACRO   HANDLE
                MOV     BX, HANDLE
                MOV     AH, 3EH
                INT     21H
                ENDM
READ_FILE       MACRO   HANDLE, BUFFER, COUNT
                MOV     DX, SEG BUFFER
                MOV     DS, DX
                MOV     DX, OFFSET BUFFER
                MOV     CX, COUNT
                MOV     BX, HANDLE
                MOV     AH, 3FH
                INT     21H
                ENDM
WRITE_FILE      MACRO   HANDLE, BUFFER, COUNT
                MOV     DX, SEG BUFFER
                MOV     DS, DX
                MOV     DX, OFFSET BUFFER
                MOV     CX, COUNT
                MOV     BX, HANDLE
                MOV     AH, 40H
                INT     21H
                ENDM
MV_FILE         MACRO   PATH
                MOV     DX, SEG PATH
                MOV     DS, DX
                MOV     DX, OFFSET PATH
                MOV     AH, 41H
                INT     21H
                ENDM
MOV_OFFSET      MACRO   HANDLE, METHOD, OFFSETLOW,
                        OFFSETHIGH
                MOV     DX, OFFSETLOW
                MOV     CX, OFFSETHIGH
                MOV     AL, METHOD
                MOV     BX, HANDLE
                MOV     AH, 42H
                INT     21H
```

```
                ENDM
FILE_ATTR       MACRO   PATH, ATTR, FUNC
                MOV     DX, SEG PATH
                MOV     DS, DX
                MOV     BX, OFFSET PATH
                MOV     CX, ATTR
                MOV     AL, FUNC
                MOV     AH, 43H
                INT     21H
                ENDM
IO_CNT          MACRO   HANDLE, BUF, GOUNT, FUNC
                MOV     BX, HANDLE
                MOV     CX, COUNT
                MOV     DX, SEG BUF
                MOV     DS, DX
                MOV     DX, OFFSET BUF
                MOV     AL, FUNC
                MOV     AH, 44H
                INT     21H
                ENDM
CP_HANDLE       MACRO   HANDLE
                MOV     BX, HANDLE
                MOV     AH, 45H
                INT     21H
                ENDM
FCP_HANDLE      MACRO   HANDLE1, HANDLE2
                MOV     BX, HANDLE1
                MOV     CX, HANDLE2
                MOV     AH, 46H
                INT     21H
                ENDM
GET_C_DIR       MACRO   BUF, DISK
                MOV     SI, SEG BUF
                MOV     DS, SI
                MOV     SI, OFFSET BUF
                MOV     DL, DISK
                MOV     AH, 47H
                INT     21H
                ENDM
ALLOC_M         MACRO   SIZE
                MOV     BX, SIZE
                MOV     AH, 48H
                INT     21H
                ENDM
```

```
FREE_M          MACRO       BLOCK
                MOV         ES, BLOCK
                MOV         AH, 49H
                INT         21H
                ENDM
REALLOC_M       MACRO       BLOCK, SIZE
                MOV         ES, BLOCK
                MOV         BX, SIZE
                MOV         AH, 4AH
                INT         21H
                ENDM
LOAD_EXEC       MACRO       PATH, BLK, FUNC
                MOV         DX, SEG PATH
                MOV         DS, DX
                MOV         DX, OFFSET PATH
                MOV         BX, SEG BLK
                MOV         ES, BX
                MOV         BX, OFFSET BLK
                MOV         AL, FUNC
                MOV         AH, 4BH
                INT         21H
                ENDM
EXIT            MACRO       CODE
                MOV         AL, CODE
                MOV         AH, 4CH
                INT         21H
                ENDM
GET_RETCODE     MACRO
                MOV         AH, 4DH
                INT         21H
                ENDM
S_MFILE         MACRO       PATH, ATTR
                MOV         DX, SEG PATH
                MOV         DS, DX
                MOV         DX, OFFSET PATH
                MOV         CX, ATTR
                MOV         AH, 4EH
                INT         21H
                ENDM
S_NFILE         MACRO
                MOV         AH, 4FH
                INT         21H
                ENDM
```

13.2.4 有关 I/O 的功能调用

功能调用 01H～0CH 是有关系统中 I/O 设备(包括键盘输入、CRT 显示、打印机输出以及辅助输入输出设备)的功能调用。下面对这些功能调用作简单的归纳和比较。

1. 有关输入的功能调用

01H：等待从键盘上输入一个字符，然后把此字符送至 AL 中并反馈至显示器上；检查此字符是否 Ctrl+C，若是，则终结程序。

03H：等待从辅助输入装置输入一个字符，把它送至 AL 中。此调用不返回状态或错误码，但检查并接收由控制台按下的 Ctrl+C，并作相应处理。

06H：此调用的功能，取决于调用时的 DL 值。

DL=FFH ---输入

若已有字符输入，字符返回至 AL 中，且标志 Z=0。

标志 Z=1，表示没有字符输入。

DL≠FFH---输出

DL 中即为要输出的字符。

此调用不检查是否按下了 Ctrl+C。

07H：此调用等待键盘输入一个字符，并把它送至 AL 中，但不把字符反馈至显示器，也不检查是否按下 Ctrl+C。

08H：此调用等待键盘输入一个字符，把它送至 AL 中，检查是否按下了 Ctrl+C，并作相应处理，但并不把输入的字符反馈至显示器。

功能调用 07H、08H，对于要求对输入的字符保密的应用情况，例如输入口令字(Password)的场合是合适的。

0AH：这是输入并缓存字符串的功能调用。此调用等待输入字符，读入并把它们放在缓冲区中，直至遇到字符 CR(回车键)。输入的字符被反馈至显示器，在输入过程中检查是否按下了 Ctrl+C，并作相应处理。

0BH：此调用检查输入缓冲区中是否已有字符输入，若有，AL=FFH；否则，AL=0。若缓冲区中有 Ctrl+C，则发出中断 23H。

0CH：此调用使键盘输入缓冲区变空，然后它的功能取决于 AL 中的值。

若 AL=1、6、7、8、0AH，则完成相应号的功能调用。

若 AL=0，则没有进一步处理。

2. 有关输出的功能调用

02H：此调用显示 DL 中的字符。若按下 Ctrl+C，则发出中断 23H。

04H：此调用把在 DL 中的字符送至辅助输出设备输出；此调用并不返回状态或错误码。

05H：此调用在标准的打印设备上打印 DL 中的字符。若控制台已经按下了 Ctrl+C，则发出中断 23H。

09H：此调用输出在指定缓冲区中的字符串(必须以字符“$”结尾)。

利用上述的有关 I/O 的功能调用就可以实现人—机对话；输入过程所必需的参数；输出机器处理的结果。

下面举一个例子，说明功能调用的使用。

电子秒表——从零开始连续显示时间，当从键盘上按下任一键时停止显示；再按下任一键，又从零连续显示。

```
        NAME        EX3
        IF1
        INCLUDE B:MACRO.LIB
        ENDIF
DATA    SEGMENT
TIME    DB          '00:00,00.00',13,10,'$'
TEN     DB          10
CRLF    DB          13,10,'$'
DATA    ENDS
STACK   SEGMENT     PARA STACK 'STACK'
        DB          100 DUP(?)
STACK   ENDS
CODE    SEGMENT
        ASSUME CS:CODE,SS:STACK,DS:DATA,ES:DATA
START:  PROC
        MOV         AX,DATA
        MOV         DS,AX
        MOV         ES,AX
FUNC_06H: SET_TIME  0,0,0,0
READ_CLOCK: GET_TIME
        CONVERT CH,TEN,TIME
        CONVERT CL,TEN,TIME
        CONVERT DH,TEN,TIME
        CONVERT DL,TEN,TIME
        DISPLAY TIME
        DISPLAY CRLF
        DIR_CONSOLE_IO 0FFH
        JNE STOP
        JMP READ_CLOCK
STOP:   READ_KBD
        JMP FUNC_06H
START   ENDP
CODE    ENDS
        END START
```

在上例中，CONVERT 是一个宏指令，其定义如下：

```
CONVERT     MACRO       VALUE,BASE,DESTINATION
            LOCAL       TABLE,START
            JMP         START
TABLE       DB          '0123456789ABCDEF'
```

```
START:    MOV     AL, VALUE
          XOR     AH, AH
          XOR     BX, BX
          DIV     BASE
          MOV     BL, AL
          MOV     AL, CS:TABLE [BX]
          MOV     DESTINATION, AL
          MOV     BL, AH
          MOV     AL, CS:TABLE [BX]
          MOV     DESTINATION [1], AL
          ENDM
```

13.3 汇编语言程序实验

13.3.1 字符输入输出及字符串显示

若在程序中要进行人机对话、结果显示或出错提示等就要利用上述的功能调用进行

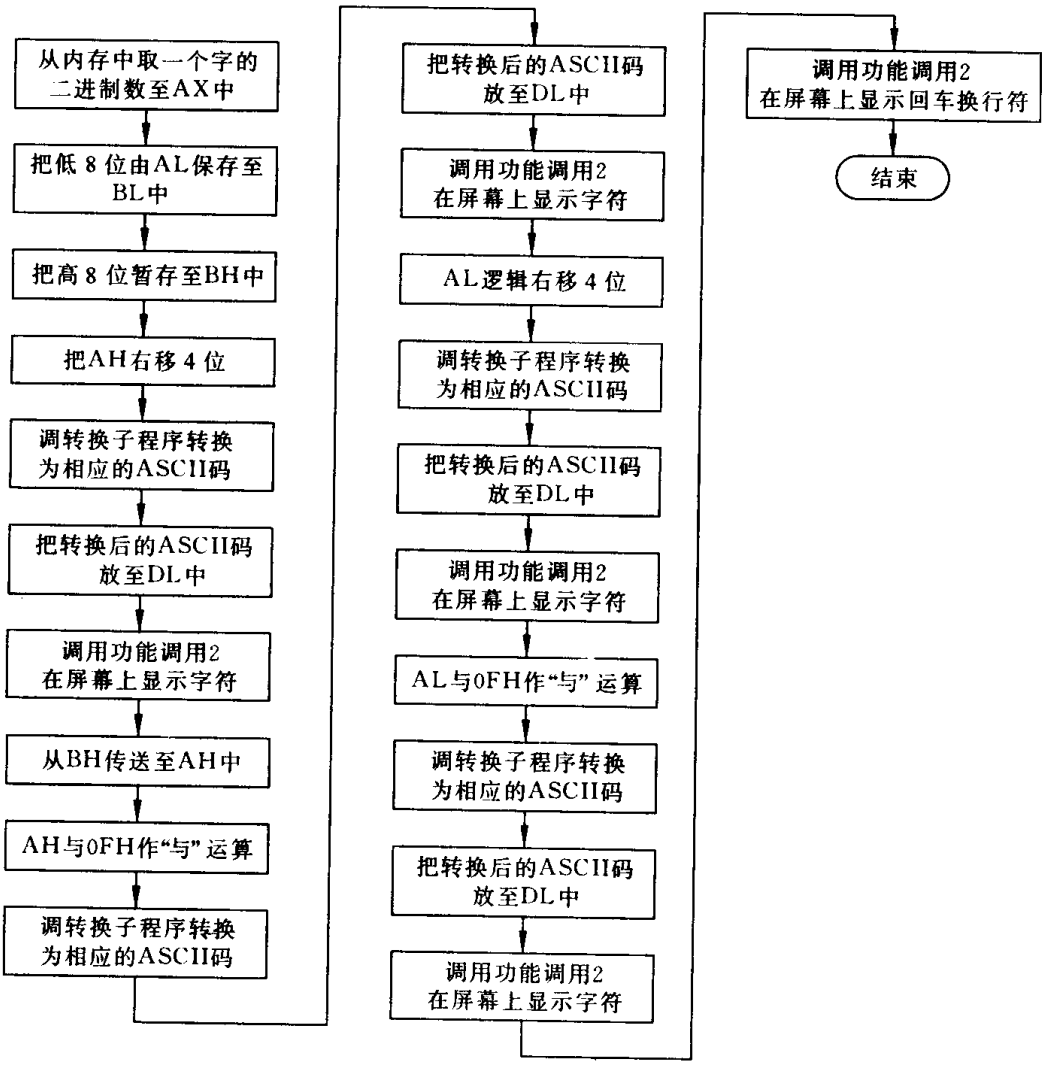


图 13-4　以 ASCII 显示 16 位二进制数

输入、输出。后面的汇编语言的程序练习也可用显示程序检查程序的操作是否正常。

(1) 把在寄存器中(或存储单元中)的一个字的二进制数,转换为 4 位十六进制数的 ASCII 码,在屏幕上显示。高位数在前。

能实现这样转换的流程图如图 13-4 所示。

若要把此程序作为一个子程序,应作哪些改变?

(2) 把在寄存器中(或存储单元中)的一个字的带符号的二进制数以十进制的形式输出。

程序中首先要检查数的符号位,然后决定在输出缓冲区中是置入"+"还是"-"号;若是负数要经过取补,找到数值。要把数值转换为 BCD 码,即要检查此二进制数中包含有多少个 10000(2710H),多少个 1000(03E8H),多少个 100(64H),多少个 10(0AH);再把这些数以及剩下的个位数分别以它们的 ASCII 码置入输出缓冲区,最后在输出缓冲区中置入字符 $ 利用系统调用 9 以输出结果。程序的流程图如图 13-5 所示。

(3) 将键盘输入的小写字母转换为大写字母。利用功能调用 1,从键盘输入一个字符,若是小写字母,把它转换为大写字母放入至行缓冲区中。遇到回车符,显示行缓冲区的内容。若打入字符 Ctrl+C,则停止程序运行,返回 DOS。程序流程图如图 13-6 所示。

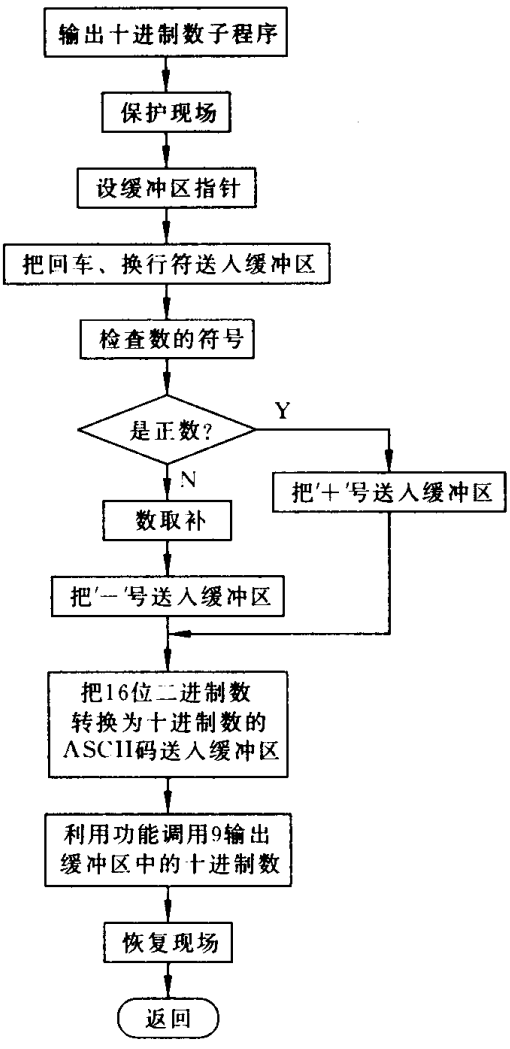


图 13-5 以十进制形式输出带符号的二进制数

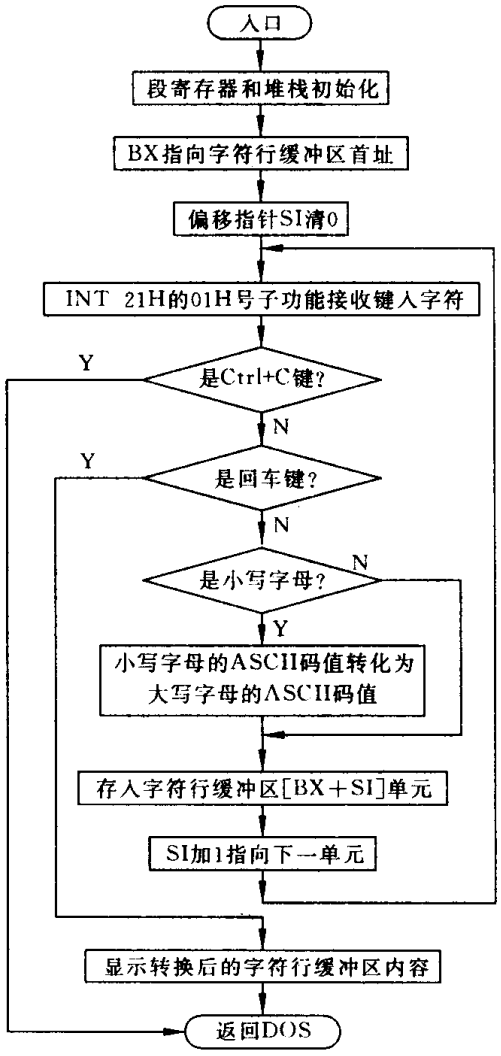


图 13-6 小写字母转换为大写字母的程序流程图

13.3.2 码转换程序

1. ASCII 码到十进制(BCD)的转换

若有一输入的 ASCII 码串(长度在串中的第一字节),要把其中的数码取出来,转换为未组合的 BCD 码,放至另一缓冲区中,并统计数码串的长度,放入此缓冲区的第一个字节。程序流程图如图 13-7 所示。

2. 把 BCD 码转换为二进制

计算机在输入输出时,都采用十进制数表示。ASCII 码与 BCD 之间的转换是方便的,通常在微型计算机中也有一些简单的十进制调整指令,可以进行简单的十进制运算。但在进行较复杂的运算时,就要把 BCD 转换为二进制。

若有一个 16 位存储单元,存放 4 位 BCD 数,要把它转换成二进制,放至另一存储单元。把 BCD 转换为二进制,常用的有两种办法,一种是用下列算式:

((千位数×10+百位数)×10+十位数)×10+个位数

就可以得到转换的结果。另一种是用千位数控制加 1000 的次数,用百位数、十位数控制加 100、加 10 的次数,最后与个位数相加。要求分别用这两种办法编写程序。

(1) 用第一种办法转换的流程如图 13-8 所示。

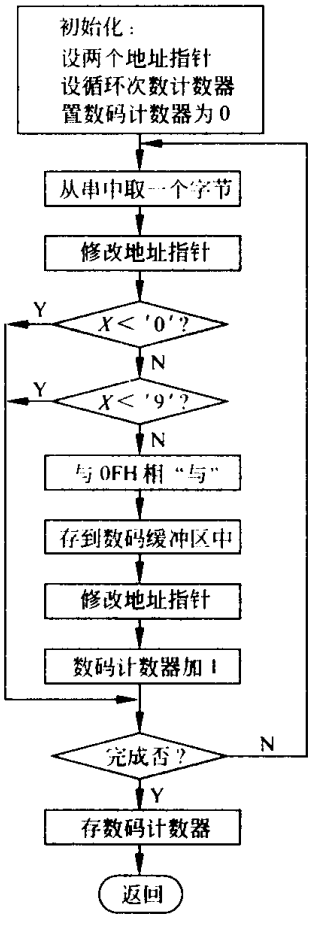


图 13-7 把 ASCII 码转换为 BCD 码的程序流程图

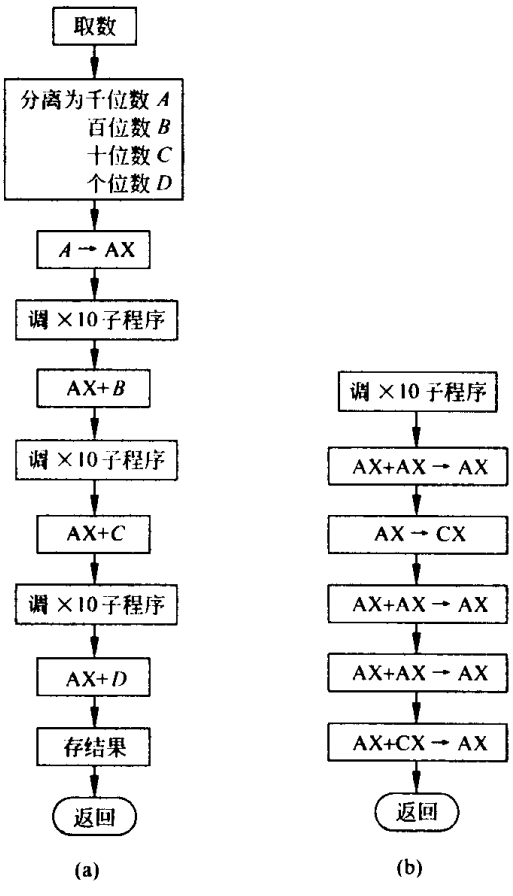


图 13-8 用乘与相加的办法实现 BCD 到二进制的转换

(2) 用第二种办法实现转换，其流程如图 13-9 所示。

3. 二进制到 BCD 的转换

这是上面介绍的 BCD 到二进制转换的逆转换，可以由找到在二进制数中所包含的 10000、1000、100、10 的个数，就可以转换为相应的 BCD 数。

若在某一个存储单元中，有一个 16 位二进制数，要把它转换为相应的十进制数的 ASCII 码存放在缓冲区中。其流程如图 13-10 所示。

4. 从二进制到 ASCII 串的转换

若要把一个二进制位串显示或输出打印，则要把位串中的每一位转换为它的 ASCII 码，例如把在内存变量 NUMBER 中的 16 位二进制数，每一位转换为相应的 ASCII 码，存入串变量 STRING 中，其流程如图 13-11 所示。

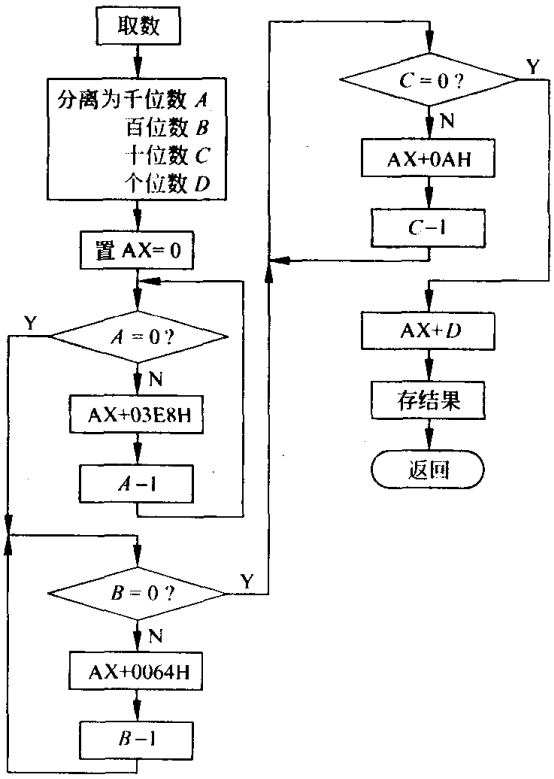


图 13-9 用千加百加十加个位数的办法实现 BCD 至二进制的转换

13.3.3 有关列和表的程序设计

列(List)是最基本的数据存储技术。它是由数据单元(一个或多个字节)组成，称之为元素，在存储器中顺序存放。这个顺序可以是连续的，这样每个元素占据一个或多个字节(如前面介绍的数据块和字符串)；顺序也可以是链接(Linked)的，这样每个数据元素除了数据单元以外，必须有指向下一个元素的指针。

表是一个有特殊目的的(查找信息——数据或地址)的数据结构，电话簿就是一个很好的例子。

有关列和表的主要操作是：查找(搜索、查询)、增加或删除。为了便于查找往往就要对列和表进行分类、排序。

1. 无序列

若列中元素的排列是无一定次序(规则)的。例如，输入的数据是按输入的先后次序排列的，图书馆中买来的书按买来的先后日期排序，而不是按类别、按书名或按作者名等加以分类和排序的。

(1) 加一个项至一个无序列

这只需要检查一下此项在列中是否已有，若无，则把它加在列的最后。所以实质上这个操作是一个查找问题，由于是无序列，查找只能从头开始一个一个地比较。

若在缓冲区中存在一个无序列，此列的长度放在缓冲区的第一个字节，要把在内存某个单元中的一项，加入到此列中。其流程图如图 13-12 所示。

(2) 从一个无序列中删去一个元素

这首先也要进行查找，找到要删去的元素时把它删去，列的长度要减 1，此元素后面

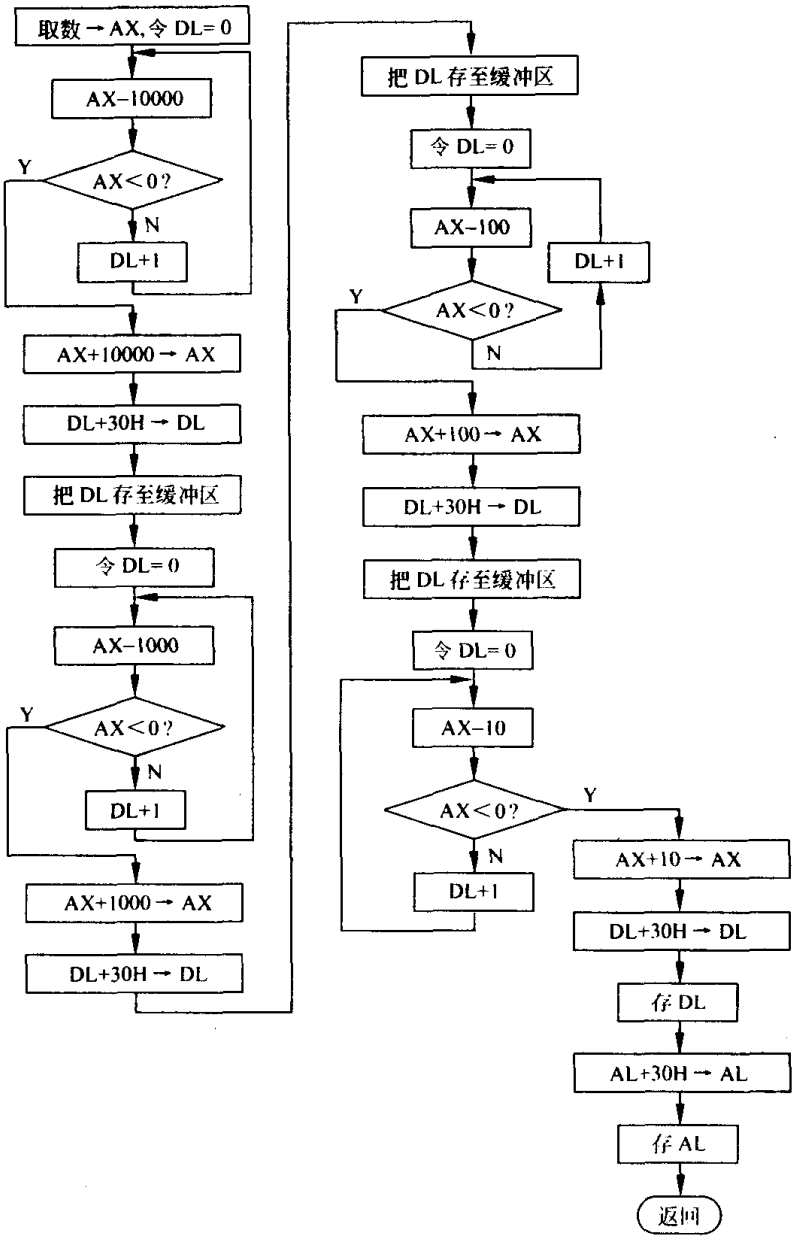


图 13-10　由二进制转换为 BCD 码的程序流程图

的元素都要上移。

若在内存缓冲区中有一个无序列，列的长度存放在第一个字节，另有一单元存放要删去的元素变量，查找此无序列，若包含要删去的元素则把它删去，并对列作相应的调整。其程序流程图如图 13-13 所示。

(3) 找出无序列中的最小值和最大值

若在内存缓冲区中有一个无序列，列的长度存放在第一个字节，要求找出此列中的最小值放在存储单元 MINVAL 中，并且找出列中的最大值放至存储单元 MAXVAL 中。程序的流程图如图 13-14 所示。

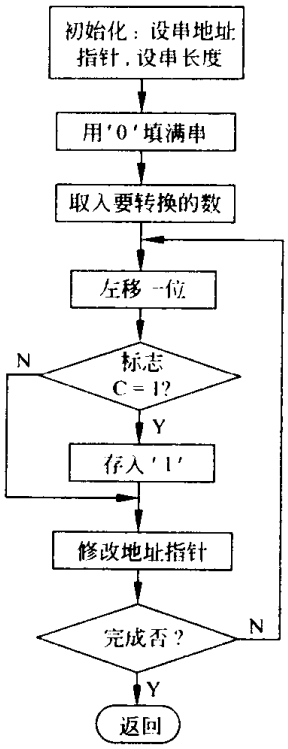


图 13-11　把二进制位串的每一位转换为 ASCII 码的程序流程图

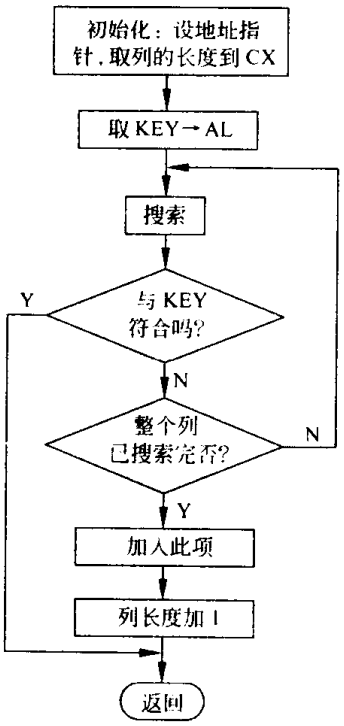


图 13-12　加一项到无序列的程序流程图

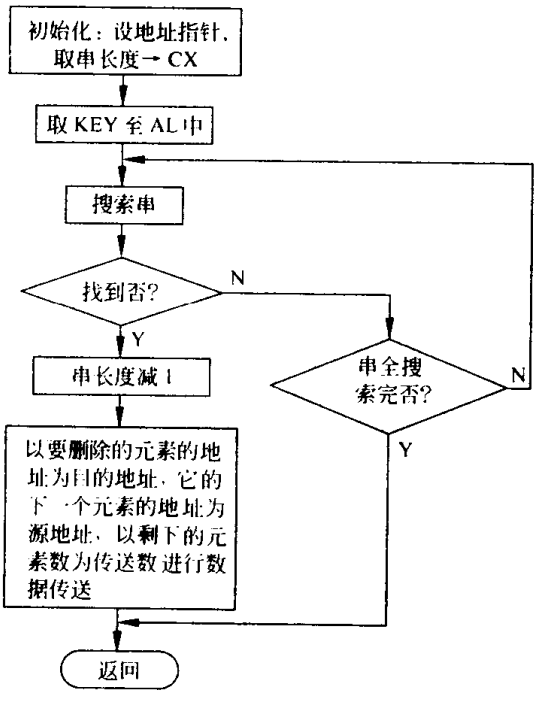


图 13-13　从无序列中删去一个元素的程序流程图

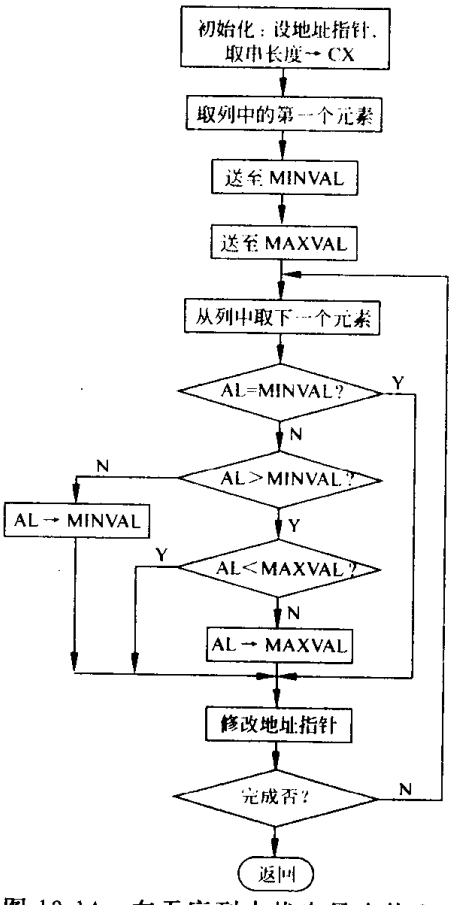


图 13-14　在无序列中找出最小值和最大值的程序流程图

上述的无序列的查找和比较操作，由于列是无序的所以都要从列的头开始，一项一项地顺序查找和比较。若列中的项数为 N，则平均的查找次数为 $N/2$，操作比较慢。

为了提高操作的效率，就要把无序列变为有序列。例如可以按每一项的值（或字符的ASCII 码值）变为一个升序或降序的有序列。这样的操作就是排序，排序的方法很多，在有关数据结构的书中有详尽的讨论，这里只讨论一种简单的排序——气泡分类法（Bubble Sort）。

2. 一种简单的排序方法——气泡分类法

若有一个数组（或列）它有 N 个数，在本例中为如下的 10 个数：22，－12，80，－6，－70，－9，127，－10，00，40。希望把它们按数的大小排列。可以采用两两比较的办法：先拿第 N 个数 e_N 与第 $N-1$ 个数 e_{N-1} 相比较，若 $e_N > e_{N-1}$ 则不交换，反之则交换；然后拿 e_{N-1} 与 e_{N-2} 相比较，按同样原则决定是否交换，这样一直比下去，最后拿 e_2 与 e_1 相比较，也按同样原则决定是否交换。则当第一趟比较结束时，数组中的最小值冒到了顶部。如图 13-15 所示。

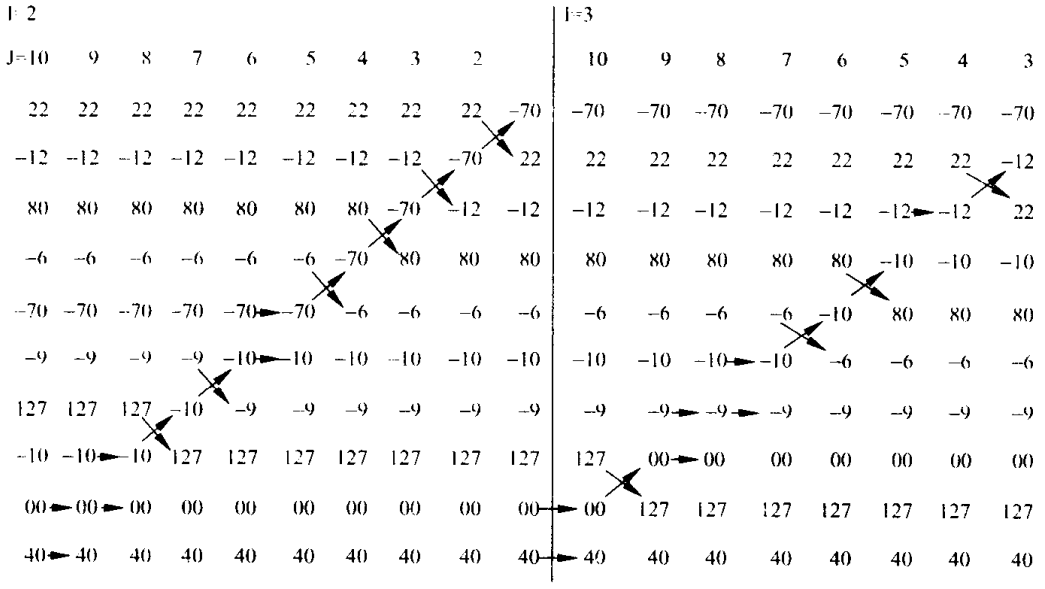


图 13-15　气泡分类法示意图

但数组尚未按大小顺序排列好，就要进行第二趟比较，这样数组中的第二个最小值升到了顶部的相应位置……。这样不断地一趟一趟比较下去，若数组的长度为 N，则最多经过 $N-1$ 趟比较，就可以使数组按大小排列有序。从图 20 中也可以看到，在每一趟比较时，数两两比较的次数，在第一趟时为 $N-1$ 次，在第二趟时，可以是 $N-2$ 次……。$N-1$趟的比较是一种最坏情况，大部分数组不需要经过 $N-1$ 趟的比较就已经排列有序了。为了在程序中除去不必要的比较（趟所造成的循环），就可以设置一个标志，在每趟（大循环）开始时，置此标志为 0；若在整个趟的比较中，未发生数的交换，则此标志仍为 0；若发生过交换，则置此标志为－1。然后在下一趟开始前，检查此标志，若不为 0，表示数组尚未排列有序。继续进行比较排序；若为 0，则表示数组已按大小排列有序（每次两两比较时，都已是大的数在下，小的数在上，故不用交换），就停止循环。一种比较简单的气

泡分类(排序)程序的流程如图 13-16 所示。

以上程序还有可能简化,首先上列程序中每一趟都进行 $N-1$ 次两两比较是没有必要的,因为第一趟比较后,数组中的最小值已升至顶部,故在第二趟比较时,已用不着进行 e_2 与 e_1 的比较,故两两比较次数可以减少一次,所以每进行了一趟比较,则下一趟的两两比较次数可以减少一次。这样就需要设置一个大循环变量 I,初值为 2,每经过一趟比较(一次大循环)I 增量,每一趟中的小循环变量初值为 N(数组长度),每两两比较一次 N 减量,当 N 减为小于 I 时就停止小循环;其次,上列程序中,若有的数组排序到某一趟,若此趟中只有 e_N 与 e_{N-1} 相比较时发生过交换,这表示数组中前 $N-2$ 个元素已经有序,在 e_{N-1} 与 e_N 比较且交换后,整个数组已经有序了。但在这种情况下,上列程序仍需进行一趟比较才能结束,这最后一趟比较是没有必要的,为了去掉这一趟不必要的比较,可以把交换标志的初值设置为 N(数组长度,也是每一趟中小循环变量的初值),而未发生交换标志不变,若发生交换,则把发生交换时的小循环变量值赋给标志。在一趟比较结束时,检查标志,若其值为 N(一趟未发生交换,或在此趟中只有 e_N 与 e_{N-1} 相比较时发生过一次交换)则停止排序。

经过改进的气泡分类程序的流程图如图 13-17 所示。

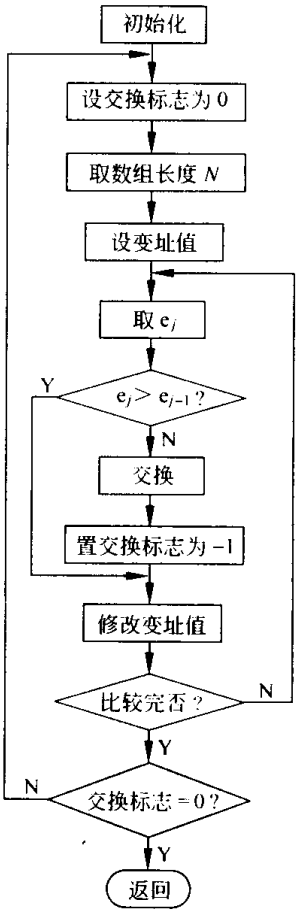


图 13-16 气泡分类程序流程图

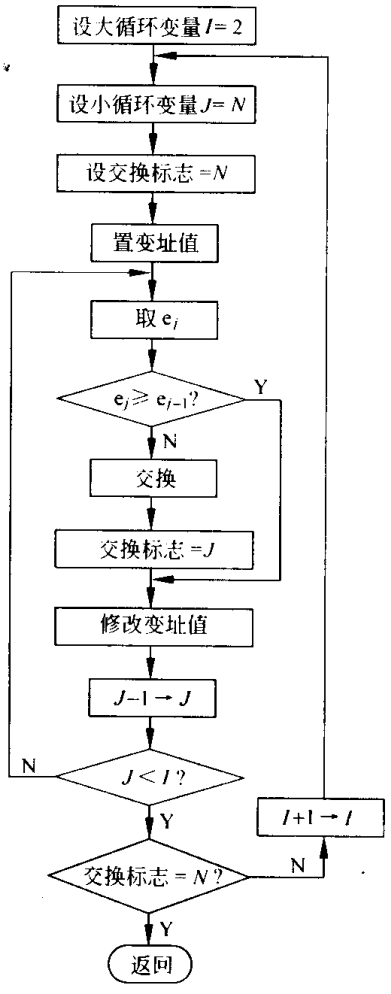


图 13-17 改进的气泡分类程序

3. 有序列操作

(1) 若要在有序列中，搜索是否有关键字 KEY，首先可以与队列中的第一个元素相比较，若 KEY 小(若有序列是按升序排列)，则此列中不会有此关键字；其次可与队列中的最后一个元素相比较，若 KEY 大，则此列中也不会有此关键字。然后就可以通过逐个比较进行查找，其流程图如图 13-18 所示。

(2) 对分搜索

在上例中，查找时仍是从头一一查找，效率仍然是很低的，在有序列中利用有序的特点，可以采用对分搜索法来提高搜索效率。

对分搜索的思路是：先取有序列中间的值 $e_{N/2}$($N/2$ 处的值)与要搜索的关键字 X 相比较，看是否相等，若相等则搜索到；若不相等则比较两个数的大小；若 $X>e_{N/2}$ 则有序列中如包含关键字，也必然在它的下半区间(值大于 $e_{N/2}$ 的区间)，故下一次取 $N/2\sim N$ 之间的中间值 $e_{3N/4}$ 与 X 相比较；若 $X<e_{N/2}$ 则有序列中如包含此关键字，也必然在它的上半区间，故下一次取 $0\sim N/2$ 之间的中间值 $e_{N/4}$ 与 X 相比较。这样每搜索一次就使区间缩小一半，如此一直进行下去，直至，或者是搜索到关键字，或者是使搜索的区间压为 0，则表示搜索不到关键字。其过程如下例所示：

若有一数组(有序列)：00，11，15，21，34，57，60，78，90，127。数组的个数 $N=10$，而数组元素的序号为 $0\sim N-1=9$。若要搜索的数为 $X=78$。

取区间上限 $L=0$，区间下限 $R=10$(即为 N，这是为了能搜索到数组中的最大值所必需的)。第一次搜索时，取 $J=N/2=(J+R)/2=(0+10)/2=5$，则 $e_{N/2}=e_J=57$，与 $X=78$ 相比，两者不等；而且 $X>e_J$，于是下一次要到下半区间去搜索，则取 $L=J=5$，$R=10$ 不变就是新的要搜索的下半区间，可求出新的序号 $J=(5+10)/5=7.5$，往下取整，取 $J=7$，则 $e_J=78$ 与 X 相等，找到。上述的搜索过程可用表 13-1 说明。

表 13-1　搜索给定数组中的数 78 的过程

搜索次数 K	1	2
区间上限 L	0	5
区间下限 R	10	10
序号 J	5	7
数 e_J	57	78
比较	$X>e_J$	$X=e_J$

若要搜索的数为 $X=20$，可用与上述类似的过程搜索，如表 13-2 所示。

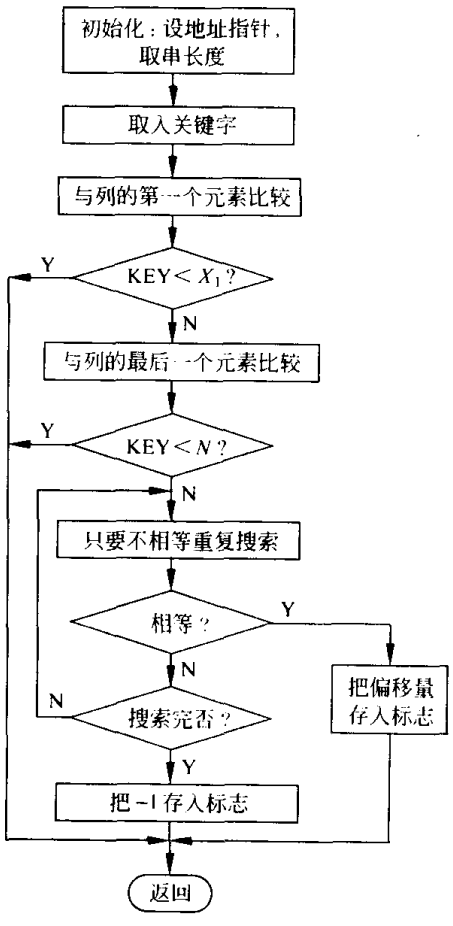


图 13-18　在一个有序列中搜索关键字

表 13-2　搜索给定数组中的数 20 的过程

搜索次数 K	1	2	3	4
区间上限 L	0	0	2	2
区间下限 R	10	5	5	3
序号 J	5	2	3	2
数 e_J	57	15	21	15
比较	$X<e_J$	$X>e_J$	$X<e_J$	$X\neq e_J$

在搜索过程中，若求出的序号 J 已与区间的上边相等，表示搜索的区间已压至最小，若此时的 e_J 仍不等于关键字 X，则表示被搜索的有序列中没有要找的关键字。按上述方法进行搜索的程序流程如图 13-19 所示。

(3) 在一个有序列中插入一项

当然首先要搜索列中是否已有此项，若无，插入需要做以下工作：

① 找到应插在何处；

② 把所有较高值元素的地址移高一个项单元；

③ 插入；

④ 修改列长度。

能实现上述要求的程序流程图如图 13-20 所示。

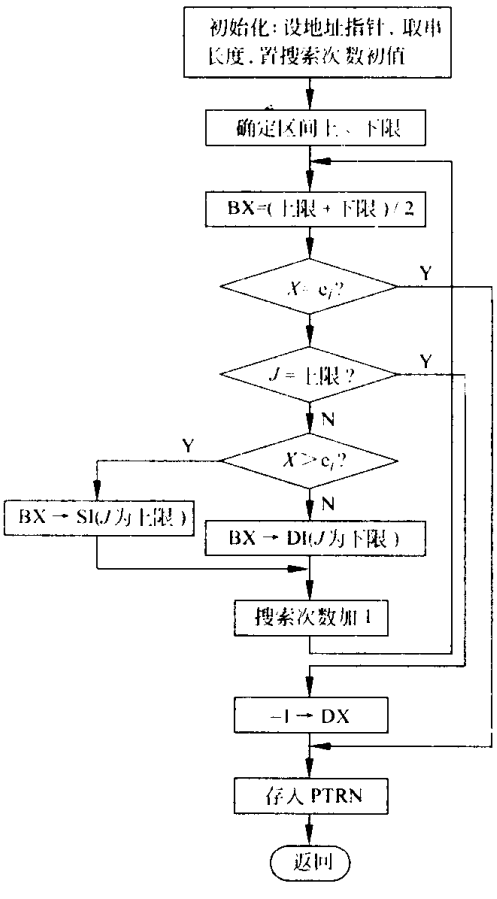


图 13-19　对分搜索程序流程图

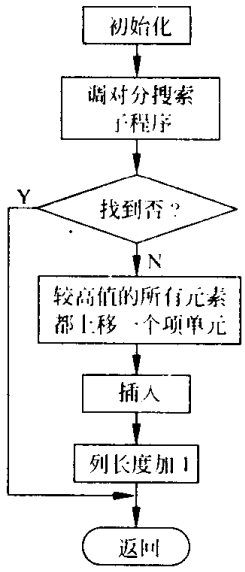


图 13-20　在一个有序列中插入一项的程序流程图

4. 链接队列

如上所述，无序队列的操作速度是很慢的，要提高效率就要使队列变为有序，气泡分类法是使一个无序队列变为有序队列的一种方法。通常队列在内存中存放时是顺序存放的，即列中的元素是一项挨着一项存放的。要在有序列中插入一项，就要使较高值元素上移(在删除时要下移)，也是要费很多时间的。另一种是采用链接(Link)的方法，列中的每一项除了数据元素以外，还包括指向下一个元素的地址指针，所以每一项要占用更多的内存空间。但这样做每一项在内存中就不必按顺序排列，而是任意的，其优点是列中每增加或减少一项，不用重新排列，而只须修改有关的地址指针即可，速度比重新排列要快。

一个链接队列(Linked Lists)的示意如图 13-21 所示。

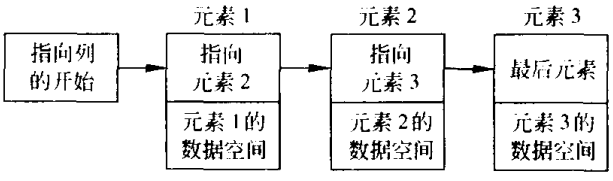


图 13-21　链接队列示意图

由于链接队列是用指针来指向的，为了改变在链中的两个数据元素的次序，不需要移动数据元素本身，而只要改变指针就行。要从链中删去一个元素，也只需要改变两个指针，而不需要移动任何一个数据元素。

若在一个链中要删除一个元素，删除前后的情况如图 13-22 所示。

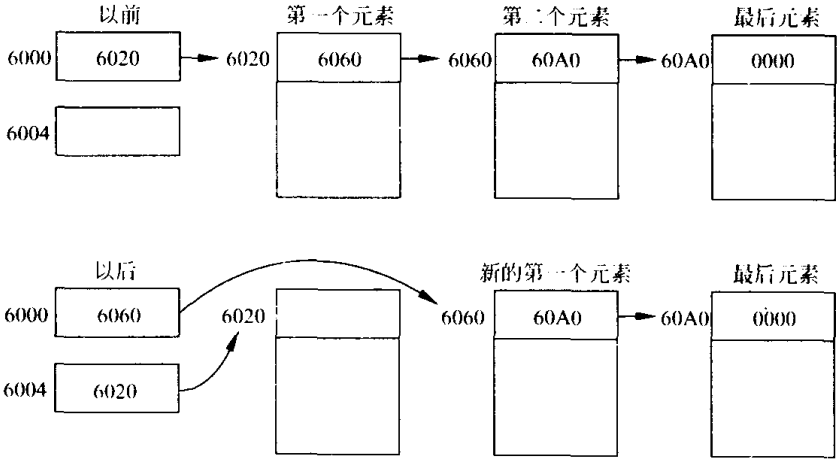


图 13-22　在链中删除一个元素的情况

调整指针的程序为：

```
MOV     QUEVE,POINTER      ;把指向项 1 的指针放至 QUEVE 中
MOV     POINTER,POINTER1   ;把项 1 中的指向项 2 的指针传送至 POINTER 中
```

双向链(Doubly Linked Lists)

有时需要用到双向链，这样每一个元素中必须包含上一个元素(逆向)和下一个元素(正向)的指针，这样就可以从任何一端访问元素。这样的数据结构可以用作先进先出方

式，也可以工作在后进先出方式，这取决于新元素是加到队列的头还是尾。

5. 表

表主要用于通过一个已知的值来得到一个信息项，如由角度可得到三角函数等。

(1) 常数表

若有一个 0°～90°的三角函数表，给定一个角度值，就可以查到相应的函数值，其流程如图 13-23 所示。

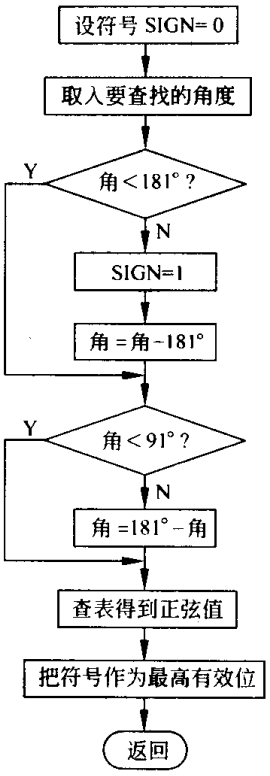


图 13-23 查三角函数表的流程图

(2) 利用表实现码之间的转换

若有 3 个表，要求把一个任意的十六进制数转换为与它等效的 ASCII 码、BCD 码和格雷码。程序流程如图 13-24 所示。

(3) 跳转表

在许多情况下，表中的项可以是地址。例如出错信息表，可以由错误号查到出错信息的起始地址；中断服务程序的入口地址表；某个命令(例如键盘命令)处理程序的入口地址表等。

若要求根据在某个寄存器中的指示码(其值为 0,1,2,3,4 中的一个)，调用 5 个用户子程序中的一个，每个子程序的入口地址占 4 个字节(两个字节的段内偏移量，两个字节的段地址)。其流程图如图 13-25 所示。

(4) 若在内存中有一个阶乘表(例如 0!～7!)，在内存的某一个单元中给定了一个值(范围为 0～7)，要通过查表，找到它的阶乘值。其程序的流程如图 13-26 所示。

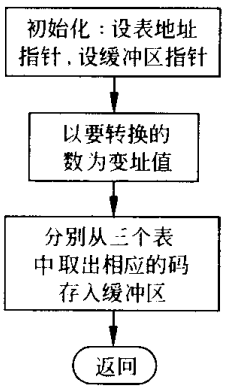


图 13-24 利用 3 个表把十六进制数转换为相应的 ASCII 码、BCD 码和格雷码

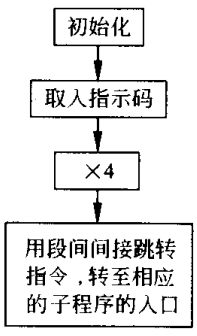


图 13-25 跳转表的一个例子的程序流程图

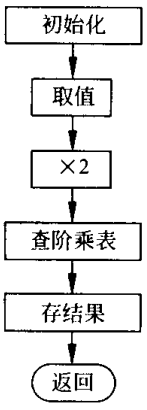


图 13-26 查表找到阶乘值的程序流程图

第 14 章　接口电路实验

14.1　TPC-H 实验装置介绍

TPC-H 实验装置是清华大学同方集团的教学仪器设备公司生产的，它比较适合本教材的教学需要。该仪器设计方案合理，通用性强，实验方案新颖，实用性强，采用了“自锁紧”插座，消除了连线接触不良现象，附加了电路保护措施，提供了 C 语言进行接口实验的参考程序。

TPC-H 实验装置的硬件包括接口卡、实验台（箱）两部分，两者之间通过 50 线扁平电缆相连。接口卡可以插入 PC 系列微机中任意一个扩展插槽。它的主要功能是将与实验有关的总线信号加以驱动后引到实验台上，同时引出的信号还有与“中断”和“DMA”实验操作有关的信号以及 +5V、+12V、−12V 电源信号。另外，接口卡上设有一个跳线开关（JP）用于选择 IQ_2、IQ_3、IQ_4、IRQ_7 中的一个作为中断请求信号。

实验台上设有 I/O 地址译码电路、总线插孔、接口实验常用的集成电路、外围电路以及通用 IC 插座等几个部分。外围电路包括逻辑电平开关电路、LED 显示电路、时钟电路、单脉冲电路、逻辑笔、复位电路、七段数码管显示电路、基本门电路、继电器及步进电机、小直流电机的驱动电路。

14.1.1　系统组成与安装

1. 系统组成

（1）硬件：接口卡一块；实验台（箱）一个；50 线扁平电缆一根；自锁紧导线 50 根；集成电路芯片（8251、74LS273、74LS244）共 3 片。

（2）软件：软盘一张。

（3）资料：使用说明书一本。

（4）附件：话筒一个、测试表笔一支。

（5）可选附件：

① 实验盒一个（包括：步进电机一个，小直流电机一个，喇叭一个，指示灯一个）；

② TPC-H 键盘显示控制器实验板一块。

2. 安装步骤

（1）关上 PC 机电源，打开微机主机箱。

（2）检查接口卡上中断请求跳线开关是否已经连好（该卡在出厂时已将 4、5 短接，选择 IRQ_7 作为中断申请信号）。接口卡的结构如图 14-1 所示。

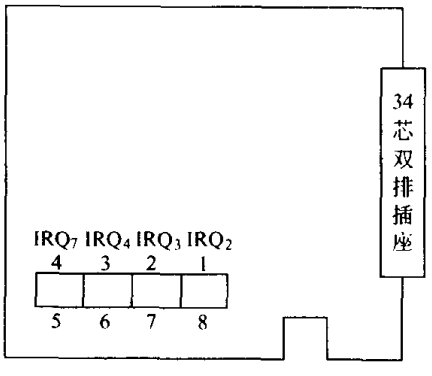


图 14-1　TPC-H 实验装置中的接口卡

（3）将 PC 总线接口卡插至主机的任一扩展插

槽中。

(4) 用 50 线扁平电缆线连接接口卡和实验台。

实验台的外观如图 14-2 所示。

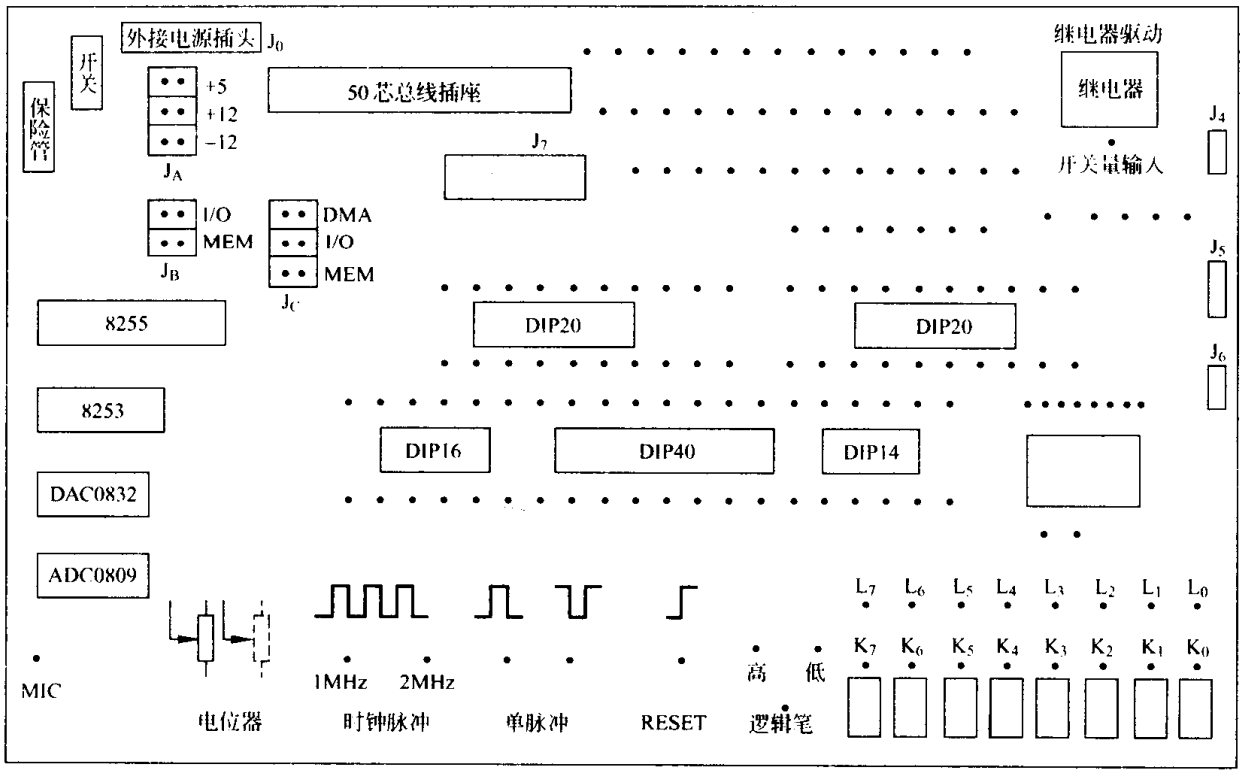


图 14-2 TPC-H 实验台外观

14.1.2 实验台结构

1. I/O 地址译码电路

如图 14-3 所示，这里选用 PC 机未用的 I/O 地址空间 280H～2BFH。共有 8 条译码输出线：Y_0 ～ Y_7。其地址分别是：Y_0：280H ～ 287H；Y_1：288H ～ 28FH；Y_2：290H ～ 297H；Y_3：298H～29FH；Y_4：2A0H～2A7H；Y_5：2A8H～2AFH；Y_6：2B0H～2B7H；Y_7：2B8H～2BFH。8 根译码输出线在实验台上标有“I/O 地址”处分别由“自锁紧”插孔引出，供实验选用。

2. 总线插孔

采用“自锁紧”插座在标有“总线”区引出的信号有：数据总线 D_7～D_0；地址总线 A_9～A_0；读、写信号 $\overline{IOR}$、$\overline{IOW}$；中断请求信号 IRQ；DMA 请求信号 DRQ_1；DMA 响应信号 $\overline{DACK}$；以及 AEN 信号。这些总线信号供学生搭试各种接口实验电路时使用。

3. 时钟电路

如图 14-4 所示，时钟电路可以输出 1MHz 和 2MHz 两种信号，供 A/D 转换器、定时器计数器、串行接口等实验使用。

4. 逻辑电平开关电路

如图 14-5 所示，实验台右下方设有 8 个开关 K_7～K_0。开关向上拨到“1”位置时，则

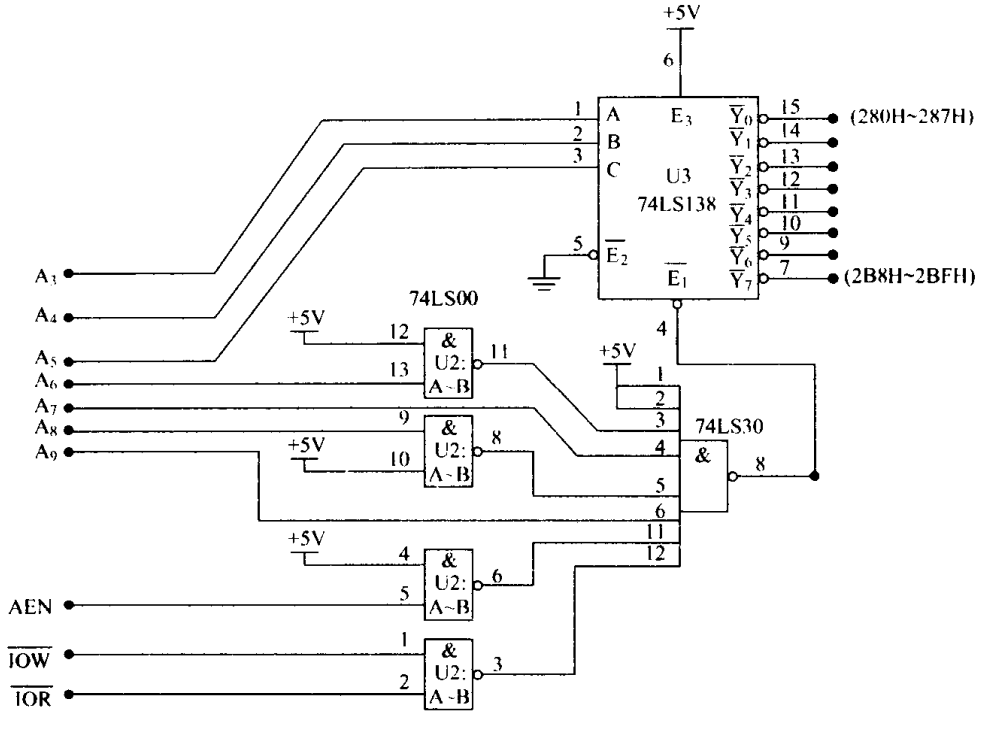


图 14-3　I/O 地址译码电路

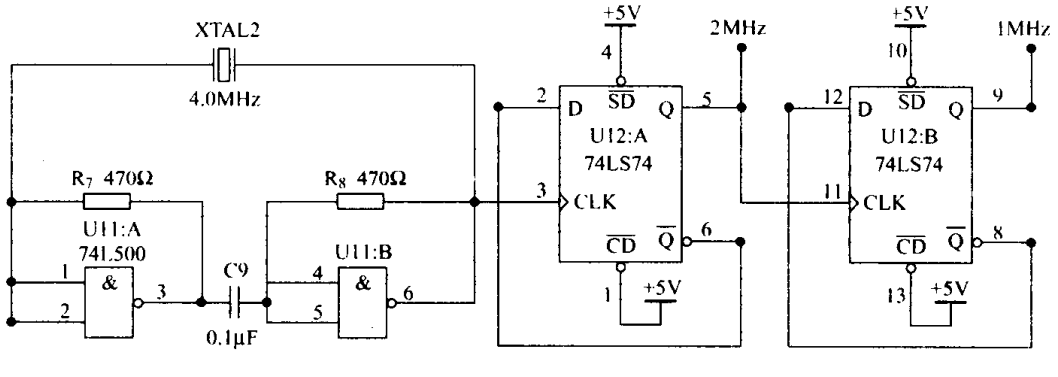


图 14-4　时钟电路

开关断开，输出高电平；向下打到"0"位置时，则开关接通并输出低电平，电路中串接了保护电阻，使接口电路不直接与+5V 电源、GND 地相连，可有效地防止学生因为误操作、误编程而损坏集成电路。

5. LED 显示电路

如图 14-6 所示。实验台上设有 8 个发光二极管 LED 及相关驱动电路（输入端 L_7～L_0），当输入信号为"1"时 LED 发光，当输入信号为"0"时 LED 熄灭。

6. 七段数码管显示电路

如图 14-7 所示，实验台上设有两个共阴极七段数码管及驱动电路。段码为同相驱动器，位码为反相驱动器，从段码与位码的驱动器输入端（段码输入端：a、b、c、d、e、f、g、dp，位码输入端：S_1、S_2）输入不同的代码即可显示不同数字或符号。

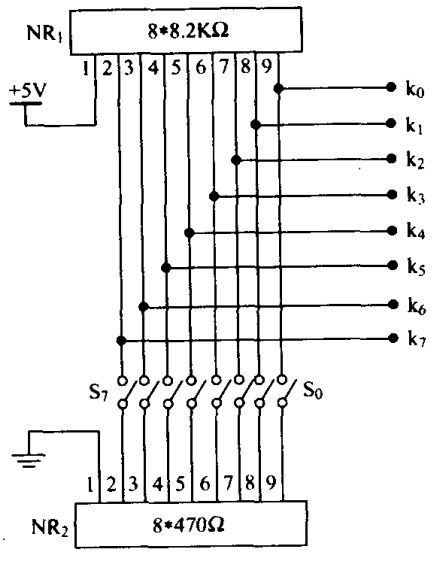


图 14-5　逻辑电平开关电路

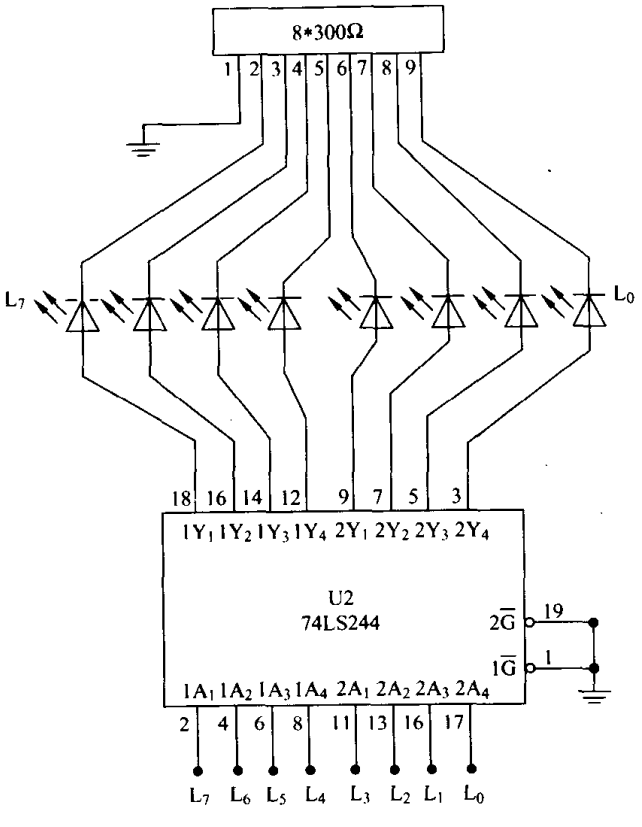


图 14-6　LED 显示电路

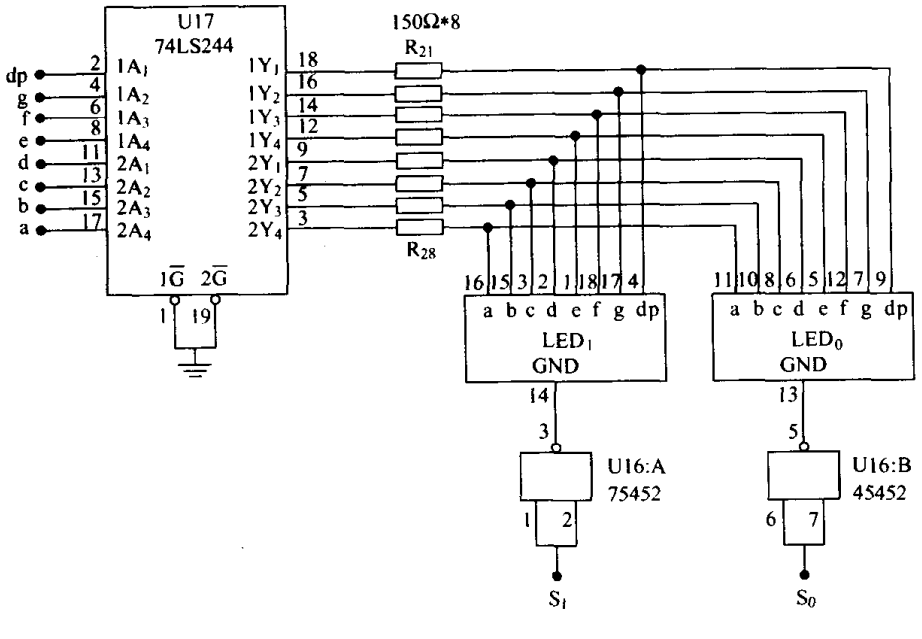


图 14-7　七段数码管显示电路

7. 单脉冲电路

如图 14-8 所示，采用 RS 触发器产生单脉冲信号，实验者每按一次开关即可从两个插座上分别输出一个正脉冲及负脉冲，以供“中断”、“DMA”、“定时器/计数器”等实验使用。

8. 逻辑笔

如图 14-9 所示，当输入端 U_i 接高电平时红灯(D_2)亮；U_i 接低电平时则绿灯亮。

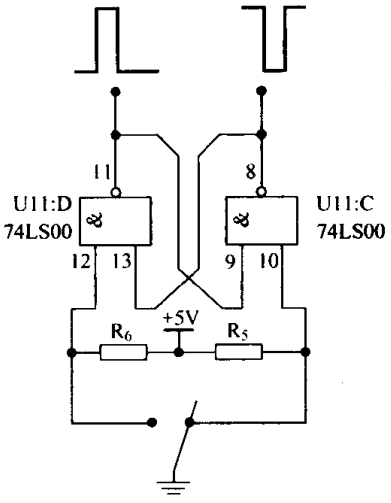


图 14-8　单脉冲电路

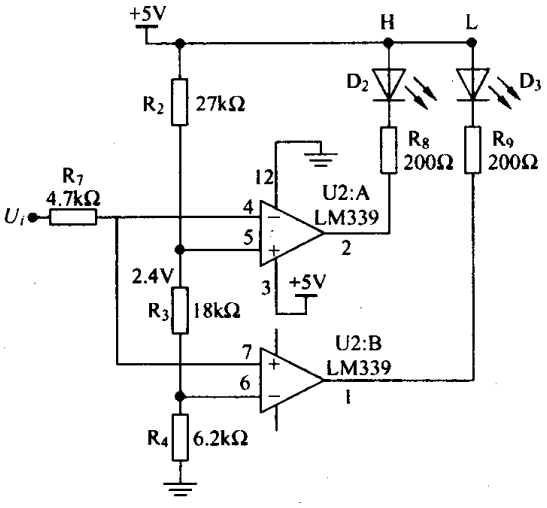


图 14-9　逻辑笔

9. 继电器及驱动电路

如图 14-10 所示。实验台上设有一个+5V 直流继电器及相应的驱动电路，当其开关量输入端输入数字量为“1”时，继电器动作：常开触点闭合、常闭触点断开。通过相应的实验，使学生了解开关量控制的一般原理。

10. 复位电路

如图 14-11 所示，复位电路能在上电时或按下复位开关 S_2 后，产生一个高电平的复位信号，以供 8255、8251 等接口芯片使用。

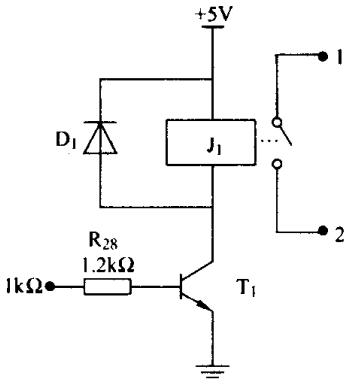


图 14-10　继电器驱动电路

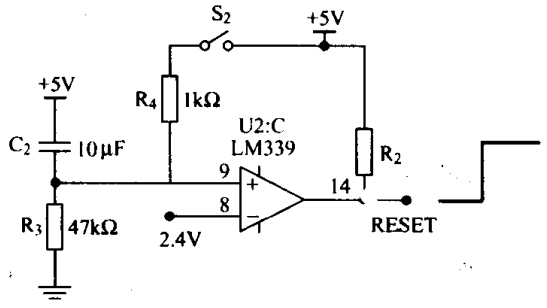


图 14-11　复位电路

11. 接口集成电路

实验台上有微型计算机原理硬件实验最常用的接口电路芯片，包括：可编程定时器/计数器(8253)、可编程并行接口(8255)、数/模转换器(DAC0832)、模/数转换器（ADC 0809)。这里芯片与 CPU 相连的引线除片选信号$\overline{CS}$外都已经连接好，与外界连接的关键引脚在芯片周围用“自锁紧”插座引出，供学生实验时使用。在本实验系统中未用到的引脚，如 8255 的 $PB_0 \sim PB_7$；8253 的 CLK_2、$GATE_2$、OUT_2；ADC 0809 的 $IN_3 \sim IN_7$ 都用小

圆插孔引出，实验时可用单股导线引出。另外，数/模转换器附有双极性输出插座，模/数转换器附有双极性输入插座。具体电路可见下面各实验说明。

12. 跳线开关(JP)

实验台上设有5个跳线开关，其中J_A、J_B、J_C在实验台的左上角，50线总线插座的左方。J_A用来选择是用主机电源还是用附加电源，当用主机电源时，应将J_A上的+5V、+12V、-12V这3个短路片插好；用户若想使用外加电源，必须首先将J_A上的3个短路片全部拔掉，才能将外接电源加入。J_B和J_C是用来选择实验类型的，I/O实验时，J_B、J_C上的短路片都应该插在标有"I/O"的位置。做存储器实验时，短路片应插在标有"MEM"的位置。做DMA实验时，J_B应该插在"I/O"的位置，J_C应该插在"DMA"的位置。实验台出厂时短路片在I/O实验位置。

实验台上另两个跳线开关为JP_1和JP_2，在实验台的左下角，分别用于模数转换器模拟量输入极性选择，将JP_2的1、2短路时，IN_2(J_2)可以输入双极性电压(-5V～+5V)，而2、3短路时为单极性输入(0～+5V)。JP_3用于选择IN_1的输入极性，方法与JP_2相同。

13. +5V电源插针

为了减少主机+5V电源的负担和各主要芯片的安全，在主要接口芯片的左上角都有相应的电源连接插针(标记为+5V)。当实验需要该芯片时，用短路块短接插针即可接通+5V电源。对用不到的芯片可将短路块拔掉以确保芯片安全。

14. 通用集成电路插座

实验台上设有4个通用数字集成电路插座，其中插座A、C为14引脚，插座B为16引脚，插座D可以插入一个24～40引脚的集成电路芯片或者2个8～20引脚的集成电路芯片。每个插座引脚附近都有对应的"自锁紧"插孔，部分实验(简单并行接口、DMA、串行通信、集成电路测试)电路是利用这些插座搭试的。要求高的学校可以让学生自己搭试更多的实验，以提高学生的动手能力。利用这些插座可以开发新的实验，也可以进行数字电路实验，以及学生毕业设计。自锁紧插座插入导线时，应稍加力并沿顺时针方向旋转一下，才能保证接触良好。拔出时，应先逆时针方向旋转，待插头完全松开后，再向上拔出。

15. 数字电路实验区

实验台上有一块数字电路实验区，设有3种基本门电路(与门、或门、非门)及D触发器，以供学生在接口实验或数字电路实验时直接使用。

16. 接线端子

如图14-2所示实验台上设有7个接线端子，标号为J_1～J_7。J_1用于外接喇叭；J_2是一个立体声插孔用于外接话筒；J_4为继电器触点，其中1接+5V，2、3为常闭触点，3、4为常开触点，5接地线；J_5用于接步进电机；J_6用于接小型直流电机；J_7是一个20芯通用插座，用于外接用户开发的实验板，本系统中8279键盘显示实验板就从此插头引出；J_7各引脚信号安排如下：

19	17	15	13	11	9	7	5	3	1
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	IOD	$\overline{CS}$
RES	+5V	+5V	$\overline{IOR}$	$\overline{IOW}$	A_0	A_1	CLK	GND	GND
20	18	16	14	12	8	6	4	2	0

17. 50 线总线插座信号安排

表 14-1 给出了 50 线总线插座各引脚信号。

表 14-1 50 线总线插座引脚信号

1	+5V	11	E245	21	A_7	31	A_1	41	ALE
2	D_7	12	$\overline{IOR}$	22	A_6	32	GND	42	T/C
3	D_6	13	$\overline{IOW}$	23	A_5	33	A_0	43	A_{16}
4	D_5	14	AEN	24	+12V	34	GND	44	A_{17}
5	D_4	15	DACK	25	A_4	35	$\overline{MEMW}$	45	A_{15}
6	D_3	16	DRQ_1	26	GND	36	$\overline{MEMR}$	46	A_{14}
7	D_2	17	IRQ	27	A_3	37	CLK	47	A_{13}
8	D_1	18	+5V	28	−12V	38	RST	48	A_{12}
9	D_0	19	A_9	29	A_2	39	A_{19}	49	A_{10}
10	+5V	20	A_8	30	GND	40	A_{18}	50	A_{11}

18. 直流电源开关(K_1)和存储器地址选择开关(K_2)

实验台上有一个直流电源开关(K_1),在实验台的左上角。在主机电源打开后,再打开实验台上开关 K_1,实验台才会加电。另外实验台上还有一个 4 位拨动开关 K_2,在实验台 8255 芯片的右方。K_2 是在做存储器实验时选择存储器地址使用的。

19. 使用外加直流电源注意事项

(1) 用户可以使用主机电源也可以使用外加直流电源,主机电源通过总线扩展卡引入到实验台的 50 芯插座上,再通过短路片 J_A 接到电源开关上。该实验台在出厂时是按使用主机电源连接的。

(2) 用户若想使用外加电源,必须首先将 J_A 上的 3 个短路片全部拔掉,将主机电源断开,才能将外部电源接入。

(3) 用户电源通过外接电源插座 J_0 接到实验台上。插入以前应仔细检查电源插头上各路电源与 J_0 上标注的是否相同,如不相同应修改电源插头,不然会损坏实验台。

(4) 外接电源插入以前应检查各路电源输出是否正确,如不正确应修改或更换电源。

(5) 以上各项都正常后,才可以打开电源进行实验。

14.2 接口实验

说明:

(1) 实验电路介绍中不加"利用通用插座"说明的,均为实验台上已经固定的电路。

(2) 实验中要连接的线均用虚线表示,实线为已经连好的电路。

实验一　I/O 地址译码

1. 实验目的

(1) 掌握 I/O 地址译码电路的工作原理；

(2) 了解实验台上 I/O 译码电路的使用方法；

(3) 复习 D 触发器的工作原理。

2. 实验内容

实验电路如图 14-12 所示。其中 74LS74 为 D 触发器，可以直接使用实验台上数字电路实验区的 D 触发器，74LS138 为地址译码器。译码输出端 Y_0～Y_7 在实验台上"I/O"地址输出端引出，每个输出端包含 8 个地址，Y_0：280H～287H，Y_1：288H～28FH，……。当 CPU 执行 I/O 指令且地址在 280H～2BFH 范围内，则译码器选中，必有一根译码线输出负脉冲。

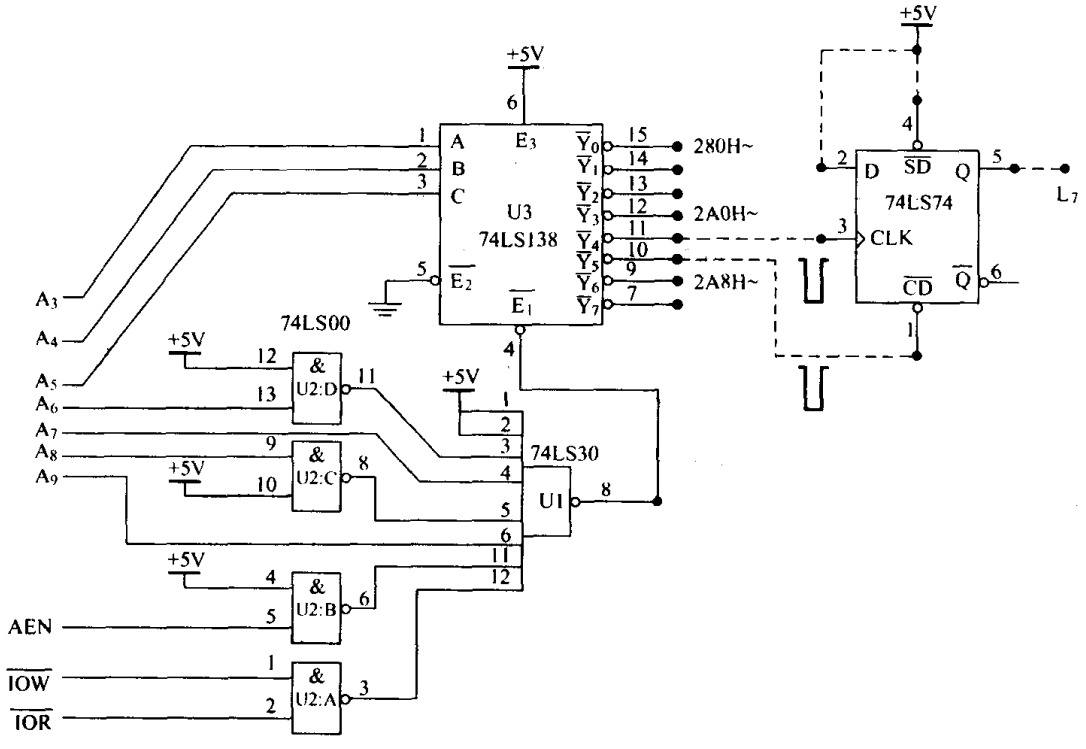


图 14-12　I/O 地址译码实验图

例如：执行下面两条指令

```
MOV DX,2A0H
OUT DX,AL(或 IN AL,DX)
```

时 Y_4 输出一个负脉冲。

利用这个负脉冲，通过 D 触发器来控制 L_7 灯闪烁发光(亮、灭、亮、灭……)，时间间隔通过软件延时来实现。

3. 编程提示

(1) 要使译码器电路输出一个负脉冲，必须使用输入或输出指令，并且其地址为译码器输出的对应地址。对于 Y_4 为 2A0H，对于 Y_5 是 2A8H。

(2) 实验电路中 D 触发器 CLK 端输入脉冲时，上升沿使 Q 端输出高电平控制 L_7 灯亮，CD 端加低电平 L_7 灯灭。

(3) 为实现亮、灭、亮、灭……，程序必须循环，而且亮和灭的间隔取决于软件延迟时间(延迟通过一段循环程序实现，通过控制循环次数，使在实验所用的主机频率下，使亮灭显示非常明显清晰)。

(4) 程序中最好检查是否有键按下，若有，则停止循环，返回 DOS。

(5) 可以通过改变连至 D 触发器的译码输出线以检验译码电路的输出。

4. 预习要求

(1) 仔细阅读 TPC-H 实验板介绍；

(2) 仔细阅读本实验指导书；

(3) 复习译码器和 D 触发器的工作原理；

(4) 编写好实验程序。

5. 报告要求

整理实验通过的程序。整理好延时子程序可以为以后使用。

实验二　简单并行接口

1. 实验目的

(1) 掌握锁存器和缓冲器的工作原理；

(2) 了解如何用锁存器和缓冲器构成简单的并行接口；

(3) 掌握实验台上的开关输入和发光二极管 LED 输出显示的使用方法。

2. 实验内容

(1) 按下面图 14-13 简单并行输出接口电路图连接线路(74LS273 插入通用插座，74LS32 用实验台上的“或门”)。74LS273 为 8D 触发器，8 个 D 输入端分别连接数据总线

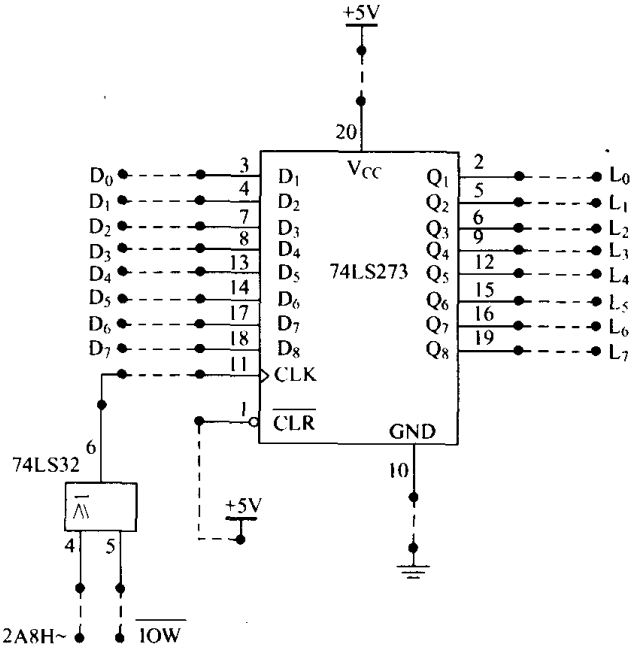


图 14-13　8D 触发器通过 LED 显示

D_0～D_7，8 个 Q 输出端连接 LED 显示电路 L_0～L_7。

（2）编程从键盘输入一个字符或数字，将其 ASCII 码通过这个输出接口输出，根据 8 个发光二极管发光情况验证正确性（灯亮为“1”，灯灭为“0”）。

（3）按图 14-14 简单并行输入接口电路图连接电路（74LS244 插入通用插座，74LS32 用实验台上的“或门”）。74LS244 为 8 缓冲器，8 个数据输入端分别连接逻辑电平开关 K 的输出 K_0～K_7，8 个数据输出端分别连接数据总线 D_0～D_7。

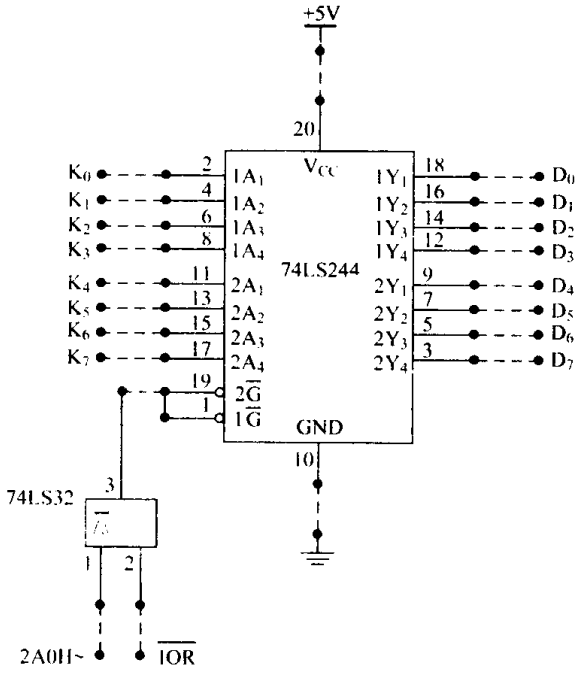


图 14-14　用开关通过缓冲器输入

（4）用逻辑电平开关预置某个字母的 ASCII 码，编程输入这个 ASCII 码，并将其对应字母在屏幕上显示出来。

3. 编程提示

（1）上述并行输出接口的地址为 2A8H，并行输入接口的地址为 2A0H。通过上述并行接口电路输出数据需要 3 条指令：

```
MOV AL,Data;          Data 为要输出的数据
MOV DX,2A8H
OUT DX,AL
```

数据可以从键盘输入（和用功能调用 1）。

通过上述并行接口输入数据需要两条指令：

```
MOV   DX,2A0H
IN    AL,DX
```

要在屏幕上显示所输入的数据，可以利用功能调用 2。

（2）参考流程图

用 8D 触发器输出的流程图如图 14-15 所示。

用 8 缓冲器输入的流程图如图 14-16 所示。

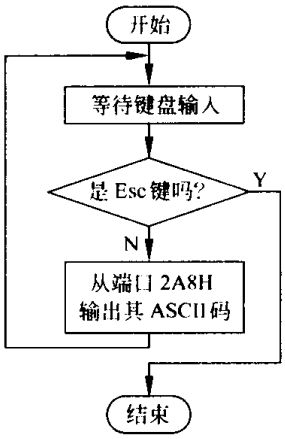


图 14-15　输出显示流程图

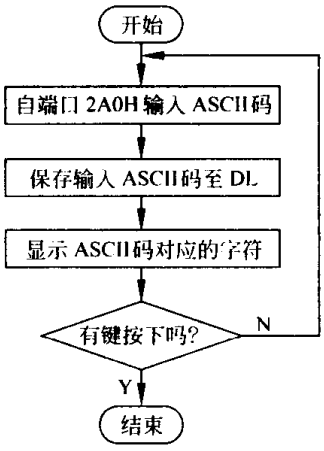


图 14-16　输入字符的流程图

4. 预习要求

(1) 阅读有关实验台的开关输入和 LED 显示部分的说明；

(2) 仔细阅读本实验书的要求；

(3) 复习 8D 触发器和 8 缓冲器的原理；

(4) 按照流程图编写相应的程序。

5. 报告要求

(1) 小结实验台上的开关输入和 LED 显示的使用方法；

(2) 掌握如何利用功能调用输入字符与显示字符。

实验三　可编程计数器/定时器(8253)

1. 实验目的

(1) 掌握可编程计数器/定时器 8253 的基本工作原理和编程方法；

(2) 熟悉单脉冲发生器和逻辑笔的使用方法。

2. 实验内容

(1) 按照图 14-17 虚线连接电路，将计数器 0 置为方式 0，计数器初值为 N($N \leqslant$ 0FH)，用手动逐个输入单脉冲，编程使计数值在屏幕上显示，并同时用逻辑笔观察 OUT0

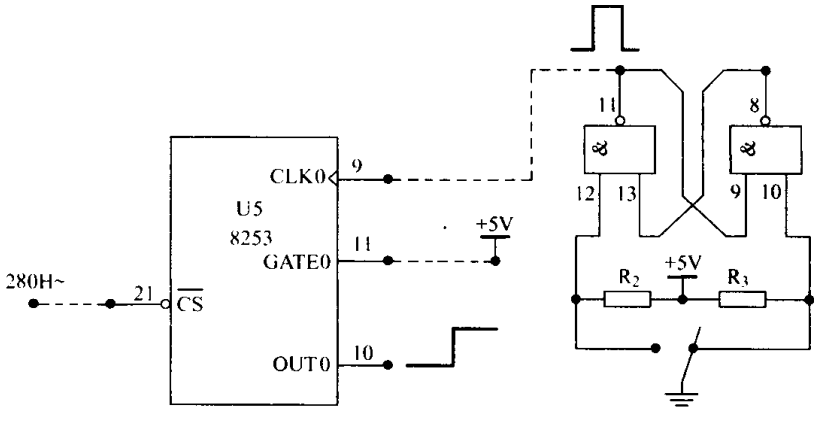


图 14-17　单脉冲发生器作为 8253 的输入

电平变化(当输入 $N+1$ 个脉冲后 OUT0 变高电平)。

(2) 按照图 14-18 连接电路,8253 工作在级连方式,用计数器 0 的输出作为计数器 1 的输入,将计数器 0、计数器 1 分别设置为方式 3,计数初值设为 1000,使用实验台上的 1MHz 时钟作为计数器 0 的时钟信号,用逻辑笔观察 OUT1 输出电平的变化(频率 1Hz)。

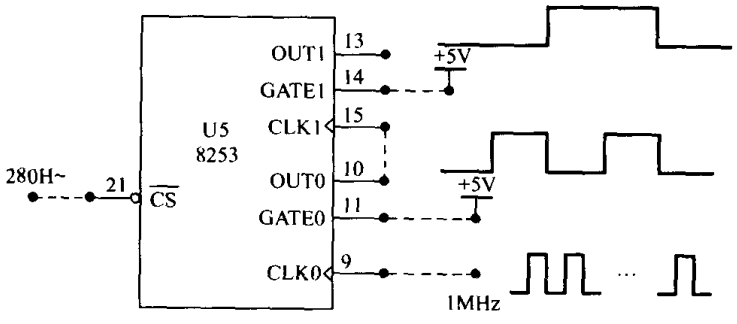


图 14-18 8253 工作在级连方式

(3) 改变连线,试其他的计数器。

(4) 用开关的输出接至 GATE 端,观察 GATE 的电平对计数器工作的影响。

(5) 在单脉冲方式下,试验改变计数值在不同计数方式下的作用。

(6) 在单脉冲方式下,观察在不同计数方式下 OUT 输出的情况。

3. 编程提示

(1) 8253 控制寄存器地址 283H

计数器 0 地址 280H

计数器 1 地址 281H

CLK0 连接时钟 1MHz

(2) 流程图

第 1 个实验即 8253 初始化编程并且读取计数值的程序流程如图 14-19 所示。8253 工作在级连方式下的程序流程如图 14-20 所示。

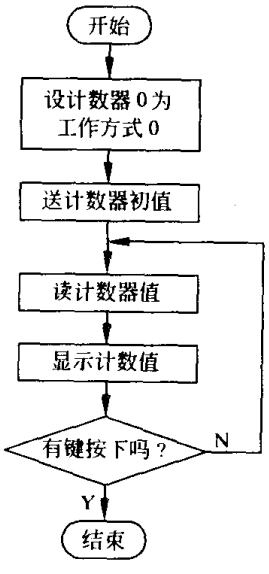


图 14-19 8253 初始化编程并且读取计数值的程序流程

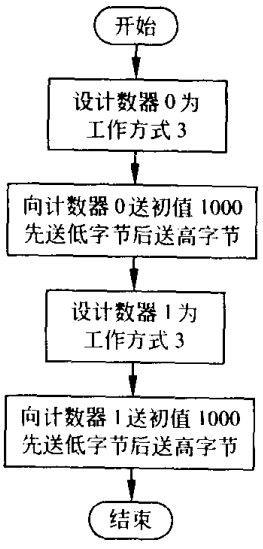


图 14-20 8253 级连方式程序流程

4. 预习要求

(1) 阅读实验台单脉冲发生器与逻辑笔的使用方法；

(2) 仔细阅读本实验指导书；

(3) 复习 8253 的工作原理、各种计数方式、计数初值及 GATE 对计数器的影响；

(4) 复习 8253 的初始化编程方法和读取计数值的方法；

(5) 分析流程图，编好程序。

5. 报告要求

(1) 根据实验结果，整理与分析 8253 的各种计数方式；

(2) 分析并总结 GATE 对计数器工作的影响；

(3) 分析并总结 8253 级连方式的特点；

(4) 总结在计数过程中如何读取计数值。考虑能否用 8253 测量一个脉冲的宽度(测量时间间隔)。

实验四　可编程并行接口(一)(8255 方式 0)

1. 实验目的

(1) 掌握可编程并行接口 8255 方式 0 的工作原理；

(2) 掌握可编程并行接口 8255 方式 0 的使用方法。

2. 实验内容

(1) 实验电路如图 14-21 所示。8255 端口 C 接逻辑电平开关 $K_0 \sim K_7$(作为输入)，端口 A 接 LED 显示电路 $L_0 \sim L_7$。

(2) 编程从 8255 端口 C 输入数据并且从端口 A 输出该数据，验证结果。

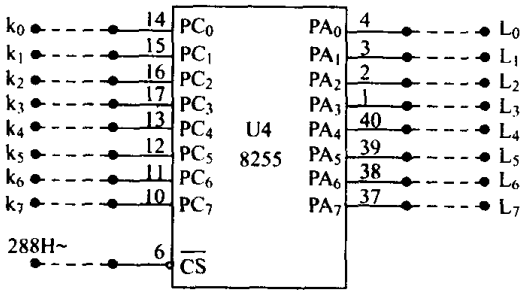


图 14-21　8255 方式 0 输入输出

3. 编程提示

(1) 8255 控制寄存器端口地址　　28BH

8255 端口 A 的地址　　288H

8255 端口 C 的地址　　28AH

(2) 流程图

可编程并行接口 8255 从端口 C 输入字符并且将输入字符从端口 A 输出的程序流程图如图 14-22 所示。

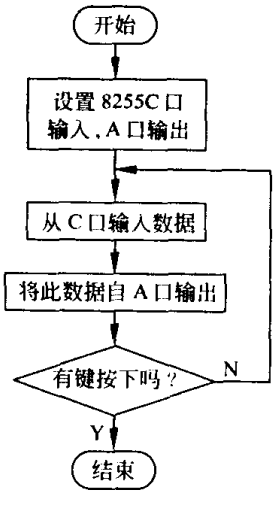


图 14-22　8255 方式 0 程序流程图

4. 预习要求

(1) 仔细阅读本实验指导书；

(2) 复习 8255 方式 0 的工作原理、使用方式和初始化编程方法；

(3) 根据流程图编好程序。

5. 报告要求

(1) 小结 8255 方式 0 的特点；

(2) 把本实验与简单并行接口实验相比较。

实验五　可编程并行接口 8255 与七段数码管

1. 实验目的

(1) 进一步熟悉可编程并行接口 8255 方式 0 的使用；

(2) 掌握数码管显示数字的原理。

2. 实验内容

(1) 静态显示：按图 14-23 连接好电路，将 8255 的端口 A 的 PA_0～PA_6 分别与七段数码管的段码驱动输入端 a～g 相连，位码驱动输入端 S_1 接＋5V(选中)，S_0、dp 接地(关闭)。编程从键盘输入一位十进制数字(0～9)，并且在七段数码管上显示出来。

(2) 动态显示：按图 14-24 连接好电路，七段数码管段码连接不变，位码驱动输入端 S_1、S_0 接 8255C 口的 PC_1、PC_0。编程在两个数码管上循环显示数字 00～99。

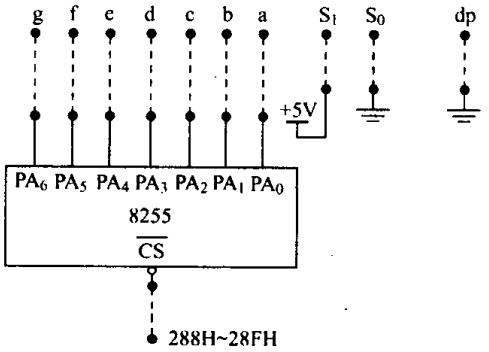


图 14-23　七段数码管静态显示

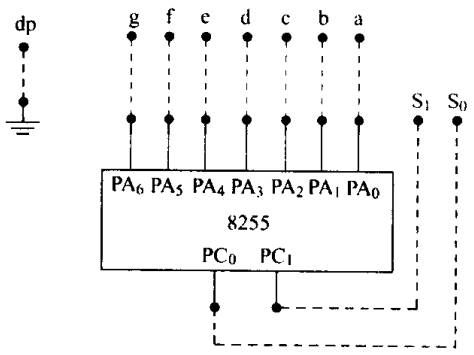


图 14-24　七段数码管动态显示

3. 编程提示

(1) 实验台上的七段数码管为共阴型，段码采用同相驱动，某一段接高电平，则相应的段亮，位码加反相驱动器，位码输入端为高电平选中相应的显示管。

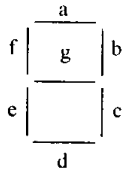


图 14-25　七段数码管字形图

(2) 七段数码管的字形如图 14-25 所示。

其显示的字形与七段的编码如表 14-2 所示。

表 14-2　七段编码表

显示字型	g	f	e	d	c	b	a	段码
0	0	1	1	1	1	1	1	3FH
1	0	0	0	0	1	1	0	06H
2	1	0	1	1	0	1	1	5BH
3	1	0	0	1	1	1	1	4FH
4	1	1	0	0	1	1	0	66H
5	1	1	0	1	1	0	1	6DH
6	1	1	1	1	1	0	1	7DH
7	0	0	0	0	1	1	1	07H
8	1	1	1	1	1	1	1	7FH
9	1	1	0	1	1	1	1	6FH

(3) 程序流程图

静态显示的流程图如图 14-26 所示。

动态显示的流程图如图 14-27 所示。

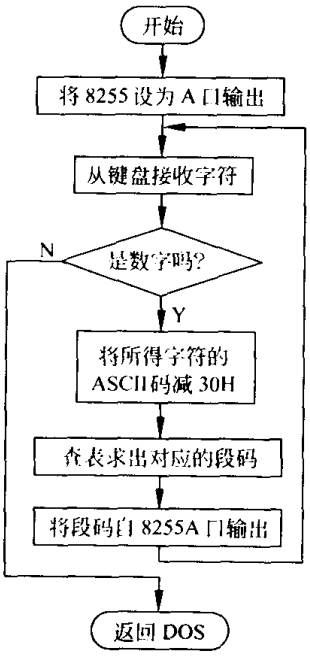


图 14-26 静态显示流程图

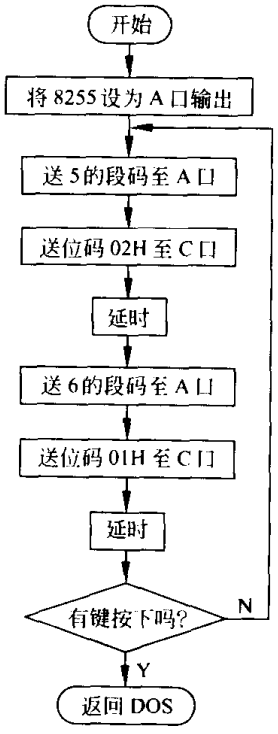


图 14-27 动态显示流程图

4. 预习要求

(1) 仔细阅读实验台七段显示器的工作原理;

(2) 仔细阅读本实验指导书;

(3) 根据流程图编好程序。

5. 报告要求

(1) 小结用七段显示管显示的编程方法;

(2) 进一步掌握 8255 方式 0 的使用。

实验六 用可编程并行接口 8255 控制继电器

1. 实验目的

(1) 了解微型计算机控制直流继电器的一般方法;

(2) 进一步熟悉 8255 和 8253 的使用。

2. 实验内容

(1) 实验电路如图 14-28 所示。按虚线连接电路: CLK0 接 1MHz,GATE0、GATE1 接 +5V,OUT0 接 CLK1,OUT1 接 PA_0,PC_0 接继电器驱动电路的开关输入端 IK,继电器输出插头 J_4 接实验盒上的继电器插头。编程使可编程定时器/计数器 8253 工作在级连方式,作为定时器使用以控制继电器周而复始的闭合 5 秒钟(指示灯亮),断开 5 秒钟(指示灯灭)。

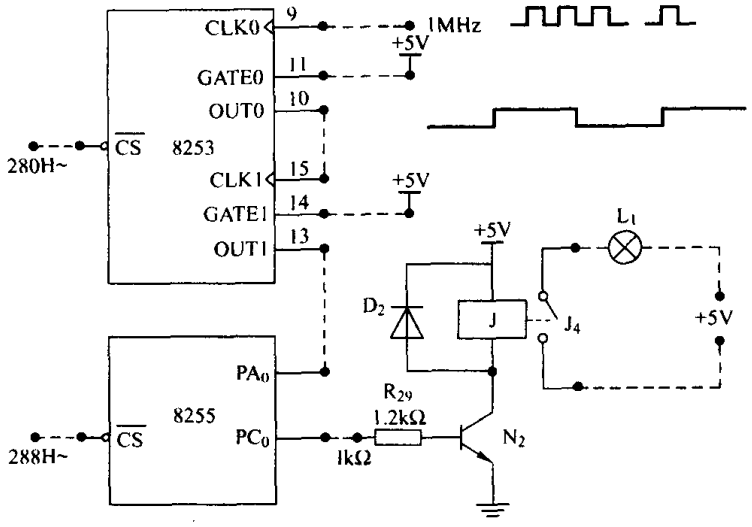


图 14-28　继电器控制

(2) 用可编程并行接口 8255 端口 C 的 PC_0 控制继电器线圈，用端口 A 的 PA_0 查询 8253 的 OUT1 的输出。

3. 编程提示

(1) 将 8253 计数器 0 设置为方式 3、计数器 1 设置为方式 0 串联使用，CLK0 接 1MHz 时钟，设置两个计数器的初值(乘积为 500000)为 5 秒，启动计数器工作后，经过 5 秒钟 OUT1 输出高电平。8255A 端口 A 的 PA_0 查询 OUT1 的输出，当其为高电平，用端口 C 的 PC_0 输出开关量控制继电器动作。

(2) 继电器开关量输入端输入“1”时，继电器常开触点闭合，电路接通，指示灯泡亮；输入“0”时，继电器断开，指示灯泡熄灭。

(3) 流程图

把程序分为主程序和延时子程序两部分，其流程分别如图 14-29 和图 14-30 所示。

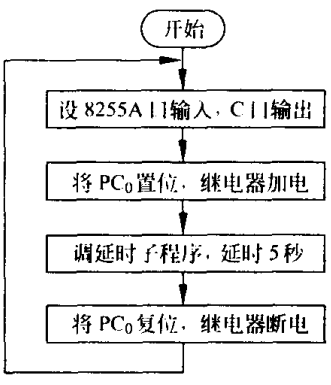


图 14-29　主程序流程图

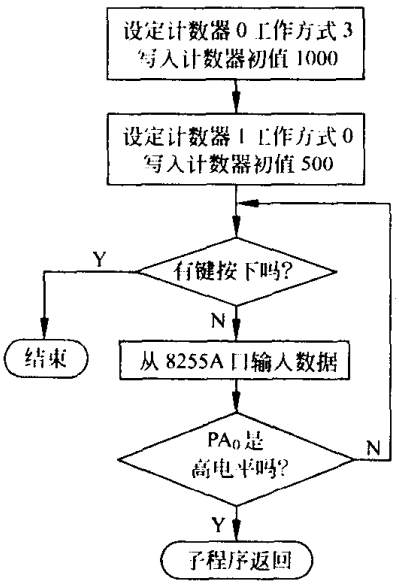


图 14-30　延时子程序流程图

4. 预习要求

(1) 了解实验台上继电器控制的使用方法；

(2) 复习 8253 采用级连方式作为定时器使用的方法；

(3) 仔细阅读本实验指导书；

(4) 了解 8253 与 8255 联合使用实现定时控制的方法；

(5) 按照流程图编好程序。

5. 报告要求

(1) 小结用微型计算机控制继电器的一般方法；

(2) 小结 8253 与 8255 联合使用实现定时控制的方法。

实验七　交通灯控制实验

1. 实验目的

通过并行接口 8255 实现十字路口交通灯的模拟控制，进一步掌握对并行口的使用。

2. 实验内容

实验电路如图 14-31 所示。用 L_7(红)、L_6(黄)、L_5(绿)作为南北路口的交通灯与 PC_7、PC_6、PC_5 相连；L_2(红)、L_1(黄)、L_0(绿)作为东西路口的交通灯与 PC_2、PC_1、PC_0 相连。编程使 6 个灯按照交通灯变化规律亮灭。

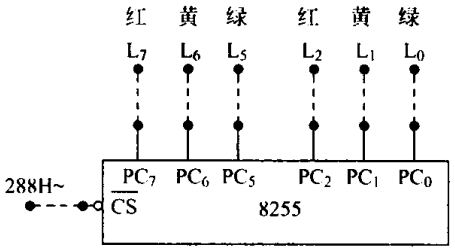


图 14-31　模拟的交通灯控制电路

3. 编程提示

(1) 十字路口交通灯的变化规律要求为：

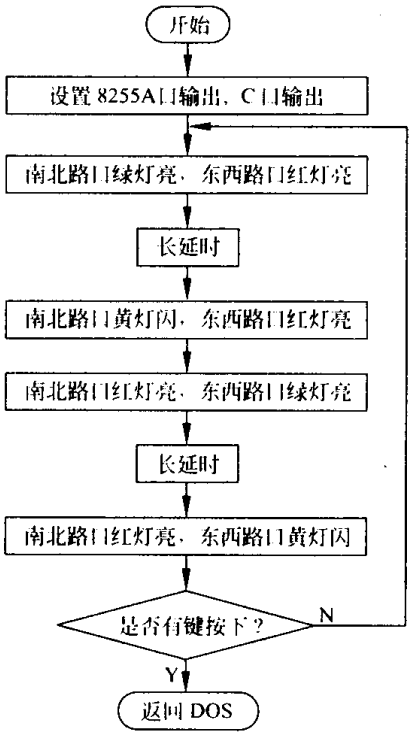


图 14-32　交通灯控制程序流程图

① 南北路口的绿灯、东西路口的红灯同时亮 30 秒左右；

② 南北路口的绿灯灭、黄灯闪烁若干次，此时东西路口的红灯继续亮；

③ 南北路口黄灯灭、红灯亮，东西路口红灯灭、绿灯亮，同时亮 30 秒左右；

④ 南北路口的红灯继续亮，同时东西路口的绿灯灭黄灯亮闪烁若干次。

⑤ 转①重复。

(2) 流程图

交通灯控制程序流程图如图 14-32 所示。

4. 预习要求

(1) 仔细阅读本实验指导书；

(2) 按照流程图编好程序。

5. 报告要求

上述的程序使红绿灯按照固定时间变化，不可改变，没有智能性。如果考虑到有大道和小道而且有检

测车辆到来的敏感元件，考虑按照如下规律编一个程序。

（1）通常情况下，大道总是绿灯亮，小道总是红灯亮。

（2）若检测到小道来车后，经过 10 秒钟使小道变为绿灯亮，则大道应变为红灯亮。但是，为了使大道的车由动到停有准备时间，在大道由绿灯亮变红灯亮之前，要有 4 秒钟的黄灯亮的时间。即小道来车后，经过 6 秒大道由绿灯亮变为黄灯亮，再过 4 秒，大道由黄灯亮变为红灯亮，在这同时小道由红灯亮变为绿灯亮。

（3）在小道变红灯亮后，如果大道没有来车，或来车但不到 3 辆车，则经过 25 秒小道变为黄灯亮，再经 4 秒小道由黄灯亮变为红灯亮，在这同时，大道由红灯亮变为绿灯亮。

（4）如果在小道绿灯亮期间，检测到大道已来 3 辆车，虽然小道亮绿灯的时间还不到 25 秒，当第三辆车到达后，小道立即换为黄灯亮，经过 4 秒，小道由亮黄灯变为红灯亮，同时大道由红灯亮变为绿灯亮。

实验八　中　　断

1. 实验目的

（1）掌握 PC 机中断处理系统的基本原理；

（2）学会编写中断服务程序。

2. 实验原理与内容

（1）实验原理

PC 机用户可以使用的硬件中断只有可屏蔽中断，由 8259 中断控制器管理。中断控制器用于接收外部的中断请求信号，经过优先级判别等处理后向 CPU 发出可屏蔽中断请求。

IBM PC、PC/XT 机内有一片 8259 中断控制器，对外可以提供 8 个中断源：

中断源	中断类型号	中断功能
IRQ_0	08H	时钟
IRQ_1	09H	键盘
IRQ_2	0AH	保留
IRQ_3	0BH	串行口 2
IRQ_4	0CH	串行口 1
IRQ_5	0DH	硬盘
IRQ_6	0EH	软盘
IRQ_7	0FH	并行打印机

8 个中断源的中断请求信号线 $IRQ_0 \sim IRQ_7$ 在主机的 62 线总线插座中可以引出，系统已经设定中断请求信号为边沿触发，普通结束方式。对于 286 以上的微型计算机又扩展了一片 8259 中断控制器，IRQ_2 已用于两片 8259 之间级连，考虑到仪器通用性，在本仪器接口卡上设有一个跳线开关(JP)，可以选择 IRQ_2、IRQ_3、IRQ_4、IRQ_7 引到实验台上的 IRQ 插座上，跳线方法前面已介绍(安装部分)过，出厂设置的是 IRQ_7。

（2）实验内容

实验电路如图 14-33 所示，直接用手动方式产生单脉冲作为中断请求信号(只需连接

一根导线)。要求每按一次开关产生一次中断,在屏幕上显示一次"THIS IS A IRQ_7 INTERRUPT",中断 10 次后程序退出。

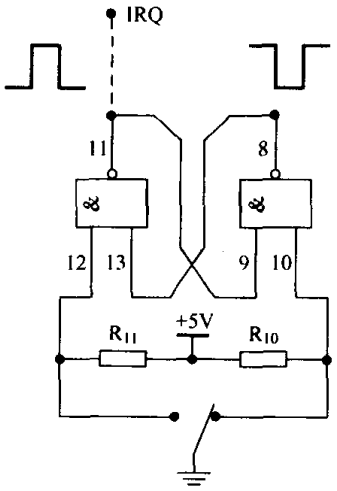


图 14-33　中断实验电路图

3. 编程提示

(1) PC 机中断控制器 8259 的地址为 20H、21H,编程时要根据中断类型号设置中断矢量,8259 中断屏蔽寄存器 IMR 对应位要清零(允许中断),中断服务结束返回前要使用中断结束命令:

```
MOV   AL,20H
OUT   20H,AL
```

中断结束返回 DOS 时应将 IMR 对应位置 1,以关闭中断。

(2) 流程图

程序分为主程序和中断服务程序两部分,其流程分别如图 14-34 和图 14-35 所示。

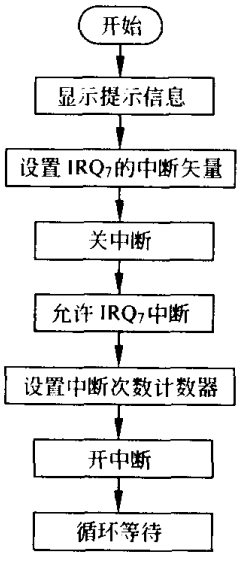


图 14-34　主程序流程图

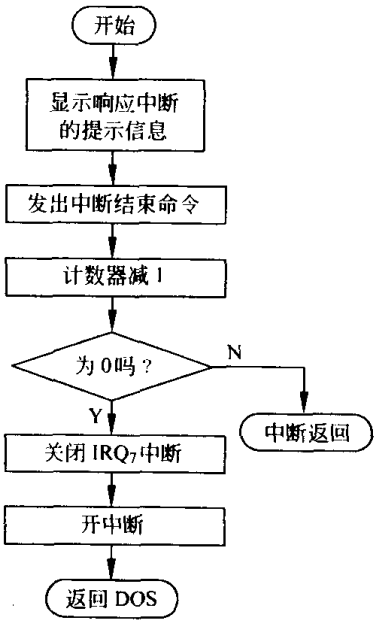


图 14-35　中断服务程序流程图

4. 预习要求

(1) 仔细阅读本实验指导书;

(2) 复习有关中断的章节;

(3) 熟悉 8259 的工作方式及编程;

(4) 根据流程图编好程序。

5. 报告要求

(1) 小结 8259 的编程方法;

(2) 小结主程序和中断服务程序的编程方法。

实验九　可编程并行接口(二)(8255 方式 1)

1. 实验目的

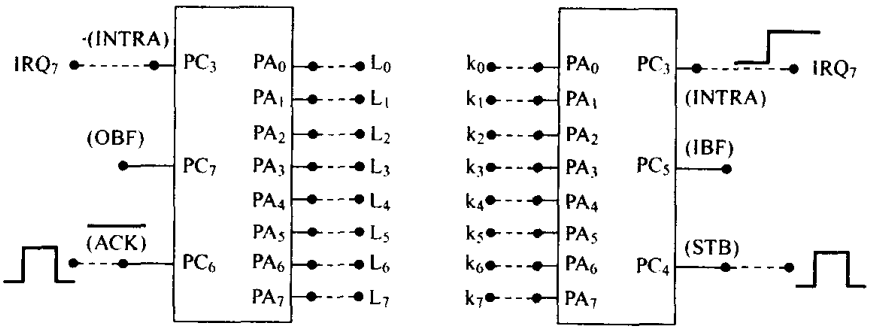
(1) 掌握 8255 工作方式 1 时的使用及编程；

(2) 进一步掌握中断服务程序的编写方法。

2. 实验内容

(1) 按图 14-36(a)8255 方式 1 的输出电路连接好线路。

(2) 编制程序。每按一次单脉冲按钮便产生一个正脉冲使 8255 产生一次中断请求，让 CPU 进行一次中断服务：依次输出 01H、02H、04H、08H、10H、20H、40H、80H 使 L_0～L_7 依次发光，中断 8 次后结束。



(a) 输出电路　　(b) 输入电路

图 14-36　8255 方式 1 输出及输入电路

(3) 按图 14-36(b)8255 方式 1 输入电路，连接好线路。

(4) 编写程序。每按一次单脉冲按钮便产生一个正脉冲使 8255 产生一次中断请求，让 CPU 进行一次中断服务：读取逻辑电平开关预置的 ASCII 码，在屏幕上显示其对应的字符，中断 8 次后结束。

3. 编程提示

可编程并行接口 8255 方式 1 输出和方式 1 输入都分为主程序和中断服务程序两部分。输出时的主程序和中断服务程序的流程如图 14-37 所示。

8255 方式 1 输入时的程序流程图如图 14-38 所示。

4. 预习要求

(1) 仔细阅读本实验指导书；

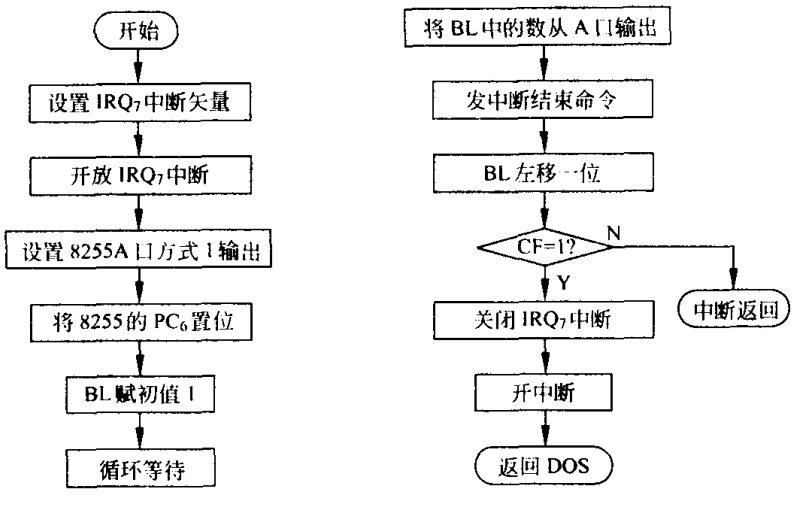
(2) 复习 8255 方式 1 的工作特点及编程方法；

(3) 按照流程图编好程序。

5. 报告要求

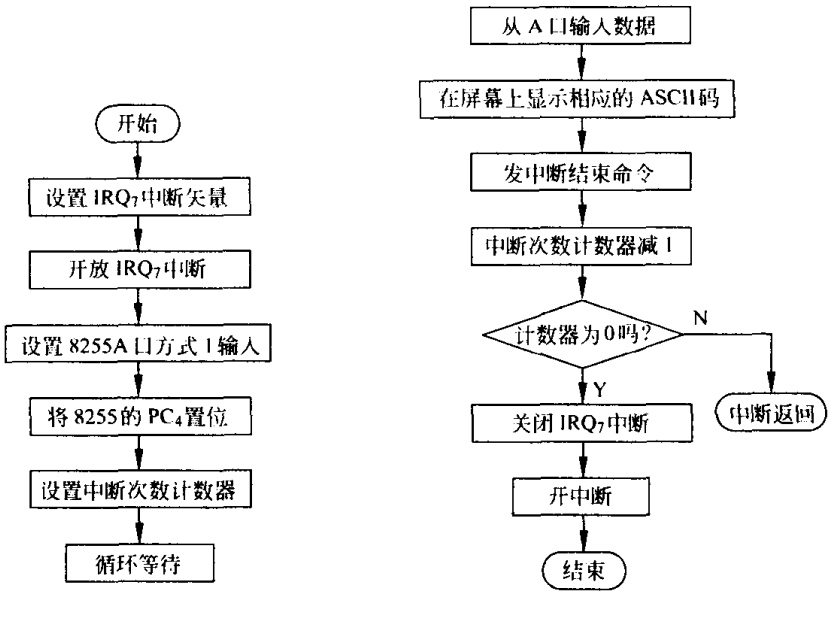
(1) 小结 8255 方式 1 的工作特点，特别是输入和输出时的联络(握手)信号线的功能及相互之间的关系；

(2) 进一步小结中断服务程序的编程方法。



(a) 输出主程序　　(b) 输出中断服务程序

图 14-37　8255 方式 1 输出时的程序流程图



(a) 输入主程序　　(b) 输入中断服务程序

图 14-38　8255 方式 1 输入时的程序流程图

实验十　数模(D/A)转换器

1. 实验目的

了解数模转换器的基本原理，掌握 DAC0832 芯片的使用方法。

2. 实验内容

(1) 实验电路原理如图 14-39 所示，DAC0832 采用单缓冲方式，具有单双极性输出端(图中的 U_a、U_b)。利用输出命令输出数据给 DAC0832，用万用表测量单极性输出端 U_a 及双极性输出端 U_b 的电压，验证数字与电压之间的线性关系。

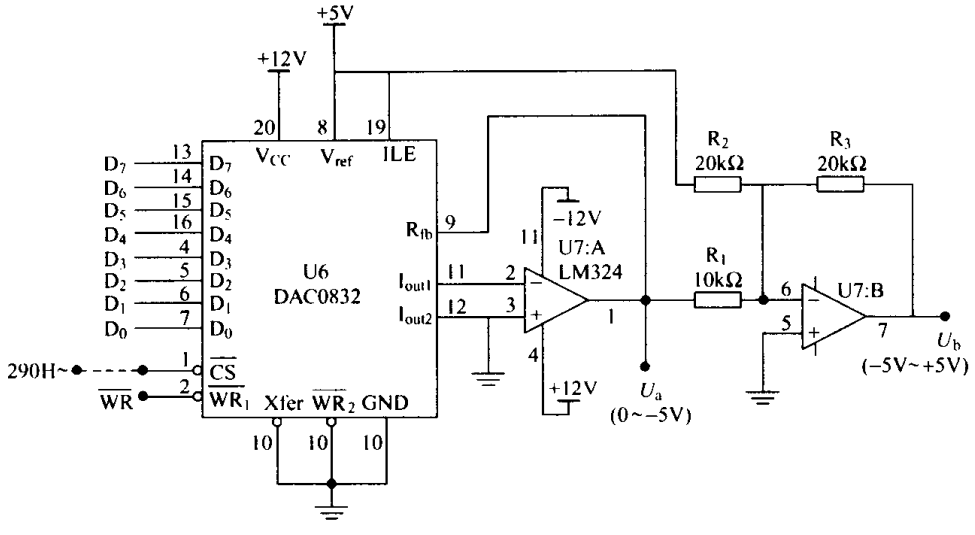


图 14-39　数模转换实验图

(2) 编写程序以产生以下波形(从 U_b 输出,用示波器观察):

① 锯齿波;

② 正弦波。

3. 编程提示

(1) 8 位 D/A 转换器 DAC0832 的端口地址为 290H,输入数据与输出电压的关系为:

$$U_a = -U_{REF}/256 \times N$$

$$U_b = 2\,U_{REF}/256 \times N - 5\text{V}$$

其中,U_{REF} 表示参考电压,N 表示数据,这里的参考电压为+5V 电源。

(2) 产生锯齿波只需将输出到 DAC0832 的数据由 0 循环递增,到最大值 FFH 后突降为 0。产生正弦波可以根据正弦函数建立一个正弦数字量表(80H、96H、0AEH、0C5H、0D8H、0E9H、0F5H、0FDH、0FFH、0FDH、0F5H、0E9H、0D8H、0C5H、0AEH、096H、80H、66H、4EH、38H、25H、15H、09H、04H、00H、04H、09H、15H、25H、38H、4EH、66H),取值范围为一个周期,表中数据为 16 个以上(32 个)。

(3) 流程图

能够产生锯齿波和正弦波的程序流程图分别如图 14-40 和图 14-41 所示。

4. 预习要求

(1) 复习数模转换器的工作原理;

(2) 仔细阅读本实验指导书;

(3) 按照流程图编好程序。

5. 报告要求

(1) 小结数模转换器的工作原理以及编程方法;

(2) 考虑若要通过 DAC0832 产生方形波或梯形波应该如何编程。

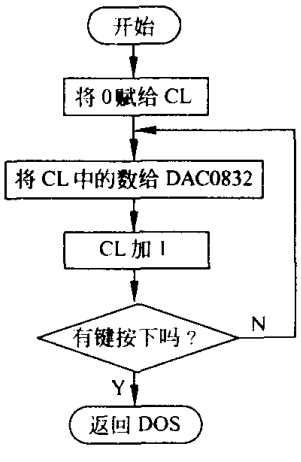


图 14-40　锯齿波流程图

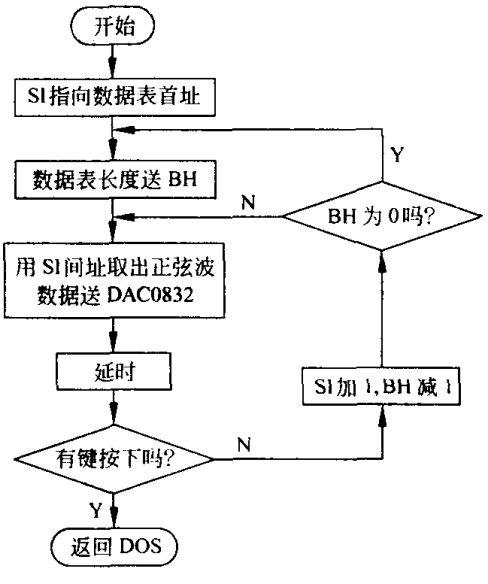


图 14-41　正弦波流程图

实验十一　模数(A/D)转换器

1. 实验目的

了解模数转换的基本原理，掌握 ADC0809 的使用方法。

2. 实验内容

(1) 实验电路原理图如图 14-42 所示。通过实验台左下角电位器 RW_1 输出 0～+5V直流电压送入 ADC0809 通道 0(IN_0)，利用输出命令启动 A/D 转换器，输入命令读取转换结果，验证输入电压与转换后的数字的关系。

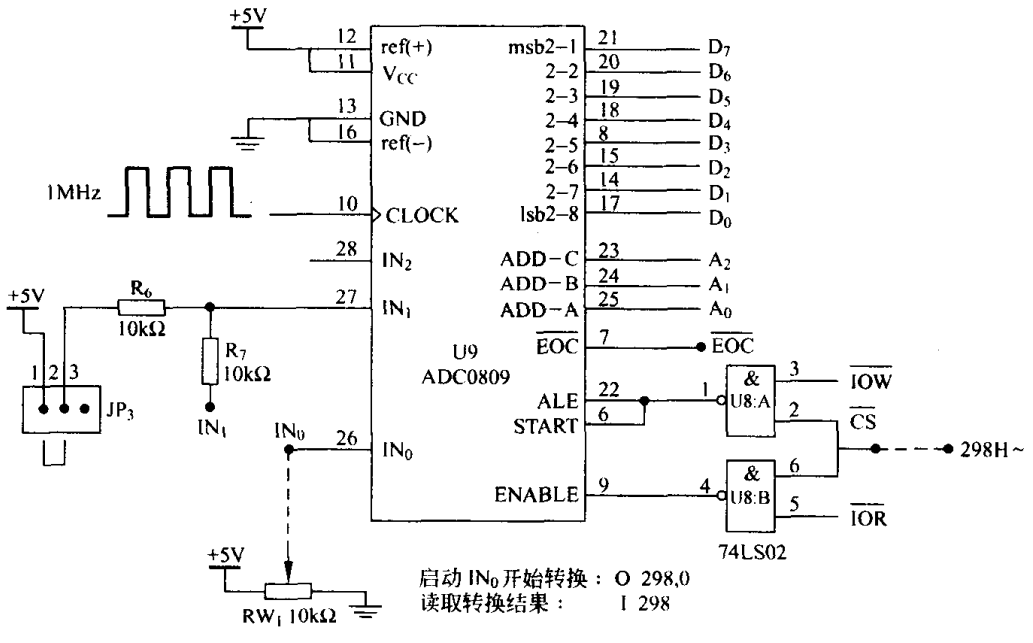


图 14-42　模数转换实验电路原理图

(2) 编程采集 IN_0 输入的电压，在屏幕上显示出转换后的数据(用十六进制数表示)。

(3) 将 JP_3 的 1、2 短接，使 IN_2 处于双极性工作方式，并给 IN_1 输入一个低频交流信号(幅度为±5V)，编程采集这个信号的数据并且在屏幕上显示波形。

3. 编程提示

(1) ADC0809 的 IN_0 端口地址为 298H，IN_1 口地址为 299H。

(2) IN_0 单极性输入电压与转换后数字的关系为：

$$N = U_i/U_{REF}/256$$

其中，U_i 为输入电压，U_{REF} 为参考电压，这里的参考电压为 PC 机的+5V 电源。

(3) 一次 A/D 转换的程序可以为：

```
MOV   DX, 端口地址
OUT   DX, AL          ;启动转换
:                     ;延时
IN    AL, DX          ;读取转换结果至 AL 中
```

(4) 流程图

主程序启动 A/D 转换，通过延时等待转换完成，读入转换后的结果，然后调用显示子程序在屏幕上显示转换的结果。

主程序的流程如图 14-43 所示。

显示子程序的流程如图 14-44 所示。

本实验第 3 个实验内容显示波形的流程图如图 14-45 所示。

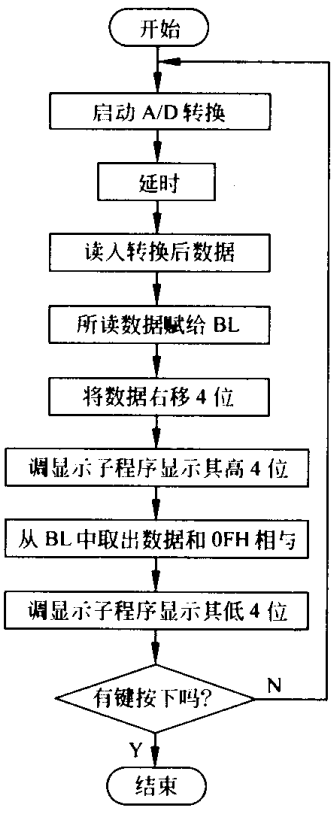


图 14-43 模数(A/D)转换主程序

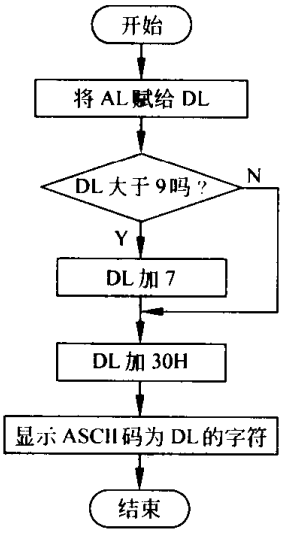


图 14-44 显示子程序

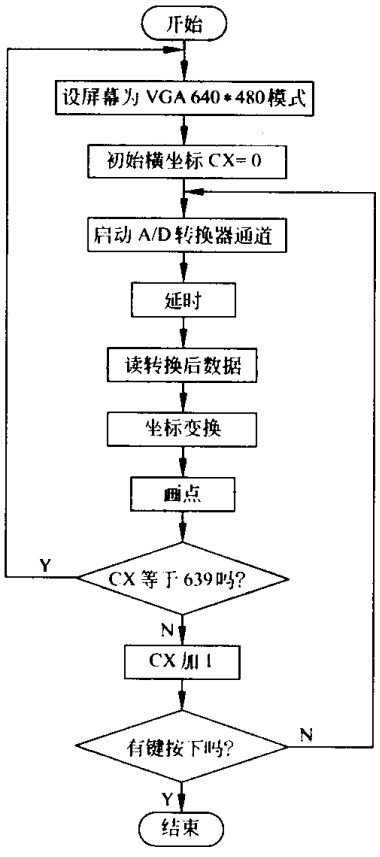


图 14-45 波形显示

4. 预习要求

(1) 复习 A/D 转换的原理,复习 ADC0809 的结构和引脚以及与 CPU 的接口方法;

(2) 仔细阅读本实验指导书;

(3) 按流程图编好程序。

5. 报告要求

(1) 整理实验的结果,把测量的输入模拟电压与数字显示列出表格;

(2) 小结 A/D 转换芯片与 CPU 的接口及编程方法;

(3) 若程序中不是用延时等待 A/D 转换完成,而是用查询或中断的方法,程序应该如何编写。

实验十二　数字录音机

1. 实验目的

(1) 了解数字录音技术的基本原理;

(2) 通过数字录音进一步掌握 A/D 转换器与 D/A 转换器的使用方法。

2. 实验内容

(1) 按图 14-46 连接电路,将声传感器接 J_2,把代表语音的电信号送给 ADC0809 通道 2(IN_2);D/A 转换器的输出端 J_1 接喇叭。

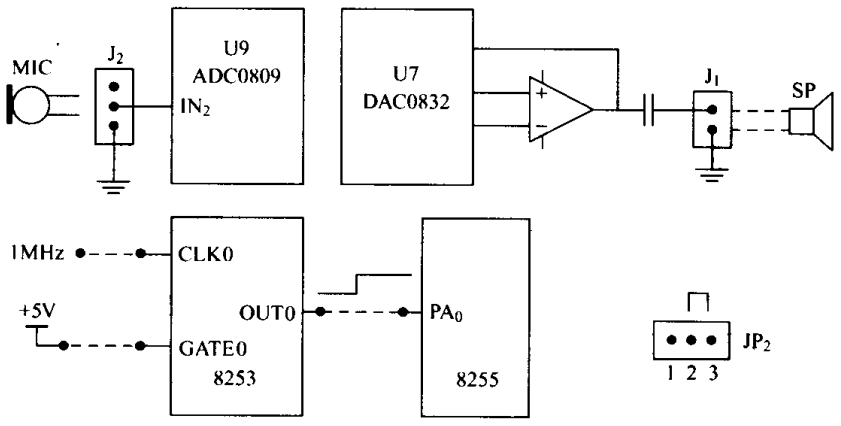


图 14-46　数字录音机实验连接电路图

(2) 编写程序,以每秒钟 5000 次的速率(以 8253 作为定时)采集 IN_2 输入的语音数据并存入内存,共采集 60000 个数据(录 12 秒),然后再以同样的速率将数据送 DAC0832 使喇叭发声(放音)。

3. 编程提示

(1) 将 8253 设置成方式 0,计数值为 200(定时 0.2ms),利用 PA_0 查询 OUT0 电平,若为高电平则表示定时时间到。

(2) ADC0809 通道 2(IN_2)的端地址为 298H。

(3) 流程图

程序分为主程序、定时(8253 定时)子程序、录音子程序和放音子程序等四个部分,它们的流程图分别如图 14-47、图 14-48、图 14-49 和图 14-50 所示。

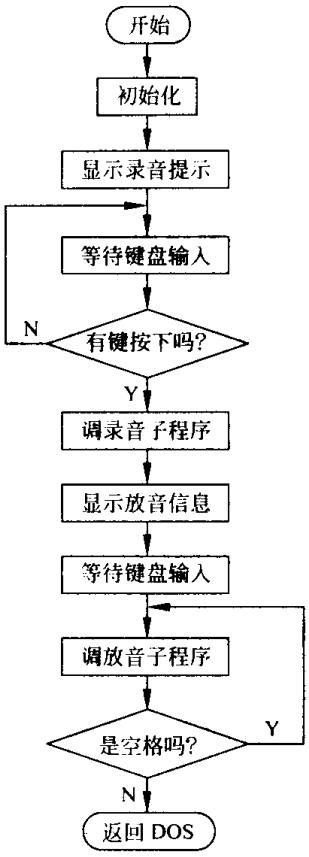


图 14-47　主程序

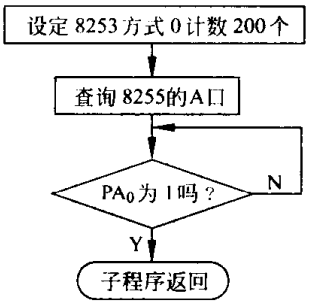


图 14-48　延时(定时)子程序

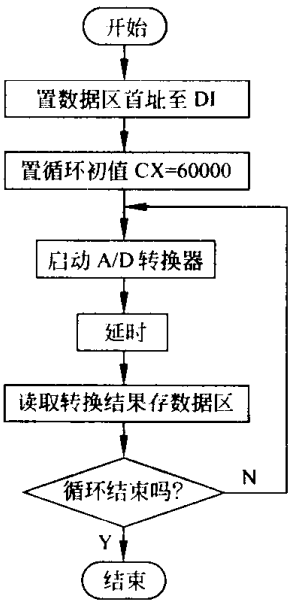


图 14-49　录音子程序

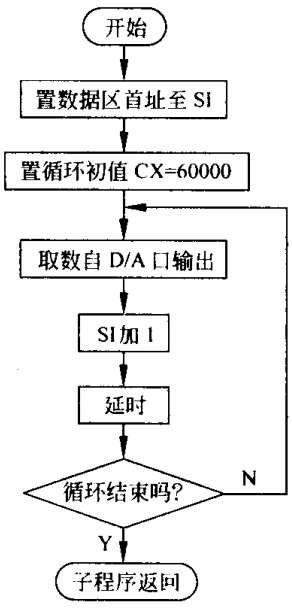


图 14-50　放音子程序

4. 预习要求

(1) 复习 A/D 转换、D/A 转换以及可编程定时器/计数器 8253 的使用；

(2) 仔细阅读本实验指导书；

(3) 按照流程图编好程序。

5. 报告要求

(1) 小结定时采样的方法；

(2) 若 8253 定时时间到，采用中断方式实现，程序应该如何编。

(3) 若设计一个实验，把 A/D 的输出作为 D/A 的输入，测量 A/D 输入的模拟电压与 D/A 输出的模拟电压，以进行验证。

实验十三　串 行 通 信

1. 实验目的

(1) 了解串行通信的基本原理；

(2) 掌握串行接口芯片 8251 的工作原理和编程方法。

2. 实验内容

(1) 按图 14-51 连接好电路(8251 插入通用插座)。其中，8253 计数器用于产生 8251 的发送和接收时钟，TxD 和 RxD 连在一起(自发自收)。

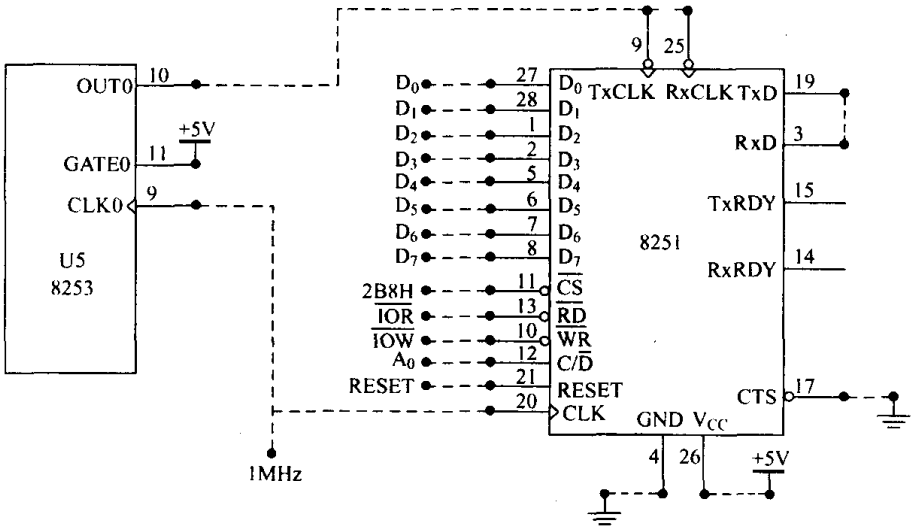


图 14-51　串行通信电路

(2) 编程：从键盘输入一个字符，将其 ASCII 码加 1 后发送出去，再接收回来并且在屏幕上显示，实现自发自收。

3. 编程提示

(1) 在图 14-51 所示电路中 8251 的控制端口地址为 2B9H，数据端口地址为 2B8H。

(2) 8253 计数器的计数初值＝时钟频率/(波特率×波特率因子)，这里的时钟频率接 1MHz，波特率若选 1200，波特率因子若选 16，则计数器初值为 52。

(3) 收发采用查询方式。

(4) 流程图如图 14-52 所示。

4. 预习要求

(1) 复习串行通信的特点以及 8251 的编程方法；

(2) 仔细阅读本实验指导书；

(3) 根据流程图编好程序。

5. 报告要求

(1) 小结 8251 的编程方法；

(2) 若对 8251 发送和接收的字符进行偶校验，则程序应该如何编写。

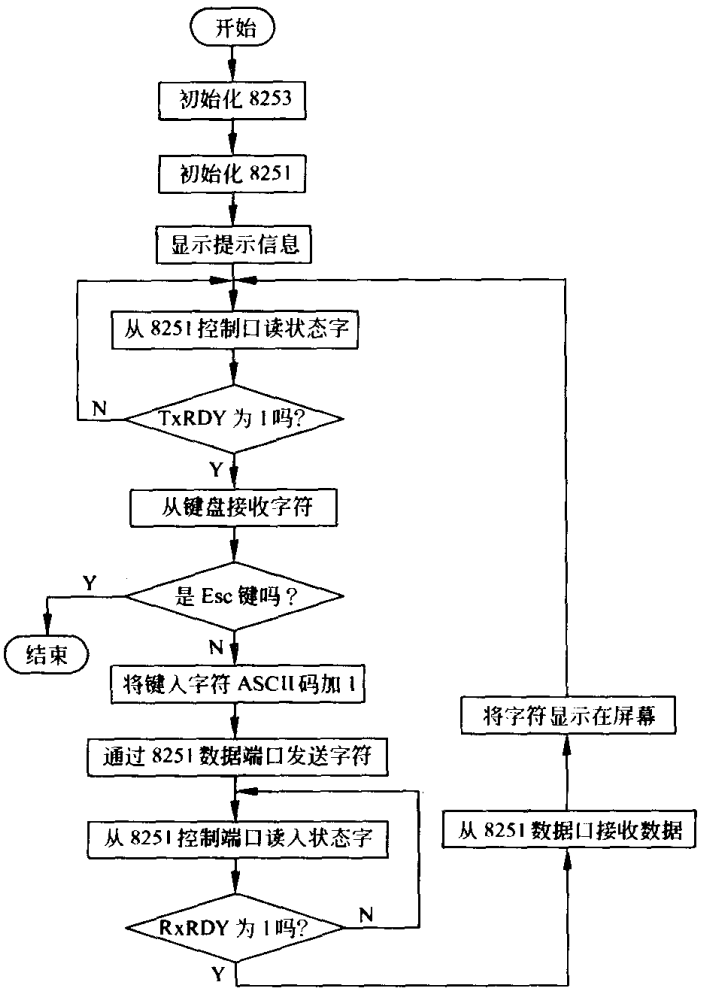


图 14-52　程序流程图

实验十四　DMA 传 送

1. 实验目的

(1) 掌握 PC 机工作环境下进行 DMA 方式传送数据的方法。

(2) 掌握 DMA 控制器 8237 的编程方法。

2. 实验内容

(1) 用通用插座按图 14-53 把电路连接好(74LS273 插通用插座，74LS74 利用实验台上的 D 触发器)，利用 DMA 控制器 8237 把内存数据传送至外设(74LS273，由 $L_0 \sim L_7$ 显示)。在内存 6000H:0 开始单元存放 10 个数据，对 DMA 控制器 8237 进行初始化编程，使每一次 DMA 请求(用单脉冲发生器通过 74LS74 D 触发器发出)从内存向外设传送一字节的数据。

(2) 用通用插座按图 14-54 把电路连接好(74LS244 插通用插座,74LS74 利用实验台上的 D 触发器),由 DMA 控制器从外设输入数据至内存。编程在内存 6000H:00 开辟一个 8 字节数据缓冲区,对 DMA 控制器 8237 进行初始化,每一次 DMA 请求(仍由单脉冲发生器通过 D 触发器发出)从外设向内存传送一个字符(ASCII 码),存入数据缓冲区,编程不断读取显示缓冲区的数据(ASCII)并且在屏幕上显示。

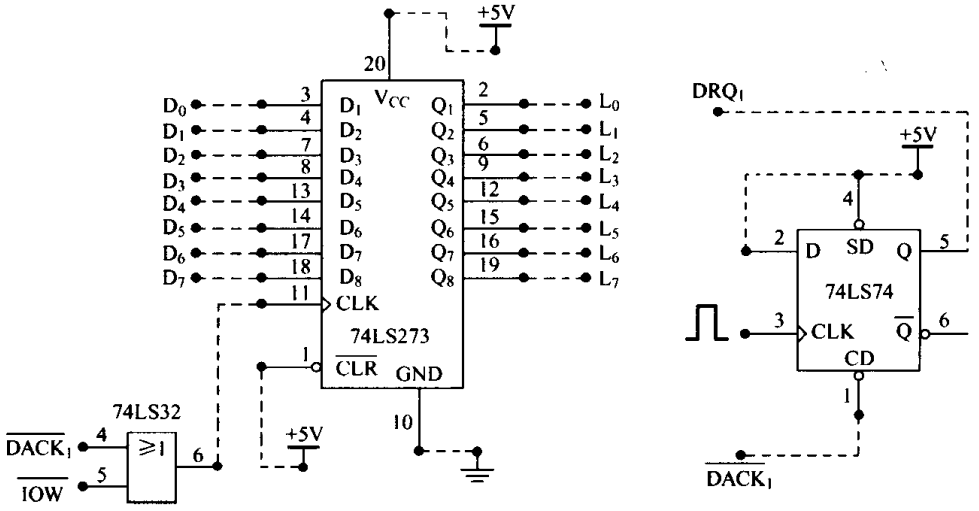


图 14-53 通过 DMA 控制器输出数据

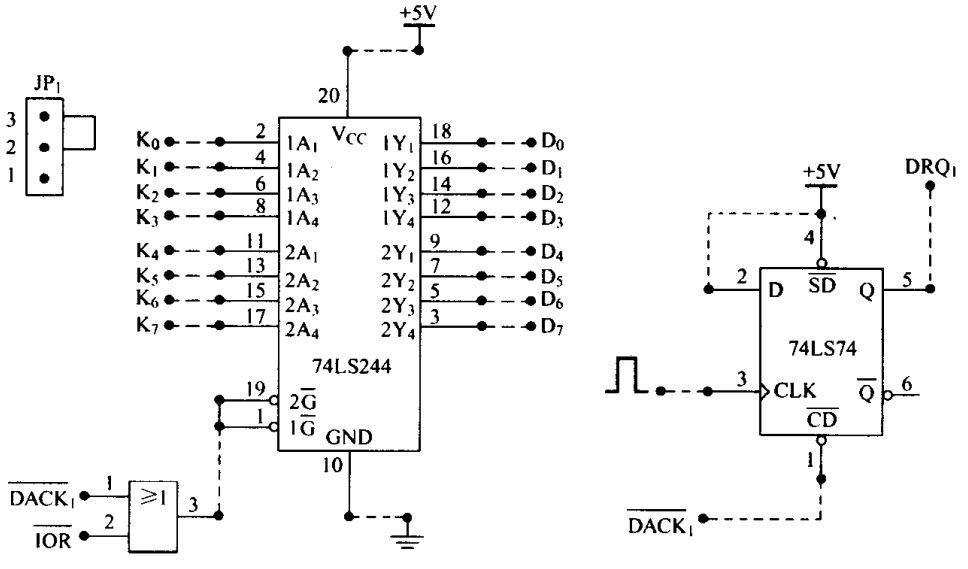


图 14-54 通过 DMA 控制器输入数据

3. 编程提示

(1) DMA 实验应该将实验台上跳线开关 JP_1 的 2、3 短接。

(2) DMA 请求是由单脉冲输入到 D 触发器,再由触发器的 Q 端向 DRQ_1 发出的,8237 在接收到 DRQ_1 后,向 CPU 发出 DMA 请求 HOLD。CPU 响应后向 8237 发出 HDLA 信号,由 8237 发出有效的 $DACK_1$ 信号将 D 触发器置成低电平以撤销请求。

(3) 流程图

通过 8237 从内存输出数据至外设的流程图如图 14-55 所示。

通过 8237 输入数据的流程图如图 14-56 所示。

4. 预习要求

(1) 复习 DMA 传送的方法和 8237 DMA 控制器的编程方法；

(2) 仔细阅读本实验指导书，了解 D 触发器、8237DMA 控制器和 74LS273、74LS274 之间的相互关系与连线；

(3) 根据流程图编好程序。

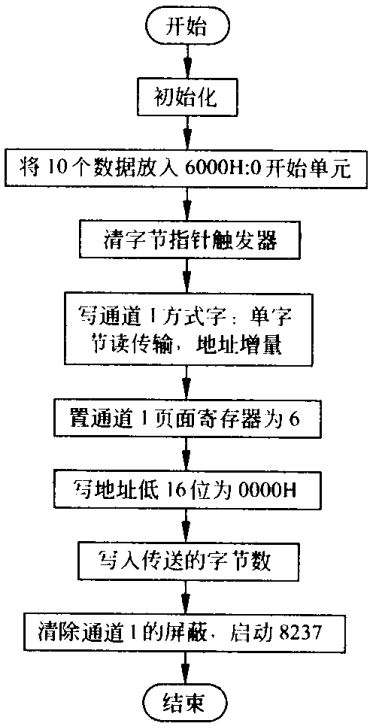


图 14-55 通过 DMA 控制器输出数据的程序流程图

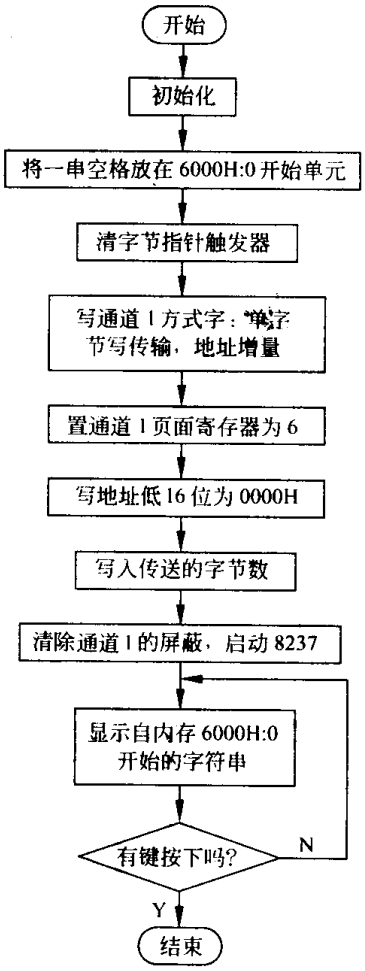


图 14-56 通过 8237 输入数据的程序流程图

5. 报告要求

(1) 小结 DMA 传送方式与程序传送方式的区别；

(2) 小结 8237DMA 控制器的编程；

(3) 考虑若用 8237 在内存间传送信息，则电路图应该如何连接，程序应该怎样编写。

实验十五 步进电机控制实验

1. 实验目的

(1) 了解步进电机控制的基本原理；

(2) 掌握控制步进电机转动的编程方法；

(3) 进一步了解 8255 的使用。

2. 实验内容

(1) 按图 14-57 连接线路，利用 8255 输出(通过 $PA_0 \sim PA_3$)的脉冲序列，以及开关 $K_0 \sim K_6$(通过 $PC_0 \sim PC_6$ 输入)控制步进电机的转速，K_7(通过 PC_7)控制步进电机的转向。实验盒上步进电机插头接实验台上 J_4。8255 的选片信号 $\overline{CS}$ 接 288H～28FH。$PA_0 \sim PA_3$ 接步进电机的线圈 BA～BD；$PC_0 \sim PC_7$ 接 $K_0 \sim K_7$。

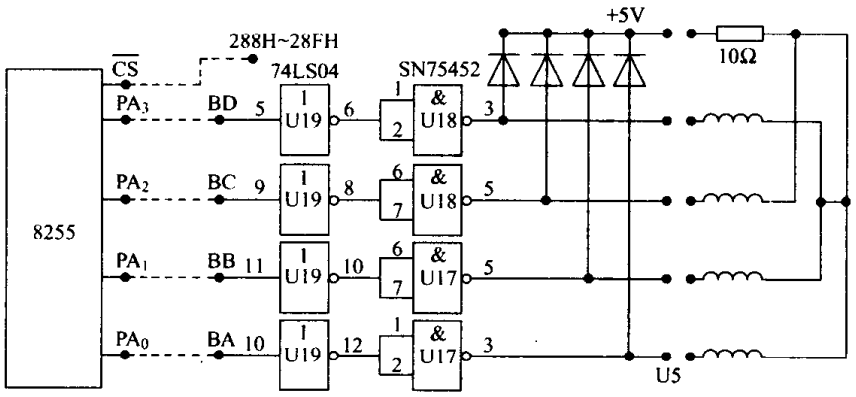


图 14-57 步进电机控制电路

(2) 编程：

当 $K_0 \sim K_6$ 中某一开关为“1”(向上拨)时，步进电机启动。K_7 向上打则电机正转，向下打则电机反转。

3. 编程提示

步进电机驱动原理是通过对每相线圈中的电流的顺序切换(实验中的步进电机有四相线圈，每次有二相线圈有电流，有电流的相顺序变化，如表 14-3 所示)来使电机作步进式旋转。驱动电路由脉冲信号来控制，所以调节脉冲信号的频率便可以改变步进电机的转速。

本实验使用的步进电机用直流＋5V 电压，每相电流为 0.16A，如图 14-58 所示。

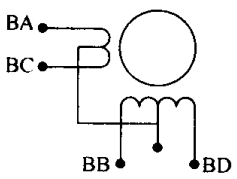


图 14-58 步进电机的线圈

电机线圈由四相组成：

即：Φ_1(BA)　Φ_2(BB)

　　Φ_3(BC)　Φ_4(BD)

表 14-3 步进电机激磁方式

顺序 \ 相	Φ_1	Φ_2	Φ_3	Φ_4
0	1	1	0	0
1	0	1	1	0
2	0	0	1	1
3	1	0	0	1

驱动方式为二相激磁方式，如表 14-3 所示。

表 14-3 中，首先使 Φ_1 线圈和 Φ_2 线圈有驱动电流，接着使 Φ_2 和 Φ_3，Φ_3 和 Φ_4 又返回到 Φ_1 和 Φ_2 有驱动电流，按照这种顺序切换，电机轴按顺时针方向旋转；若按 Φ_1 和 Φ_4、Φ_4 和 Φ_3……这样的顺序向线圈供电，则电机按照逆时针方向旋转。

实验可通过不同长度的延时（由 $K_0 \sim K_6$ 不同开关的闭合来控制）来得到不同频率的步进电机输入脉冲，从而得到多种步进速度。

程序的流程图如图 14-59 所示。

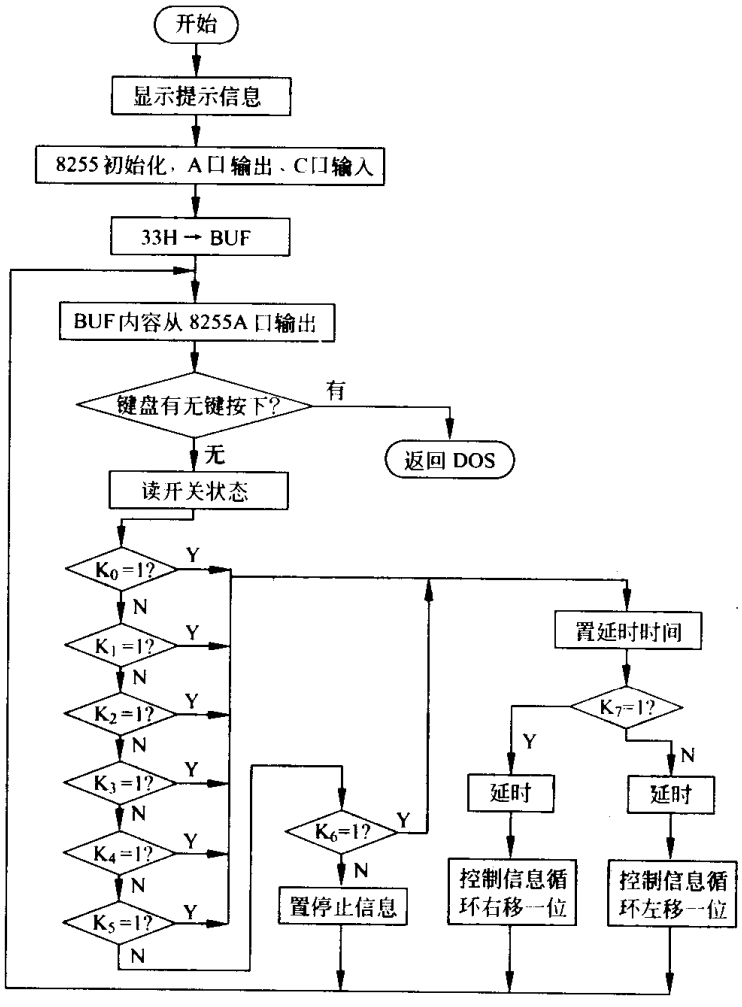


图 14-59　步进电机控制实验程序流程图

4. 预习要求

（1）仔细阅读本实验指导书，了解步进电机的工作原理以及实验电路和实验要求；

（2）根据流程图编好程序。

5. 报告要求

（1）小结步进电机的编程方法以及如何改变步进电机的转速与转向；

（2）考虑若有六相步进电机，采用四相激磁法，电路应该如何连接，如何编程。

实验十六　小直流电机转速控制实验

1. 实验目的

(1) 进一步了解 DAC0832 的性能及编程方法；

(2) 了解直流电机控制的基本方法。

2. 实验内容

(1) 按照图 14-60 线路接线。DAC0832 的$\overline{CS}$接 290H～297H，U_b 接 DJ 插孔，实验盒上直流电机插头接实验台 J_6，8255 的选片信号$\overline{CS}$接 288H～28FH。

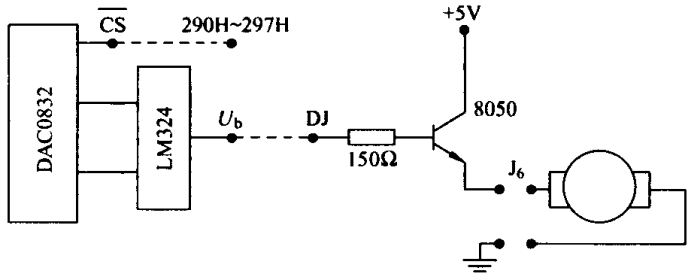


图 14-60　直流电机控制电路

(2) 编程，利用 DAC0832 输出一串脉冲，经放大后驱动小直流电机，利用开关 K_0～K_5 控制改变输出脉冲的电平及持续时间，达到使电机加速或减速之目的。

3. 实验原理简述

小直流电机的转速是由 U_b 输出脉冲的占空比来决定的，正向占空比越大则转速越快，反之则越慢，如图 14-61 所示。

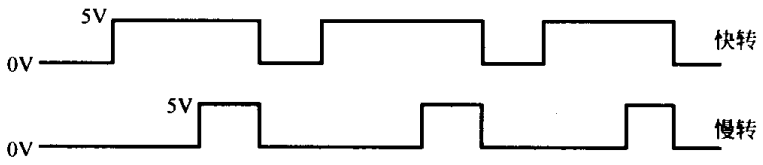


图 14-61　控制电机转速的脉冲

在本实验中，模拟量输出 U_b 为双极性，当输入数字量小于 80H 时则输出为负，输入等于 80H 时则输出为 0V，输入大于 80H 时则输出为正，因而在本实验中，DAC0832 输入数字量只有 2 个(80H 和 FFH)，通过不同的延迟时间达到改变小电机转速的目的。

程序的流程图如图 14-62 所示。

4. 预习要求

(1) 仔细阅读本实验指导书，了解控制小直流电机转速的原理；

(2) 根据流程图编好程序。

5. 报告要求

(1) 小结控制直流电机转速的方法；

(2) 若要使电机从停止状态平稳上升直至最高速，或者由最高速逐步下降直至停止，应该如何编程？

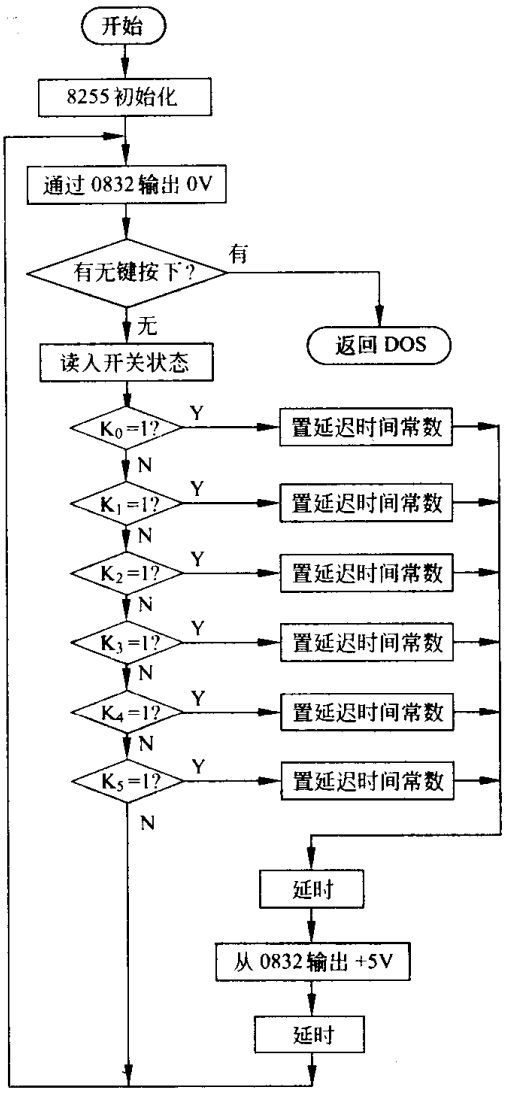


图 14-62　控制电机转速的流程图

参 考 文 献

1 Intel 公司. IA-32 Intel Architecture Software Developer's Manual, Volume 1: Basic Architecture

2 Intel 公司. IA-32 Intel Architecture Software Developer's Manual, Volume 2: Instruction Set Reference

3 Intel 公司. IA-32 Intel Architecture Software Developer's Manual, Volume 3: System Programming Guide

4 易先清，莫松海，喻晓峰等编著. 微型计算机原理与应用. 北京：电子工业出版社，2001

5 周明德，张淑玲. 80x86、80x87 结构与汇编语言程序设计. 北京：清华大学出版社，1993

Document generated by Anna’s Archive around 2023-2024 as part of the DuXiu collection (https://annas-blog.org/duxiu-exclusive.html).

Images have been losslessly embedded. Information about the original file can be found in PDF attachments. Some stats (more in the PDF attachments):

{
  "filename": "6auY562J5a2m5qCh5pWZ5p2QICDpq5jliIblrZDnp5HlrabkuI7lt6XnqIvns7vliJfmlZnmnZAgIOmrmOWIhuWtkOeJqeeQhuWunumqjF8xMTQ1MzE3MS56aXA=",
  "filename_decoded": "\u9ad8\u7b49\u5b66\u6821\u6559\u6750 \u9ad8\u5206\u5b50\u79d1\u5b66\u4e0e\u5de5\u7a0b\u7cfb\u5217\u6559\u6750 \u9ad8\u5206\u5b50\u7269\u7406\u5b9e\u9a8c_11453171.zip",
  "filesize": 11444569,
  "md5": "39cc5028de40952147ae8748bc23fb30",
  "header_md5": "99198bf96ca7a440f65d43773b71bcb5",
  "sha1": "433cf3b7aa69b0f61c2aa7594899b2db8de3b9ac",
  "sha256": "bbe154dd3048ed4b775a00d5d26d0374b2473f33950c1ed12ae4a47d8b3477b8",
  "crc32": 4263161629,
  "zip_password": "",
  "uncompressed_size": 12542740,
  "pdg_dir_name": "\u2555\u2580\u2561\u255a\u2564\u00ba\u2568\u00fa\u255c\u2560\u2593\u2500 \u2555\u2580\u2556\u2553\u256b\u2559\u2510\u255e\u2564\u00ba\u2559\u03b4\u2563\u00f1\u2502\u2560\u2567\u2561\u2534\u2568\u255c\u2560\u2593\u2500 \u2555\u2580\u2556\u2553\u256b\u2559\u256c\u2229\u2514\u03c6\u2569\u2561\u2564\u0398_11453171",
  "pdg_main_pages_found": 193,
  "pdg_main_pages_max": 193,
  "total_pages": 201,
  "total_pixels": 1290516480,
  "pdf_generation_missing_pages": false
}